
ADVANTEST®

ADVANTEST CORPORATION

ATL-51
Pattern Program Reference
Manual

MANUAL NUMBER 8256292-27

Applicable Operating System
ASX/U-51

Applicable Systems

T5593
T5592
T5591
T5591R
T5586
T5585
T5581
T5581H
T5581P
T5571P

Trademarks and Registered Trademarks

The following are trademarks and registered trademarks used in this manual.

Other product names not listed here are as a general rule trademarks or registered trademarks of their respective owners.

- ADVANTEST, ADVANsite, ATLworks, and ATL are trademarks of Advantest Corporation.
- Direct RDRAM is a trademark of Rambus Inc.

Revision History

Rev.	Date	Notes
01	Feb 15/96	
02	Oct 18/96	
03	Mar 18/97	
04	Jun 30/97	
05	Jul 23/97	
06	Oct 30/97	
07	Mar 10/98	
08	May 8/98	
09	Jul 17/98	
10	Nov 5/98	
11	Jan 20/99	
12	Feb 10/99	
13	Jun 30/99	
14	Oct 25/99	
15	Mar 3/00	
16	Jun 20/00	
17	Dec 1/00	
18	Apr 10/01	
19	Aug 20/01	
20	Sep 25/01	
21	Jan 10/02	
22	Mar 18/02	
23	Apr 17/02	
24	Aug 9/02	
25	Nov 12/02	

Rev.	Date	Notes
26	May 15/03	
27	Jul 25/03	

Table of Contents

Preface	Preface-1
Important Points	Preface-1
Overview	Preface-3
Relevant Manuals	Preface-3
 1. General Information	1-1
1. 1 General Information of Pattern Program Reference Manual	1-1
1. 2 ALPG Configuration	1-2
 2. Pattern Program Instructions	2-1
2. 1 Pattern Program Configuration	2-1
2. 2 Compilation Control Instructions	2-5
2. 2. 1 MPAT Statement	2-5
2. 2. 2 ILMODE Statement	2-6
2. 2. 3 RDX Statement	2-6
2. 2. 4 MODE Statement, MAIN MODE Statement, SUB MODE Statement	2-7
2. 2. 5 REGISTER Statement, MAIN REGISTER Statement, SUB REGISTER Statement	2-22
2. 2. 6 ARIRAM DEFINE Statement	2-32
2. 2. 7 ADDRESS DEFINE Statement	2-36
2. 2. 8 BURST BLOCK Statement	2-39
2. 2. 9 DPUTYPE Statement	2-41
2. 2.10 SDEF Statement	2-42
2. 2.11 SCRAM Statement/PRESCRAM Statement	2-43
2. 2.12 START Statement	2-43
2. 2.13 PCC Statement	2-44
2. 2.14 DATA DEFINE Statement	2-46
2. 2.14. 1 PDS DATA Statement	2-46
2. 2.15 MODULE Statement	2-48
2. 2.16 AFM PATTERN Statement	2-49
2. 2.17 DBM PATTERN Statement	2-49
2. 2.18 CBM PATTERN Statement	2-49
2. 2.19 BWPE ADDRESS Statement	2-50
2. 2.20 BANK SAVE ADDRESS Statement	2-52

2. 2.21	BANK LOAD ADDRESS Statement	2-54
2. 2.22	BYTEMASK DEFINE Statement	2-56
2. 2.23	END Statement	2-66
2. 2.24	Test Pattern Programming Format	2-66
2. 2.25	%REMARK Statement	2-71
2. 2.26	%IFE Statement, %IFN Statement, %ENDC Statement	2-71
2. 2.27	INSERT Statement	2-75
2. 3	Pattern Sequence Controller	2-78
2. 3. 1	Index Loop	2-82
2. 3. 2	Subroutine	2-85
2. 3. 3	Operation Register Setting Instruction (SET reg. data)	2-86
2. 3. 4	Condition Flag Reference Branch (JNCn)	2-88
2. 3. 5	Data Flag Reference Branch (JZD)	2-89
2. 3. 6	Timer Flag Reference Branch (JET)	2-90
2. 3. 7	Match Flag Sense Instruction	2-91
2. 3. 8	Burst Data Transfer Instruction (OUT)	2-92
2. 3. 9	Pattern Sequence Control Register Set Instruction (STISP)	2-92
2. 3.10	WAIT	2-93
2. 3.11	STPS, STFL	2-93
2. 3.12	Control Bit	2-93
2. 4	Address Pattern Generation Instructions	2-96
2. 4. 1	Hardware Configuration of Address Generator Section	2-96
2. 4. 2	Configuration of Address Generation Instruction	2-99
2. 4. 3	Outline of Address Generation Instruction	2-100
2. 4. 4	Base Address Field	2-104
2. 4. 5	Current Address Field	2-112
2. 4. 6	Source Address Field	2-132
2. 4. 7	N Register Field	2-135
2. 4. 8	B Register Field	2-136
2. 4. 9	Address Inversion Field	2-137
2. 4.10	Refresh Counter Field	2-138
2. 4.11	D3, D4 Register Field	2-139
2. 4.12	Address Swap Field	2-142
2. 4.13	Instructions and Execution Cycle	2-143
2. 4.14	Notes on Program Generation	2-144
2. 5	Data Pattern Generating Instructions	2-146
2. 5. 1	Hardware Configuration for Data Pattern Generator Section	2-146
2. 5. 2	Configuration of Data Generation Instruction	2-149
2. 5. 3	Outline of the Data Generation Instruction	2-149
2. 5. 4	TP, TP2 Register Field	2-151
2. 5. 5	FP Function Field	2-153
2. 5. 6	DSD Field	2-162
2. 5. 7	Temporary Inversion of Data (/D, /D2)(Data Inversion Field)	2-163

2. 5. 8	Inversion with Data Flag (DFLG)	2-164
2. 5. 9	Regional Inversion	2-164
2. 5.10	Saving Output Data (PD)(Data Hold Field)	2-164
2. 5.11	Source Data Field	2-164
2. 5.12	BWPE(Block Write Pseudo Emulator) Field	2-165
2. 5.13	CBM Address Field	2-169
2. 5.14	DSC Field	2-169
2. 5.15	Instructions and Execution Cycle	2-172
2. 5.16	Notes on Program Generation	2-173
2. 6	Control Bit	2-174
2. 6. 1	Configuration of Control Instruction	2-174
2. 6. 2	TS Field	2-175
2. 6. 3	MUT Field	2-175
2. 6. 4	Data Pattern on the DUT Control Terminals	2-176
2. 6. 5	Specifying the Cycles for Performing Logical Comparison	2-177
2. 6. 6	Specifying to Turn Driver ON or OFF in I/O Operation	2-178
2. 6. 7	Specifying Real Time Inversion of the Driver	2-179
2. 6. 8	Address Multiplexing Control	2-180
2. 6. 9	Specifying Real Time Swap Cycle	2-181
2. 6.10	Specifying the Cycle to Perform Hi-Z Comparison in the Realtime Hi-Z Mode	2-181
2. 6.11	Specifying the Cycle which Controls the STRB and Clock in Auto-scan Mode	2-184
2. 6.12	Address Control Field	2-185
2. 6.13	SCRAM Control Field	2-185
2. 6.14	PRESCRAM Control Field	2-186
2. 6.15	ARIRAM Control Field	2-186
2. 6.16	PM Control Field	2-186
2. 6.17	Cycle Palette Field	2-187
2. 6.18	DBM Control Field	2-189
2. 6.19	X Address Save Field	2-190
2. 7	ROM Test Pattern Data Program Section	2-193
2. 7. 1	Programming Format	2-193
2. 7. 1. 1	AFM PATTERN Statement	2-193
2. 7. 1. 2	BITSIZE Statement	2-194
2. 7. 1. 3	AMAX Statement	2-195
2. 7. 1. 4	AMIN Statement	2-196
2. 7. 1. 5	FMA Statement	2-196
2. 7. 1. 6	ROM Test Pattern Data	2-197
2. 7. 2	Example of Programming	2-200
2. 8	DBM Pattern	2-202
2. 8. 1	Programming Format	2-202
2. 8. 2	DBM CHANNEL Statement	2-203

2. 8. 3	DBM PINSEL Statement	2-204
2. 8. 4	DBMA Statement	2-218
2. 8. 5	DBM Data Field	2-219
2. 8. 5. 1	REPEAT Statement	2-222
2. 8. 6	Control of Driver Enable/Comparator Enable	2-225
2. 8. 7	DBM STORAGE Statement	2-229
2. 9	CBM Pattern	2-230
2. 9. 1	Programming Format	2-231
2. 9. 2	CBM CHANNEL Statement	2-232
2. 9. 3	CBMA Statement	2-233
2. 9. 4	CBM Data Field	2-235
2. 9. 4. 1	REPEAT Statement	2-238
2.10	STISP, STIn Burst Test Condition Statement	2-239
2.10. 1	STISP Statement	2-239
2.10. 2	STIn Statement	2-240
2.11	Packet Instruction	2-242
2.11. 1	PKT Statement	2-242

3. Pattern Generation Instructions for Test Plan Programs 3-1

3. 1	Instructions	3-2
3. 2	Functional Test Execution	3-6
3. 3	Register Setting and Modification	3-11
3. 4	Using Failure Analysis Memory	3-12
3. 5	Shmoo Test	3-14
3. 6	Scramble Program Transfer	3-16
3. 7	Specifying Window Addresses	3-20

4. Programming I 4-1

4. 1	Power Supply Current Measurement	4-1
4. 2	Output Voltage Measurement (When output voltage is maintained)	4-5
4. 3	DRAM Test	4-11
4. 3. 1	Address Multiplexing	4-11
4. 3. 2	Pause Test	4-16
4. 3. 3	Refresh Test	4-17

4. 3. 4	Regional Inversion Test (ARIRAM)	4-41
4. 4	Bump Test (BURST BLOCK)	4-43
4. 4. 1	How to Specify Conditions in the Test Plan Program	4-43
4. 4. 2	How to Specify Conditions in the Pattern Program	4-46
4. 5	Partial Test (Z Address)	4-51
4. 5. 1	Partial Test Using Z Address	4-51
4. 5. 2	Programming	4-51
5.	Programming II	5-1
5. 1	Page Mode Test	5-1
5. 1. 1	Page Mode Test	5-1
5. 1. 2	Waveform Control by Control Bits	5-3
5. 2	Test for Multi-bit Memory Chips	5-7
5. 2. 1	Data Generation	5-7
5. 2. 2	I/O Control	5-9
5. 2. 3	Generation of Parity Data	5-11
5. 3	Read Modify Write Test	5-14
5. 3. 1	Read/Write Data Inversion	5-14
5. 3. 2	Programming	5-15
5. 4	EPROM Test	5-18
5. 4. 1	Using the Match Flag Sense Instruction	5-18
5. 5	Test for Dual Port Memory	5-19
5. 5. 1	Test for Dual Port Memory	5-19
5. 5. 2	MAIN/SUB PG Synchronous Mode Test	5-19
6.	Programming for Interleave Mode	6-1
6. 1	Specification of Interleave Mode	6-1
6. 2	Operation in Interleave Mode	6-2
6. 3	Programming Format for Interleave Mode	6-8
6. 4	Outline of Restrictions in Interleave Mode	6-11
6. 4. 1	Concept of Successive Two or Four Steps	6-12
6. 4. 2	Restriction-related Instruction and Unrelated Instruction	6-13
6. 4. 3	Instructions whose Use is Prohibited in Interleave Mode	6-15
6. 5	Restrictions on Each Instruction in Interleave Mode	6-16
6. 5. 1	DSET (Direct data SET) Instruction (register name<m m:set value) Restriction	6-16
6. 5. 2	XB and YB Register Operating Instruction Restriction	6-18

6. 5. 3	Z Register Operating Instruction Restriction	6-19
6. 5. 4	N Register Operating Instruction Restriction	6-21
6. 5. 5	B Register Operating Instruction Restriction	6-24
6. 5. 6	D1 to D4 Register Operating Instruction Restriction	6-26
6. 5. 7	D5 and D6 Register Specifying Restriction	6-28
6. 5. 8	XC and YC Register Operating Instruction Restriction	6-29
6. 5. 9	Carry (BX, CY)-attached Operating Instruction Restriction	6-35
6. 5.10	TP and TP2 Register Operating Instruction Restriction	6-43
6. 5.11	DSD, RCD, and CCD Register Operating Instruction Restriction	6-46
6. 5.12	RF Register Operating Instruction Restriction	6-48
6. 5.13	DSC Register Operating Instruction Restriction	6-50
6. 5.14	CBM Register Operating Instruction Restriction	6-52
6. 5.15	SWAP Instruction (LMAX<>HMAX) Describing Restriction	6-52
6. 5.16	Restriction on the RTN Instruction	6-54
6. 5.17	Restriction on Refresh Test	6-55
6. 5.18	Restrictions on BWPE control bit	6-55
6. 5.19	Restrictions when Specifying the XASV Instruction	6-57
6. 6	Conversion from Normal Mode to Interleave Mode	6-61
6. 7	Conversion from PCC 2 Mode to 2WAY Interleave Mode	6-62
6. 8	Maximum Pattern Generating Frequency in Each Mode, Normal, PCC, or Interleave	6-63
6. 9	Coexistence Among Normal, PCC and Interleaved Modes	6-64

Appendix 1.List of Pattern Program Instructions	A1-1
--	-------------

List of Figures	F-1
------------------------------	------------

List of Tables	T-1
-----------------------------	------------

Index	I-1
--------------------	------------

Preface

Important Points

- Test system descriptions

The test system descriptions in this manual apply to the following models unless specified otherwise.

Table P-1 Models Adapted for Test System Descriptions

Test system descriptions	Adapted models
T5500	T5593, T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P
T5591	T5591R
T5581	T5586, T5585, T5581H, and T5581P

- ALPG configuration

In a system that is equipped with two lines (four lines) of address/data generator sections inside the algorithmic pattern generator (ALPG), patterns can be generated with operating frequencies two times (four times) higher than that when operated by only one system, by interleaving generated address/data of two (four) systems. In addition, two types of patterns can be generated by allocating two (four) systems as the MAIN PG and SUB PG. In order to express these two (four) systems of address/data generator sections separately, they are expressed as PG UNIT1, and PG UNIT2 (PG UNIT1, PG UNIT2, PG UNIT3, and PG UNIT4). PG UNIT is allocated to MAIN PG and SUB PG by interleave mode as follows:

Table P-2 ALPG Configuration and Allocation of MAIN PG and SUB PG for Each Interleave Mode

Test system	Number of ALPG address/data generators	Interleave mode	PG UNIT allocation	
			MAIN PG	SUB PG
T5593 No SUBPG option	Two systems	2WAY	1, 2	None
		1WAY	1	2
T5593 With SUBPG option	Four systems	2WAY	1, 2	3, 4
		1WAY	1	2

Test system	Number of ALPG address/data generators	Interleave mode	PG UNIT allocation	
			MAIN PG	SUB PG
T5592	Four systems	4WAY	1, 2, 3, 4	None
		2WAY	1, 3	None
		1WAY	1	None
T5591	Four systems	4WAY	1, 2, 3, 4	None
		2WAY	1, 3	2, 4
		1WAY	1	2
T5586, T5585, T5581, T5581H, and T5581P	Two systems	2WAY	1, 2	None
		1WAY	1	2
T5571P	One system	1WAY	1	None



Tip

- In a T5593 with SUBPG option, there are four systems of address/data generator sections, but 4WAY interleave mode cannot be specified.
- In the T5592, there are four systems of address/data generator sections, but SUB PG cannot be specified.
- When there is no SUB PG, SUB cannot be specified in each control instruction.

It also applies to a scrambler, prescrambler, and ARIRAM because each PG UNIT has them.

- Compatibility of pattern programs

The pattern program is divided into the following three types depending on the test system:

1. For the T5500 series (ASX/U-51)
2. For the T5377, T5376, T5375, T5371, T5336, and T5311 (ASX/U-51)
3. For the T5300 series (MOST-50, ASX/U-50)

These types of pattern program are not compatible.

However, since they have many common functions, the pattern program of other test systems can be used with partial modification.

A pattern object file, which does not correspond to the test system, cannot be executed.

- Specifying interleave mode

1WAY, 2WAY, and 4WAY, which are described in this manual, are the specifications of interleave mode. The test system operates in each interleave mode within the following operation frequency range:

Table P-3 Specification of the Interleave Mode and Corresponding Operation Frequency

Interleave mode	T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P	T5592	T5593
4WAY	500 MHz or less	533 MHz or less	Cannot be specified
2WAY	250 MHz or less	266 MHz or less	533 MHz
1WAY	125 MHz or less	133 MHz or less	266 MHz

➔ For information on programming in interleaved mode, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Overview

Chapter 1 General Information

Chapter 1 describes the configuration of the whole ALPG.

Chapter 2 Pattern Program Instructions

Chapter 2 describes instructions, which are grouped according to their purposes. It also contains block diagrams of hardware for sequence control and for address and data generation.

Chapter 3 Pattern Generation Instructions for Test Plan Programs

Chapter 3 describes test plan program statements, which are related to pattern generation, and statements, which are described in the test plan program for controlling pattern program execution.

Chapter 4 and 5 Programming I and II

Chapters 4 and 5 describe methods of programming memory device tests. They also show the correspondence between the methods and test plan programs, as required.

Chapter 6 Programming for Interleave Mode

Chapter 6 describes programming methods, limitations and so on of the pattern program for ALPG interleaved mode which are adopted newly in T5500 series.

Relevant Manuals

ATL-51 Test Plan Program Reference Manual(No. : 8431498)

ATL-51 Pattern Program Reference Manual (T5377, T5376, T5375, T5371, T5336 and T5311)(No. : 8289058)

ATL-51 Socket Program Reference Manual(No. : 8256293)

ATL-51 Scramble Program Reference Manual(No. : 8256294)

T5500 Series Tester Utility Program Reference Manual(No. : 8256295)

T5500 Series Controller Command Reference Manual(No. : 8256297)

ATLworks Tool User's Manual(No. : 8311174)

ASX/U-51 Hybrid Controller User's Manual(No. : 8409598)

ASX/U-51 Hybrid Controller Reference Manual(No. : 8409600)

Error Code Table Reference Manual(No. : 8141091)

XATL/U-51 Cross Compiler User's Manual(No. : 8256237)

ADVANsite Test System Software Installation/System Start User's Manual Vol.1 Basic OS Installation(No. : 8227728)

ADVANsite Test System Software Installation/System Start User's Manual Vol.2 Tester Environment Software Installation(No. : 8227729)

T5500 Series Memory Test System Control Box/Trigger Panel User's Manual(No. : 8256298)

T5300 Series Flash Memory Parallel Measurement Function Description(No. : 8239605)

T5500 Series Memory Test System Calibration Program Reference Manual(No. : 8256296)

T5500 Series Memory Test System Precision Calibration Program Reference Manual(No. : 8311137)

T5500 Series Memory Test System Precision Calibration2 Program Reference Manual(No. : 8350393)

T5500 Series MPATsim User's Manual(No. : 8311103)

1. General Information

This chapter describes an overview of this manual and ALPG configuration.

1. 1 General Information of Pattern Program Reference Manual

This manual explains how to create a test pattern program for the T5500 series memory test system and outlines the algorithmic pattern generator (ALPG), which generates test patterns for each function.

1. 2 ALPG Configuration

The ALPG mainly consists of the three sections described below. [Figure 1-2](#) shows the ALPG function block diagram.

Control section

The control section controls the flow of the address patterns and data patterns entered into the DUT. It consists of the instruction memory, Writable Control Storage (WCS) in which operation or selection instructions to be entered in the address or data generator section are stored, the sequence control section, and the DUT control signal generation circuit which generates the DUT control signal.

➔ For more information on the function block diagram, refer to [2. 3 "Pattern Sequence Controller" on page 2-78](#).

Address generator section

The address generator section, which consists of the X, Y, and Z address generators, generates address pattern entered in the DUT at testing. The address generator section calculates addresses in real time with the address operation instruction from the control section at testing, and generates the address patterns.

➔ For more information on the function block diagram, refer to [2. 4. 1 "Hardware Configuration of Address Generator Section" on page 2-96](#).

Data generator section

The data generator section generates data patterns to be entered in the DUT when writing, and expected patterns used for determining values read from the DUT during reading.

➔ For more information on the function block diagram, refer to [2. 5. 1 "Hardware Configuration for Data Pattern Generator Section" on page 2-146](#).

Figure 1-1 ALPG Function Block Diagram (T5593)

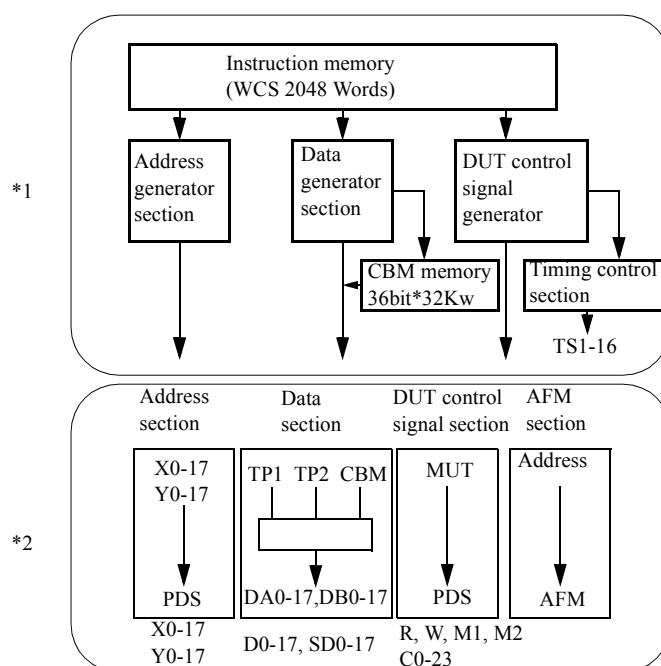
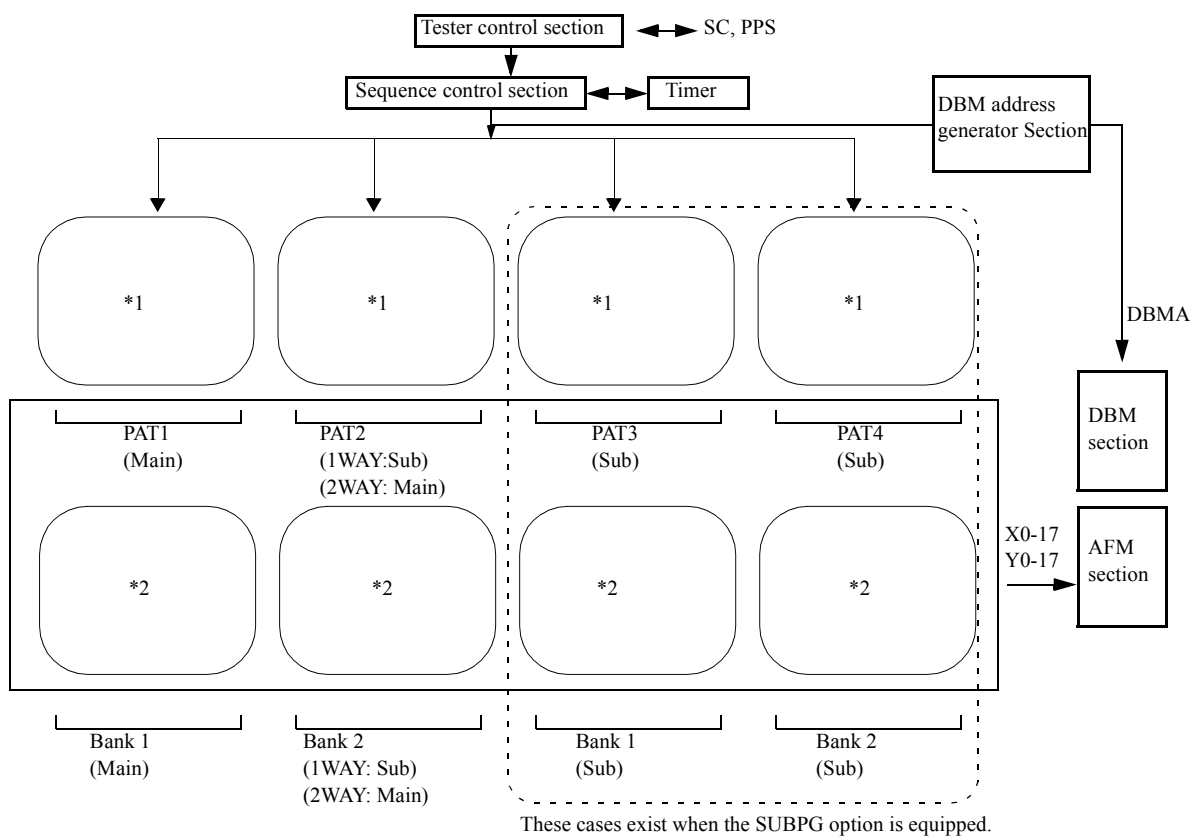
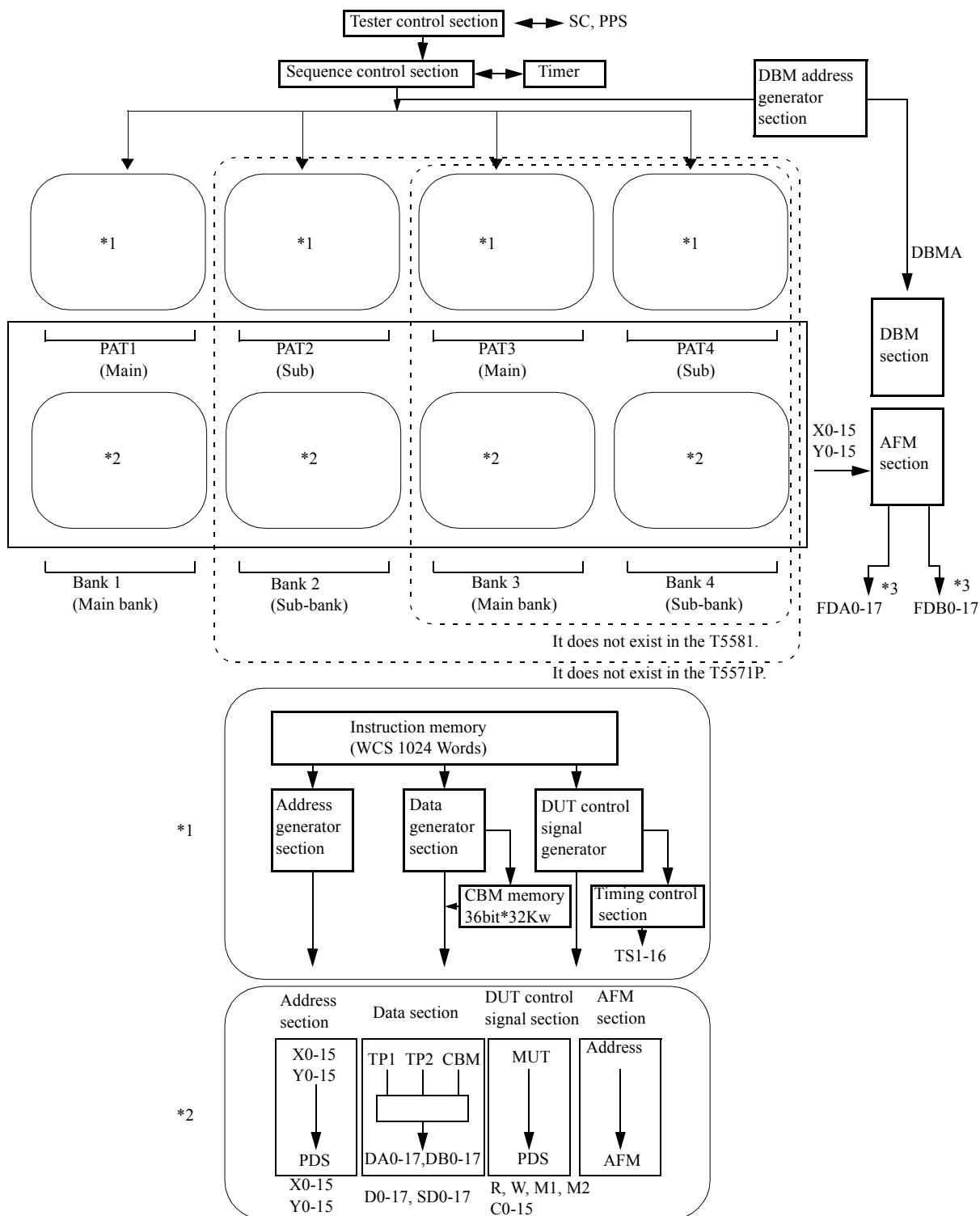


Figure 1-2 ALPG Function Block Diagram (T5592, T5591, T5586, T5585, T5581T, T5581H, T5581P, and T5571P)



2. *Pattern Program Instructions*

This chapter explains pattern program configuration and instructions.

This chapter also shows the block diagrams of hardware used to generate patterns.

2. 1 Pattern Program Configuration

The pattern program begins with the MPAT statement and ends with the END statement. It consists of the common section and the module section as shown in ["The Common Section and Module Section of a Pattern Program" on page 2-2](#).

Each pattern program is described in the module section. Settings of data common to modules (for example, settings of data and registers such as XMAX and YMAX defining the memory capacity) is described in the common section.

Up to 29 modules can be coded in the module section.

Instructions can be written in a module as shown in ["Sample MODULE Description" on page 2-3](#).

Data common to modules is set in the common section. Instructions can be written in the section as shown in ["Sample Common Section Description" on page 2-4](#).

Conditions for setting a pattern program can be changed by the test plan program.

➔ For more information, refer to [3. "Pattern Generation Instructions for Test Plan Programs" on page 3-1](#) and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

A pattern program to be executed at testing is selected according to the following instructions in the test plan program:

```
REG MPAT PC = n
```

The above n is an address specified by the START statement in the MODULE section.

If data set for the same item is coded by the module section, the common section, and the test plan program, the data priority follows the order shown below:

Test plan program > Module section > Common section

♦ The Common Section and Module Section of a Pattern Program

MPAT P64K

REGISTER

XMAX=#7F

YMAX=#1FF

IDX2=#FFFE

Common section

```
SDEF    INIT=XB<0  YB<0  TP<0
```

```
SDEF  BIN C=XB<XB+1  YB<YB+1^BX
```

```
MODULE BEGIN      ;CHECKER
```

START #0

IDXI1 #6 INIT

```
ST0:  JN12  ST0  W  BIN  FP1
```

```
ST1:  JNI2  ST1  R  BINC  FP1
```

JZD ST0

STPS

MODULE END

```
MODULE BEGIN          ; PAUSE
```

MODE PAUSE

REGISTER

TIMER=2MS

START #10

IDXI1 #6 INIT

```
ST0:      JNI2      ST0  W      BINC  FP1
```

NOP T

```
ST1:      JNI2      ST1  R      BINC  FP1
```

JZD ST0

STPS

MODULE END

END

Module sectionUp to 29 modules
can be written

◆ Sample MODULE Description

```
MODULE BEGIN

    RDX  16

    REGISTER
    XMAX=#FF
    YMAX=#3FF

    ARIRAM DEFINE
    X=X7
    INV=X

    DATA DEFINE
    PDS DATA=D/D

    BURST BLOCK BEGIN (A)
    VS1=4.5V

    BURST BLOCK END

    SDEF  INIT=XB<0 YB<0 TP<0
    SDEF  INC=XB<XB+1 YB<YB+1^BX

    SCRAM S256K

    START  #0

    IDXI1  #6      INIT
ST0:     JN12  ST0  W  INC  FP1
ST1:     JN12  ST1  R  INC  FP1
        JZD   ST0
        STPS

MODULE END
```

◆ Sample Common Section Description

```
MPAT program-name

RDX    16

REGISTER
  XMAX=#FF
  YMAX=#3FF

ARIRAM DEFINE
  X=X7
  INV=X

DATA DEFINE
  PDS DATA=D/D

BURST BLOCK BEGIN (A)
  VS1=4.5V

BURST BLOCK END

SDEF  INIT=XB<0 YB<0 TP<0
SDEF  INC=XB<XB+1 YB<YB+1^BX

SCRAM S256K

MODULE BEGIN

    :

MODULE END

MODULE BEGIN

    :

MODULE END

END
```

2. 2 Compilation Control Instructions

2. 2. 1 MPAT Statement

Function: MPAT statement specifies a pattern program name and an interleaved mode.

Syntax

```
MPAT program-name <IL-mode>
```

Argument: program-name

It is the name of a pattern program.

This name is used as the name of an object file name after compilation and becomes an identification name when a pattern file is specified in the test plan program.

A pattern program name must be specified by using up to 32 characters which include alphanumeric characters, periods (.), and dollar signs (\$).

<IL-mode>

It is the specification of the ALPG interleave mode. The following can be specified:

<1WAY> : Specification of 1WAY mode (normal mode)

<2WAY> : Specification of 2WAY mode

<4WAY> : Specification of 4WAY mode

Always enclose with < > when specifying modes.

Interleaved mode can be omitted. In that case, 1WAY mode is set.

➔ For information on programming in interleaved mode, refer to [6. "Programming for Interleave Mode" on page 6-1](#).

Tip

- In the following situations, MAIN or SUB cannot be specified individually by using each control instruction. Also, be cautious about a pattern writing syntax difference and so on.

T5593 (SUBPG option not equipped) 2WAY interleave mode

T5592

T5591 4WAY interleave mode

T5581 2WAY interleave mode

- The interleave mode can be specified with the ILMODE statement too. The specification by the ILMODE statement takes precedence over the specification in the title statement.

- *The PCC statement cannot be specified in the pattern program where the 2WAY/4WAY interleaving was specified.*
- *T5571P does not support the 2WAY or 4WAY mode pattern program. T5593 and T5581 do not support the 4WAY mode pattern program.*

2. 2. 2 ILMODE Statement

Function: ILMODE statement specifies the interleaved mode.

Syntax

```
ILMODE 1WAY
ILMODE 2WAY
ILMODE 4WAY
```

Description

Specify either 1WAY, 2WAY, or 4WAY before the START statement, [MAIN/SUB] DBM PATTERN statement or [MAIN/SUB] CBM PATTERN statement as above.

The interleaved mode can be specified in the line of the title statement.

Also, it is possible to specify in modules. In that case, the order of priority is as follows.

1. ILMODE statement in the module part
2. ILMODE statement in the common part
3. Interleaved mode specified in the line of title statement

Smaller number takes precedence.

The specification of title statement line or ILMODE statement specified for the common part becomes default value.

Tip

When a PCC statement has been specified in the pattern program, ILMODE 2WAY or ILMODE 4WAY cannot be specified.

2. 2. 3 RDX Statement

Function: RDX statement defines the radix of a constant entered in the pattern program.

Syntax

```
RDX  n
```

Argument: n

A decimal number $2 \leq n \leq 16$ can be specified. If this instruction is omitted, the default value is 16.

However, when the constant is written in hexadecimal, write "0" or "#" before the first character if the first character is A to F.

2. 2. 4 MODE Statement, MAIN MODE Statement, SUB MODE Statement

Function: The MODE statement is used to specify the operation mode of ALPG.
Specify before the START statement or the [MAIN/SUB] CBM PATTERN statement.

Syntax

(1) When Specified by the MODE Statement

```
MODE mode1 [ mode2 ....]
```

A mode specified both in main PG and sub PG becomes valid.

(2) When Specified by the MAIN MODE Statement

```
MAIN MODE mode1 [ mode2 ....]
```

A mode specified in main PG only becomes valid (except common modes for MAIN and SUB).

(3) When Specified by the SUB MODE Statement

```
SUB MODE mode1 [ mode2 ....]
```

A mode specified in sub PG only becomes valid.

Argument: mode

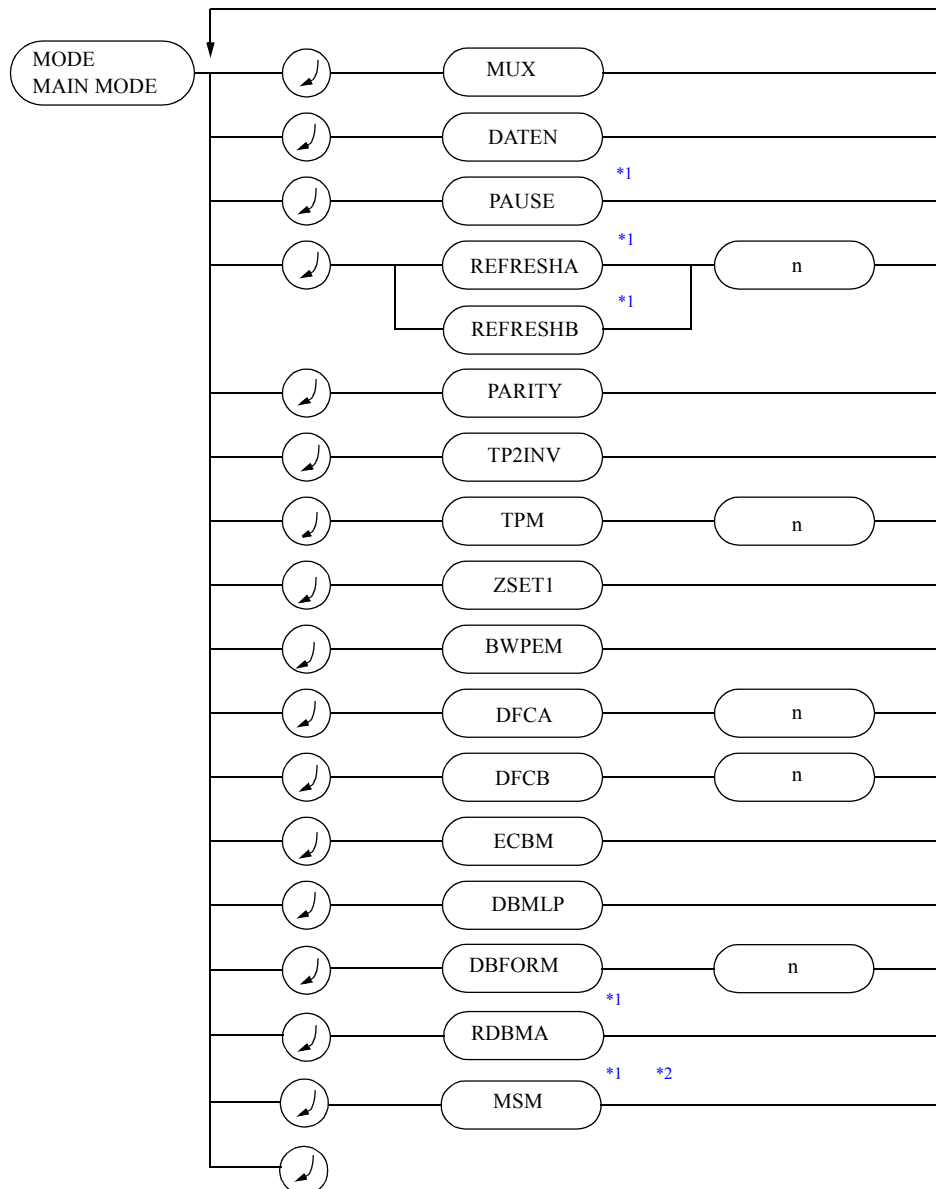
[Figure 2-1](#) and [Figure 2-2](#) show the operation modes that can be specified by each MODE statement.

— Tip —

When an operation mode is set in both module section and common section

The mode specified becomes valid in both sections. When different operation modes are specified in both the module section and the common section in REFRESHA and REFRESHB, the operation mode specified in the module section becomes valid.

Figure 2-1 Specifiable Operation Modes (T5593)



*1 This mode becomes valid for both MAIN and SUB even when MAIN is specified in MAIN and SUB command modes.

*2 MSM cannot be specified as DFCA_n, DFCB_n or TPM36 at the same time.

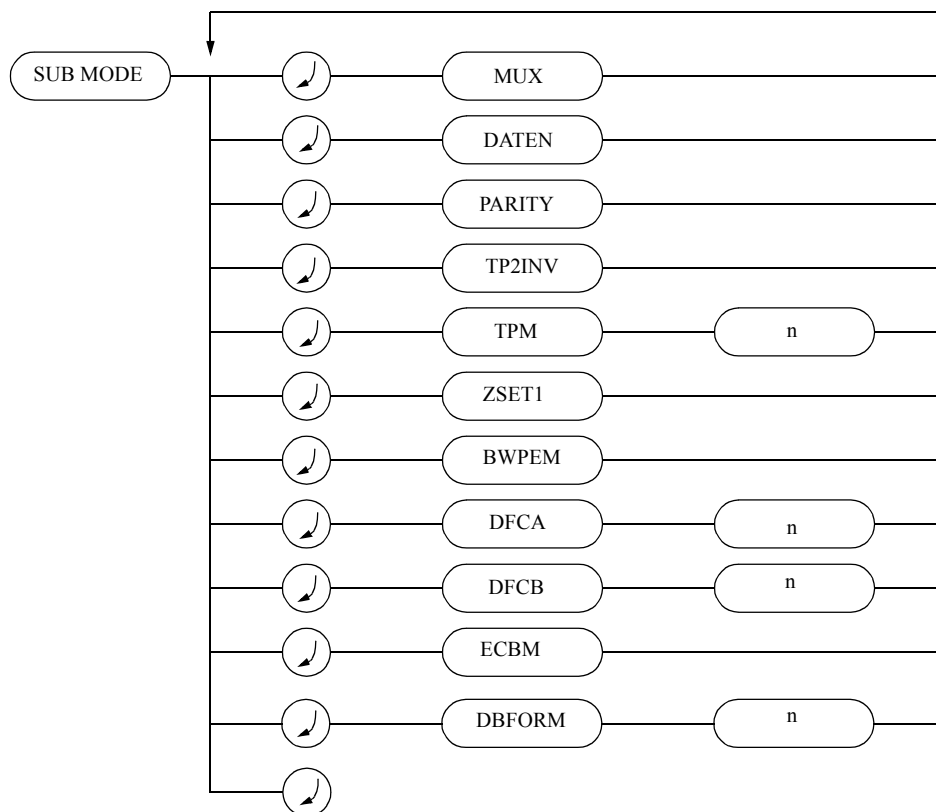
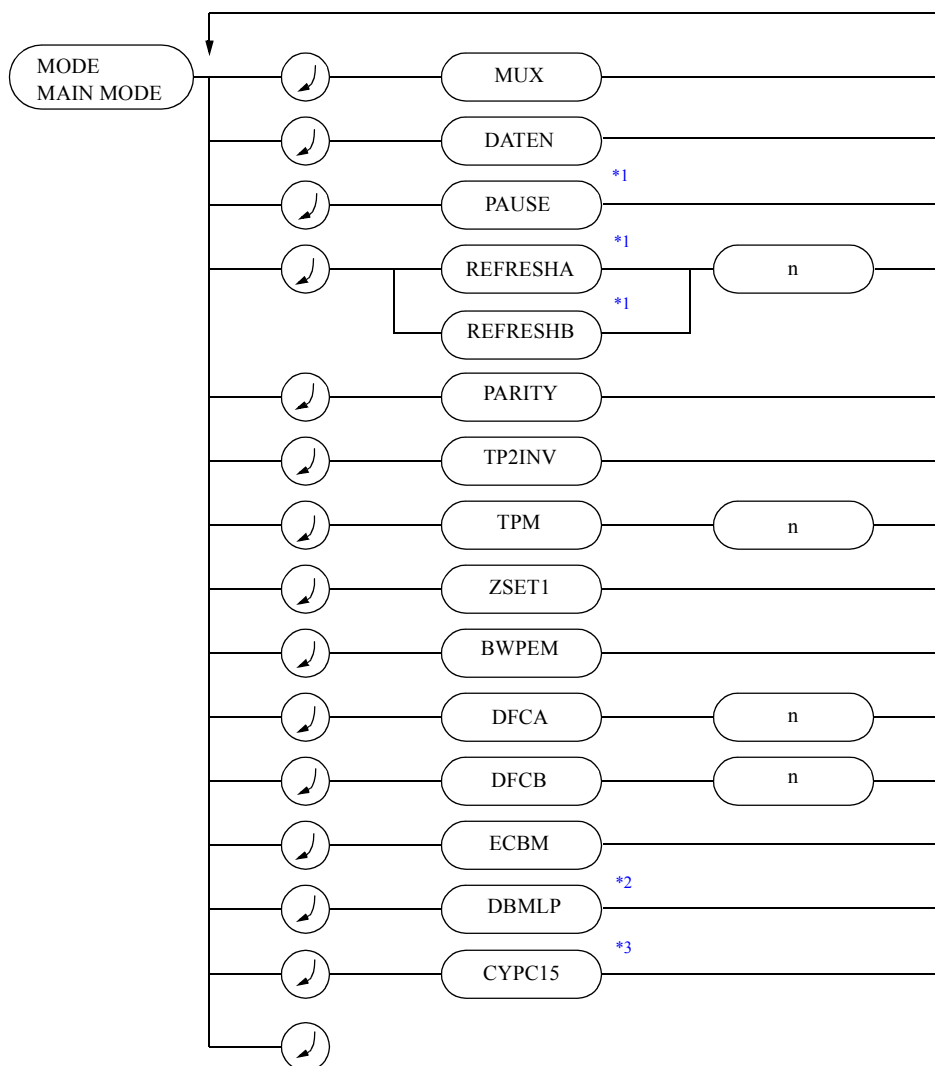


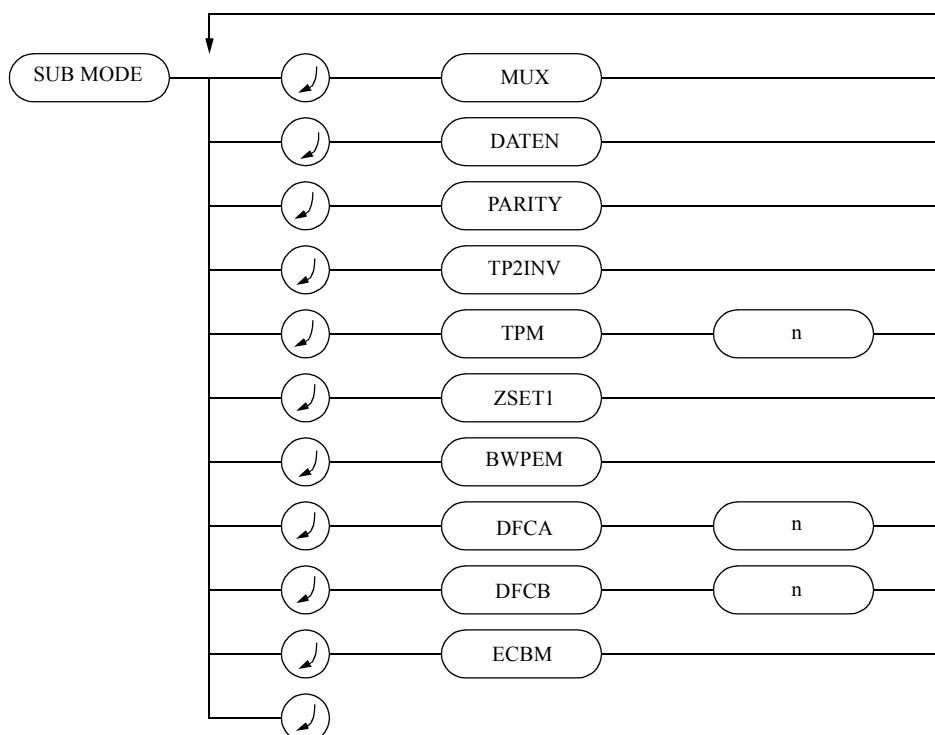
Figure 2-2 Specifiable Operation Modes (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)



*1 This mode becomes valid for both MAIN and SUB even when MAIN is specified in MAIN and SUB command modes.

*2 This mode can be specified in the T5592.

*3 This mode can be used with a T5592 equipped with cycle palette 16 (expansion option).



MUX

MUX specifies address multiplex mode.

Since SDM declares the address multiplex mode in a pin condition statement, it is not necessary to specify MUX mode.

To execute address multiplexing, specify as follows with the pin statement:

◆ Example

```
PD1 = IN1, XOR, ACLK1, BCLK1, SDM, <X0, Y0>
```

DATEN

DATEN specifies the real time data swap mode.

Since RDSM declares real-time data swap mode in a pin condition statement, it is not necessary to specify DATEN.

PAUSE

PAUSE specifies the pause test. In PAUSE mode, generation of a pattern stops, for the time set in PAUSE timer (TPAUSE), for the cycle in which control bit T or TM2 is entered in the pattern program.

If this operation mode is used, PAUSE timer (TPAUSE) cannot be set to 0.

If PAUSE mode is specified with REFRESHA mode or REFRESHB mode at the same time, write the control bit I in the cycle where the control bit T or TM2 is written to avoid an internal interrupt.

REFRESHA n, REFRESHB n

(n=64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144)

The modes are used for the auto refresh test with the universal timer (TREFRESH) and the interrupt function. The choice of mode A or B depends on the relationship between the TREFRESH setting time and the interrupt timing. When the REFRESHB mode is used together with the burst commanded by the OUT command, "NOP T," a timer restarting command, is necessary after the OUT command.

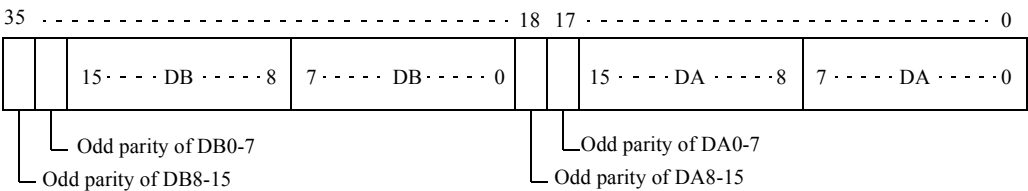
When the REFRESHB mode is used together with the PAUSE mode, there needs "NOP TM1," REFRESH timer restarting command, after the PAUSE timer starting command.

n=131072, and 262144 can be specified in T5593.

PARITY

Used to declare a mode for automatically generating an odd parity.

If TPM18 is specified with the MODE statement, the odd parity of DA0-7 is output to DA16, the odd parity of DA8-15 is output to DA17, the odd parity of DB0-7 is output to DB16, and the odd parity of DB8-15 is output to DB17, respectively.



If TPM36 is specified with the MODE statement, the odd parities of DA0-7 is output to DA16, the odd parities of DA8-15 is output to DA17, respectively, and no parity is output to DB16 and DB17.

The way to output DA0-17 and DB0-17 to D0-17 and SD0-17 of the PDS can be selected with the Source data Field (D<TPn).

➔ For more information, refer to [2. 5. 11 "Source Data Field" on page 2-164](#).

TP2INV

TP2INV specifies the mode to control the data inversion by ARIRAM, FP function, and DFLG for TP2 register data.

If this mode is not specified, TP2 register data cannot be the object of the above control.

TPM n(n=18 or 36)

TPM n specifies which mode is used for TP1 and TP2, 18-bit mode or 36-bit mode. If this mode is not specified, the 18-bit mode is set automatically.

ZSET1

ZSET1 specifies the mode to set the operation of DSET instruction to Z register to event time 1.

When this mode is specified, SET Data to Z register becomes valid at the step where the instruction is written. (The specification of $Z < 0$ is not admitted as the DSET instruction.)

This specification is invalid in the pattern written under the interleave mode, and the SET instruction to Z register becomes event time 2 the same as other registers. If this mode is not specified, event time 2 is set the same as other registers and the result is output to the following step where the instruction is specified.

◆ **In the case of Event Time 1**

Output

```

NOP Z<#1 ; Z=1
NOP Z<#2 ; Z=2
NOP Z<#3 ; Z=3
NOP      ; Z=3

```

◆ **In the case of Event Time 2**

Output

```

NOP Z<#1 ; Z=undefined
NOP Z<#2 ; Z=1
NOP Z<#3 ; Z=2
NOP      ; Z=3

```

BWPEM

BWPEM specifies the mode to use TP2 value as mask data when an expected value is generated with Block Write Pseudo Emulator (BWPE).

When this mode is omitted, MD register value in BWPE is used as the mask data.



Tip

The BWPEM mode and BYTEMASK DATA=MD cannot be specified simultaneously.

DFCA n, DFCB n (n=1 to 6)

DFCA n and DFCB n specify 72 bit data mode.

➔ For more information, refer to [2. 5. 14 "DSC Field" on page 2-169](#).

DFCAn : Specifies data format on DA side (D0-17 of PDS).

DFCBn : Specifies data format on DB side (SD0-17 of PDS).

n is the number to specify operation mode, and the range is $1 \leq n \leq 6$.

(n = 5, and 6 can be specified in T5593.)

2. 2 Compilation Control Instructions

The pattern data format to be used is determined from the following two depending on the specification of n and the pattern of DFLG, /D1, /D2.

Data format 1					Data format 2			
71 —54	53 —36	35 —18	17 —0		71 —54	53 —36	35 —18	17 —0
all 0	all 0	all 0	D17-0	DSC=#1	all 1	all 1	all 1	D17-0
all 0	all 0	D17-0	all 0	DSC=#2	all 1	all 1	D17-0	all 1
all 0	D17-0	all 0	all 0	DSC=#4	all 1	D17-0	all 1	all 1
D17-0	all 0	all 0	all 0	DSC=#8	D17-0	all 1	all 1	all 1

n	DFLG	/D1, /D2 ^{*1}	Data format
1	0	0	Data format 1
	0	1	Data format 2
	1	0	Data format 2
	1	1	Data format 1
2	0	0	Data format 2
	0	1	Data format 1
	1	0	Data format 1
	1	1	Data format 2
3	- ^{*2}	-	Data format 1
4	-	-	Data format 2
5	0	-	Data format 1
	1	-	Data format 2
6	0	-	Data format 2
	1	-	Data format 1

^{*1} In the case of DFCA_n, it is controlled by /D1.

In the case of DFCA_n, it is controlled by /D2.

^{*2} - shows that it is not controlled by DFLG, /D1, or /D2.

DFCA and DFCA can be specified at the same time.

Block switching between D0-17, D18-35, D36-53, and D54-71 is performed by DSC Control Field instruction.

The default is DFCA 1 and DFCA 1.

The specification with a pin statement discloses in which block the pin is located by specifying DSS 1 to 4.

◆ Example

```
PC33-50  =OUT1, STRB1, DSS1, ....<D0-17>
PC97-114 =OUT1, STRB1, DSS2, ....<D0-17>
PC161-178=OUT1, STRB1, DSS3, ....<D0-17>
PC225-242=OUT1, STRB1, DSS4, ....<D0-17>
```

Tip

- If CYPn (the cycle palette instruction) is written in the T5592, the 72 bit data mode cannot be used.
- If the MSM mode is specified, the 72 bit data mode cannot be used.
- DFCA1 to 4 and DFCB5, DFCB6 or DFCA5, as well as DFCA6 and DFCB1 to 4 cannot be specified in the same module simultaneously.

ECBM

ECBM specifies to extend and use CBM memory.

This operation mode can be specified in 2WAY/4WAY interleaved mode.

Interleave mode	Not in ECBM mode	In ECBM mode
1WAY	32Kw × 36bit	Cannot be specified
2WAY	32Kw × 36bit	64Kw × 36bit
4WAY	32Kw × 36bit	128Kw × 36bit

In ECBM mode, data of 64 K words × 36 bits/128 K words × 36 bits can be generated by storing different data to each CBM memory in PG UNIT 1 to PG UNIT 4.

(However, when this mode is specified, the CBM Field instruction can only be specified once in the second or fourth step which are interleaved.)

DBMLP

This is used to declare the DBM loop mode.

If DBMLP is specified, the DBM patterns (written at the address where a loop begins to where it ends) can be output repeatedly making a loop.

The address where a loop begins and ends can be specified using the DBMLPB and DBMLPE register, respectively.



Tip

DBMLP can be specified in the T5593 and T5592.

◆ If the MODE DBMLP is Specified

```

MPAT program-name
MODE DBMLP
REGISTER
  DBMA  = #0      ; DBM ADDRESS POINTER
  DBMLPB = #10     ; DBM LOOP BEGIN
  DBMLPE = #FF    ; DBM LOOP END

START #0
JMP . DBMINC      ; The DBM address pointer is incremented.
STPS

DBM PATTERN
DBM CHANNEL 0-7
DBM PINSEL (CHTYPE10,BITMODE1)
D0=P1
:
DBMA #0          ; DBMA
1000 LLLL        ; #0
0100 LLLL        ; #1
:
0001 LLLL        ; #F
0000 HLLL        ; #10
0000 LHLL        ; #11
:
0000 LLLH        ; #FF
END

```

The DBM patterns output repeatedly in the loop

CYPC15

CYPC15 declares the use of cycle palettes CYP9 to CYP16 in T5592.

When CYPC15 is specified with the T5592 equipped with cycle palette 16 (expansion option), CYP1 to CYP16 can be used.



Tip

- If CYPC15 is specified, MUT signal C15 cannot be used.
- In the T5593, specifying CYPC15 is unnecessary even when using CYP9 to 16.

DBFORMn(n=1 to 4)

Selects the address used by FP function inversion conditions and ARIRAM in the double speed mode by using the prescrambler.

In the double speed mode using the prescrambler, two Y addresses (for the first and second halves) correspond to two STRB in one cycle.

The two sets of FP functions, which correspond to two Y addresses, and ARIRAMs are equipped.

In FP function and ARIRAM, individual Y addresses (one for the first and second halves of the double speed mode) can be used as the inversion conditions for the TP1 and TP2 data.

In this mode, specify which of the addresses for the first half and the latter half of the double speed mode shall be used as the inversion conditions.

Table 2-1 Relation between TP1 Data and FP Function and ARIRAM Data

MODE statement		TP1			
TPM	DBFORM	35 27	26 18	17 9	8 0
18	1	-----	-----	17..ARIA...9	8..ARIA...0
	2	-----	-----	17..ARIB...9	8..ARIA...0
	3	-----	-----	8..ARIA...0	8..ARIA...0
	4	-----	-----	17..ARIA...9	8..ARIA...0
36	1	17..ARIA...9	8..ARIA...0	17..ARIA...9	8..ARIA...0
	2	17..ARIB...9	8..ARIA...0	17..ARIB...9	8..ARIA...0
	3	8..ARIB...0	8..ARIB...0	8..ARIA...0	8..ARIA...0
	4	17..ARIB...9	8..ARIB...0	17..ARIA...9	8..ARIA...0

Table 2-2 Relation between TP2 Data and FP Function and ARIRAM Data

MODE statement		TP2			
TPM	DBFORM	35 27	26 18	17 9	8 0
18	1	-----	-----	17..ARIA...9	8..ARIA...0
	2	-----	-----	17..ARIB...9	8..ARIA...0
	3	-----	-----	8..ARIB...0	8..ARIB...0
	4	-----	-----	17..ARIB...9	8..ARIB...0
36	1	17..ARIA...9	8..ARIA...0	17..ARIA...9	8..ARIA...0
	2	17..ARIB...9	8..ARIA...0	17..ARIB...9	8..ARIA...0
	3	8..ARIB...0	8..ARIB...0	8..ARIA...0	8..ARIA...0
	4	17..ARIB...9	8..ARIB...0	17..ARIA...9	8..ARIA...0

ARIA: Inversion by the FP function and ARIRAM for the first half of the double speed mode

ARIB: Inversion by the FP function and ARIRAM for the second half of the double speed mode



Tip

- Default is DBFORM1.
- DBFORM2 to 4 can be specified in the T5593.
- In the double speed mode, which uses the prescrambler, two ARIRAMs are used. The same ARIRAM data is written in the two ARIRAMs.



Important

If DBFORM4 is specified, and one of the following AFM operation mode types is specified in the AFM ACTION statement, failure analysis is performed incorrectly.

FZ>FMX
FO>FMX
FZSBn>FMX
FOSBn>FMX
FZSB (v) >FMX
FOSB (v) >FMX

RDBMA

RDBMA declares the mode which sets data to the DBMA register in realtime.

When the RDBMA mode is specified, data can be set to the DBMA register in realtime by using a DBMSET instruction.

Tip

- When the RDBMA mode is specified, the usable capacity in the DBM memory is reduced to half.
- The RDBMA mode can be specified in the T5593.

MSM

MSN declares the mode in which the sub PG is used in the 2WAY interleave mode of the T5593.

This mode is used for testing high-speed SRAM which is equipped with two ports that operate independently.

By specifying the MSM mode, the data bits to be input in the PDS are allocated as follows:

MSM mode	D0-8	D9-17	SD0-8	SD9-17
Not specified	Main PG DA0-8	Main PG DA9-17	Main PG DB0-8	Main PG DB9-17
Specified	Main PG DA0-8	Sub PG DA0-8	Main PG DB0-8	Sub PG DB0-8

When the MSM mode is specified, the bit width of the data is 18 bits for the main PG and the sub PG. In this case, specify D9-17 and SD9-17 as the pin data of the pin statement to select the data in the sub PG. When using the FP function and ARIRAM, specify DBFORM3.

As failse can be captured into the FM only at the address where the fail occurred in the main PG, the main PG side is used to generate comparator patterns and the sub PG side is used to generate driver patterns.

Tip

When the SUBPG option is equipped in the 2WAY interleave mode in T5593, the MSM mode can be specified.

Limitation

- The 72 bit data mode (MODE DFCA_n, DFCA_n, DSC instruction) and the MSM mode cannot be specified in the same module simultaneously.

- The 36 bit mode (MODE TPM36) and the MSM mode cannot be specified in the same module simultaneously.
- The MSM mode cannot be specified for a test which uses the DBM.
- In other modules in which MSM mode is specified, the 1WAY normal mode module, which uses the sub PG, cannot be specified.

2. 2. 5 REGISTER Statement, MAIN REGISTER Statement, SUB REGISTER Statement

The REGISTER statement is used to initialize each register in ALPG. [Table 2-3](#) or [Table 2-4](#) shows the initial values of the registers in ALPG.

Set the registers by the following three statements.

- REGISTER statement:
Initialization is performed to all the units.
- MAIN REGISTER statement:
Initialization is performed to the units assigned as MAIN PG.
- SUB REGISTER statement:
Initialization is performed to the units assigned as SUB PG.

Default values of each register in the common section are as shown in [Table 2-3](#) or [Table 2-4](#), but the default values in the module section are as follows:

- The common section MAIN register is default for the MAIN register.
- The common section SUB register is default for the SUB register.

When the data is set in the module section, the setting of the module section becomes valid.

Table 2-3 List of Registers (T5593)

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
TIMER	#FFFF	#100F	----	----	Time can be specified (100 ns to 409.5 s)
TREFRESH	#FFFF	#100F	----	----	Time can be specified (100 ns to 409.5 s)
TPAUSE	#FFFF	#100F	----	----	Time can be specified (100 ns to 409.5 s)
PC	#7FF	#0	----	----	----
ISP	#7FF	#0	----	----	----
IDXn	#FFFFFFF	#0	----	----	n=1 to 8

2. 2 Compilation Control Instructions

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
IDXm	#FFFFFFF	#0	----	----	m=9 to 16
CFLG	#FFFF	#0	----	----	----
CPEn	Signal name ^{*1}	R	Signal name ^{*1}	R	n=1 to 4
DRE(DRE1)	Signal name ^{*1}	W	Signal name ^{*1}	W	Can be described as DRE1
DRE2	Signal name ^{*1}	W	Signal name ^{*1}	W	----
RINV	Signal name ^{*1}	R	Signal name ^{*1}	R	----
LMAX	#3FFFF	XMAX	#3FFFF	Sub XMAX	----
HMAX	#3FFFF	YMAX	#3FFFF	Sub YMAX	----
XMAX	#3FFFF	#FFF	#3FFFF	#FFF	----
YMAX	#3FFFF	#FFF	#3FFFF	#FFF	----
ZMAX	#3FFFF	#FF	#3FFFF	#FF	----
XMASK	#3FFFF	#FFD	#3FFFF	#FFD	----
YMASK	#3FFFF	#FFD	#3FFFF	#FFD	----
XOS	#3FFFF	#0	#3FFFF	#0	----
YOS	#3FFFF	#0	#3FFFF	#0	----
XT	#3FFFF	#0	#3FFFF	#0	----
YT	#3FFFF	#0	#3FFFF	#0	----
XH	#3FFFF	#0	#3FFFF	#0	----
YH	#3FFFF	#0	#3FFFF	#0	----
ZH	#3FFFF	#0	#3FFFF	#0	----
NH	#FF	#0	#FF	#0	----
BH	#FF	#0	#FF	#0	----
D1n	#3FFFF	#0	#3FFFF	#0	n=A,B,C,D,E,F,G,H (When n is omitted, same as D1A)
D2n	#3FFFF	#0	#3FFFF	#0	n=A,B,C,D (When n is omitted, same as D2A)
D3B	#3FFFF	#0	#3FFFF	#0	----
D4B	#3FFFF	#0	#3FFFF	#0	----
RCD	#3FFFF	#0	#3FFFF	#0	----
CCD	#3FFFF	#0	#3FFFF	#0	----

2. 2 Compilation Control Instructions

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
DSD	#3FFFF	#0	#3FFFF	#0	----
TPH(TPH1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as TPH1
TPH2	#FFFFFFFF	#0	#FFFFFFFF	#0	----
DCMR (DCMR1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as DCMR1
DCMR2	#FFFFFFFF	#0	#FFFFFFFF	#0	----
D5	#FFFFFFFF	#0	#FFFFFFFF	#0	----
D6	#FFFFFFFF	#0	#FFFFFFFF	#0	----
DBMA ^{*2}	#3FFFFE	#0	----	----	----
DBMLPB ^{*2}	#3FFFFE	DBMA	----	----	----
DBMLPE ^{*2}	#3FFFFE	Maximum value	----	----	----
RLMAX	#3FFFF	#FF	#3FFFF	#FF	#3F,#7F,#FF,#1FF,#3FF,#7FF, #FFF,#1FFF,#3FFF,#7FFF, #FFFF,#1FFFF, and #3FFFF can be specified.
ZD	#3FFFF	#0	#3FFFF	#0	----
NDn	#FF	#0	#FF	#0	n=1 to 7(when n is omitted, same as n=1)
BDn	#FF	#0	#FF	#0	n=1 to 7(when n is omitted, same as n=1)
RHZA	Signal name ^{*1}	FL	Signal name ^{*1}	FL	----
RHQB	Signal name ^{*1}	FL	Signal name ^{*1}	FL	----
ASCNT	Signal name ^{*1}	FL	----	----	----
ASINT	Signal name ^{*1}	FL	----	----	----
FPSEL	Address Function name ^{*3}	DISABLE	Address Function name ^{*3}	DISABLE	----

"----" is a register which can not specify SUB PG independently.

^{*1} Specify by using one of the following signals.

FIXL(FL), FIXH(FH), R(RD), W(WT), M1-2, C0-23

2. 2 Compilation Control Instructions

*2 The maximum value depends on the kinds of interleave mode, DBM board, and RDBMA mode (MODE RDBMA).

DBM board	Specifying RDBMA mode	Interleave mode	Bit size of DBMA, DBMLPB and DBMLPE
2 M board	None	1WAY	#0 ≤ n ≤ #1FFFFFF
		2WAY	#0 ≤ n #3FFFFFFE and is a multiple of 2
	Specified	1WAY	#0 ≤ n ≤ #FFFFFF
		2WAY	#0 ≤ n ≤ #1FFFFFFE and is a multiple of 2

Specify DBMLPB and DBMLPE to satisfy $DBMLPB \leq DBMLPE$.

Specify the setting range of DBMA, DBMLPB, and DBMLPE by using the address range of the DBM pattern selected.

The default value of DBMLPE is the maximum address of the DBM pattern transferred.

*3 Specify one of the following address function names or DISABLE.

FP0, FP1, FP3, FP4, FP5, FP6, FP7, FP8, FP9, FP10

Table 2-4 List of Registers (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
TIMER	#FFFF	#100F	----	----	Time can be specified (100 ns to 409.5 s)
TREFRESH	#FFFF	#100F	----	----	Time can be specified (100 ns to 409.5 s)
TPAUSE	#FFFF	#100F	----	----	Time can be specified (100 ns to 409.5 s)
PC	#3FF	#0	----	----	Up to #1FF in PCC mode.
ISP	#3FF	#0	----	----	Up to #1FF in PCC mode.
BAR	#3FF	#0	----	----	Up to #1FF in PCC mode.
IDXn	#FFFFFFF	#0	----	----	n=1 to 2
IDXm	#FFFFFFF	#0	----	----	m=3 to 8
CFLG	#FF	#0	----	----	----
CPEn	Signal name ^{*1}	R	Signal name ^{*1}	R	n=1 to 4
DRE(DRE1)	Signal name ^{*1}	W	Signal name ^{*1}	W	Can be described as DRE1
DRE2	Signal name ^{*1}	W	Signal name ^{*1}	W	----

2. 2 Compilation Control Instructions

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
RINV	Signal name ^{*1}	R	Signal name ^{*1}	R	----
LMAX	#FFFF	XMAX	#FFFF	Sub XMAX	----
HMAX	#FFFF	YMAX	#FFFF	Sub YMAX	----
XMAX	#FFFF	#FFF	#FFFF	#FFF	----
YMAX	#FFFF	#FFF	#FFFF	#FFF	----
ZMAX	#FFFF	#FF	#FFFF	#FF	----
XMASK	#FFFF	#FFD	#FFFF	#FFD	----
YMASK	#FFFF	#FFD	#FFFF	#FFD	----
XOS	#FFFF	#0	#FFFF	#0	----
YOS	#FFFF	#0	#FFFF	#0	----
XT	#FFFF	#0	#FFFF	#0	----
YT	#FFFF	#0	#FFFF	#0	----
XH	#FFFF	#0	#FFFF	#0	----
YH	#FFFF	#0	#FFFF	#0	----
ZH	#FFFF	#0	#FFFF	#0	----
NH	#F	#0	#F	#0	----
D1	#FFFF	#0	#FFFF	#0	----
D2	#FFFF	#0	#FFFF	#0	----
D3B	#FFFF	#0	#FFFF	#0	----
D4B	#FFFF	#0	#FFFF	#0	----
RCD	#FFFF	#0	#FFFF	#0	----
CCD	#FFFF	#0	#FFFF	#0	----
DSD	#FFFF	#0	#FFFF	#0	----
TPH(TPH1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as TPH1
TPH2	#FFFFFFFF	#0	#FFFFFFFF	#0	----
DCMR (DCMR1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as DCMR1
DCMR2	#FFFFFFFF	#0	#FFFFFFFF	#0	----
D5	#FFFFFFFF	#0	#FFFFFFFF	#0	----
D6	#FFFFFFFF	#0	#FFFFFFFF	#0	----

2. 2 Compilation Control Instructions

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
DBMA ^{*2}	#3FFFC	#0	----	----	----
DBMLPB ^{*2}	#3FFFC	DBMA	----	----	----
DBMLPE ^{*2}	#3FFFC	Maximum value	----	----	----
RLMAX	#FFFF	#FF	#FFFF	#FF	#3F, #7F, #FF, #1FF, #3FF, #7FF, #FFF, #1FFE, #3FFF, #7FFF, and #FFFF can be specified.

"----" is a register which can not specify SUB PG independently.

^{*1} 1 needs to be specified by using one of the following signals:

FIXL(FL), FIXH(FH), R(RD), W(WT), M1-2, C0-15, PREF(For PREF, CPE1 to 4 only)

^{*2} The maximum value depends on the type of interleaved mode and DBM board.

DBM board	Interleave mode	Bit size of DBMA, DBMLPB and DBMLPE
64 K board	1WAY	$\#0 \leq n \leq \#FFFF$
	2WAY	$\#0 \leq n \leq \#1FFFE$ and is a multiple of 2
	4WAY	$\#0 \leq n \leq \#3FFFC$ and is a multiple of 4
256 K board	1WAY	$\#0 \leq n \leq \#3FFFF$
	2WAY	$\#0 \leq n \leq \#7FFFE$ and is a multiple of 2
	4WAY	$\#0 \leq n \leq \#FFFFC$ and is a multiple of 4
1 M board	1WAY	$\#0 \leq n \leq \#FFFFF$
	2WAY	$\#0 \leq n \leq \#1FFFFE$ and is a multiple of 2
	4WAY	$\#0 \leq n \leq \#3FFFFC$ and is a multiple of 4

Specify DBMLPB and DBMLPE to satisfy $DBMLPB \leq DBMLPE$.

TIMER

Time can be set (in cycles or in time) for time tests, such as the pause test (Timer 2), refresh test (Timer 1), with three timers. It can be set in cycles or times. When PAUSE mode is specified by the MODE statement, however, time cannot be set in cycles.

TREFRESH

Set time with Timer 1 for the refresh test (Timer 1). It can be set in cycles or times.

TPAUSE

Set time with Timer 2 for the pause test (Timer 2). It can also be set in times.

PC(Program Counter)

Set the start address of ALPG.

When described dividedly by a MODULE statement, data is stored into ALPG from the address indicated by the START statement. In practice, however, the start address of a pattern is specified by the REG MPAT PC statement of the test plan program.

ISP(Interrupt Save Pointer)

Set the branch destination address to which control branches if an interrupt is caused by the internal timer in the auto refresh test. For the specification, an address can be set directly or a label can be set.

Tip

The FLGLI1 conditional branch instructions cannot be used as the first executable instruction for the branch address specified by ISP.

→ For more information, refer to [4. 3. 3 "Refresh Test" on page 4-17](#).

BAR(Branch Address Register)

The BAR defines the branch address of the flag sense instruction FLGLI1.

IDXn(n = 1 to 8)

Set the number of loops in a loop instruction of ALPG, such as JNIn.

→ For more information, refer to [2. 3. 1 "Index Loop" on page 2-82](#).

IDXm(m = 9 to 16)

The IDXm registers store the number of loops. They are used to change the number of loops of IDX1 to 8 by the LDIn instruction.

→ For more information, refer to [2. 3. 1 "Index Loop" on page 2-82](#).

CFLG

Set the JNCn branch condition. The bit of the CFLG register corresponds to JNC1 to JNC16 as shown below. If the corresponding CFLG bit is "1," the JNCn instruction jumps to the specified address (label).

→ For information on the JNCn instruction, refer to [2. 3. 4 "Condition Flag Reference Branch \(JNCn\)" on page 2-88](#).

CFLG register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JNC16	JNC15	JNC14	JNC13	JNC12	JNC11	JNC10	JNC9	JNC8	JNC7	JNC6	JNC5	JNC4	JNC3	JNC2	JNC1

The CFLG register is used to change the execution sequence in a pattern program. In this case, set the CFLG register with the REG MPAT statement in a test plan program.

CPE_n(n = 1 to 4)

Set a control bit for specifying a cycle for logical comparison. R, W, FIXL, FIXH, M1, M2, C0 to C23, or PREF can be selected as the control bit.

➔ For a sample program, refer to [2. 6. 5 "Specifying the Cycles for Performing Logical Comparison" on page 2-177](#).

DRE(DRE1), DRE2

Specify a control bit which turns a driver for the I/O pin of the DUT on or off. It can be selected from R, W, FIXL, FIXH, M1, M2, and C0 to C23. DRE1 has a same meaning as DRE.

➔ For a sample program, refer to [2. 6. 6 "Specifying to Turn Driver ON or OFF in I/O Operation" on page 2-178](#).

RINV

Set a control bit for inverting a data pattern in real time for the pin that is set as P_n =, RINV,, in the test plan program. It can be selected from R, W, M1, M2, and C0 to C23.

LMAX, HMAX

Specify valid bits of registers when register groups (X, Y, and Z groups) for address generation are to be concatenated.

➔ For more information, refer to [2. 4 "Address Pattern Generation Instructions" on page 2-96](#).

XMAX, YMAX, ZMAX

Specify valid bits of addresses X, Y, and Z to be entered in the DUT.

➔ For more information, refer to [2. 4 "Address Pattern Generation Instructions" on page 2-96](#).

XH, YH, ZH, NH, BH

XH, YH, ZH, NH and BH store initial values of registers XB, YB, Z, N and B. They are loaded into the registers with the following operations:

$$XB < XH \quad YB < YH \quad Z < ZH \quad N < NH \quad B < BH$$

D1(D1A to D1H), D2(D2A to D2D), D3B, D4B

D1, D2, D3B, and D4B are used for the operation of registers XC, XS, XK, YC, YS, and YK.

D1(D1A to D1H), and D2(D2A to D2D) are also used for the operation of registers XB and YB.

TPH(TPH1), TPH2

The TPH (TPH1) initializes register TP (TP1).

It is stored in register TP (TP1) when the operation is TP < TPH (TP1 < TPH1).

TPH and TPH1 have the same meaning. Similarly, TP and TP1 have the same meaning.

The TPH2 initialize register TP2.

It is stored in register TP2 when the operation is TP2 < TPH2.

DCMR, DCMR2(Data Complement Mask Register)

Set the mask bit which determines whether or not to invert data in the TP or TP2 register with the /D or /D2 instruction. D0 to D35 corresponding to a 0 bit is inverted.

➔ For a sample program, refer to [2. 5. 7 "Temporary Inversion of Data \(/D, /D2\)\(Data Inversion Field\)" on page 2-163.](#)

XMASK, YMASK

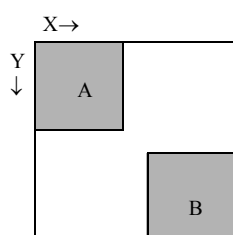
XMASK and YMASK are used for address functions FP7 (mask parity), FP8 (row bar), FP9 (column bar), and FP10 (row/column bar).

➔ For more information, refer to [2. 5. 5 "FP Function Field" on page 2-153.](#)

XOS, YOS

XOS and YOS are the registers to set offset values.

The offset values are added to the results of address operating instructions executed. Owing to it, the following partial test becomes possible.



If XOS and YOS in the pattern testing the area A are specified, the area B will be tested.
(Specify XMAX, YMAX, LMAX, HMAX using values corresponding to the area A.)

XT, YT

XT and YT are used to generate optional addresses with a specified cycle.

The address data set in XT and YT register is output to the cycle where the XT and YT instructions are specified.

◆ Example

```

REGISTER
  XT=#3F
  YT=#10
  :
NOP      XB<XB+1  YB<YB+1
NOP      XT      ← X address is #3F regardless of XB value.
NOP      YT      ← Y address is #10 regardless of YB value.
NOP      ← XB and YB are output for X and Y.

```

D5, D6

The D5 register is used for TP1 operation.

The D6 register is used for TP2 operation.

RCD, CCD(Row Address Complement Data, Column Address Complement Data)

RCD and CCD are used for address functions FP8 (row bar), FP9 (column bar) and FP10 (row/column bar), respectively.

➔ For more information, refer to [2. 5. 5 "FP Function Field" on page 2-153](#).

DSD(Diagonal Shift Data)

DSD is used as the shift data for FP3 (diagonal), FP4 (invert diagonal), FP5 (diagonal 2) and FP6 (invert diagonal 2).

➔ For more information, refer to [2. 5. 5 "FP Function Field" on page 2-153](#).

DBMA

DBMA is the register to initialize address pointer of DBM.

When data is generated from DBM, the address pointer of the DBM memory is initialized by this register and incremented by the DBMINC instruction.

RLMAX

Set the counter in refresh mode. If the value is not specified with this register, the default is the value of n in REFRESHA n (or REFRESHB n) in the MODE statement.

DBMLPB, DBMLPE

DBMLPB and DBMLPE are registers used to specify the addresses where a loop begins and ends, respectively, when the DBM loop mode (MODE DBMLP) is specified and the DBM patterns are output repeatedly in the loop.

DBMLPB is the address where a loop begins.

DBMLPE is the address where a loop ends.

These are available when DBMLP is specified in the MODE statement.

ZD, ND(ND1 to ND7), BD(BD1 to BD7)

ZD, ND and BD are registers used to operate the Z, N and B registers.

RHZA

RHZA selects a control bit to specify the cycle for Hi-Z comparison in the real time Hi-Z mode (specified by pin statements RHZA1, RHZA2, RHZA129 and RHZA130).

R, W, FIXL, FIXH, M1, M2, or C0 to C23 can be selected as the control bit.

The control bit specified here executes the Hi-Z comparison by using the specified cycle and the STRB, which is specified by pin statements RHZA1, RHZA2, RHZA129 and RHZA130.

➔ For more information, refer to [2. 6. 10 "Specifying the Cycle to Perform Hi-Z Comparison in the Realtime Hi-Z Mode" on page 2-181](#).

RHQB

RHQB selects a control bit to specify the cycle for Hi-Z comparison in the real time Hi-Z mode (specified by pin statements RHQB1, RHQB2, RHQB129 and RHQB130).

R, W, FIXL, FIXH, M1, M2, or C0 to C23 can be selected as the control bit.

The control bit specified here executes the Hi-Z comparison by using the specified cycle and the STRB which is specified by pin statements RHZB1, RHZB2, RHZB129 and RHZB130.

➔ For more information, refer to [2. 6. 10 "Specifying the Cycle to Perform Hi-Z Comparison in the Realtime Hi-Z Mode" on page 2-181.](#)

ASCNT

ASCNT selects the control bit to specify the cycle for incrementing/decrementing the STRB and clock value in the auto scan mode.

R, W, FIXL, FIXH, M1, M2, or C0 to C23 can be selected as the control bit.

ASINI

ASINI selects the control bit to specify the cycle for initializing the STRB and clock value in the auto scan mode.

R, W, FIXL, FIXH, M1, M2, or C0 to C23 can be selected as the control bit.

FPSEL

The FPSEL register is used to select the address function in the cycle in which the FPX instruction is described.

➔ For more information, refer to [2. 5. 5 "FP Function Field" on page 2-153.](#)

2. 2. 6 ARIRAM DEFINE Statement

ARIRAM DEFINE statement is set for inversion of the data pattern with the regional inversion function.

Set ARIRAM DEFINE statement by the following three statements:

- ARIRAM DEFINE statement:
All units are set.
- MAIN ARIRAM DEFINE statement:
The unit assigned as MAIN PG is set.
- SUB ARIRAM DEFINE statement:
The unit assigned as SUB PG is set.

If this instruction is not specified in the common part, ARIRAM in the common becomes DISABLE.

If this instruction is not specified in the module part, MAIN ARIRAM DEFINE statement becomes default in the common part of MAIN ARIRAM.

In a like manner, SUB ARIRAM DEFINE statement becomes default in the common part of SUB ARIRAM.

ARIRAM DEFINE statement can be defined by the scramble program. When this is carried out, the priorities are as follows: (The lower the number the higher the priority.)

1. SCRAM specified in test plan program
2. ARIRAM DEFINE statement specified in pattern program module

3. SCRAM specified in pattern program module
 4. ARIRAM DEFINE statement specified by common section of pattern program
 5. SCRAM specified by common section of pattern program
- ➔ For more information on ARIRAM DEFINE statement and regional inversion, refer to [4. 3. 4 "Regional Inversion Test \(ARIRAM\)" on page 4-41](#).

There are three types of area inversion.

- Area inversion using the INV statement
According to X and Y address conditions and inverting condition given by the INV statement, pattern data (D0-35) is inverted.

```

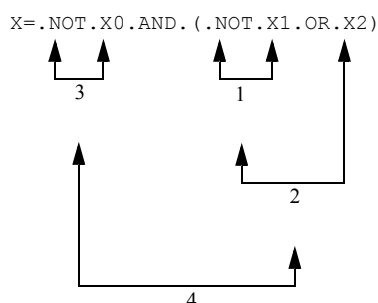
ARIRAM DEFINE (MAIN ARIRAM DEFINE, SUB ARIRAM DEFINE)

X = Xk1 O1 Xk2..... } Specification of address range for inversion
Y = Yl1 O2 Yl2..... }
INV = X O3 Y..... } The data pattern is inverted when inverting condition
                        INV = 1 is specified.

0 ≤ kn, lm ≤ 11
On  &= . AND.
    |= . XOR.
    ' = . OR.
    %= . NOT.

```

◆ Example



In this case, operations are executed in the order of 1 to 4.

The maximum usable parentheses number is 127.

Tip

Number types of X and Y elements described in the right sides is a total of 12 in conditional equations.

• Area inversion on the data bit basis 1

Specifying address conditions inverts desired data bits.

```
ARIRAM DEFINE (MAIN ARIRAM DEFINE, SUB ARIRAM DEFINE)
```

```
Dn      = Xh1 O1 Yi1 ...
```

```
Dn, m   = Xh2 O1 Yi2 ... (a)
```

```
Dn-m    = Xh3 O1 Yi3 ... (b)
```

(a) and (b) can be specified at the same time.

n and m specify the data bit. n or m cannot be omitted.

Specification range

$0 \leq n, m \leq 17$

For T5593

$0 \leq h \leq 17$

$0 \leq i \leq 17$

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

$0 \leq h \leq 15$

$0 \leq i \leq 15$

For operators, operational elements and "(", see area inversion using the INV statement on preceding pages.

Tip

- Number of types of X and Y element in the right sides should be 12 or less in total.

◆ Example

```
D0,1=X1.XOR.(Y0.AND..NOT.X7)
D2  =X1.XOR.X7.XOR.(Y0.AND.X6)
D3  =X1.XOR.X7.XOR.(Y0.AND.X6)
```

In the above example, 4 types of X and Y variables are used in the right sides.

- *In the same module and to the same PG (MAIN/SUB), this function cannot be used with the area inversion function using the INV or ARIn statement at the same time.*
- *If a data bit is given inverting conditions twice or more, the conditions given last are effective.*

- Area inversion on the data bit basis 2

```
ARIRAM DEFINE(MAIN ARIRAM DEFINE, SUB ARIRAM DEFINE)
ARIl = Xh1 O1 Yi1 ...
Dn = ARI
Dn, m = ARIl --- (a)
Dn-m = ARIl --- (b)
```

(a) and (b) can be specified at the same time.

For operators, operational elements and "()," see area inversion using the INV statement on preceding pages.

n and m specify the data bit.

Specification range

$$0 \leq n, m \leq 17$$

$$0 \leq l \leq 7$$

For T5593

$$0 \leq h \leq 17$$

$$0 \leq i \leq 17$$

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

$$0 \leq h \leq 15$$

$$0 \leq i \leq 15$$

For operators, operational elements and "()," see area inversion using the INV statement on preceding pages.

 Tip

- *Number of types of X and Y element described in the right side is a total of 12 in ARI statement.*
- *In the same module and to the same PG (MAIN/SUB), this function cannot be used with the area inversion function using the INV statement or Dn = equations at the same time.*
- *If the ARIn is defined twice or more to the same data bit, the last defined value are effective.*

- For $Dn = ARIn$, if $ARIn$ is not defined in the upper line, a compiler error is occurred.

2. 2. 7 ADDRESS DEFINE Statement

Data specified by ADDRESS DEFINE statement is used to generate the test patterns below.

Set ADDRESS DEFINE statement by the following three statements:

- ADDRESS DEFINE statement:
All units are set.
- MAIN ADDRESS DEFINE statement:
The unit assigned as MAIN PG is set.
- SUB ADDRESS DEFINE statement:
The unit assigned as SUB PG is set.

If this instruction is not specified in the common part, ADDRESS data in the common becomes DISABLE.

If this instruction is not specified in the module part, MAIN ADDRESS DEFINE statement becomes default in the common part of MAIN ADDRESS DEFINE statement.

In a like manner, SUB ADDRESS DEFINE statement becomes default in the common part of SUB ADDRESS DEFINE statement.

◆ N, Z, B

The address data stored in the N, Z, or B register specified by the ADDRESS DEFINE statement can be output to the bit of each address of X0 to X17 and Y0 to Y17 (in T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, X0 to 15 and Y0 to 15) to be impressed to the DUT.

```
ADDRESS DEFINE (MAIN ADDRESS DEFINE, SUB, ADDRESS DEFINE)
```

```
Xl-m = Zp-q, Nr-s, Bt-u
```

```
Yl-m = Zp-q, Nr-s, Bt-u
```

```
XY = XY .OR. ZN or ZN (ZN can be described as ZNB.)
```

In p-q, r-s, and t-u address bits are set contiguously. In this case, the values must be $p < q$, $r < s$, and $t < u$.

l, m, p, q, r, s, t, and u can be specified in [Table 2-5](#).

Table 2-5 Specification Range for l, m, p, q, r, s, t, and u

Specification range	T5593	T5592/T5591/T5586/T5585/ T5581/T5581H/T5581P/T5571P
l, m	0 to 17	0 to 15
p, q	0 to 17	0 to 15

Specification range	T5593	T5592/T5591/T5586/T5585/ T5581/T5581H/T5581P/T5571P
r, s	0 to 7	0 to 3
t, u	0 to 7	Cannot be specified

The bits specified by $Xl-m = Zp-q$, $Nr-s$, $Bt-u$ or $Yl-m = Zp-q$, $Nr-s$, $Bt-u$ are set to whether X or Y address will be output by the OR operation, or the specified Z, N or B address will be output preferentially.

XY .OR. ZN(ZNB)

The X or Y address and the N, Z or B address are output by the OR operation.

ZN(ZNB)

Inhibits the X or Y address and outputs the N, Z or B address only.

When XY is not specified as being equal to either XY.OR.ZN (ZNB) or just ZN(ZNB), the X or Y address and the N, Z or B address are output by the OR operation.



Tip

- When defining Z addresses to X/Y addresses, specify them with the following combinations:

For T5593

X (same Y)																	
17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Z17	Z16	Z15	Z14	Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0
Z16	Z15	Z14	Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	
Z15	Z14	Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0		
Z14	Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0			
Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0				
Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0					
Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0						
Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0							
Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0								
Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0									
Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0										
Z6	Z5	Z4	Z3	Z2	Z1	Z0											
Z5	Z4	Z3	Z2	Z1	Z0												
Z4	Z3	Z2	Z1	Z0													
Z3	Z2	Z1	Z0														
Z2	Z1	Z0															
Z1	Z0																
Z0																	

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

X (same Y)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Z15	Z14	Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0
Z14	Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	
Z13	Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0		
Z12	Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0			
Z11	Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0				
Z10	Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0					
Z9	Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0						
Z8	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0							
Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0								
Z6	Z5	Z4	Z3	Z2	Z1	Z0									
Z5	Z4	Z3	Z2	Z1	Z0										
Z4	Z3	Z2	Z1	Z0											
Z3	Z2	Z1	Z0												
Z2	Z1	Z0													
Z1	Z0														
Z0															

When a subscript of X or Y is n , only the bits under Z_n can be specified for X_n .

- When defining N addresses to X/Y addresses, specify them with the following combinations:

For T5593

X (same Y)																	
17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0
N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3
N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2
N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1
N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0
N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7
N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6
N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5
N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4
N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3
N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2
N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

X (same Y)															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0
N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3
N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2
N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1

- When defining B addresses to X/Y addresses, specify them with the following combinations:

For T5593

										X (same Y)							
17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0
B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7
B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6
B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5
B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4
B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3
B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2
B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

Cannot be specified.

◆ Example 1:

ADDRESS DEFINE

X4-7=Z0-3 ;Z0-3 is output to X4-7 by the OR operation.

Y8-9=Z8-9 ;Z8-9 is output to Y8-9 by the OR operation.

XY=XY.OR.ZN

◆ Example 2:

ADDRESS DEFINE

X0-7=Z0-3,N0-3 ;Z0-3 is output to X0-3 and X4-7 is output to X4-7.

Y12-15=Z12-13,N0-1 ;Z12 and Z13 are output to Y12 and Y13
;and N0 and 1 are output to Y14 and Y15.

XY=ZN

◆ Example 3:

ADDRESS DEFINE

X0-3= B1-4 ;B1-4 is output to X0-3 by the OR operation.

Y14-17=B0-3 ;B0-3 is output to Y14-17 by the OR operation.

XY=XY.OR.ZNB

2. 2. 8 BURST BLOCK Statement

Function: BURST BLOCK statement sets, in the on-the-fly mode, the condition for the setting alteration test, for example, a bump test of the measurement section specified by the OUT instruction.

Syntax

```

BURST BLOCK BEGIN(Block-name)
           : Setting test condition
BURST BLOCK END

```

Argument: Block-name

Name referenced by OUT instruction.

Specify characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

Description

Sixteen sets can be entered in a pattern program.

Data in [Table 2-6](#) can be set. The number of test conditions which can be described in each block depends on the total number of words shown in [Table 2-6](#). The total can be up to 31.

➔ For information on STISP, STIn, refer to [2. 10 "STISP, STIn Burst Test Condition Statement" on page 2-239](#).

Table 2-6 Number of Burst Data Words

Mnemonic	Number of words
VS _n = vs, R (vr) (Refer to Tip) VS _n = vs	2
WAIT TIME t	1
STISP m	1
STIn m	1

Tip

- "n" of VS_n should be specified in the range 1 to 2 and 9 to 12 for T5581 and T5581H, and in the range 1 to 12 for T5593, T5592, T5591, T5586, T5585, T5581P, and T5571P.
- For the T5593, T5592, T5591, T5586, T5585, T5581P, and T5571P, it is not possible to simultaneously specify VS5 and VS9, VS6 and VS10, VS7 and VS11, and VS8 and VS12.
- Do not specify VS other than that described in the test plan program.
- To specify VS, always specify the DPUTYPE statement.

2. 2. 9 DPUTYPE Statement

Specify one of the following to describe VS in the burst test condition:

For the T5581 and T5581H

DPUTYPE1

For T5593, T5592, T5591, T5586, T5585, T5581P, and T5571P

DPUTYPE2

The default is DPUTYPE1.

Once either of the above is specified in the pattern program, it will be valid in all common sections and module sections. Multiple specifications can be described in the common sections and module sections. In this case, use the same specification for all.



Tip

The DPUTYPE statement cannot be described in the BURST BLOCK instruction.

When the DPUTYPE statement has been omitted, the pattern program can be compiled by the compilation execution command shown below as if DPUTYPE2 was specified in the pattern program. The command can also be used to specify the range of the voltage source (VS) for the burst test.

trans -V DPUTYPE2 source

It performs the same way as if DPUTYPE2 has been specified in the pattern program.

trans -V DPUTYPE2R5 source

It performs the same way as if DPUTYPE2 has been specified in the pattern program with R(5 V) specified for the VS voltage range. In the T5593, it performs the same way as if R(16V) had been specified.

trans -V DPUTYPE2R16 source

It performs the same way as if DPUTYPE2 has been specified in the pattern program with R(16 V) specified for the VS voltage range.

The source file name is set by "source" in the compilation execution command statements.

➔ For more information on the compilation command, refer to [XATL/U-51 Cross Compiler User's Manual\(No. : 8256237\)](#).

The following explains the relationship between the pattern program and the compilation execution command shown above.

When the DPUTYPE statement is specified in the pattern program

Specification on the compilation execution command becomes invalid.

When the DPUTYPE statement is not specified in the pattern program

Specification on the compilation execution command becomes valid.

However, when the VS voltage range is specified in the pattern program, the VS voltage range specified in the compilation execution command is ignored. This relationship is listed below.

In pattern program VS voltage range specification R(Vr)		Compilation execution command	Compiled burst test conditions	
			DPUTYPE statement	VS voltage range
None ^{*2}		None	DPUTYPE1	Optimum range ^{*1}
		DPUTYPE2	DPUTYPE2	Optimum range ^{*1}
		DPUTYPE2R5	DPUTYPE2	R(5V)
		DPUTYPE2R16	DPUTYPE2	R(16V)
Specified ^{*3}	0V ≤ Vr ≤ 5V	DPUTYPE2 or DPUTYPE2R5 or DPUTYPE2R16	DPUTYPE2	R(5V)
	5V < Vr ≤ 16V			R(16V)

^{*1} It is automatically set based on the applicable voltage.

^{*2} The VS voltage range specified in the compilation execution command is valid.

^{*3} The VS voltage range specified in the pattern program is valid. The VS voltage range specified in the compilation execution command is ignored.

2. 2.10 SDEF Statement

Function: SDEF statement is used to define the address generation instruction, the data generation instruction and the packet instruction with an arbitrary mnemonic. A pattern program can be described using the mnemonics specified by this instruction. Control bits (I, T, TM1, TM2, and DBMINC) are not specified.

Syntax

```
SDEF name = instructions
```

Argument: name

Specify characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

Up to 500 codes can be specified.

instructions

Data generation instruction and packet instruction

Example

(1) When Instructions are Defined for INIT and INC

```

SDEF INIT = XB<0  YB<0  TP<0
SDEF INC = XB<XB+1  YB<YB+1^BX
      :
START      #0

      IDXI1 #6          INIT
ST0 :JN12 ST0 W INC FPI
ST1 :JN12 ST1 R INC FPI
      JZD  ST0
      STPS
      :

```

2. 2.11 SCRAM Statement/PRESCRAM Statement

SCRAM statement

Specifies the name of the SCRAM file containing data of the address descrambler.

PRESCRAM statement

Specifies the name of the SCRAM file containing data of the prescrambler.

SCRAM filename

Specifies the scramblers in all the units.

MAIN SCRAM filename

Specifies the scramblers assigned as MAIN SCRAM.

SUB SCRAM filename

Specifies the scramblers assigned as SUB SCRAM.

PRESCRAM filename

Specifies the scramblers in all the units.

MAIN PRESCRAM filename

Specifies the scramblers assigned as MAIN PRESCRAM.

SUB PRESCRAM filename

Specifies the scramblers assigned as SUB PRESCRAM.

2. 2.12 START Statement

The START statement defines the start address which stores test pattern generating instructions. The start address can be specified in the following range.

T5593

#0 to #7FF

T5592/T5591/T5586/T5585/T5581/T5581H/T5581P/T5571P

#0 to #3FF

Up to 100 codes can be specified.

2. 2.13 PCC Statement

Function: PCC statement is an instruction to specify the value of n, the number of cycles of address data generating instructions to be executed, against the execution of a one cycle instruction (NOP, JMP, etc.) to control pattern generation.

Syntax

```
PCC n (n=1 to 2)
```

Description

In the T5300 series memory test system, the maximum operation frequency can be made twice the normal mode by using PCC 2 mode. However, even if PCC 2 mode is specified in T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, the operation frequency is the same as the normal mode.

In T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, the PCC 1 and PCC 2 modes are supported in order to maintain compatibility with the T5300 series. Limitation and pattern describing format in PCC mode are the same as T5300 series.

If operating frequencies which exceed the maximum allowable value in the normal mode are required for T5593, T5592, T5591 or T5581, specify the 2WAY or 4WAY interleaved mode, or the BMUX mode using the PIN statement.

➔ For the description format when PCC mode is specified, refer to [2. 2. 24 "Test Pattern Programming Format" on page 2-66](#).

PC instructions (e.g., NOP, JMP) to control pattern generation when PCC 2 mode is specified become valid in the latter cycle of PCC to be executed.

◆ Example

```

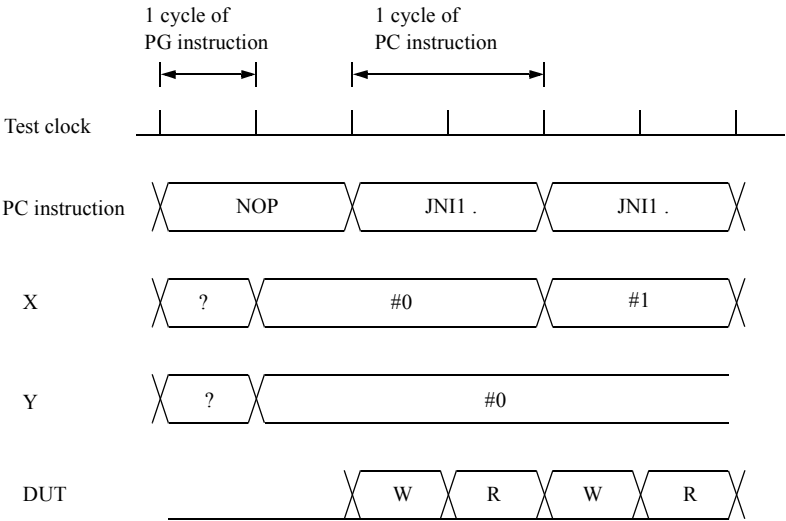
MPAT    PCCMODE
REGISTER
        XH = #0
        YH = #0
        IDX1 = #3E
SDEF    BINC = XB<XB+1  YB<YB+1^BX
PCC      2
START   #0
        NOP    % XB<XH YB<YH TP<0
        JN11   . %                W
        % BINC  R
        STPS
END

```

PC instruction ALPG instruction

Tip

- In T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, when PCC n is not written in the program, the PC value is #0 to #3FF. If, however, PCC n is written in the program, the PC value is #0 to #1FF.
- 2WAY/4WAY interleaved mode and PCC mode cannot be specified at the same time.
- PCC n can be specified only in the normal mode (1WAY).
- In T5593, the PCC mode cannot be specified.



Timing chart of MPAT PCC MODE

The effective cycle of the control bits of "I," "T," "TM1," "TM2," "DBMINC," to be described in the instruction part of PC is as follows:

- In PCC 1 mode

```
NOP I T TM1 TM2 DBMINC          % ; I T TM1 TM2 DBMINC
```

- In PCC 2 mode

```
NOP I T TM1 TM2 DBMINC          % ; I          DBMINC
                                % ; I T TM1 TM2 DBMINC
```

In PCC2 mode, DBMINC instruction is related to 2 step specification.

2. 2.14 DATA DEFINE Statement

DATA DEFINE statement is used to define the data selection of PDS.

PDS shows the programmable data selector.

Set DATA DEFINE statement by the following three statements:

- DATA DEFINE statement:
All units are set.
- MAIN DATA DEFINE statement:
The unit assigned as MAIN PG is set.
- SUB DATA DEFINE statement:
The unit assigned as SUB PG is set.

2. 2.14. 1 PDS DATA Statement

Function: PDS DATA statement is used to define the data selection of PDS to output to PDS (D0-17 and SD0-17 in the pin statement).

Before writing the PDS DATA statement, the specification of DATA DEFINE statement (MAIN DATA DEFINE statement/SUB DATA DEFINE statement) is required.

Syntax

```
PDS DATA = d/sd
```

Argument: d

Test data to output to pin data D0-17

sd

Test data to output to pin data SD0-17

The following can be specified for d and sd.

d	sd
D CBM1 FDA	D CBM2 FDB

The data combination to be selected for d and sd is free.

However, in the case of definition by SUB DATA DEFINE statement, FDA and FDB cannot be specified. Also, FDA or FDB cannot be specified in the T5581 2WAY interleaved mode.

On the T5593, T5592 and T5591, the FDA or FDB specification is processed as D if specified.

◆ Example

```
DATA DEFINE
PDS DATA=FDA/D
```

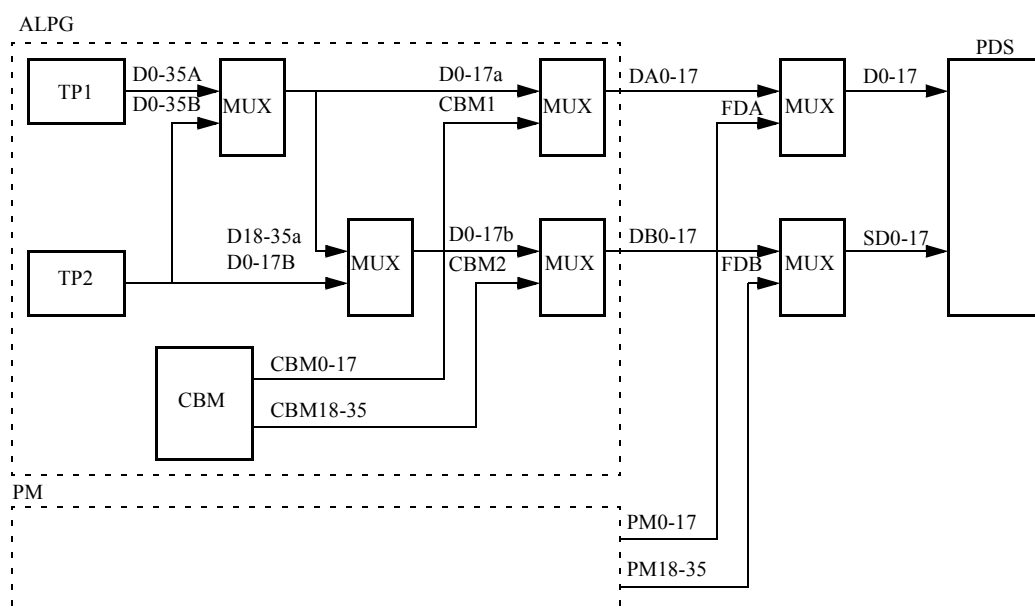
Test data of D, CBM1, CBM2, FDA, and FDB mean the following data.

D	The data output by MODE TPM and data generating instruction. (The data from TP1, TP2, and CBM)
CBM1	Bit 0-17 of CBM data (Always output)
CBM2	Bit 18-35 of CBM data (Always output)
FDA	Output data bit0-17 in the pattern mode of PM expected value However, it cannot be specified to the unit assigned as SUB PG.
FDB	Output data bit 18-35 in the pattern mode of PM expected value However, it cannot be specified to the unit assigned as SUB PG.

When there is no specification in the common part, PDS DATA = D/D becomes default of MAIN and SUB.

When there is no specification in the module part, the specification in the common part becomes default.

The flow of D, CBM1, CBM2, FDA, and FDB data is as follows.



Tip

- In the T5593, T5592 and T5591, data generation from PM cannot be performed because no PM was installed.
- The specification in this statement can be rewritten by the SELECT PDS DATA statement in the test plan program. In other words, the specification in the test plan program has priority.
- When the data is specified from DBM option, specify PPAT in PIN statement.

2. 2.15 MODULE Statement

MODULE statement is a separator when two or more patterns are entered in one pattern program. The format always begins with MODULE BEGIN and terminates with MODULE END.

Normally, the START statement explicitly indicates from which address of WCS the module is allocated, and the REG MPAT PC statement in the test plan program specifies the module whose pattern is used for the test.

The section prior to the first module section in the pattern program is called the common section. A specification by the module section has priority over one by the common section, if the two sections specify different data.

In case of the common section is assigned and the module section is not assigned then the assignment in the common section is reflected to the module section.

➔ For information on MODULE statement description examples, refer to ["The Common Section and Module Section of a Pattern Program" on page 2-2](#) and ["Sample MODULE Description" on page 2-3](#).

2. 2.16 AFM PATTERN Statement

This statement declares the start of an ROM test pattern.

This statement is used to write an ROM test pattern in a pattern program. One pattern can be written in the command part and in each module.

➔ For the description format of the ROM test pattern data, refer to [2. 7 "ROM Test Pattern Data Program Section" on page 2-193](#).

This statement is effective in the system having the PM option.

2. 2.17 DBM PATTERN Statement

This statement declares the start of a DBM test pattern.

When a random logic pattern is wanted to use in DUT test, this statement is used to write the random logic pattern, which will be stored in DBM (option), in a pattern program.

Single-specification for MAIN or SUB is also possible. One pattern can be written in the command part and in each module.

➔ For the description format of DBM, refer to [2. 8 "DBM Pattern" on page 2-202](#).

This statement can be used in the system equipped with DBM option.

2. 2.18 CBM PATTERN Statement

This statement declares the start of a CBM test pattern.

When the random logic pattern is wanted to use instead of the data of TP register, this statement is used to write the random logic pattern which will be stored in CBM (standard equipped) in a pattern program.

Single-specification for MAIN or SUB is also possible. One pattern can be written in the command part and in each module.

➔ For the description format of CBM, refer to [2. 9 "CBM Pattern" on page 2-230](#).

2. 2.19 BWPE ADDRESS Statement

When an expected value is generated by the Block Write Pseude Emulator (BWPE), BWPE ADDRESS statement selects the address to be applied to each register file of BD REG-FILE, CD REG-FILE and MD REG-FILE.

➔ For the expected value generation by BWPE, refer to [2. 5. 12 "BWPE\(Block Write Pseudo Emulator\) Field" on page 2-165](#).

Set BWPE ADDRESS by the following three statements:

- BWPE ADDRESS a0 [,a1,a2,a3,a4,a5]
All units are set.
- MAIN BWPE ADDRESS a0 [,a1,a2,a3,a4,a5]
The unit assigned as MAIN PG is set.
- SUB BWPE ADDRESS a0 [,a1,a2,a3,a4,a5]
The unit assigned as SUB PG is set.

When there is no specification in the common section, the BWPE ADDRESS data in the common section does not select the address.

When there is no specification in the module section, common section MAIN BWPE ADDRESS become the default of MAIN BWPE ADDRESS.

Also, common section SUB BWPE ADDRESS become the default of SUB BWPE ADDRESS.

When the data is set in the module section, the setting of the module section becomes valid.

For a0 to a5, specify the address to be applied to each register file.

a0

Select the address applied to BD REG-FILE address bit 0.

a1

Select the address applied to BD REG-FILE address bit 1.

a2

Select the address applied to CD REG-FILE address bit 0.

a3

Select the address applied to CD REG-FILE address bit 1.

a4

Select the address applied to MD REG-FILE address bit 0.

a5

Select the address applied to MD REG-FILE address bit 1.

For a0 to a5, specify as follows:

X(Y)n

Specification of the address applied to the register file.

0

Specification of no address selection for the register file.

Specify n within the following range.

For T5593

$$0 \leq n \leq 17$$

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

$$0 \leq n \leq 15$$

◆ **Example**

```
BWPE ADDRESS X1,X2,0,0,Y1,Y2
```

a0 to a5 can also be specified continuously.

```
X(Y)n-m  
X(Y)n-X(Y)m
```

Specify n and m within the following range.

For T5593

$$0 \leq n, m \leq 17$$

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

$$0 \leq n, m \leq 15$$

◆ **Example**

```
BWPE ADDRESS X0-X3,Y1-2
```

In this case, it is the same as the specification of BWPE ADDRESS X0, X1, X2, X3, Y1, Y2.
Also, above specifications can be combined to specify the BWPE ADDRESS.

◆ **Example**

```
BWPE ADDRESS X1,X2,0,0,Y1-2
```

a1 to a5 can be omitted. For the register file corresponding to the omitted part, the address selection is not done. (It is the same as the specification of "0.")

◆ **Example**

```
BWPE ADDRESS X1,X2
```

In this case, it is the same as the specification of BWPE ADDRESS X1, X2, 0, 0, 0, 0.

 Tip

In the same common section/module section, BWPE ADDRESS statement and MAIN BWPE ADDRESS statement, or BWPE ADDRESS statement and SUB BWPE ADDRESS statement cannot be described at the same time.

◆ Example

```
BWPE ADDRESS X1,X2,Y1,Y2,0,0
```

X1 is selected to the BD REG-FILE address bit 0.

X2 is selected to the BD REG-FILE address bit 1.

Y1 is selected to the CD REG-FILE address bit 0.

Y2 is selected to the CD REG-FILE address bit 1.

Address of MD REG-FILE address bit 0 is not selected.

Address of MD REG-FILE address bit 1 is not selected.

2. 2.20 BANK SAVE ADDRESS Statement

The T5593 is provided with a function which saves and loads the low address of each bank by using the XASV instruction and the XALD instruction in order to test memory devices which have multiple banks.

➔ For more information, refer to [2. 6. 19 "X Address Save Field" on page 2-190](#).

By specifying the bank address on the low side by using the BANK SAVE ADDRESS statement, the low address (X-address from ALPG) of the cycle in which the XASV instruction is specified can be saved for each bank address.

(Up to 64 low addresses can be saved.)

The syntax of the BANK SAVE ADDRESS statement is as follows:

- BANK SAVE ADDRESS a0 [a1, a2, a3, a4, a5]
All units are set.
- MAIN BANK SAVE ADDRESS a0 [a1, a2, a3, a4, a5]
The unit assigned as the MAIN PG is set.
- SUB BANK SAVE ADDRESS a0 [a1, a2, a3, a4, a5]
The unit assigned as a SUB PG is set.

a0

An address, which corresponds to the low-side bank address bit 0, is selected.

a1

An address, which corresponds to the low-side bank address bit 1, is selected.

a2

An address, which corresponds to the low-side bank address bit 2, is selected.

a3

An address, which corresponds to the low-side bank address bit 3, is selected.

a4

An address, which corresponds to the low-side bank address bit 4, is selected.

a5

An address, which corresponds to the low-side bank address bit 5, is selected.

a0 to a5 select the address bits of X and Y addresses which are used in the low-side bank address. The following address bits can be specified:

X0 to X17, Y0 to Y17, 0

"0" shows the Fix Low bit regardless of the bank address. (Shows that no address bit is selected.)

a1 to a5 can be omitted. The bank address for the omitted bit is not selected. (Same as specifying "0.")

◆ **Example**

```
BANK SAVE ADDRESS X13,X14
```

a0 to a5 can be specified as shown below.

Single specification

Specify separate address bits one by one by using commas.

◆ **Example**

```
BANK SAVE ADDRESS X13,X14,0,0,0,0
```

Continuous specification

Specify by connecting address bits with a hyphen.

Continuous specification in ascending or descending numerical order.

X_{n-m} $0 \leq n, m \leq 17$

Y_{n-m}

X_n-X_m

Y_n-Y_m

◆ **Example**

```
BANK SAVE ADDRESS X12-X17
```

Combination of single specification and continuous specification

Single specification and continuous specification can be combined and specified.

◆ Example

```
BANK SAVE ADDRESS X12-X15,0,0
```

The BANK SAVE ADDRESS statement can be specified in the common or module sections.

When transferring the common section, the bank address is not selected without specifying the BANK SAVE ADDRESS statement. (It is the same as the specification of "0" for a0 to a5.) If the common section is specified, the bank address is selected by the specification for the common section.

When transferring the module section, if the module section BANK SAVE ADDRESS statement is not specified, the bank address is selected by specifying the common section. If the module is specified, the bank address is selected by specifying the module section.

Tip

- BANK SAVE ADDRESS statement can be used in the T5593.
- In the same common or module section, a BANK SAVE ADDRESS statement and a MAIN BANK SAVE ADDRESS statement, or a BANK SAVE ADDRESS statement and a SUB BANK SAVE ADDRESS statement cannot be described at the same time.
- The addresses selected for the X and the Y addresses specified by the BANK SAVE ADDRESS statement have not been through the PRESCRAM part.

2. 2.21 BANK LOAD ADDRESS Statement

The T5593 is provided with a function which saves and loads the low address of each bank by using the XASV instruction and the XALD instruction in order to test memory devices which have multiple banks.

➔ For more information, refer to [2. 6. 19 "X Address Save Field" on page 2-190](#).

By specifying the column side bank address by using a BANK LOAD ADDRESS statement, the low address, which is saved by an XASV instruction, can be loaded for each bank address by using a cycle that specifies an XALD instruction.

The syntax of the BANK LOAD ADDRESS statement is as follows:

- BANK LOAD ADDRESS a0 [,a1, a2, a3, a4, a5]
All units are set.
- MAIN BANK LOAD ADDRESS a0 [,a1, a2, a3, a4, a5]
The unit assigned as the MAIN PG is set.
- SUB BANK LOAD ADDRESS a0 [,a1, a2, a3, a4, a5]
The unit assigned as a SUB PG is set.

a0

An address, which corresponds to the column-side bank address bit 0, is selected.

a1

An address, which corresponds to the column-side bank address bit 1, is selected.

a2

An address, which corresponds to the column-side bank address bit 2, is selected.

a3

An address, which corresponds to the column-side bank address bit 3, is selected.

a4

An address, which corresponds to the column-side bank address bit 4, is selected.

a5

An address, which corresponds to the column-side bank address bit 5, is selected.

a0 to a5 the select address bits of X and Y addresses which are used in the column-side bank address. The following address bits can be specified:

X0 to X17, Y0 to Y17, 0

"0" shows the Fix Low bit regardless of the bank address. (Shows that no address bit is selected.)

a1 to a5 can be omitted. The bank address for the omitted bit is not selected. (Same as specifying "0.")

◆ Example

```
BANK LOAD ADDRESS X13,X14
```

The specification method of a0 to a5 and the specification of the common or module section are the same as in the BANK SAVE ADDRESS statement.

➔ For more information, refer to [2. 2. 20 "BANK SAVE ADDRESS Statement" on page 2-52.](#)

Tip

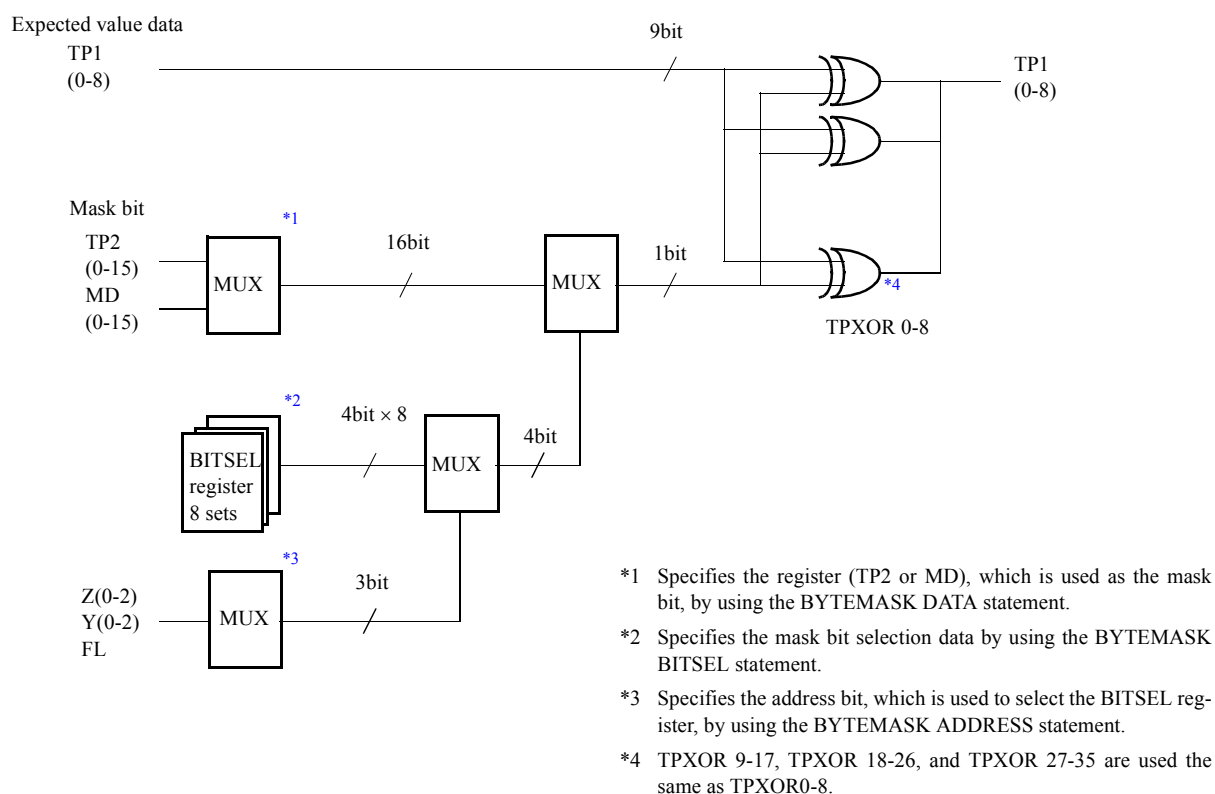
- *BANK LOAD ADDRESS statements can be used in the T5593.*
- *In the same common or module section, a BANK LOAD ADDRESS statement and a MAIN BANK LOAD ADDRESS statement, or a BANK LOAD ADDRESS statement and a SUB BANK LOAD ADDRESS statement cannot be described at the same time.*
- *The addresses selected for the X and the Y addresses specified by the BANK LOAD ADDRESS statement have not been through the PRESCRAM part.*

2. 2.22 BYTEMASK DEFINE Statement

The memory device provides a function which conducts mask control (byte mask control) for each byte data by using the 1 bit mask-bit during write operation.

A BYTEMASK DEFINE statement masks the expected value data (TP1 data) byte by byte by using 1 bit mask bits (a specific bit of TP2 or MD REG-FILE) to generate the expected value data by emulating the byte mask control. This function is called the byte mask mode.

Figure 2-3 Block Diagram of Byte Mask Mode Outline



The syntax of the BYTEMASK DEFINE statement is as follows:

```
BYTEMASK DEFINE
  BYTEMASK DATA = TP2/MD
  BYTEMASK MD ADDRESS = ma0,ma1
  BYTEMASK ADDRESS = a0,a1,a2
  BYTEMASK BITSEL0 = b00,b01,b02,b03,b04,b05,b06,b07
  BYTEMASK BITSEL1 = b10,b11,b12,b13,b14,b15,b16,b17
  BYTEMASK BITSEL2 = b20,b21,b22,b23,b24,b25,b26,b27
  BYTEMASK BITSEL3 = b30,b31,b32,b33,b34,b35,b36,b37
```

The BYTEMASK DEFINE statement can be specified in the common or module section.

When transferring the common section, if there is no specification by a BYTEMASK DEFINE statement, byte mask mode is not specified. If the common section is specified, the byte mask mode is specified by specifying the common section.

When transferring the module section, if the module section BYTEMASK DEFINE statement is not specified, the byte mask mode is specified by specifying the common section. If the module section is specified, the byte mask mode is selected by specifying the module section.

BYTEMASK DEFINE

BYTEMASK DEFINE declares the byte mask mode.

Each data declaration statement of the byte mask mode is described after this declaration.

When using the byte mask mode, specify D<TPXOR in the cycle in which byte mask is performed.

Tip

BYTEMASK DEFINE statements can be used in the T5593.

BYTEMASK DATA = TP2 or MD

A register, which is used as the mask bit, is selected.

TP2

TP2 register is used as the mask bit.

MD

MD REG-FILE is used as the mask bit.

MD REG-FILE is a 4-word register file. To select 1 word out of 4 words, specify the BYTEMASK MD ADDRESS statement.

Tip

- *If the BYTEMASK DEFINE statement is described and the body text is omitted, TP2 is selected.*
- *A MODE BWPEM or BWPE ADDRESS statement cannot be specified in the common section or module section in which BYTEMASK DATA = MD is specified.*

BYTEMASK ADDRESS = a0,a1,a2

The address bit, which is used as mask bit selection, is specified.

The mask bit is stored in the 8-word BITSEL register by the BYTEMASK BITSEL statement.

The address bit specified in the body text is used to select one BITSEL register.

The address bit, which is used to generate the burst address, is specified.

a0

Selection of address bit 0

a1

Selection of address bit 1

a2

Selection of address bit 2

For a0 to a2, specify one of Y0 to Y3, Z0 to Y3, and 0.

"0" is a specification not to perform address selection resulting in Fix Low.

a0 to a2 can be specified as shown below.

Single specification

Specify separate address bits one by one by using commas.

◆ **Example**

```
BYTEMASK ADDRESS = Y0,Y1,Y2
```

Continuous specification

Specify by connecting address bits with a hyphen.

Continuous specification in ascending or descending numerical order.

◆ **Example**

```
BYTEMASK ADDRESS = Y0-2
```

Combination of single specification and continuous specification

Single specification and continuous specification can be specified in combination.

◆ **Example**

```
BYTEMASK ADDRESS = Y0-1,Y2
```

 Tip

- If the BYTEMASK DEFINE statement is described and the body text is omitted, a0, a1, and a2 lead to the same result as when "0" has been specified.
- a1 and a2 can be omitted. If omitted, the result is the same as the result when "0" is specified for a1 and a2.
- If Y0 to Y3 is specified in the double speed mode by using PRESCRAM, the Y address in the first half of the double speed mode is used.

BYTEMASK MD ADDRESS = ma0,ma1

If BYTEMASK DATA = MD is specified, the address to be used for selection one word is specified among 4-work MD REG-FILE.

In the MD REG-FILE, it is necessary to write data of the TP2 register by a BMW instruction in advance.

➔ For information on BMW instruction, refer to [2. 5. 12 "BWPE\(Block Write Pseudo Emulator\) Field" on page 2-165](#).

The specification in the body text is also used for word selection for the BMW instruction.

ma0

MD REG-FILE address bit 0 is selected.

ma1

MD REG-FILE address bit 1 is selected.

Specify X0 to X17, Y0 to Y17, or 0 for ma0 and ma1.

"0" is specified to not perform address selection, and MD REG-FILE address bit becomes Fix Low.

ma0, and ma1 can be set to single specification, continuous specification in ascending and descending numeric order, and any of combination.

➔ For information on single and continuous specification, refer to ["BYTEMASK ADDRESS = a0,a1,a2" on page 2-57](#).

◆ Example

```
BYTEMASK MD ADDRESS = X0,X1
BYTEMASK MD ADDRESS = X0-1
```

Tip

- When *BYTEMASK DATA = TP2* is described, no body text can be described.
Describe the body text after describing *BYTEMASK DATA = MD*.
- If *BYTEMASK DEFINE* statement is described and the body text is omitted, ma0, and ma1 lead to the same result as when "0" has been specified.
- ma1 can be omitted. If omitted, the result is the same as the result when "0" is specified for ma1.
- If Y0 to Y17 is specified in the double speed mode using *PRESCRAM*, the Y address in the first half of the double speed mode is used.

BYTEMASK BITSELn = bn0,bn1,bn2,bn3,bn4,bn5,bn6,bn7

Information on selecting mask bits to select a specific TP2/MD data bit as the mask bit is specified.

The mask bits which correspond to the address specified by the TP1 data byte block and the *BYTEMASK ADDRESS* statement is specified.

n

The number of TP1 data byte blocks, which are targeted for masking, is specified.

The specification range for n is 0 to 3.

➔ For information on correspondence between n and TP1 data, see [Table 2-7](#).

bn0,bn1,bn2,bn3,bn4,bn5,bn6,bn7

Specify a specific TP2/MD data bit by using the bit number as the mask bit for the byte block of TP1 which is specified by n.

bn0 to bn7 is a mask bit that corresponds to the address bit specified by the BYTEMASK ADDRESS statement.

➔ For information on correspondence between bn0 to bn7 and the address bit, see [Table 2-7](#).

For bn0 to bn7, bit number 0 to 15 can be specified.

Describe the bit number after "B."

◆ Example

If the bit number is 15, describe it as "B15."

bn0 to bn7 can be set to single specification, continuous specification in ascending and descending numeric order and any combination.

➔ For information on single and continuous specification, refer to ["BYTEMASK ADDRESS = a0,a1,a2" on page 2-57](#).

◆ Example

```
BYTEMASK BITSEL0 = B0,B1,B2,B3
BYTEMASK BITSEL0 = B0-3
BYTEMASK BITSEL0 = B0-2,B3
```

The mask bit of the BYTEMASK BITSEL statement can be specified for each TP1 byte block, which is targeted for masking, and for each address bit specified by the BYTEMASK ADDRESS statement.

Mask bit allocation is shown in [Table 2-7](#).

For example, if TP1 bits 0 to 8 are output to D0 to D8, which are defined in the pin statement, by byte-masking at the 15th bit of TP2/MD when address #7 is specified in the BYTEMASK ADDRESS statement in the MODE TPM18 mode, specify n=0 and b07=B15 in the BYTEMASK BITSEL statement.

Table 2-7 Allocation of Mask Bit (b00 - b37)

BYTE-MASK BITSEL _n	Output destination of TP1. XOR. mask bit	Byte block of masking target TP1		BYTEMASK ADDRESS=a0,a1,a2							
				aaa 210	aaa 210	aaa 210	aaa 210	aaa 210	aaa 210	aaa 210	aaa 210
		MODE TPM18	MODE TPM36	000	001	010	011	100	101	110	111
n=0	D0-8	TP1 0-8	TP1 0-8	b00	b01	b02	b03	b04	b05	b06	b07
n=1	D9-17	TP1 9-17	TP1 9-17	b10	b11	b12	b13	b14	b15	b16	b17
n=2	SD0-8	TP1 0-8	TP1 18-26	b20	b21	b22	b23	b24	b25	b26	b27
n=3	SD9-17	TP1 9-17	TP1 27-35	b30	b31	b32	b33	b34	b35	b36	b37

Tip

- If the BYTEMASK DEFINE statement is described, the body text cannot be omitted.
- Any BYTEMASK BITSEL statement of block *n* can be omitted.
The mask bit of the omitted block becomes the same as when the 0 bit of TP2/MD data are specified together with *bn*0 to *bn*7.
- Specification of mask bit of either one of *bn*1 to *bn*7 can be omitted.
The omitted mask bit becomes the same as when the 0 bit of TP2/MD data is specified.

◆ Example

In the following example, the 0 bit of TP2/MD data is specified for b04 to b07, b14 to b17, b20 to b27, and b30 to b37.

```
BYTEMASK BITSEL0 = B0-3
BYTEMASK BITSEL1 = B4,B5,B6,B7
```

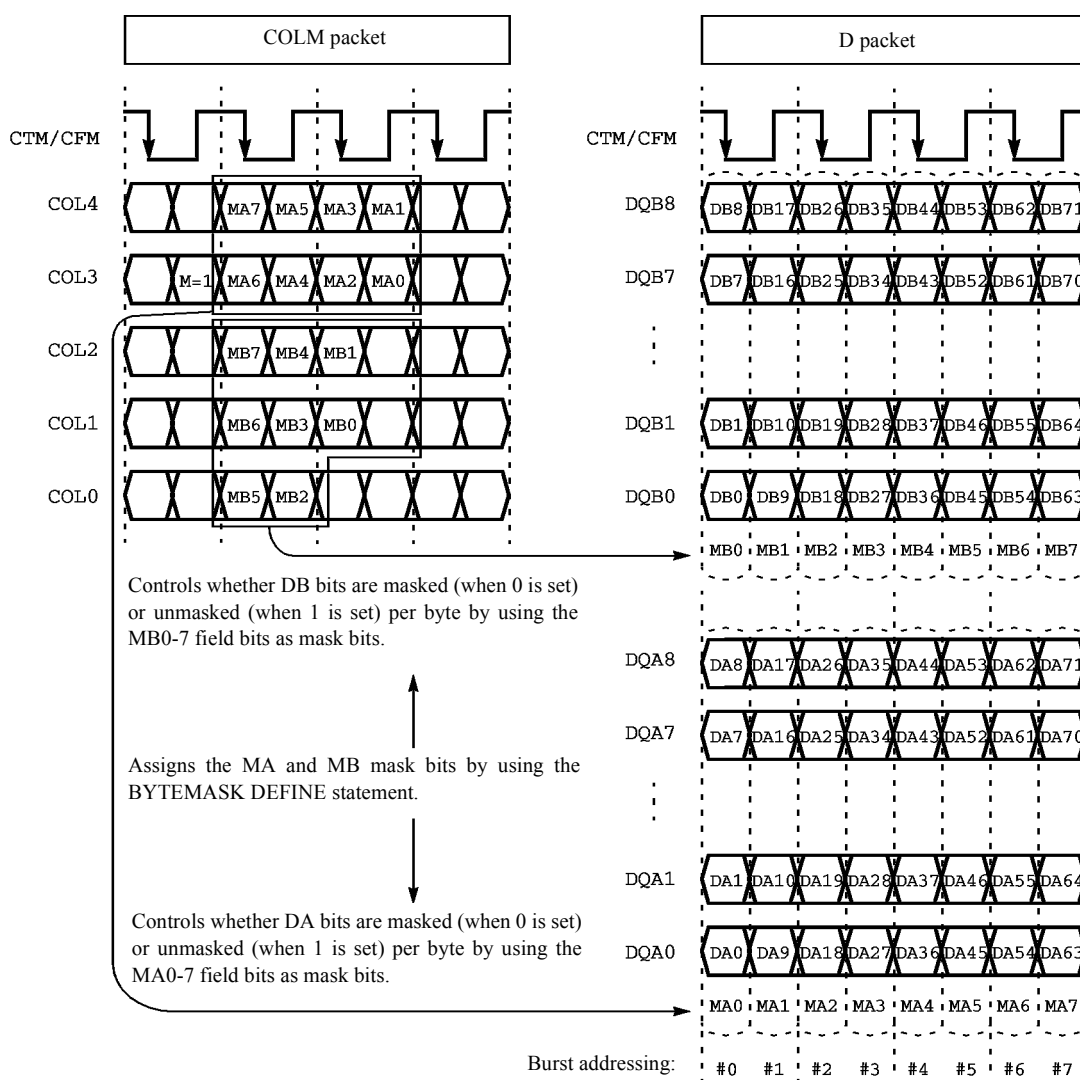
- If *D*<TPXOR is specified in byte mask mode, all 36 bits are XORed, regardless of the specification of 18 bit mode (TPM18) or 36 bit mode (TPM36) of the MODE statement.

◆ Using Example

The following example shows a case in which the expected value is generated by emulating byte mask control during the write operation of the Direct RDRAM.

The formats for the COLM packet and D packet in the case of byte mask control during write operation are described below.

Figure 2-4 Format for COLM Packet and D Packet



During data writing, the TP1 register is allocated to the data byte A(DA) and data byte B(DB) of the D packet. A specific bit of the TP2 register is allocated as the mask bit to each 16 bit of MA0 to 7, and MB0 to 7 of the COLM packet.

When outputting the expected values while reading the data, each TP1 data byte must be masked by using the TP2 data mask bit.

2. 2 Compilation Control Instructions

The correspondence of DA and DB of DQA0 to 8 pin, and DQB0 to 8 and TP1(D) data, MA and MB and TP2(SD) data, and expected value data is shown below.

Burst address *1	#0 (#0)	#1 (#0)	#2 (#1)	#3 (#1)	#4 (#2)	#5 (#2)	#6 (#3)	#7 (#3)
DQA0-8 pin								
DA	DA0-8	DA9-17	DA18-26	DA27-35	DA36-44	DA45-53	DA54-62	DA63-71
TP1(D)	D0-8	D9-17	D0-8	D9-17	D0-8	D9-17	D0-8	D9-17
MA	MA0	MA1	MA2	MA3	MA4	MA5	MA6	MA7
TP2(SD)	SD0	SD1	SD2	SD3	SD4	SD5	SD6	SD7
Expected value	D0-8 <u>.XOR.</u> SD0	D9-7 <u>.XOR.</u> SD1	D0-8 <u>.XOR.</u> SD2	D9-17 <u>.XOR.</u> SD3	D0-8 <u>.XOR.</u> SD4	D9-17 <u>.XOR.</u> SD5	D0-8 <u>.XOR.</u> SD6	D9-17 <u>.XOR.</u> SD7
DQB0-8 pin								
DB	DB0-8	DB9-17	DB18-26	DB27-35	DB36-44	DB45-53	DB54-62	DB63-71
TP1(D)	D0-8	D9-17	D0-8	D9-17	D0-8	D9-17	D0-8	D9-17
MB	MB0	MB1	MB2	MB3	MB4	MB5	MB6	MB7
TP2(SD)	SD8	SD9	SD10	SD11	SD12	SD13	SD14	SD15
Expected value	D0-8 <u>.XOR.</u> SD8	D9-7 <u>.XOR.</u> SD9	D0-8 <u>.XOR.</u> SD10	D9-17 <u>.XOR.</u> SD11	D0-8 <u>.XOR.</u> SD12	D9-17 <u>.XOR.</u> SD13	D0-8 <u>.XOR.</u> SD14	D9-17 <u>.XOR.</u> SD15

*1 The burst address is generated by using the double PRESCRAM speed mode.

In this example, the sequential address is generated by using the YC register and Z register.

The value of the Z register is shown in brackets. With the BYTEMASK ADDRESS statement, the Z register is specified.

An example of the program to generate expected values under the above-described conditions is shown below.

Test plan program

```
PRO SAMPLE1 <2WAY>
:
DQA = ..., CYP1,<D0-8,D9-17>, CYP2,<D0-8, D9-17>
DQB = ..., CYP1,<D0-8,D9-17>, CYP2,<SD0-8,SD9-17> *1
:
           ↑           ↑
       For writing data   For reading data

SELECT PRESCRAM ENABLE
SELECT PRESCRAM MODE ADDZ2 ; Specification of the double speed mode
SELECT PRESCRAM ADDRESS FM,FP
SET WRAP ADDRESS = Y0-2
:
MEAS MPAT SAMPLE2
:
END
```

Pattern program

```

MPAT SAMPLE2 <2WAY>

MODE TPM18 *1

REGISTER
  XH =0      ; BURST START ADDRESS
  YH =0
  ZH =0
  D1 =8      ; BURST LENGTH = 8
  TPH1=#000F
  TPH2=#5555 ; MASK BIT

BYTEMASK DEFINE
  BYTEMASK DATA = TP2 *2
  BYTEMASK ADDRESS = Z0,Z1,0 *3
  BYTEMASK BITSEL0 = B0, B2, B4, B6 ; MA0 MA2 MA4 MA6
  BYTEMASK BITSEL1 = B1, B3, B5, B7 ; MA1 MA3 MA5 MA7
  BYTEMASK BITSEL2 = B8, B10,B12,B14 ; MB0 MB2 MB4 MB6
  BYTEMASK BITSEL3 = B9, B11,B13,B15 ; MB1 MB3 MB5 MB7
] *4

:

START #0
:
  NOP      : XB<XH YB<YH Z<ZH   TP1<TPH1 TP2<TPH2
            : XC<XB YC<YB
ST0:  NOP      : W                      Z<Z+1 X<XC Y<YC D<TP1 CYP1
            : W                      Z<Z+1 X<XC Y<YC D<TP1 CYP1
JN11 ST0 : W                      Z<Z+1 X<XC Y<YC D<TP1 CYP1
            : W  XC<XC+1 YC<YC+D1^CY Z<Z+1 X<XC Y<YC D<TP1 CYP1
] *5

:

  NOP      : XB<XH YB<YH Z<ZH   TP1<TPH1 TP2<TPH2
            : XC<XB YC<YB
ST1:  NOP      : R                      Z<Z+1 X<XC Y<YC D<TPXOR CYP2 *1
            : R                      Z<Z+1 X<XC Y<YC D<TPXOR CYP2 *1
JN11 ST1 : R                      Z<Z+1 X<XC Y<YC D<TPXOR CYP2 *1
            : R  XC<XC+1 YC<YC+D1^CY Z<Z+1 X<XC Y<YC D<TPXOR CYP2 *1
] *6

:

STPS

END

```

*1 In the cycle in which $D<TPXOR$ is described in MODE TPM18, the following data is output to D0 to 17, and SD0 to 17.

D0 to 8 = Mask bit of TP0 to 8 .XOR.BYTEMASK BITSEL0

D9 to 17 = Mask bit of TP9 to 17.XOR.BYTEMASK BITSEL1

SD0 to 8 = Mask bit of TP0 to 8 .XOR.BYTEMASK BITSEL2

SD9 to 17 = Mask bit of TP9 to 17.XOR.BYTEMASK BITSEL3

*2 Select TP2 as the register which stores the mask bit.

*3 Select Z0 and Z1 as addresses to be used for the burst address.

*4 Specify the mask bit that corresponds to MA0 to 8 and MB0 to 8.

**5 TP1 data is output during data writing. The device writes data by performing mask control in accordance with the mask bit of MA and MB fields.*

**6 During data reading, the mask bit of TP2 is determined in accordance with the specification of *4 and TP1 data and the data obtained by XORing for each byte are taken as the expected values.*

2. 2.23 END Statement

This statement terminates the program compilation.

Write END at the end of all programs.

2. 2.24 Test Pattern Programming Format

The description format for the pattern program is as shown below.

field A

PC operation code, operand, I, T, TM1, TM2, DBMINC

field B

Main PG instruction

field C

Sub PG instruction

Specification format when the normal mode (1WAY).

Normal

```
---A--- ---B---!---C---
```

PCC = 1

```
---A---%---B---!---C---
```

PCC = 2

```
---A---%---B---!---C---
      %---B---!---C---
```

Tip

- When setting PCC mode, separate field A from field B by a % mark.
- The field C (SUB PG instruction) cannot be specified for the T5592 and T5571P.
- Separate field B from field C by the symbol !.
- When description of field B and field C are omitted, default values are inserted.
The default value is an instruction to put all the data on hold.

$TS1\ XB<XB\ YB<YB\ XC<XC\ XS<XS\ XK<XK\ YC<YC\ YS<YS\ YK<YK$
 $D3<D3\ D4<D4\ Z<Z\ N<N\ RF<RF\ X<XB\ Y<YB\ TP1<TP1\ TP2<TP2$
 $DSC<DSC\ CBMA<CBMA\ D<TP1\ B<B$

- When field C is omitted, "!" may be omitted.
When "!" is written and field C is not specified, field C is set to the default (hold) condition.
- When field B, "!", and field C are omitted, "%" may be omitted, too.
- When "!" and field C are not specified, field C is set to the condition below.

When ! is written after the START statement

Field C is set to the default (hold state).

◆ Example

```

START #10
NOP  XB<XB+1
NOP  YB<YB+1^BX
NOP  XB<XB+1    ! TP<TP+1
NOP  YB<YB+1^BX
STPS

```

In the example above, field C is not written at addresses #10, #11, #13, and #14.

In this case, each of the field C is set to the default.

When ! is not written after the START statement

Field C is set to the same instruction as field B.

◆ Example

```

START #10
NOP  XB<XB+1
NOP  YB<YB+1^BX
NOP  XB<XB+1
NOP  YB<YB+1^BX
STPS

```

In the example above, field C is not written at addresses #10 to #14.

Field C is set to the same instruction as field B of each address.

As the above, the system determines whether patterns are the same between MAIN and SUB for every START statement.

- The 2WAY/4WAY interleaved mode and PCC mode cannot be set at the same time.

Specification format when the interleaved mode is 2WAY.

In the 2WAY mode, there are two types of specifications, a normal specification and a separate specification.

- Separate format

Separate specification is to specify 2 steps of address and data generating instructions (field B, field C) for 1 sequence control instruction (field A). (Use a colon ":" as a field separating character.)

```
---A---:---B---!---C---
      :---B---!---C---
```

◆ **Example 1:**

```
START #0
NOP      :XB<0 YB<0 TP1<0
      :
L1:  JN11 L1 :XB<XB+1 YB<YB+1 W C0
      :R /D C1 TS2
STPS
```

• Normal format

A normal specification is identical to a usual specification (not in the interleaved mode).

In this case, one step of the address and data generation instruction is specified for one sequence control instruction, but the operation is the same as the one specified in two steps by the MPAT compiler with a few exceptions.

```
---A--- ---B---!---C---
```

◆ **Example 2:**

```
START #0
NOP      XB<0 YB<0 TP1<0
L1:  JN11 L1 XB<XB+1 YB<YB+1 W C0
NOP      R /D C1 TS2
STPS
```

The above specification has the same effect as the one specified below.

```
START #0
NOP      :XB<0 YB<0 TP1<0
      :XB<0 YB<0 TP1<0
L1:  JN11 L1 :XB<XB+1 YB<YB+1 W C0
      :XB<XB+1 YB<YB+1 W C0
NOP      :R /D C1 TS2
      :R /D C1 TS2
STPS      :
      :
```

Tip

- *field C(SUB PG instruction) can be described in the T5591 2WAY interleave mode, or the T5593 2WAY interleave mode in which the MSM mode (MODE MSM) is specified. This instruction cannot be specified in the T5592 and T5581 2WAY interleaved mode. Cautions for field C are the same as those for the normal mode.*
- *When field B is omitted, the default value is entered.
The default value is an instruction to put all the data on hold.
TS1 XB<XB YB<YB XC<XC XS<XS XK<XK YC<YC YS<YS YK<YK
D3<D3 D4<D4 Z<Z N<N RF<RF X<XB Y<YB TP1<TP1 TP2<TP2
DSC<DSC CBMA<CBMA D<TP1 B<B*
- *In the separate specification, write the field separating character ":" even if there is no instruction to specify in the second step.*
- *When attaching a label, write it in the field A line.*

Specification format when the interleaved mode is 4WAY.

In the 4WAY mode, there are two types of specifications, a normal specification and separate specification.

• Separate format

A separate specification specifies four steps of an address and data generation instructions (field B) for one sequence control instruction (field A). (Use a colon ":" as a field separating character.)

```
---A---:---B---
      :---B---
      :---B---
      :---B---
```

◆ Example 1:

```
START #0
      NOP      :XB<0 YB<0 TP1<0
              :
              :
              :
L1:   JN11 L1  :XB<XB+1 YB<YB+1 W C0
              :R /D C1 TS2
              :
              :
      STPS
```

• Normal format

A normal specification is identical to a usual specification (not in the interleaved mode).

In this case, one step of the address and data generation instruction is specified for one sequence control instruction, but the operation is the same as the one specified in four steps by the MPAT compiler with a few exceptions.

---A--- ---B---

◆ Example 2:

```
START #0
      NOP      XB<0 YB<0 TP1<0
L1:   JN11 L1  XB<XB+1 YB<YB+1 W C0
      NOP      R /D C1 TS2
      STPS
```

The above specification has the same effect as the one specified below.

```
START #0
      NOP      :XB<0 YB<0 TP1<0
      :XB<0 YB<0 TP1<0
      :XB<0 YB<0 TP1<0
      :XB<0 YB<0 TP1<0
L1:   JN11 L1  :XB<XB+1 YB<YB+1 W C0
      :XB<XB+1 YB<YB+1 W C0
      :XB<XB+1 YB<YB+1 W C0
      :XB<XB+1 YB<YB+1 W C0
      NOP      :R /D C1 TS2
      :R /D C1 TS2
      :R /D C1 TS2
      :R /D C1 TS2
      STPS      :
      :
      :
      :
```

Tip

- field C instruction (SUB PG instruction) cannot be specified in the 4WAY interleaved mode.
- When field B is omitted, the default value is entered.

The default value is an instruction to put all data on hold.

TS1 XB<XB YB<YB XC<XC XS<XS XK<XK YC<YC YS<YS YK<YK
D3<D3 D4<D4 Z<Z N<N RF<RF X<XB Y<YB TP1<TP1 TP2<TP2
DSC<DSC CBMA<CBMA D<TP1 B<B

- In the separate specification, write the field separating character ":" even if there is no instruction to specify in steps 2 to 4.
- When attaching a label, write it in the field A line.

- *PCC n cannot be specified in the 4WAY interleaved mode.*

2. 2.25 %REMARK Statement

By this statement, a character string following %REMARK statement and ending with the carriage return is output at the end of an object as a revision data.

Alphanumeric characters, asterisks (*), underlines (-), periods (.) and dollar signs (\$) are usable.

The maximum number of characters is 80.

This statement can be used only once in a program.

Tip

- *This statement can be described in both the common section and module section. It cannot follow the START statement.*
- *If more than 80 characters are given, 81st character and followings are ignored.*
- *If this statement is used twice or more in a program, the second and following statements are ignored.*

2. 2.26 %IFE Statement, %IFN Statement, %ENDC Statement

Function: The %IFE, %IFN, and %ENDC statements are used for conditional compilation.

Syntax

- (1) When a Global Switch Used as a Conditional Switch is Specified by an Alphabetic Character

```
%IFE X
    Group of statements to be compiled conditionally.
%ENDC

%IFN X
    Group of statements to be compiled conditionally.
%ENDC
```

(2) When a Global Switch Used as a Conditional Switch is Specified by an Operation Expression

```
%IFE Operation
    Group of statements to be compiled conditionally.
%ENDC

%IFN Operation
    Group of statements to be compiled conditionally.
%ENDC
```

Argument: x

Global switch used as a conditional switch. Specify it using one alphabetical character.

Operation expression

Global switch for logical operation. The maximum number of characters is 384.

Type of operation (Logical operators)

Logical operators can be specified for the global switch used as a conditional switch. The following operators can be used:

Parentheses can be used to prioritize the operators.

.OR.	OR
.XOR.	Exclusive OR
.AND.	AND
.NOT.	NOT

Priority of the logical operation

() $\rightarrow$.NOT. $\rightarrow$.AND. $\rightarrow$.OR.

.XOR.

Priority between .OR. and .XOR. is written order.

Description

A single global switch

Since it is True that the global switch corresponding to a conditional switch of a conditional compilation statement is specified in a compilation command and it is False that the switch is not specified in the command, conditional compilation is performed as follows:

%IFE x

:

%ENDC

%IFN x

:

%ENDC

} If true, executes compilation.

} If false, no compilation.

} If true, no compilation.

} If false, executes compilation.

Logical operation

As a conditional switch, define it is true when the result of a logical operation of the specified global switch is 1, and false when the result is 0. Then perform conditional compilation.
Specify the global switch in the compilation command.

%IFE Operation expression

:

%ENDC

%IFN Operation expression

:

%ENDC

} If true, executes compilation.

} If false, no compilation.

} If true, no compilation.

} If false, executes compilation.

◆ Example

trans -C AF

%IFE F.AND. (A.OR.H)

:

%ENDC

%IFN F.AND. (A.OR.H)

:

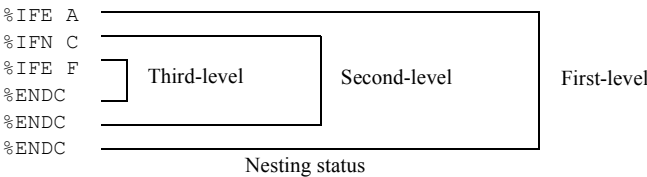
%ENDC

} Executes compilation.

} No compilation.

Tip

- Up to ten levels of nesting are allowed for conditional compilation statements.



2. 2 Compilation Control Instructions

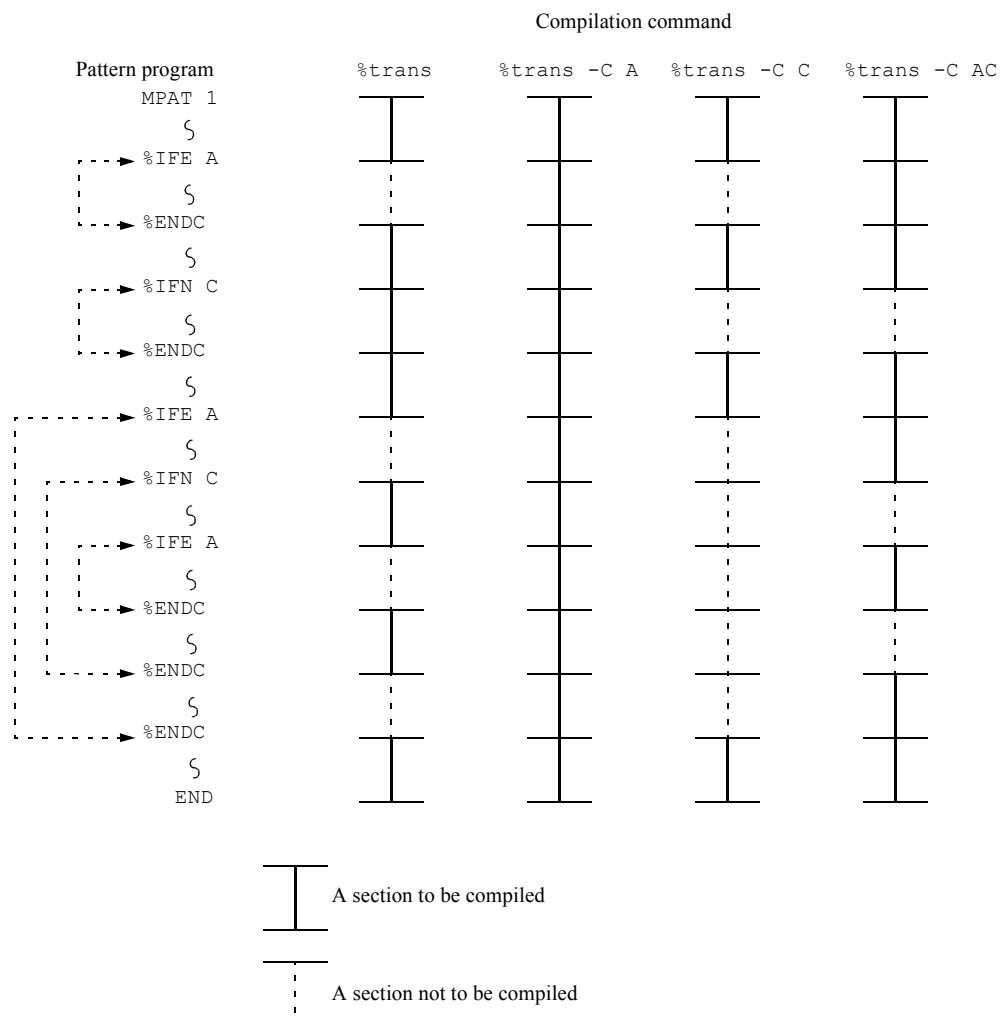
- *The total number of %IFE statements and %IFN statements need not be the same as the number of %ENDC statements, but if the number of %ENDC statements is larger, a compilation error occurs.*
- *Statements that would not be compiled according to a conditional compilation statement can be compiled by*

`%IFE, %IFN, %ENDC, %INSERT`

- ➔ For information on the %INSERT statement, refer to [2. 2. 27 "INSERT Statement" on page 2-75](#).
 - *Conditional compilation statements must be at the left side of the lines.*
-

Example

(1) Sections of a Pattern Program Compiled by Conditional Compilation Statements



2. 2.27 INSERT Statement

Function: The INSERT statement inserts another source file to be compiled during compilation of a source file.

Syntax

```
INSERT file-name
%INSERT file name
```

Argument: file-name:

Specifies the name of the source file to be inserted. Specify as follows:

◆ **Example**

```
INSERT  ABC001
```

ABC001 is the source file name.

Description

The specified source file is inserted into the line and then compiled.

There are two types of INSERT statements: %INSERT and INSERT. The %INSERT statement is not affected by conditional compilation statements (%IFE, %IFN, and %ENDC). This means that the %INSERT file inserts the source file even though it will not be compiled by a conditional compilation statement.



Tip

- *Up to five levels of nesting is allowed for INSERT statements.*

◆ When ASC AO is Compiled

ASC AO

```
MPAT  A0
REGISTER
  IDX1=#1
  IDX2=#2
INSERT BO
  IDX7=#7

START #0
NOP
STPS
END
```

ASC BO

```
IDX3=#3
INSERT CO
nmIDX6=#6
```

ASC CO

```
IDX4=#4
IDX5=#5
```

Compilation results

```
Line 1      MPAT  A0
Line 2      REGISTER
Line 3      IDX1 = #1
Line 4      IDX2 = #2
Line 5
Line 6      IDX3 = #3
Line 7
Line 8      IDX4 = #4
Line 9      IDX5 = #5
Line 10     IDX6 = #6
Line 11     IDX7 = #7
Line 12
Line 13     START #0
Line 14     NOP
Line 15     STPS
Line 16     END
```

- *Up to 49 INSERT statements can be used in one program.*

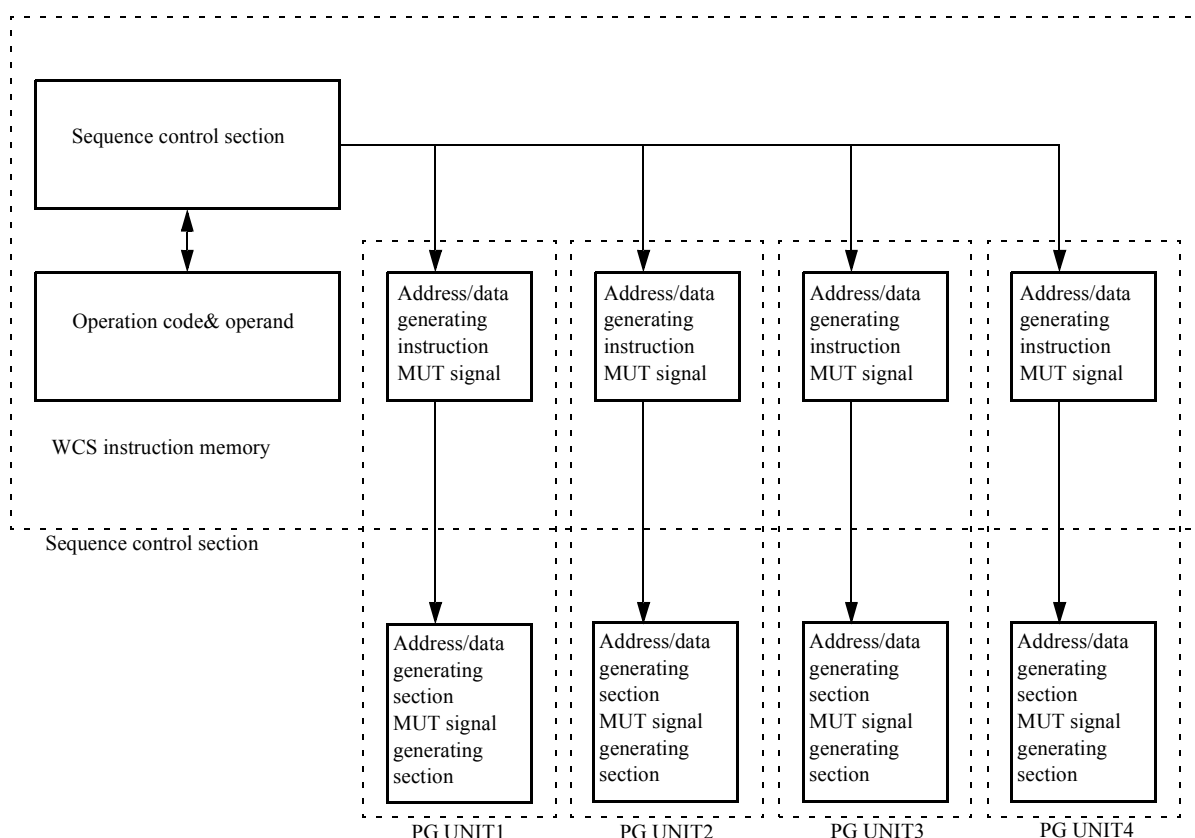
2. 3 Pattern Sequence Controller

The pattern sequence controller controls PG UNIT1, PG UNIT2, PG UNIT3 and PG UNIT4 as shown in [Figure 2-5](#), Block Diagram of ALPG Outline. (It controls PG UNIT1 and PG UNIT2 on T5593 in systems in which SUBPG option is not equipped and T5581, and only PG UNIT1 on T5571P.)

The test pattern generating instructions (address/data generating instructions and MUT signal) described in the pattern program are stored in the instruction memory of PG UNIT1, PG UNIT2, PG UNIT3 and PG UNIT4 (WCS).

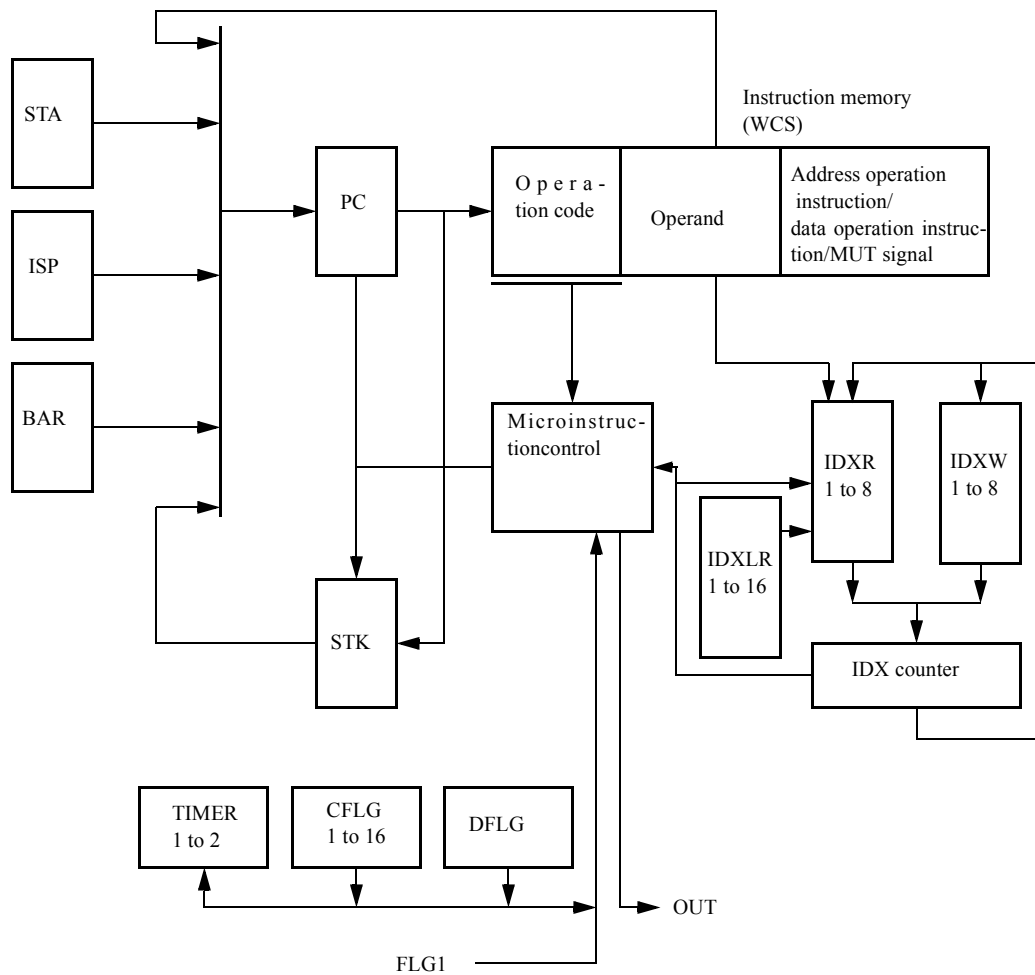
As shown in [Figure 2-6](#), the pattern sequence controller is comprised of registers, memory, and a timer. The controller applies each operation instruction stored in WCS (instruction described in field B or field C) to the address generator, data generator, and MUT signal generator in PG UNIT1, PG UNIT2, PG UNIT3 and PG UNIT4 respectively, by decoding and executing the operation code (instruction described in field A) stored in WCS.

Figure 2-5 Block Diagram of ALPG Outline



* PG UNIT3 and PG UNIT4 are not included in the T5593 without the SUBPG option installed and the T5581.

* T5571P does not have PG UNIT2, PG UNIT3, and PG UNIT4.

Figure 2-6 Function Block Diagram of Sequence Control Section

IDXLR1 to IDXLR16 and CFLG9 to CFLG16 are included in the T5593.

T5593

m=1 to 16

T5592/T5591/T5586/T5585/T5581/T5581H/T5581P/T5571P

m=1 to 8



Tip

- *FLGLII* can be executed in *T5586*, *T5585*, *T5581*, *T5581H*, *T5581P* and *T5571P*.
- *LDIn*, *STIn*, *STISP*, and *DBMSET* can be executed in *T5593*.

A branch instruction (conditional and unconditional branch instructions) is one of the sequence control instructions. Its branch address can be specified in the following three ways:

- Specification with label

```
ST0: JNII1 ST0
```

The label name must be specified by using characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

The same label can be used for different modules.

- Specification of absolute address in WCS

```
START #0
NOP
JNII1 #11
```

- Specification of relative address

```
JNII1 .
JNII1 .±n
```

Branches to an address before the current one (.-n), or an address after the current one (.+n).

If only a period is entered, the current address is specified.

The sequence for the test pattern is generated by the combination of the instructions in the sequence control.

Because test pattern generation is stopped by the STPS or STFL instruction, enter STPS or STFL at the end of the test pattern.

PASS stops STPS.

FAIL stops STFL.

2. 3. 1 Index Loop

Use the index reference instruction for pattern program loops.

Table 2-8 shows the index reference instructions of the sequence control section.

Table 2-8 Index Reference Instructions of the Sequence Control Section

Mnemonic	Action
STIn m (n = 1 to 8) This mode can be executed in T5593.	(PC) + 1 → PC m → IDXRn
JNIn m (n = 1 to 8)	1st m → PC (IDXRn) → IDX (IDX) → IDXWn 2nd if (IDXWn) = 0 then else m → PC (IDXRn) → IDX (IDX) - 1 → IDXWn
IDXIn m (n = 1 to 8)	1st (PC) → PC m → IDX 2nd if (IDX) = 0 then else (PC) + 1 → PC (PC) → PC (IDX) - 1 → IDX
LDIn m (n = 1 to 8, m = 9 to 16 or m = n) This mode can be executed in T5593.	(PC) + 1 → PC IDXLm → IDXRn

The index reference instruction performs loops using the IDXR1 to 8 registers shown in Figure 2-4, IDXW1 to 8 registers and IDX counter.

The registers (IDXR1 to 8) store the number of loops. The number of loops can be set by the following instructions:

- STIn instruction
- LDIn instruction
- Setting by the REGISTER statement in the pattern program.

```
REGISTER
  IDXn=FFFFFFFF n=1 to 8
```


- Setting by REG statement in the test plan program.

```
REG MPAT IDXn=#FFFFFFF n=1 to 8
```

IDXW1 to 8 registers are used to save data of the IDX counter when nesting loops.

Each index reference instruction is explained below.

STIn m (n=1 to 8)

Use this instruction for setting the number of loops m to IDXRn.

The specification range for m is #0 to #FFFFFFF.

JNIn m (n=1 to 8)

This instruction is used to jump to the PC shown by the operand m for (n + 1) times (the number of setting loops indicated by IDXRn), and execute PC+1 → PC at the (n + 2)th setting loop. Use this instruction to create a pattern program loop.

IDXIn m (n=1 to 8)

This is a one pattern loop instruction that holds PC (m+2) times indicated by the operand, and proceeds to the next step.

The specification range for m is #0 to #FFFFFFF in T5593 and #0 to #3FF in T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P.

LDIn m (n=1 to 8, m=9 to 16 or m=n)

This instruction sets the number of loops stored in IDXLm to IDXRn.

The number of loops is stored in IDXLm by using the following methods.

- Setting by the REGISTER statement in the pattern program.

```
REGISTER
  IDXm=#0 to #FFFFFFF (m=1 to 16)
```

- Setting by REG statement in the test plan program.

```
REG MPAT IDXm=#0 to #FFFFFFF (m=1 to 16)
```

When the number of loops of IDXRn is changed by the LDIn or STIn instruction, the restoration to the number of loops before the change can be performed by execution of LDIn instruction with m=n.

◆ Example of Program Using the LDIn Instruction

```

MPAT mpat_file_name

REGISTER
  IDX1 =#6      ; IDXR1 and IDXHR1 setting
  IDX2 =#FD     ; IDXR2 and IDXHR2 setting
  IDX3 =#3D     ; IDXR3 and IDXHR3 setting
  IDX4 =#2      ; IDXR4 and IDXHR4 setting
  IDX5 =#6      ; IDXR5 and IDXHR5 setting
  IDX9 =#D      ; IDXHR9 setting
  IDX10=#3D     ; IDXHR10 setting
  IDX11=#FD     ; IDXHR11 setting
  IDX12=#6      ; IDXHR12 setting
  IDX13=#2      ; IDXHR13 setting

START #0
ST0: NOP
  JN11 ST0      ; Executes the loop #6 + 2 times
  NOP
  JN12 ST0      ; Executes the loop #FD + 2 times
  NOP
  JN13 ST0      ; Executes the loop #3D + 2 times
  NOP
  JN14 ST0      ; Executes the loop #2 + 2 times
  NOP
  JN15 ST0      ; Executes the loop #6 + 2 times

  LDI1 9        ; IDXR1 ← IDXHR9
  LDI2 10       ; IDXR2 ← IDXHR10
  LDI3 11       ; IDXR3 ← IDXHR11
  LDI4 12       ; IDXR4 ← IDXHR12
  LDI5 13       ; IDXR5 ← IDXHR13

  NOP
ST1: JN11 ST1    ; Executes the loop #D + 2 times
  NOP
  JN12 ST1      ; Executes the loop #3D + 2 times
  NOP
  JN13 ST1      ; Executes the loop #FD + 2 times
  NOP
  JN14 ST1      ; Executes the loop #6 + 2 times
  NOP
  JN15 ST1      ; Executes the loop #2 + 2 times

  LDI1 1        ; IDXR1 ← IDXHR1
  LDI2 2        ; IDXR2 ← IDXHR2
  LDI3 3        ; IDXR3 ← IDXHR3
  LDI4 4        ; IDXR4 ← IDXHR4
  LDI5 5        ; IDXR5 ← IDXHR5
} Resets the loop execution count to the value which is set in the
  REGISTER statement.

JZD ST0
STPS

```



Tip

Follow the points below when creating programs using the index reference instructions.

The combination of the following instructions does not operate.

```

      :
      JNIn ST0
      :
ST0:  JNIm ST1
      :

```

When JNIn is issued successively for $n=m$, and when the specification of the above ST1 is ST0, the instructions operate normally.

2. 3. 2 Subroutine

Subroutine call instructions are shown in [Table 2-9](#).

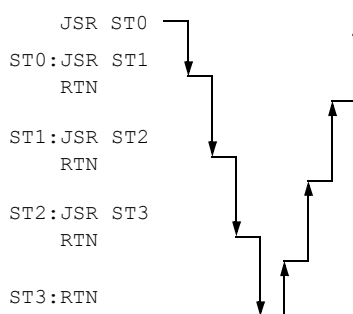
Table 2-9 Subroutine Call Instructions

Mnemonic	Action
JSR m	(PC) + 1 → STK ↓ (Push) m → PC
RTN	STK ↑ (pop) → PC

Up to four nestings can be made.

JSR m

This instruction pushes (PC)+1 to the subroutine stack and jumps to the PC shown by operand m.



RTN

The instruction used to return from a subroutine-called subroutine. The instruction jumps to the PC shown in STK and pops the subroutine stack.

2. 3. 3 Operation Register Setting Instruction (SET reg. data)

SET register name m(m : set value)

The contents of the registers used for address operation or data operation in ALPG can be set with the SET instruction.

Data, set in a register with the SET instruction, is valid in the next and subsequent steps.

```

D1 = #1
:
SET D1 #2      YC < YC + D1 ← Operate with D1=#1
NOP            YC < YC + D1 ← Operate with D =#2

```

Table 2-10 shows the registers that can be set by the SET instruction.

Table 2-10 Registers the SET Instruction can Set

Register	Bit size	
	T5593	T5592/T5591/T5586/T5585/T5581H/ T5581P/T5571P
XH	18	16
YH	18	16
D1 (D1A)	18	16
D2 (D2A)	18	16
D3B	18	16
D4B	18	16
NH	8	4
ZH	18	16
Z	18	16
XOS	18	16
YOS	18	16
XT	18	16
YT	18	16
XMASK	18	16
YMASK	18	16
RCD	18	16
CCD	18	16
DSD	18	16

Register	Bit size	
	T5593	T5592/T5591/T5586/T5585/T5581/T5581H/ T5581P/T5571P
DSC	4	4
TP1	18(36)	18(36)
TP2	18(36)	18(36)
TPH1	18(36)	18(36)
TPH2	18(36)	18(36)
D5	18(36)	18(36)
D6	18(36)	18(36)
DCMR1	18(36)	18(36)
DCMR2	18(36)	18(36)
CBMA	15(16/17)	15(16/17)
ND (ND1)	8	-
BD (BD1)	8	-
BH	8	-
ZD	18	-

The default for TP1, TP2, TPH1, TPH2, D5, D6, DCMR1 and DCMR2 is 18 bits. However, it can be specified up to 36 bits by the setting of MODE TPM36.

CBMA can be specified to 16 bits for 2WAY and 17 bits for 4WAY when MODE ECBM is specified.

Tip

*SET register name m cannot be specified in 2WAY/4WAY interleaved mode.
Use DSET instruction (register name < m).*

Register name<m (DSET instruction, m: set value)

For the registers which can be specified with SET instruction, the values can be set without using SET instruction.

When this specification is used, different group registers can be set at the same time.

Group A	Group B	Group C	Group D
XH D1 D3B NH XOS XT ND ZD	YH D2 D4B ZH Z YOS YT BH BD	TP1 TPH1 D5 DCMR1 XMASK CBMA RCD DSD	TP2 TPH2 D6 DCMR2 YMASK DSC CCD

However, when MODE TPM36 (36-bit data mode) is specified, the registers of group C and group D cannot be specified at the same time.

(This instruction is called DSET instruction in order to distinguish it from the SET instruction.)

◆ **Example**

```

NOP    XH<#100 YH<#0 TPH1<#0 TPH2<#F
NOP    : D3B<#3
        : YOS<#200

```

 Tip

- Caution must be taken when DSET instruction is used in 2WAY/4WAY interleaved mode.
→ For more information, refer to [6. "Programming for Interleave Mode" on page 6-1.](#)
- CLEAR instruction ($F<0$) is not DSET instruction except $DSC<0$.
- D1A to D1H, D2A to D2D, ND1 to ND7, and BD1 to BD8 registers exist in T5593, but only D1A, D2A, ND1, and BD1 can be specified by the SET instruction or DSET instruction.

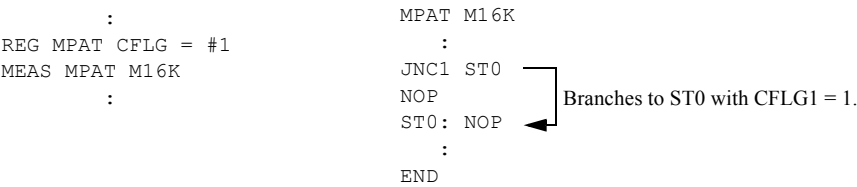
2. 3. 4 Condition Flag Reference Branch (JNCn)

Condition flag reference branch instructions are shown in [Table 2-11](#).

Table 2-11 Condition Flag Reference Branch Instructions

Mnemonic	Action
JNCn m T5593 (n = 1 to 16) T5592/T5591/T5586/T5585/T5581/T5581H/T5581P/T5571P (n = 1 to 8)	if CFLGn = 0 then (PC) + 1 → PC else m → PC

The JNCn instruction references CFLG (condition flag). Use it when the sequence of the pattern program is changed from the test plan program.



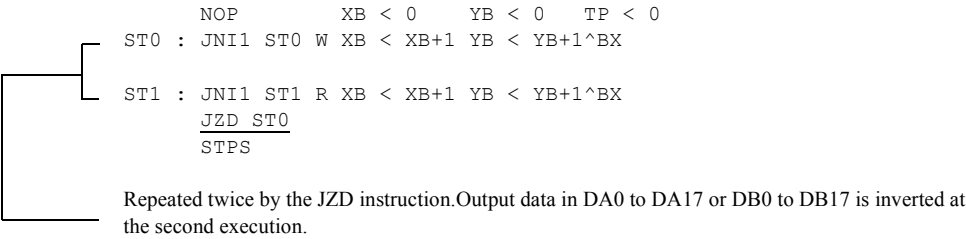
2. 3. 5 Data Flag Reference Branch (JZD)

Data flag reference branch instructions are shown in [Table 2-12](#).

Table 2-12 Data Flag Reference Branch Instructions

Mnemonic	Action
JZD m	if DFLG = 0 then m → PC 1 → DFLG else (PC) + 1 → PC 0 → DFLG

The JZD instruction references DFLG (data flag). Use it when a data pattern to be entered in the DUT is inverted for a test. DA0 to DA17 or DB0 to DB17 output from the pattern generator are inverted when DFLG is set to 1.



 **Tip**

When a TP (TP2) operation instruction is specified in the loop of the JZD instruction, the inverted patterns, which are opposite to the first patterns, may not be generated in the second loop, depending on the operation result.

The TP (TP2) register must be initialized at the beginning of the loop before the TP (TP2) operation instruction is specified.

- ◆ **This Example Shows the Program that cannot Generate the Inverted Patterns.**
Inverted data will be inverted again by the JZD instruction and TP</TP in the second loop. That is, patterns end up being equal to the original ones used in the first loop.

				First TP value	Second TP value
	NOP	XB<0	YB<0	TP<0	;
ST0:	JNI1	. W	XB<XB+1	YB<YB+1^BX	; 0
	JNI1	. R	XB<XB+1	YB<YB+1^BX	; 0
	NOP			TP</TP	; 0
	JNI1	. W	XB<XB+1	YB<YB+1^BX /X /Y	; 1
	JNI1	. R	XB<XB+1	YB<YB+1^BX /X /Y	; 1
	JZD	ST0			; 1
	STPS				

2. 3. 6 Timer Flag Reference Branch (JET)

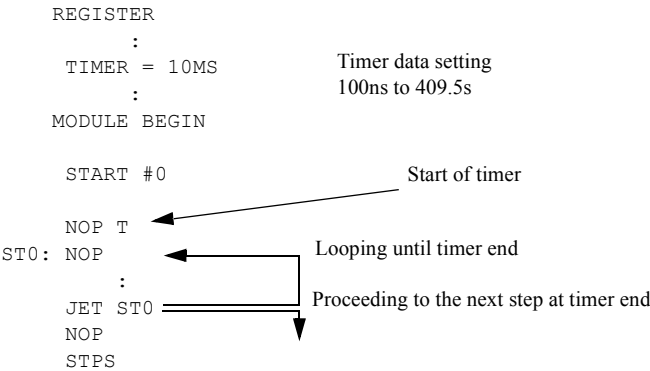
Timer flag reference branch instructions are shown in [Table 2-13](#).

Table 2-13 Timer Flag Reference Branch Instructions

Mnemonic	Action
JET m	if TFLG = 0 then m → PC else (PC) + 1 → PC 0 → TFLG

The JET instruction references TFLG (Timer flag). Use it when the time program set in the internal timer is executed repeatedly.

TFLG is set when the internal timer ends and is cleared when it is referenced by this instruction.



Tip

The *JET* loop execution time is as follows:
The set value of the timer + RATE in the *JET* loop *n
The value of n is as follows:

Test system	Interleave mode		
	1WAY	2WAY	4WAY
T5593	186	372	-
T5592/T5591/T5586/T5585/T5581/ T5581H/T5581P/T5571P	125	250	500

2. 3. 7 Match Flag Sense Instruction

The match flag sense instruction is a conditional branch instruction based on a flag (FLG1) that indicates the result of a logical comparison. [Table 2-14](#) shows the match flag sense instructions.

Table 2-14 Match Flag Sense Instruction

Mnemonic	Action
FLGLI1 m	1st (IDXR8)→ IDX (IDX) - 1→ IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC 2nd If (IDXW8) = 0 then (BAR) → PC else (IDXW8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC

FLG1 becomes "1" when it matches the expected value and becomes "0" when it does not.

In the cycles in which match flag sense instructions are described, set RATE so that it satisfies "RATE $\geq$ set value of STRB1 + 5.0 μ s."

➔ For more information, refer to [5. 4. 1 "Using the Match Flag Sense Instruction" on page 5-18.](#)

Tip

- The match flag sense instruction cannot be executed on T5593, T5592 and T5591.
- The match flag sense instruction cannot be executed if description of an IDX18 #0 instruction is omitted in the step that follows the match flag sense instruction.
No sequence control instructions except NOP can be described between a match flag sense instruction and the IDX18 #0 instruction.
- When you have described a match flag sense instruction, specify a value other than 0 for the IDX8 register in the REGISTER statement or the REG MPAT statement in the test plan program.
- In the loop specified by the match flag sense instruction, no instruction can be described as the index loop instruction (JN18 and IDX18) using the IDX8 register except the IDX18 #0 instruction described after the match flag sense instruction.

2. 3. 8 Burst Data Transfer Instruction (OUT)

```
OUT m (m: burst block name)
```

The burst data transfer instruction changes the unit setting condition by using the real time trigger at pattern declaration, for the burst block data entered in the pattern program.

It is used for a bump test for DRAM.

2. 3. 9 Pattern Sequence Control Register Set Instruction (STISP)

Mnemonic	Action
STISP m	(PC) + 1 $\rightarrow$ PC m $\rightarrow$ ISP

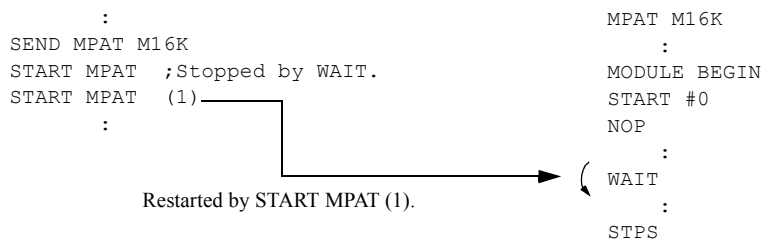
This statement sets the value m to the ISP (Interrupt Save Pointer) register. For the ISP register, specify the address which PC is branched at the interrupt generation from the inside timer.

This is used for the refresh test etc.

2. 3.10 WAIT

The WAIT instruction stops pattern generation; it can be restarted by an instruction from the test plan program.

Neither the control bits "I," "T," "TM1," "TM2," "DBMINC," "DBMSET," an address or data operation instruction nor an MUT signal can be described in a cycle in which a WAIT instruction is described.



2. 3.11 STPS, STFL

STPS (Set Pass) sets pass, or STFL (Set Fail) sets fail as the result when a test terminates. Then control of the program is returned to the test plan program. Be sure to enter STPS or STFL at the end of each module.

Use STPS for a normal functional test.

Neither the control bits "I," "T," "TM1," "TM2," "DBMINC," "DBMSET," an address or data operation instruction nor an MUT signal can be described in a cycle in which STPS or STFL is described.

Tip

The STPS or STFL instruction is executed for 2 cycles according to the hardware configuration.

In the T5592, T5591, T5591R, T5586, T5585, T5581, T5581H, T5581P, and T5571P, the XB register value is output as the X address and Y address in the second cycle of the STPS or STFL instruction.

2. 3.12 Control Bit

The control bits in the main sequence controller are "I," "T," "TM1," "TM2," "DBMINC," and "DBMSET."

I

In a cycle in which control bit "I" is described, an interruption into the interruption by the internal timer is inhibited.

T, TM1, TM2

These control bits start the internal timer at a described cycle.

The timers to be started by T, TM1, and TM2 are as follows:

T

Simultaneously starts timers (Timer 1^{*1}, Timer 2^{*2}) specified by the MODE statement.

TM1

Starts Timer 1 only.

TM2

Starts Timer 2 only.

**1 A timer used in MODE REFRESHA, B.*

**2 A timer used in MODE PAUSE.*

DBMINC

With the cycle of DBMINC specified, DBM address pointer (DBMA) is incremented by 1.

The incremented value becomes effective from the next step. (Event Time 2)

DBMSET(m)

The value m is set to the DBM address pointer (DBMA) in the cycle in which DBMSET(m) is described. The set value becomes effective from the next step. (Event Time 2)

The specification range for m is as follows:

DBM board	Interleave mode	The setting range for m
2 M board	1WAY	#0 ≤ m ≤ FFFFF
	2WAY	#0 ≤ m ≤ 1FFFFE and is a multiple of 2

◆ **An example of DBMSET(m)**

```

:
REGISTER
DBMA=#0

START #0                                ; Value of DBMA
NOP DBMSET (#10) : XB<XB+1             ; #0
                  : XB<XB+1             ; #0
NOP DBMINC       : XB<XB+1             ; #10
                  : XB<XB+1             ; #10
NOP DBMSET (#20) : XB<XB+1             ; #11
                  : XB<XB+1             ; #11
NOP              :                     ; #20
                  :                     ; #20
:

```

 Tip

- When describing DBMSET(m), specify the RDBMA mode (MODE RDBMA).
- DBMSET(m) can be used in T5593.

- *DBMINC and DBMSET(m) cannot be described in the same step.*
 - *If the DBMLP mode (MODE DBMLP) is specified and the DBMSET(m) is described for the step of DBM loop end (DBMLPE), DBMSET(m) in the step is invalid.*
-

2. 4 Address Pattern Generation Instructions

2. 4. 1 Hardware Configuration of Address Generator Section

Figure 2-7 and Figure 2-8 show the address generator section function block diagram.

Figure 2-7 Address Generator Section Function Block Diagram (T5593)

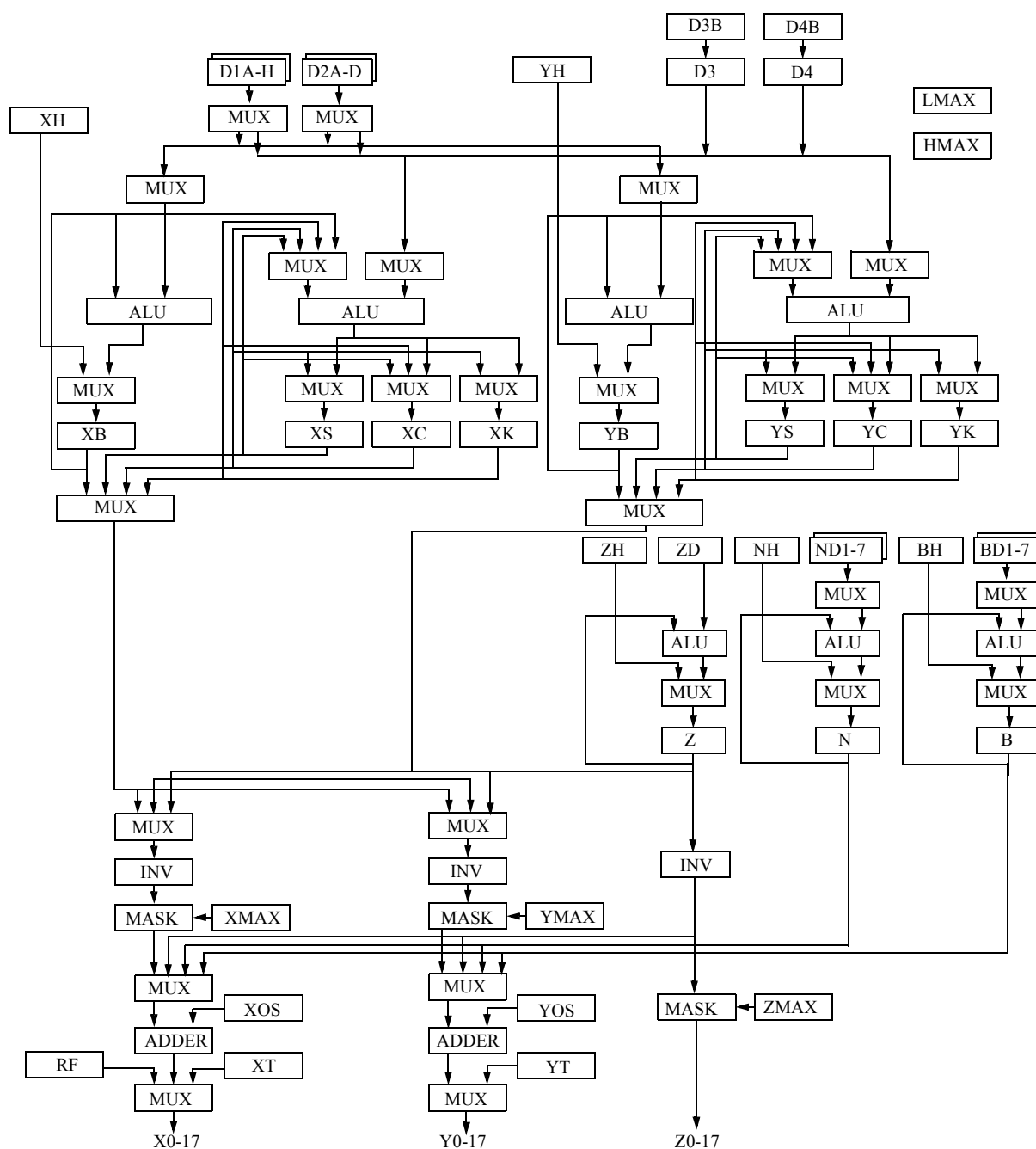
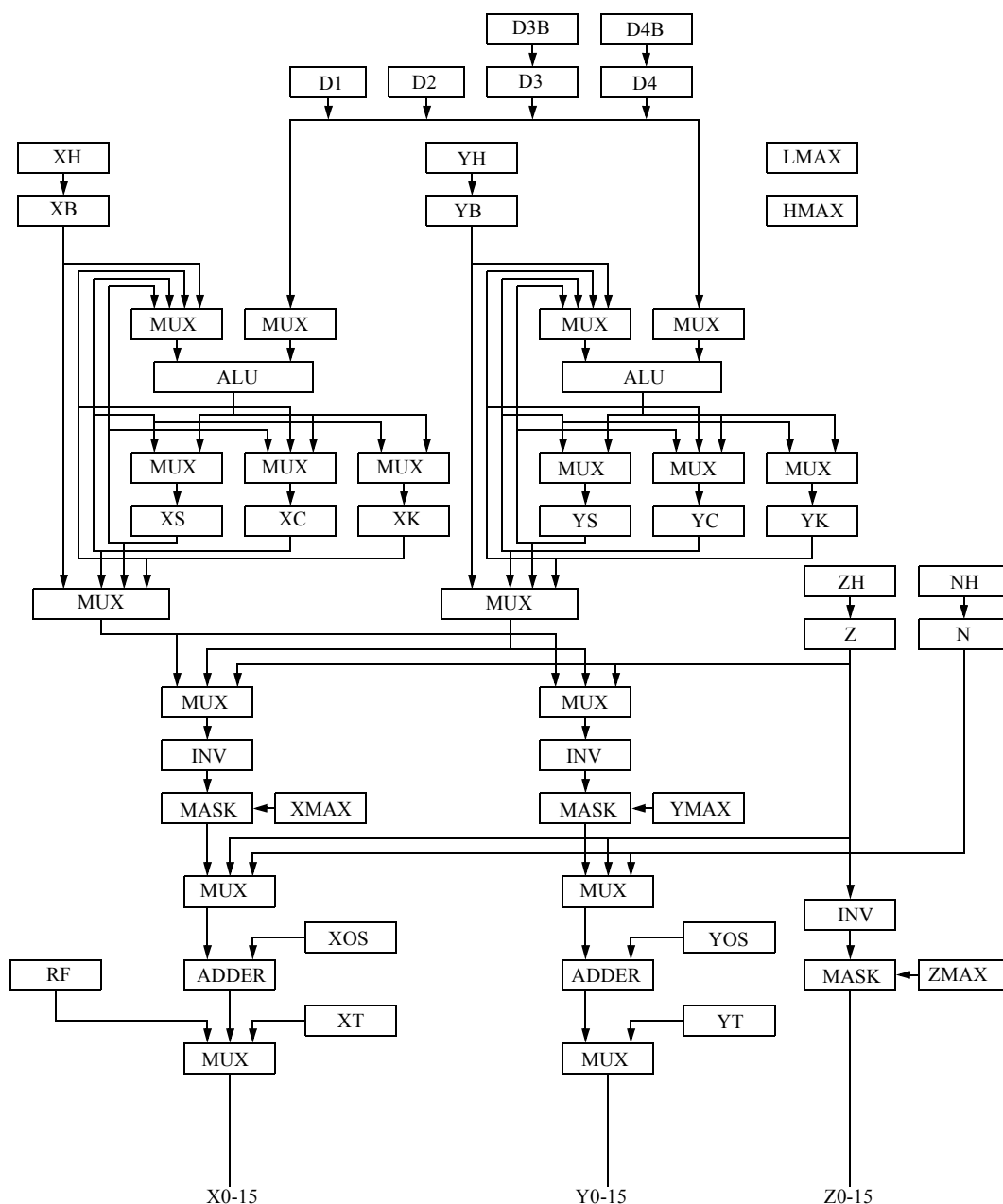


Figure 2-8 Address Generator Section Function Block Diagram (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)



XB, YB, Z

Base registers for addresses. It is used mainly for generating addresses of cells to be tested.

XH, YH, ZH

Registers to store the initial values to be loaded into the base register of each address. These can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program and by the REG statement in the test plan program.

XC, YC

These are used to generate addresses (disturb addresses) for testing the relation to the cells to be tested, and to generate test pattern addresses that cannot be generated by base registers.

XS, XK, YS, YK

These are used mainly as save registers to save each address generated by XC and YC. Since XS, XK, YS and YK registers have functions of the same level as XC and YC registers, more complicated addresses can be generated by combining the XS, XK, YS and YK with XC and YC registers.

D1A(D1) to D1H, D2A(D2) to D2D

These are used as the offset data when performing address operation of XB, YB, XC, YC, XS, YS, XK, and YK.

D1A(D1) and D2A(D2) can be set by using the SET instruction, or the DSET instruction in the pattern program. D1A to D1H and D2A to D2D can be set by using the REGISTER statement in the pattern program and the REG statement in the test plan program.

D3, D4

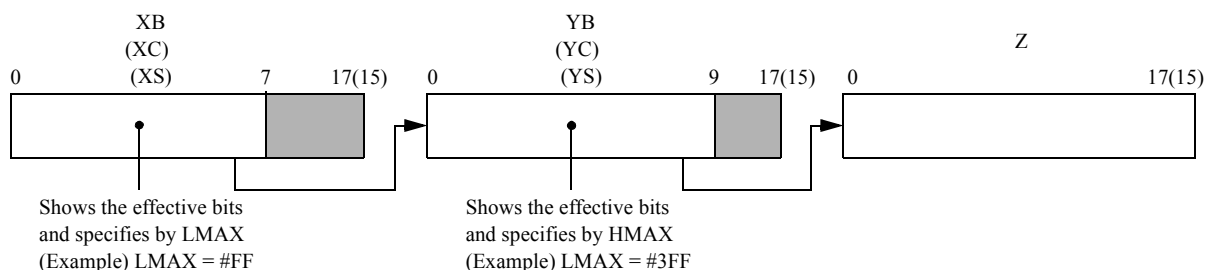
These are used as the offset data when performing address operation of XC, YC, XS, YS, XK, and YK.

D3B, D4B

These are registers to store the initial values to be loaded into D3, and D4 registers. These can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program and by the REG statement in the test plan program.

LMAX, HMAX

LMAX and HMAX are registers to specify effective bits of X and Y registers for using a conditional operation instruction. They can set 2n-1 data (n: the number of effective bits) by the REGISTER statement in the pattern program and the REG statement in the test plan program.

**N, NH, ND1 to ND7**

These are used mainly for bank address generation.

NH is the register which stores the initial values to be loaded into the N register.

ND1 to ND7 are registers used to operate the N register.

NH and ND1 can be set by using the SET instruction or the DSET instruction in the pattern program. NH and ND1 to ND7 can be set by using the REGISTER statement in the pattern program or the REG statement in the test plan program.

B, BH, BD1 to BD7

These are used mainly for bank address generation.

BH is the register which stores the initial values to be loaded into the B register.

BD1 to BD7 are registers used to operate the B register.

BH and BD1 can be set by using the SET instruction or the DSET instruction in the pattern program. BH and BD1 to BD7 can be set by using the REGISTER statement in the pattern program or the REG statement in the test plan program.

ZD

ZD is the register used to operate the Z address.

The Z address cannot be impressed to the DUT. However, the Z address can be generated inside the ALPG and it can be used to interrupt the X/Y address and to generate the burst address.

ZD can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program, and by the REG statement in the test plan program.

RF

RF is a register to generate refresh addresses. Refresh addresses are output to refresh cycle as X addresses.

XMAX, YMAX, ZMAX

These are the registers that show the effective bits of each address to be output to X0-17, Y0-17, and Z0-17 (X0-15, Y0-15, and Z0-15 in T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P).

They can set 2n-1 data (n: the number of effective bits) by the REGISTER statement in the pattern program and the REG statement in the test plan program.

XOS, YOS

Register to store the offset data to be added to the addresses. These are used to move the programmed pattern whose origin are addresses (0, 0) in the small area to the other area.

XOS and YOS can be set by the SET instruction, the DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test plan program.

XT, YT

Registers to store the logical data when temporally outputting the logical data to the address line. These are used to measure the device to be executed the mode setting from address line.

XT and YT can be set by the SET instruction, the DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test plan program.

2. 4. 2 Configuration of Address Generation Instruction

Address generation part comprises the following instructions.

- Base address field
- Current address field
- Source address field

- N register field
- B register field
- Address inversion field
- Refresh counter field
- D3, D4 register field
- Address swap field

2. 4. 3 Outline of Address Generation Instruction

Shown below are the mnemonics of instructions in the address generation instruction part.

One instruction can be selected in the same cycle from among those in a group delimited by parentheses. The square in parentheses indicates that one instruction can be selected from among those in the group.

Base address field

- X base address field

XB<XB	A<D1A *1
XB<XB+1	A<D1B *1
XB<XB-1	A<D1C *1
XB<XH	A<D1D *1
XB<0	A<D1E *1
XB<A *1	A<D1F *1
XB<XB+A *1	A<D1G *1
XB<XB-A *1	A<D1H *1
	A<D2A *1
	A<D2B *1
	A<D2C *1
	A<D2D *1

- Y base address field

YB<YB	A<D1A *1
YB<YB+1	A<D1B *1
YB<YB-1	A<D1C *1
YB<YH	A<D1D *1
YB<YB+1^BX	A<D1E *1
YB<YB-1^BX	A<D1F *1
YB<YB+1+BX	A<D1G *1
YB<YB-1-BX	A<D1H *1
YB<0	A<D2A *1
YB<A *1	A<D2B *1
YB<YB+A *1	A<D2C *1
YB<YB-A *1	A<D2D *1
YB<YB+A^BX *1	
YB<YB-A^BX *1	
YB<YB+A+BX *1	
YB<YB-A-BX *1	

- Z base address field

Z<Z
Z<Z+1
Z<Z-1
Z<ZH
Z<Z*2
Z</Z
Z<0
Z<Z+1^BX
Z<Z-1^BX
Z<Z+1^CY
Z<Z-1^CY
Z<Z+ZD *1
Z<Z-ZD *1
Z<Z+ZD^BX *1
Z<Z-ZD^BX *1
Z<Z+ZD^CY *1
Z<Z-ZD^CY *1

*1 This instruction can be executed in T5593.

Tip

- D1A can be described as D1.
- D2A can be described as D2.
- Two or more kinds of D1A to D1H registers cannot be described in the same step.
- Two or more kinds of D2A to D2D registers cannot be described in the same step.

Current address field

- X current address field

X<XC	XS<XS	XK<XK	F<A	A<XB	B<D1A
X<F	XS<XS	XK<XK	F<A+1	A<XC	B<D1B *2
X<XS	XS<XS	XK<XK	F<A-1	A<XS	B<D1C *2
X<XK	XS<XS	XK<XK	F<A*2	A<XK	B<D1D *2
X<XC	XS<F	XK<XK	F</A		B<D1E *2
X<XC	XS<XC	XK<XK	F<B		B<D1F *2
X<XC	XS<XS	XK<F	F<A+B		B<D1G *2
X<XC	XS<XS	XK<XC	F<A-B		B<D1H *2
X<S<F *1		XK<XK	F<A&B		B<D2A
X<K<F *1	XS<XS		F<A"B		B<D2B *2
X<SK<F *1			F<A'B		B<D2C *2
X<XS<F		XK<XK	F<0		B<D2D *2
XS<XC<F		XK<XK			B<D3
X<XK<F	XS<XS				B<D4
XK<XC<F	XS<XS				
X<>XS		XK<XK			
X<>XK	XS<XS				

2. 4 Address Pattern Generation Instructions

- Y current address field

YC<YC	YS<YS	YK<YK	F<A	A<YB	B<D1A
YC<F	YS<YS	YK<YK	F<A+1	A<YC	B<D1B *2
YC<YS	YS<YS	YK<YK	F<A-1	A<YS	B<D1C *2
YC<YK	YS<YS	YK<YK	F<A*2	A<YK	B<D1D *2
YC<YC	YS<F	YK<YK	F</A		B<D1E *2
YC<YC	YS<YC	YK<YK	F<B		B<D1F *2
YC<YC	YS<YS	YK<F	F<A+B		B<D1G *2
YC<YC	YS<YS	YK<YC	F<A-B		B<D1H *2
YCS<F *1		YK<YK	F<A&B		B<D2A
YCK<F *1	YS<YS		F<A"B		B<D2B *2
YCSK<F *1			F<A'B		B<D2C *2
YC<YS<F		YK<YK	F<0		B<D2D *2
YS<YC<F		YK<YK	F<A+1+CY		B<D3
YC<YK<F	YS<YS		F<A-1-CY		B<D4
YK<YC<F	YS<YS		F<A+B+CY		
YC<>YS		YK<YK	F<A-B-CY		
YC<>YK	YS<YS		F<A+1+BX		
			F<A-1-BX		
			F<A+B+BX		
			F<A-B-BX		
			F<A+1^CY		
			F<A-1^CY		
			F<A+B^CY		
			F<A-B^CY		
			F<A+1^BX		
			F<A-1^BX		
			F<A+B^BX		
			F<A-B^BX		

*1 Load statements which load operation results of F to XC, XS and XK registers in the same operation cycle.

$XCS<F$; Loads to XC and XS registers

$XCK<F$; Loads to XC and XK registers

$XCSK<F$; Loads to XC, XS, and XK registers

The same applies to YC, YS and YK.

A, B, and F in the mnemonics are formal registers for explanation. In programming, describe mnemonics by replacing the selected A, B, and F with the mnemonics on their right side.

Example: When $XCS<F$ $F<A+B$ $A<XB$ $B<D1$ are selected

↓
 $XCS<XB+D1$; Mnemonics in programming

*2 This instruction can be executed in the T5593.

Tip

- D1A can be described as D1.
- D2A can be described as D2.
- Two or more types of D1A to D1H registers cannot be described in the same step.
- Two or more types of D2A to D2D registers cannot be described in the same step.

Source address field

X<XB	Y<YB
X<XC	Y<YC
X<XS	Y<YS
X<XK	Y<YK
X<YB	Y<XB
X<YC	Y<XC
X<YS	Y<XS
X<YK	Y<XK
X<XB' Z	Y<XB' Z
X<XC' Z	Y<XC' Z
X<XS' Z	Y<XS' Z
X<XK' Z	Y<XK' Z
X<Z	Y<Z
X<RF	



Tip

In a group of X address generation registers (XB, XC, XS, XK) and a group of Y address generation registers (YB, YC, YS, YK), different registers in the same group cannot be selected as a combination of the X and Y output selections.

X<XB Y<XC (Not allowed)

X<YB Y<YC (Not allowed)

X<XB Y<XB (Allowed)

X<XB Y<YB (Allowed)

N register field

N<N	A<ND1 *1
N<N+1	A<ND2 *1
N<N-1	A<ND3 *1
N<NH	A<ND4 *1
N<0	A<ND5 *1
N</N	A<ND6 *1
N<A	A<ND7 *1
N<N+A *1	
N<N-A *1	

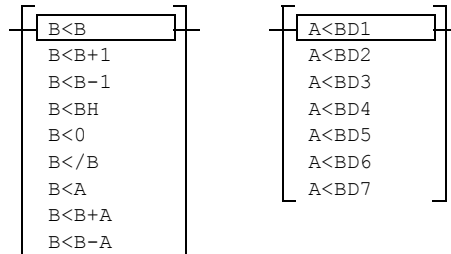
*1 This instruction can be executed in T5593.



Tip

ND1 can be described as ND.

B register field



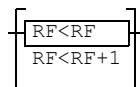
Tip

- *BD1 can be described as BD.*
- *B register field can be executed in T5593.*

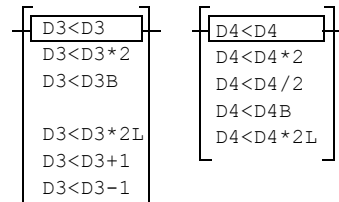
Address inversion field

[/X] [/Y] [/Z]

Refresh counter field



D3, D4 register field



Address swap field

[LMAX<>HMAX]

2. 4. 4 Base Address Field

The base address field comprises the operation instructions, as described in the following, used to generate test target cell addresses for the generation of N^2 or $N^{3/2}$ type test patterns.

Hold (XB<XB YB<YB Z<Z)

Used to execute nothing.

This instruction can be omitted.

```

NOP XB<0 YB<0
ST0:JNi1 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
  NOP XC<XB+1 YC<YB+1^BX X<XB Y<YB
  || Equivalent. (Specifications can be omitted.)
  NOP XB<XB YB<YB XC<XB+1 YC<YB+1^BX X<XB Y<YB

```

Clear (XB<0 YB<0 Z<0)

Used to initialize XB, YB and Z.

MPAT CHECK

```

REGISTER
XMAX=#7F
YMAX=#7F
IDX1=#3FFF
PC=#0
START #00

```

When using XB, YB, and Z, always initialize them at the first step.

```

NOP XB<0 YB<0

```

```

ST0:JNi1 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB

```

Load (XB<XH YB<YH Z<ZH XB<Dn YB<Dn)

Used to initialize XB, YB and Z. This instruction is used to load the contents of registers XH, YH, ZH and Dn to XB, YB, and Z.

```

XB<XH    XH → XB
YB<YH    YH → YB
Z<ZH     ZH → Z
XB<Dn    Dn → XB
YB<Dn    Dn → YB

```

Dn shows D1A to D1H, and D2A to D2D.

INC (XB<XB+1 YB<YB+1 Z<Z+1)

Used to increment each address of XB, YB and Z. It executes the following operations:

```

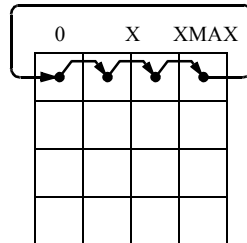
XB<XB+1    XB+1 → XB
YB<YB+1    YB+1 → YB
Z<Z+1      Z+1 → Z

```

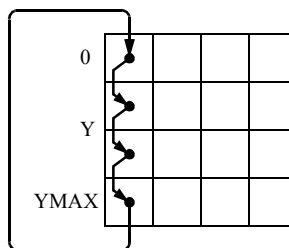
When the values of XB, YB and Z registers are output to X, Y and Z addresses, the effective bit length is the same as XMAX, YMAX and ZMAX registers.

The following address sequences can be generated by using this instruction.

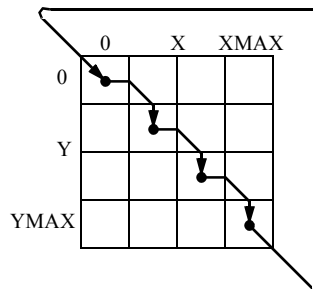
- ST0: JINn ST0 XB<XB+1 X<XB



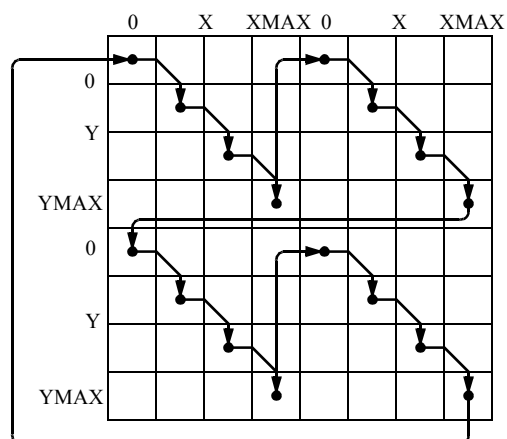
- ST0: JINn ST0 YB<YB+1 Y<YB



- ST0: JINn ST0 XB<XB+1 YB<YB+1 X<XB Y<YB



- ST0: JINn ST0 XB<XB+1 YB<YB+1 X<XB Y<YB
JINm ST0 XB<XB+1 YB<YB+1 Z<Z+1 X<XB Y<YB



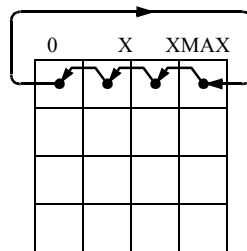
Z=0	Z=1
Z=2	Z=3

DEC (XB<XB-1 YB<YB-1 Z<Z-1)

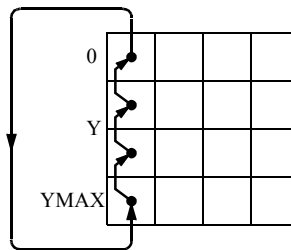
Used to decrement each address of XB, YB and Z. It executes the following operations:

$XB < XB-1 \quad (XB)-1 \rightarrow XB$
 $YB < YB-1 \quad (YB)-1 \rightarrow YB$
 $Z < Z-1 \quad (Z)-1 \rightarrow Z$

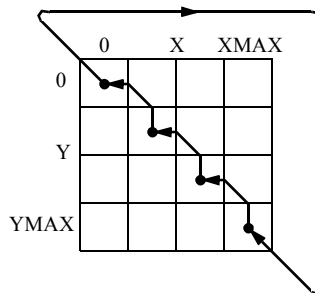
- ST0: JNIn ST0 XB<XB-1 X<XB



- ST0: JNIn ST0 YB<YB-1 Y<YB

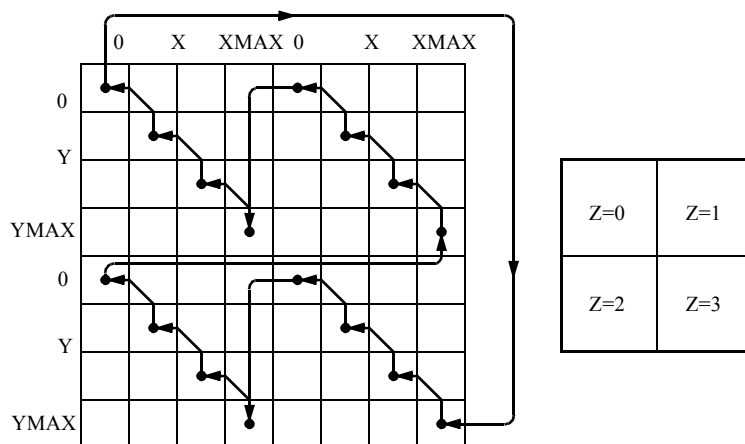


- ST0: JNIn ST0 XB<XB-1 YB<YB-1 X<XB Y<YB



- ST0: JNIn ST0 XB<XB-1 YB<YB-1 X<XB Y<YB

JNIm ST0 XB<XB-1 YB<YB-1 Z<Z-1 X<XB Y<YB



Add/subtract instructions (XB<XB±Dn YB<YB±Dn Z<Z±ZD)

Used for the addition or subtraction of each XB, YB and Z address. The command executes the following operations:

XB<XB±Dn	XB±Dn → XB
YB<YB±Dn	YB±Dn → YB
Z<Z±ZD	Z±ZD → Z

Dn shows D1A to D1H, and D2A to D2D.

 Tip

When the values of XB, YB and Z registers are output to X, Y and Z addresses, the effective bit length is the same as XMAX, YMAX and ZMAX registers.

Conditional Operation Instruction

This instruction is used to generate addresses by combining each of the X, Y and Z addresses. It executes the following operations:

Conditional Operation Instruction	Condition	Judgment	Operation
YB<YB+1^BX	Generation of carry/ borrow by XB operation	then	YB+1 → YB
		else	YB → YB
YB<YB-1^BX	Generation of carry/ borrow by XB operation	then	YB-1 → YB
		else	YB → YB
YB<YB+1+BX	Generation of carry/ borrow by XB operation	then	YB+2 → YB
		else	YB+1 → YB
YB<YB-1-BX	Generation of carry/ borrow by XB operation	then	YB-2 → YB
		else	YB-1 → YB
Z<Z+1^BX	Generation of carry/ borrow by XB or YB operation	then	Z+1 → Z
		else	Z → Z
Z<Z-1^BX	Generation of carry/ borrow by XB or YB operation	then	Z-1 → Z
		else	Z → Z
Z<Z+1^CY	Generation of carry/borrow by XC(XS)(XK), or YC(YS)(YK) operation	then	Z+1 → Z
		else	Z → Z
Z<Z-1^CY	Generation of carry/borrow by XC(XS)(XK), or YC(YS)(YK) operation	then	Z-1 → Z
		else	Z → Z
YB<YB+Dn^BX	Generation of carry/ borrow by XB operation	then	YB+Dn → YB
		else	YB → YB
YB<YB-Dn^BX	Generation of carry/ borrow by XB operation	then	YB-Dn → YB
		else	YB → YB
YB<YB+Dn+BX	Generation of carry/ borrow by XB operation	then	YB+Dn+1 → YB
		else	YB+Dn → YB
YB<YB-Dn-BX	Generation of carry/ borrow by XB operation	then	YB-Dn-1 → YB
		else	YB-Dn → YB
Z<Z+ZD^BX	Generation of carry/ borrow by XB or YB operation	then	Z+ZD → Z
		else	Z → Z
Z<Z-ZD^BX	Generation of carry/ borrow by XB or YB operation	then	Z-ZD → Z
		else	Z → Z

Conditional Operation Instruction	Condition	Judgment	Operation
$Z < Z + ZD \wedge CY$	Generation of carry/borrow by XC(XS)(XK), or YC(YS)(YK) operation	then	$Z + ZD + 1 \rightarrow Z$
		else	$Z + ZD \rightarrow Z$
$Z < Z - ZD \wedge CY$	Generation of carry/borrow by XC(XS)(XK), or YC(YS)(YK) operation	then	$Z - ZD - 1 \rightarrow Z$
		else	$Z - ZD \rightarrow Z$

Dn shows D1A to D1H, and D2A to D2D.

Carry/borrow means carry/borrow by the following register operation.

- Carry/borrow by XB or YB operation

T5593

Carry/borrow generated by the highest level of effective bits stored in LMAX and HMAX in operation of XB and YB registers described in the same cycle as the conditional operation instruction

T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

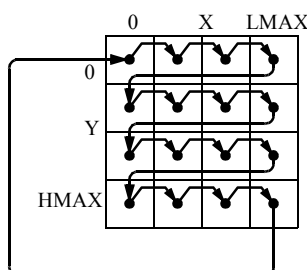
Carry/borrow generated under the condition $XB = LMAX$ and $YB = HMAX$ in operation of XB and YB registers described in the same cycle as the conditional operation instruction.

- Carry/borrow by XC(XS)(XK), or YC(YS)(YK) operation

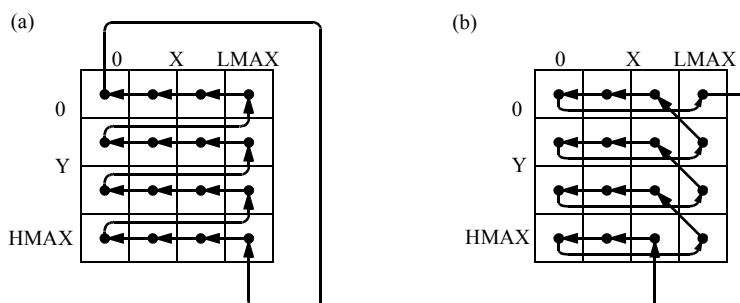
Carry/borrow generated by the highest level of effective bits stored in LMAX and HMAX in operation of XC(XS)(XK) and YC(YS)(YK) registers described in the same cycle as the conditional operation instruction

This instruction can generate addresses of concatenated X, Y, and Z. As an example, the generation of the address sequence using this instruction is shown below.

- ST0: JNIn ST0 $XB < XB + 1$ $YB < YB + 1 \wedge BX$ $X < XB$ $Y < YB$



- ST0: JNIn ST0 XB<XB-1 YB<YB-1^BX X<XB Y<YB

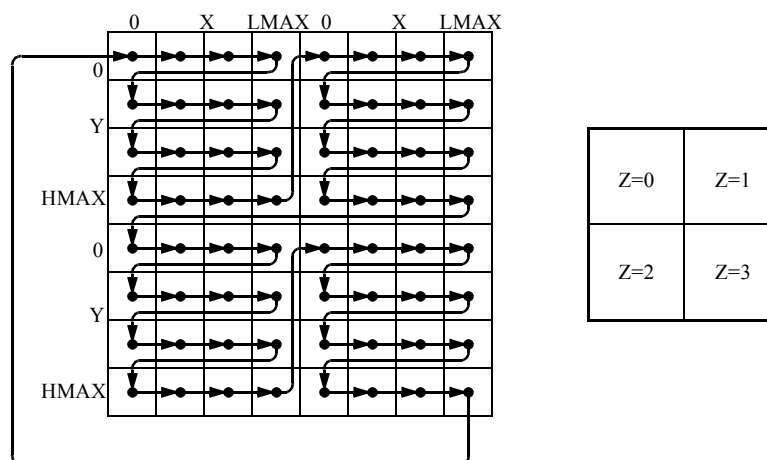


In 1WAY normal mode in T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, the address sequence is as shown in (b).

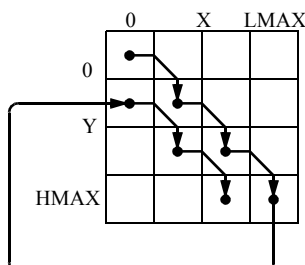
To apply the address sequence shown in (a), use the following instruction:

ST0: JNIn ST0 XB<XB+1 YB<YB+1^BX X<XB Y<YB /X /Y

- ST0: JNIn ST0 XB<XB+1 YB<YB+1^BX Z<Z+1^BX X<XB Y<YB



- ST0: JNIn ST0 XB<XB+1 YB<YB+1+BX X<XB Y<YB



Shift ($Z < Z * 2$)

This instruction doubles (shifts left by 1 bit) the data of the Z register.

$Z < Z * 2$ $Z * 2 \rightarrow Z$ Shift left (LSB=0)

Complement (Z</Z)

This instruction reverses the internal window of Z register.

Z</Z NOT. (Z)→Z

2. 4. 5 Current Address Field

The current address field is used to generate addresses (disturb addresses) to test interrelation with test target cells in N^2 and $N^{3/2}$ type test pattern, and to generate the addresses of test target cells in N type test patterns that cannot be generated by a base field operation.

The operation instruction of the current address field comprises the following three types of instruction groups.

- C[S][K]<f (B)
- C[S][K]<f (C[S][K])
- C[S][K]<f (B, C[S][K], t)

B represents XB or YB.

C[S][K] represents XC, YC, XS, YS or XK, YK.

Carry/borrow used in descriptions in the Current Address Field is subject to the following register operation.

- Carry/borrow by XB or YB operation

T5593

Carry/borrow generated by the highest level of effective bits stored in LMAX and HMAX in operation of XB and YB registers described in the same cycle as the conditional operation instruction

T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

Carry/borrow generated under the condition XB=LMAX and YB=HMAX in operation of XB and YB registers described in the same cycle as the conditional operation instruction.

- Carry/borrow by XC(XS)(XK), or YC(YS)(YK) operation

Carry/borrow generated by the highest level of effective bits stored in LMAX and HMAX in the operation of XC(XS)(XK) and YC(YS)(YK) registers described in the same cycle as the conditional operation instruction

◆ $C[S][K] < f(B)$

$C[S][K] < f(B)$ is a group of instructions used to obtain the addresses of disturb cells to be stored in XC, YC or in XS, YS, XK, YK through the operation from the address of the test target cells to be stored in XB, YB. These include the following instructions:

$C[S][K] < B$	
$C[S][K] < B \pm 1$	
$C[S][K] < B \pm Dn$	
$C[S][K] < B * 2$	Shift left
$C[S][K] < / B$	Complement
$C[S][K] < B \& Dn$	AND
$C[S][K] < B \text{ " } Dn$	Exclusive OR
$C[S][K] < B \text{ ' } Dn$	OR
$C[S][K] < B \pm 1 \wedge BX$	} Only YC or YS, YK is valid
$C[S][K] < B \pm Dn \wedge BX$	
$C[S][K] < B \pm 1 \pm BX$	
$C[S][K] < B \pm Dn \pm BX$	
$C[S][K] < B \pm 1 \wedge CY$	
$C[S][K] < B \pm Dn \wedge CY$	
$C[S][K] < B \pm 1 \pm CY$	
$C[S][K] < B \pm Dn \pm CY$	

- $C[S][K] < B$ (XC(XS)(XK) < XB, YC(YS)(YK) < XB)

These are used to initialize the registers of XC(XS)(XK) and YC(YS)(YK). The contents of the registers XB and YB are loaded into the registers of XC(XS)(XK) and YC(YS)(YK).

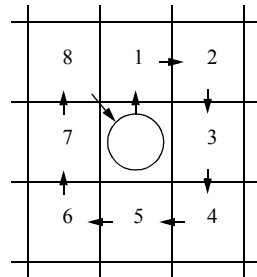
- $C[S][K] < B \pm 1$

Use this instruction to generate the addresses of before and after (± 1) the address of the test target cells stored in XB and YB.

The values of $XB \pm 1$ and $YB \pm 1$ are loaded into XC(XS)(XK) and YC(YS)(YK). Use this instruction to initialize the generation of the surround-disturb pattern used for accessing only the periphery of test target cells, and the generation of the address of the disturb cell in N2 type test patterns such as Galloping and Walking.

Examples are shown below.

◆ Surround Disturb

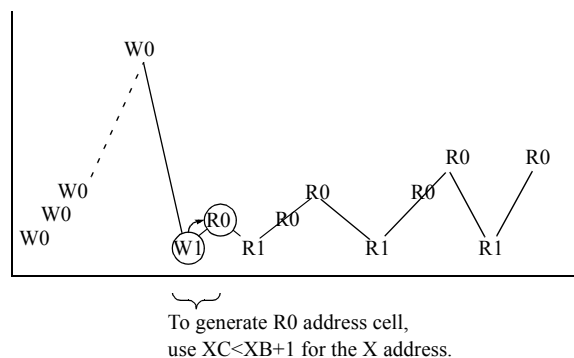


```

      :
NOP  XC<XB      YC<YB-1
NOP  XC<XB+1    YC<YB-1
NOP  XC<XB+1    YC<YB
NOP  XC<XB+1    YC<YB+1
NOP  XC<XB      YC<YB+1
NOP  XC<XB-1    YC<YB+1
NOP  XC<XB-1    YC<YB
NOP  XC<XB-1    YC<YB-1
      :

```

◆ Galloping

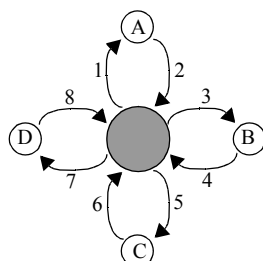


```
NOP          XB<0 YB<0 TP<TPH
ST0: JNl1 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP     W XC<XB+1 YC<YB+1^BX X<XB Y<YB /D
ST2: NOP     R X<XC Y<YC
           :
```

- $C[S][K] \leq B \pm D_n \quad (XC(XS)(XK) \leq XB \pm D_n \quad YC(YS)(YK) \leq YB \pm D_n)$

Used to generate different cell addresses based on the test target cell addresses stored in XB and YB. When this instruction is executed, the values of XB±Dn, YB±Dn(Dn=D1A to D1H, D2A to D2D, D3, D4) are loaded into XC(XS)(XK) and YC(YS)(YK).

This instruction is used in combination with the operation function of the D3 register for butterfly pattern generation. A butterfly pattern generation example is shown in the following.



After making access in the order of 1 to 8, increment the distance from the test target cell and repeat the above actions.

● : test target cell XB, YB
○ : disturbing cells XC, YC

```
REGISTER
D3B=#1
:
START #00

NOP XB<0 YB<0 TP<TPH D3<D3B
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP W                   YC<YB-D3 X<XB Y<YB /D
ST2: NOP R                   X<XB Y<YC
NOP R XC<XB+D3               X<XB Y<YB /D
NOP R                        X<XC Y<YB
NOP R                   YC<YB+D3 X<XB Y<YB /D
NOP R                        X<XB Y<YC
NOP R XC<XB-D3               X<XB Y<YB /D
NOP R                        X<XC Y<YB D3<D3+1
JN12 ST2 R                   YC<YB-D3 X<XB Y<YB /D
JN13 ST1 W XB<XB+1 YB<YB+1 X<XB Y<YB D3<D3B
```

- $C[S][K] < B \pm 1, Dn^{\wedge}BX \ (YC(YS)(YK) < YB \pm 1^{\wedge}BX, YC(YS)(YK) < YB \pm Dn^{\wedge}BX)$

This instruction is used to link XB to YC or YS or YK. It operates as follows.

$YC(YS)(YK) < YB \pm 1, Dn^{\wedge}BX$

When no XB carry/borrow is generated $YB \rightarrow YC, YS \text{ or } YK$

When XB carry/borrow is generated $YB \pm 1, Dn \rightarrow YC, YS \text{ or } YK$

2. 4 Address Pattern Generation Instructions

The use of the instruction becomes necessary because, whereas the disturb cell starting Y address is the same as that of the target cell as long as the target cell X address has not reached the LMAX, the Y address is incremented after the target cell X address reaches the LMAX.

MPAT PING

```
REGISTER
XMAX=#7F
YMAX=#7F
TPH=#0
DCMR=#0
IDX1=#3FFF
IDX2=#3FFD
```

PC=#00

START #00

NOP XB<0 YB<0 TP<TPH

ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB

ST1: NOP W XC<XB+1 YC<YB+1^BX X<XB Y<YB /D

ST2: NOP R X<XC Y<YC

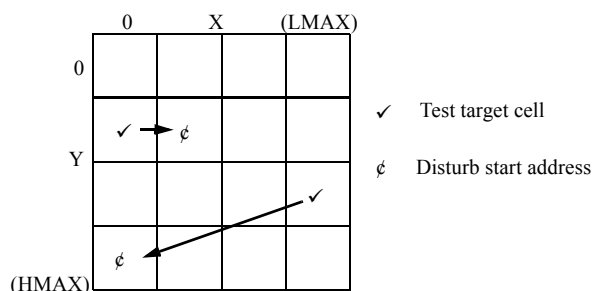
JN12 ST2 R XC<XC+1 YC<YC+1^CY X<XB Y<YB /D

JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB

NOP

STPS

END



The Y address of the disturb start address is incremented only when the X address of the test target cell reaches the LMAX.

- $C[S][K] < B \pm 1, Dn \pm BX$ ($YC(YS)(YK) < YB \pm 1 \pm BX, YC(YS)(YK) < YB \pm Dn \pm BX$)

Use this instruction to add or subtract carry and borrow to be generated by XB operation in the operation instructions of YC, YS, and YK.

$YC(YS)(YK) < YB \pm 1, Dn \pm BX$ ($Dn = D1A$ to $D1H, D2A$ to $D2D, D3, D4$)

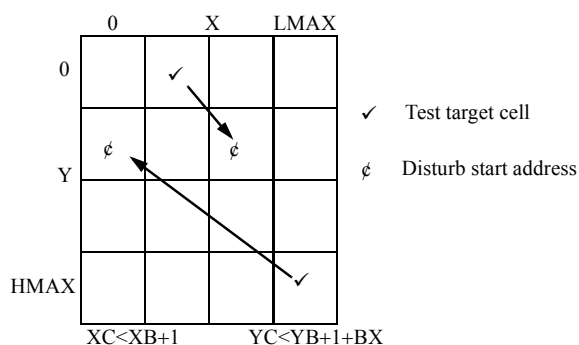
When no XB carry/borrow is generated

$YB \pm 1, Dn \rightarrow YC, YS$ or YK

When XB carry/borrow is generated

$YB \pm 1, Dn \pm 1 \rightarrow YC, YS$ or YK

This instruction is required mainly for setting the following disturb-start address from target addresses.



- $C[S][K] < B \pm 1, Dn \wedge CY \quad (YC(YS)(YK) < YB \pm 1 \wedge CY, YC(YS)(YK) < YB \pm Dn \wedge CY)$

The conditional operation instructions of YC, YS, and YK control the operation of YC, YS, and YK by a carry and borrow generated in the operation of XC or XS or XK. Such carry and borrow are generated at the most significant effective bit stored in LMAX in the operation of XC, XS, and XK.

$YC(YS)(YK) < YB \pm 1, Dn \wedge CY \quad (Dn = D1A \text{ to } D1H, D2A \text{ to } D2D, D3, D4)$

When no XC(XS)(XK) carry/borrow is generated

$YB \rightarrow YC, YS \text{ or } YK$

When XC(XS)(XK) carry/borrow is generated

$YB \pm 1, Dn \rightarrow YC, YS \text{ or } YK$

Use this instruction to specify the disturb start address from test target addresses just like $C[S][K] < B \pm 1 \wedge BX$.

$C[S][K] < B \pm 1, Dn \wedge BX$ executes $YB \pm 1, Dn$ only when carry or borrow is generated by XB operation in the target address. Whereas this instruction executes $YB \pm 1, Dn$ when the carry of XC or XS or XK is generated, even if no carry or borrow is generated by XB operation in the target address.

- $C[S][K] < B \pm 1, Dn \pm CY \quad (YC(YS)(YK) < YB \pm 1 \pm CY, YC(YS)(YK) < YB \pm Dn \pm CY)$

This instruction adds and subtracts the carry and borrow generated by the operation of XC or XS or XK in the operation instruction of YC, YS, and YK.

Use this instruction to specify the disturb start address from test target addresses just like $C[S][K] < B \pm 1 \pm BX$.

$YC(YS)(YK) < YB \pm 1, Dn \pm CY \quad (Dn = D1A \text{ to } D1H, D2A \text{ to } D2D, D3, D4)$

When no XC(XS)(XK) carry/borrow is generated

$YB \pm 1, Dn \rightarrow YC, YS \text{ or } YK$

When XC(XS)(XK) carry/borrow is generated

$YB \pm 1, Dn \pm 1 \rightarrow YC, YS \text{ or } YK$

- $C[S][K] < B * 2 \quad (XC(XS)(XK) < XB * 2, YC(YS)(YK) < YB * 2)$

Use this instruction to shift left ($XB * 2, YB * 2, \text{LSB} = 0$) the addresses in XB and YB and to store the left-shifted addresses to XC(XS)(XK), YC(YS)(YK).

- $C[S][K] \leftarrow B$ ($XC(XS)(XK) \leftarrow XB$, $YC(YS)(YK) \leftarrow YB$)
Use this instruction to store in $XC(XS)(XK)$ and $YC(YS)(YK)$ the addresses obtained by reversing the addresses stored in XB and YB .
- $C[S][K] \leftarrow B \& D_n$ ($XC(XS)(XK) \leftarrow XB \& D_n$, $YC(YS)(YK) \leftarrow YB \& D_n$)
($D_n = D1A$ to $D1H$, $D2A$ to $D2D$, $D3$, $D4$)
Use this instruction to store in $XC(XS)(XK)$ and $YC(YS)(YK)$ the addresses stored in XB and YB , and the addresses of the AND of data indicated by D_n .
- $C[S][K] \leftarrow B' D_n$ ($XC(XS)(XK) \leftarrow XB' D_n$, $YC(YS)(YK) \leftarrow YB' D_n$)
($D_n = D1A$ to $D1H$, $D2A$ to $D2D$, $D3$, $D4$)
Use this instruction to store in $XC(XS)(XK)$ and $YC(YS)(YK)$ the addresses stored in XB and YB , and the addresses of the OR of data indicated by D_n .
- $C[S][K] \leftarrow B'' D_n$ ($XC(XS)(XK) \leftarrow XB'' D_n$, $YC(YS)(YK) \leftarrow YB'' D_n$)
($D_n = D1A$ to $D1H$, $D2A$ to $D2D$, $D3$, $D4$)

Use this instruction to generate patterns such as the bit-varying patterns to generate addresses obtained by reversing the bits indicated by D_n of the addresses stored in XB and YB . It executes the following operations:

$XC \leftarrow XB'' D_n$	$XB \oplus D_n \rightarrow XC$	
$XS \leftarrow XB'' D_n$	$XB \oplus D_n \rightarrow XS$	
$XK \leftarrow XB'' D_n$	$XB \oplus D_n \rightarrow XK$	
$YC \leftarrow YB'' D_n$	$YB \oplus D_n \rightarrow YC$	(Dn= D1A to D1H, D2A to D2D, D3, D4)
$YS \leftarrow YB'' D_n$	$YB \oplus D_n \rightarrow YS$	
$YK \leftarrow YB'' D_n$	$YB \oplus D_n \rightarrow YK$	

⊕: Exclusive OR

An example of bit changing pattern generation performed by use of this instruction is shown in the following.

Example of bit changing pattern

Address selection errors and multiple address selection faults occurring in the decoder circuit can be efficiently detected by using bit changing patterns. An example of bit changing pattern for testing the row address decoder is described in the following.

- Pattern sequence
 1. "0" is written in every cell.
 2. "0" is written in the focus cell.
 3. "1" is written in the address generated by inverting a bit of the focus cell address.
 4. "0" is read from the focus cell.
 5. Steps 2 to 4 are repeated after changing the bit to be inverted.
 6. Steps 2 to 5 are repeated after changing the focus cell.

◆ Test Program

```

MPAT BIT      ;ROW
REGISTER
XMAX=#7F
YMAX=#7F
TPH=#0
DCMR=#0
PC=#0
D3B=#1
IDX1=#3FFE
IDX2=#3
IDX3=#7E

START #0
NOP          XB<0 YB<0 TP<TPH D3<D3B
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP      W XC<XB"D3          X<XB Y<YB
      NOP      W                  X<XC Y<YB /D
      JN12 ST1 R                  X<XB Y<YB D3<D3*2
      NOP      W XC<XB"D3          X<XB Y<YB
      NOP      W                  X<XC Y<YB /D
      JN13 ST1 R XB<XB+1          X<XB Y<YB D3<D3B
      NOP
      STPS
END

```

◆ $C[S][K] < f(C[S][K])$

$C[S][K] < f(C[S][K])$ is an instruction group for carrying out the address generation of disturb cell through operation using the registers XC(XS)(XK) and YC(YS)(YK). The group is comprised of the following instructions:

$C < C, S < S \text{ or } K < K$	
$C[S][K] < C[S][K] \pm 1$	
$C[S][K] < Dn$	
$C[S][K] < 0$	
$C[S][K] < C[S][K] * 2$	Shift left
$C[S][K] < \sim C[S][K]$	Complement
$C[S][K] < C[S][K] \& Dn$	AND
$C[S][K] < C[S][K] \text{ " } Dn$	Exclusive OR
$C[S][K] < C[S][K] \text{ ' } Dn$	OR
$C[S][K] < C[S][K] \pm 1 \wedge BX$	} Only YC or YS, YK is valid
$C[S][K] < C[S][K] \pm Dn \wedge BX$	
$C[S][K] < C[S][K] \pm 1 \pm BX$	
$C[S][K] < C[S][K] \pm Dn \pm BX$	
$C[S][K] < C[S][K] \pm 1 \wedge CY$	
$C[S][K] < C[S][K] \pm Dn \wedge CY$	
$C[S][K] < C[S][K] \pm 1 \pm CY$	
$C[S][K] < C[S][K] \pm Dn \pm CY$	

C means XC or YC.

S means XS or YS.

K means XK or YK.

C[S][K] means XC, XS or XK and YC, YS or YK.

Dn represents D1A to D1H, D2A to D2D, D3, and D4.

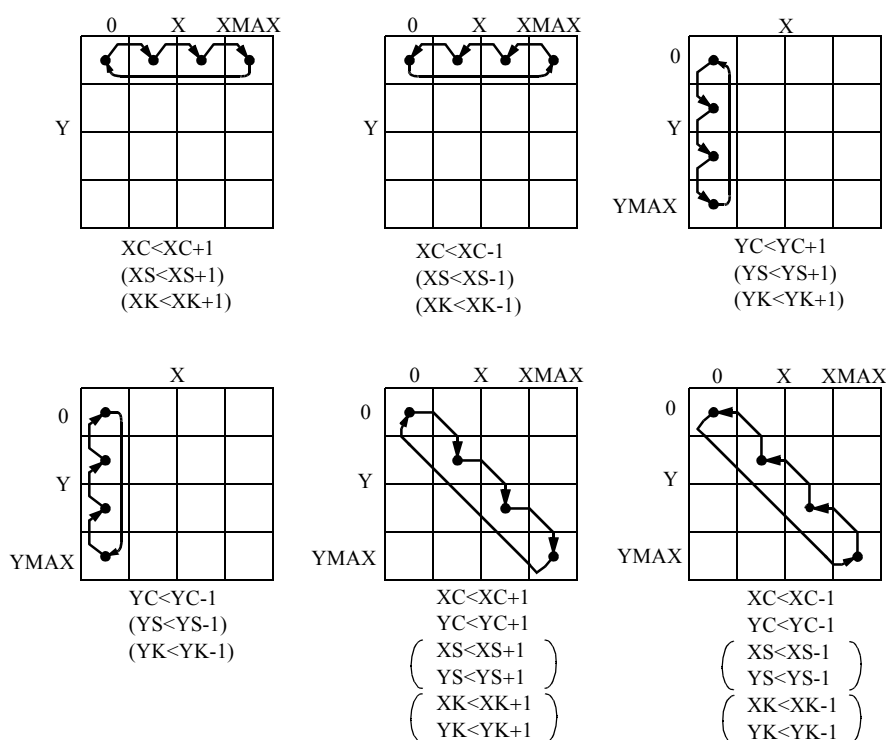
- $C < C$, $S < S$ or $K < K$ ($XC < XC$, $XS < XS$, $XK < XK$, $YC < YC$, $YS < YS$, $YK < YK$)

Use this instruction when nothing is executed due to the hold instruction. This instruction may be omitted.

- $C[S][K] < C[S][K] \pm 1$ ($XC(XS)(XK) < XC(XS)(XK) \pm 1$, $YC(YS)(YK) < YC(YS)(YK) \pm 1$)

Use this instruction to increment or decrement addresses.

This instruction can generate the following sequences.



- $C[S][K] < C[S][K] \pm Dn$ ($XC(XS)(XK) < XC(XS)(XK) \pm Dn$, $YC(YS)(YK) < YC(YS)(YK) \pm Dn$)

Use this instruction to increase or decrease the disturb addresses stored in $XC(XS)(XK)$ and $YC(YS)(YK)$.

It executes the following operations:

$C < C + Dn$ ($XC(XS)(XK) < XC(XS)(XK) + Dn$, $YC(YS)(YK) < YC(YS)(YK) + Dn$)

When $C + Dn < MAX + 1$ ($C + Dn$) $\rightarrow$ C

When $C + Dn > MAX$ ($C + Dn - MAX$) $\rightarrow$ C

MAX represents XMAX and YMAX.

Dn represents D1A to D1H, D2A to D2D, D3, and D4.

$C < C - D_n$ ($XC(XS)(XK) < XC(XS)(XK) - D_n$, $YC(YS)(YK) < YC(YS)(YK) - D_n$)

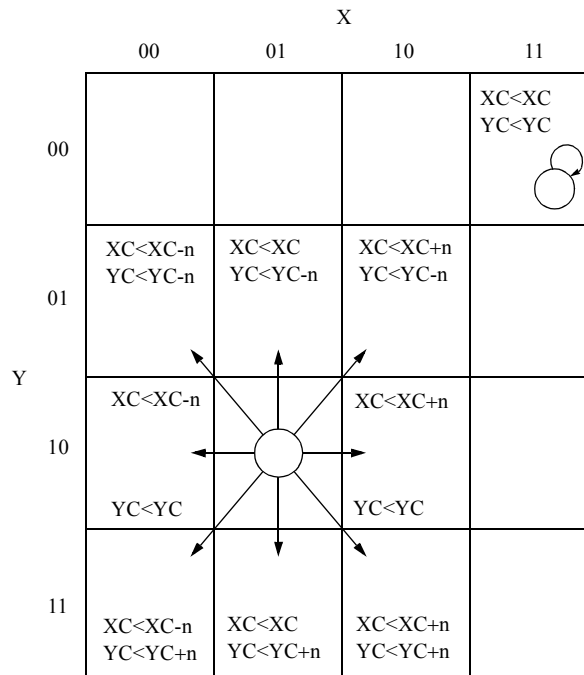
When $C - D_n \geq 0$ $(C - D_n) \rightarrow C$

When $C - D_n < 0$ $XMAX + 1 - (C - D_n) \rightarrow C$

MAX represents XMAX and YMAX.

D_n represents D1A to D1H, D2A to D2D, D3, and D4.

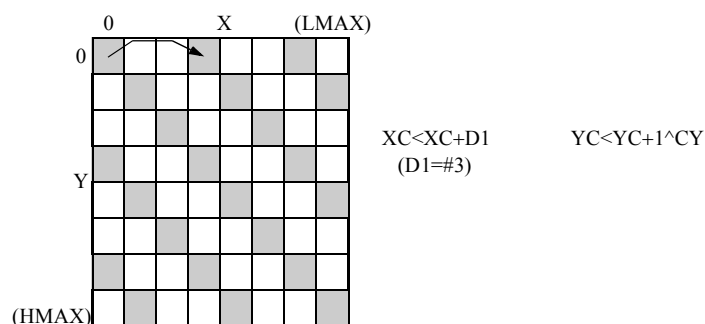
Use this instruction when generating addresses as follows:



n represents D1A to D1H, D2A to D2D, D3, and D4.

2. 4 Address Pattern Generation Instructions

This instruction is used, for example, when sequentially accessing the addresses shown by hatching below.



```
MPAT SAMPLE
REGISTER
XMAX=#7
YMAX=#7

D1=#3      Operand

IDX1=#14
PC=#0

START #0
NOP XB<0 YB<0
NOP XC<XB YC<YB ; Initializes XC, YC

ST0: JN11 ST0 XC<XC+D1 YC<YC+1^CY X<XC Y<YC
:

END
```

- $C[S][K] < C[S][K] \pm 1, Dn^{BX} \text{ (YC(YS)(YK) < YC(YS)(YK) \pm 1^{BX})}$
 $(YC(YS)(YK) < YC(YS)(YK) \pm Dn^{BX})$

This instruction executes the following operations in conditional operation instructions when carry or borrow is generated in XB operation.

$YS(YS)(YK) < YC(YS)(YK) \pm 1^{BX}$

When no XB carry/borrow is generated $YC(YS)(YK) < YC(YS)(YK)$

When XB carry/borrow is generated $YC(YS)(YK) < YC(YS)(YK) \pm 1$

$$YS(YS)(YK) < YC(YS)(YK) \pm Dn \wedge BX$$

When no XB carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK)$
When XB carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm Dn$

- $C[S][K] < C[S][K] \pm 1, Dn \pm BX$ ($YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm BX$)
($YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm BX$)

Use this instruction to add or subtract carry and borrow to be generated by XB operation in the operation instructions of YC, YS, and YK.

$$YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm BX$$

When no XB carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm 1$
When XB carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm 2$

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm BX$$

When no XB carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm Dn$
When XB carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm 1$

- $C[S][K] < C[S][K] \pm 1, Dn \wedge CY$ ($YC(YS)(YK) < YC(YS)(YK) \pm 1 \wedge CY$)
($YC(YS)(YK) < YC(YS)(YK) \pm Dn \wedge CY$)

This instruction is a concatenation instruction for each address of XC(XS)(XK) and YC(YS)(YK), executing the following operations.

This instruction is valid only when the combination of add/subtract instructions of XC is $(XC(XS)(XK) < XC(XS)(XK) \pm 1, Dn)$.

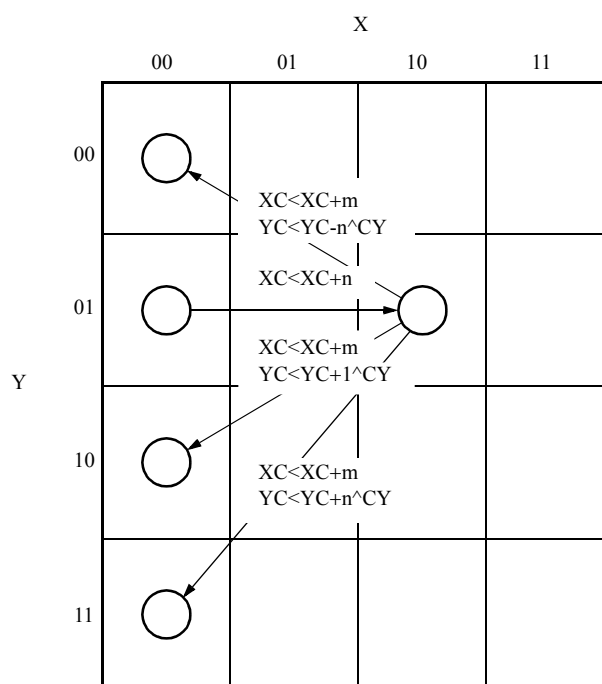
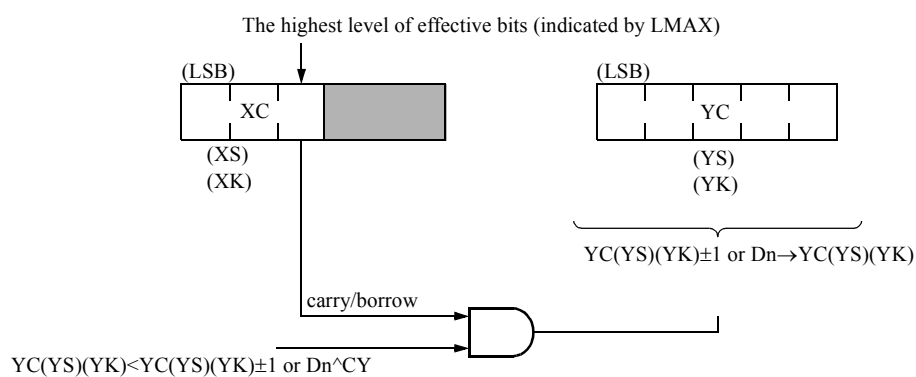
$$YC(YS)(YK) < YC(YS)(YK) \pm 1 \wedge CY$$

When no XC(XS)(XK) carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK)$
When XC(XS)(XK) carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm 1$

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \wedge CY$$

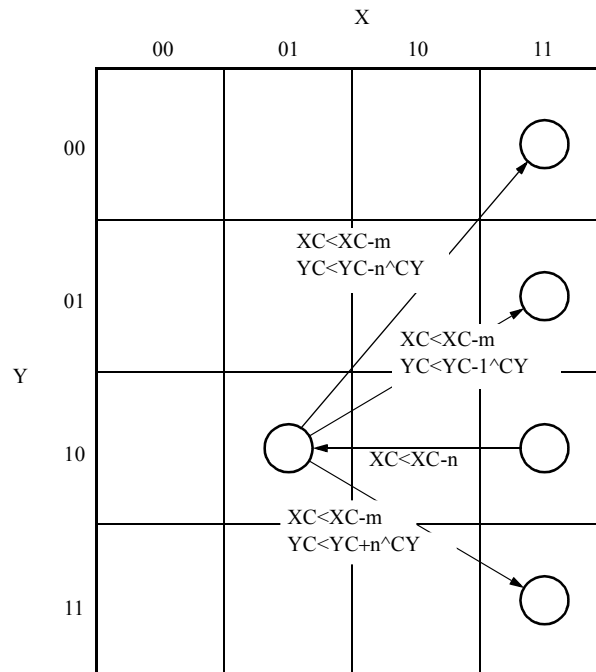
When no XC(XS)(XK) carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK)$
When XC(XS)(XK) carry/borrow is generated	$YC(YS)(YK) < YC(YS)(YK) \pm Dn$

This instruction executes $YC(YS)(YK) \pm 1$ or $D_n \rightarrow YC(YS)(YK)$ only when carry or borrow is generated in the operation $XC(XS)(XK)$ described in the same cycle. When the above condition is not satisfied, the instruction $YC(YS)(YK) \rightarrow YC(YS)(YK)$ is executed.



n and m represent D1A to D1H, D2A to D2D, D3, and D4.

m is the value such that the result of $XC + m$ satisfies the condition $LMAX < XC + m$.



n and m represent D1A to D1H, D2A to D2D, D3, and D4.

m is the value such that the result of $XC-m$ satisfies the condition of $0 > XC-m$.

2. 4 Address Pattern Generation Instructions

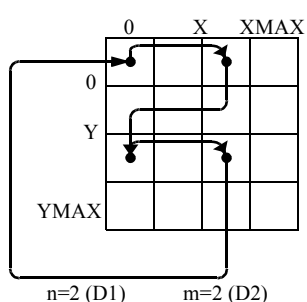
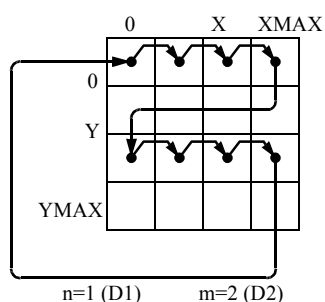
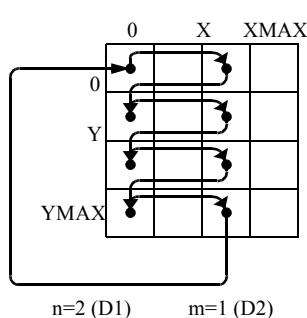
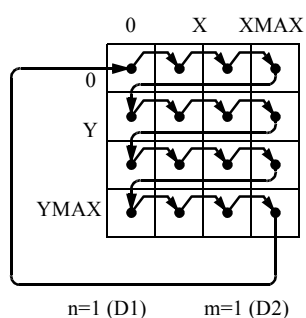
Use $YC(YS)(YK) < YC(YS)(YK) \pm Dn \wedge CY$ when the number of operations used in the address operation is other than "1" in a N type test pattern such as Waltzing. Examples are shown below.

```

MPAT SAMPLE
REGISTER
:
D1=n
D2=m
:
START #00

ST0: JN11 ST0 XC<XC+D1 YC<YC+D2^CY X<XC Y<YC
:
END

```



- $C[S][K] < C[S][K] \pm 1, Dn \pm CY$ ($YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm CY$)
($YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm CY$)

This instruction adds and subtracts carries and borrows generated by the operation of XC, XS, and XK in the operation instruction of YC, YS, and YK. This instruction is valid only when the combination of add/subtract instructions of XC(XS)(XK) is $(XC(XS)(XK) < XC(XS)(XK) \pm 1, \text{ or } Dn)$.

$YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm CY$

When no XC(XS)(XK) carry/borrow is generated

$YC(YS)(YK) < YC(YS)(YK) \pm 1$

When XC(XS)(XK) carry/borrow is generated

$YC(YS)(YK) < YC(YS)(YK) \pm 2$

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm CY$$

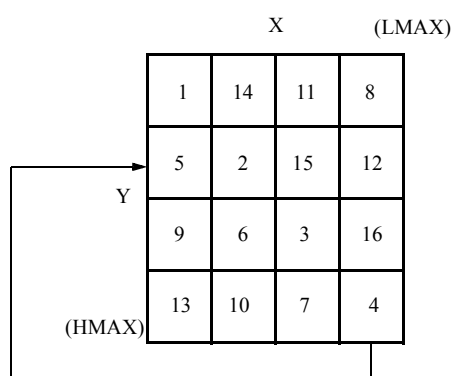
When no XC(XS)(XK) carry/borrow is generated

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn$$

When XC(XS)(XK) carry/borrow is generated

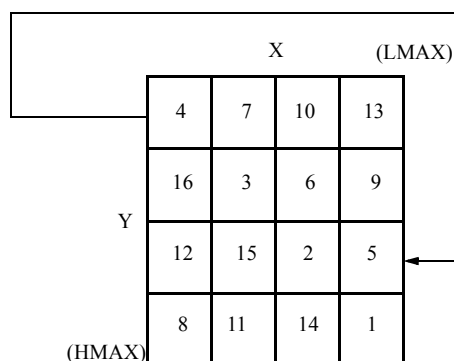
$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm 1$$

Use this instruction to generate disturb addresses such as shift diagonal.



ST0 : JN11 ST0 XC<XC+1 YC<YC+1+CY X<XC Y<YC

Only when XC carry = 1 (4, 8, 12, 16 in the left figure), execute YC<YC+2. Otherwise, execute YC<YC+1.



ST0 : JN11 ST0 XC<XC-1 YC<YC-1-CY X<XC Y<YC

Only when XC borrow = 1 (4, 8, 12, 16 in the left figure), execute YC<YC-2. Otherwise execute YC<YC-1.

- $C[S][K] < Dn$ ($XC(XS)(XK) < Dn$, $YC(YS)(YK) < Dn$)
This instruction initializes each register of XC(XS)(XK) and YC(YS)(YK) and loads the contents of the registers Dn (D1A to D1H, D2A to D2D, D3, D4).
- $C[S][K] < 0$ ($XC(XS)(XK) < 0$, $YC(YS)(YK) < 0$)
This instruction zeros (#0) each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < C[S][K] * 2$ ($XC(XS)(XK) < XC(XS)(XK) * 2$, $YC(YS)(YK) < YC(YS)(YK) * 2$)
This instruction left-shifts (LSB=0) the contents of the specified register and loads them into each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < /C[S][K]$ ($XC(XS)(XK) < /XC(XS)(XK)$, $YC(YS)(YK) < /YC(YS)(YK)$)
This instruction reverses the contents of the specified register and loads them into each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < C[S][K] \& Dn$ ($XC(XS)(XK) < XC(XS)(XK) \& Dn$, $YC(YS)(YK) < YC(YS)(YK) \& Dn$)

This instruction obtains the AND of the contents of the specified registers of XC(XS)(XK) and YC(YS)(YK) and the data indicated by Dn, and loads it into each register of XC(XS)(XK) and YC(YS)(YK).

- $C[S][K] < C[S][K] \text{ "Dn" } (XC(XS)(XK) < XC(XS)(XK) \text{ "Dn" }, YC(YS)(YK) < YC(YS)(YK) \text{ "Dn" })$

This instruction obtains the exclusive OR of the contents of the specified registers of XC(XS)(XK) and YC(YS)(YK) and the data indicated by Dn, and loads it into each register of XC(XS)(XK) and YC(YS)(YK).

- $C[S][K] \text{ 'Dn' } (XC(XS)(XK) < XC(XS)(XK) \text{ 'Dn' }, YC(YS)(YK) < YC(YS)(YK) \text{ 'Dn' })$

This instruction obtains the OR of the contents of the specified registers of XC(XS)(XK) and YC(YS)(YK) and the data indicated by Dn, and loads it into each register of XC(XS)(XK) and YC(YS)(YK).

◆ $C[S][K] < f(B, C[S][K], t)$

ALPG has the save registers of XS, XK, YS, and YK to save each address of XC and YC by instruction because the address sequence generates complex test patterns.

The following instructions are provided to control these save registers.

$C < S$

$S < C$

$C \diamond S$

$CS < OPE$

$C < S < OPE$

$S < C < OPE$

$C < K$

$K < C$

$C \diamond K$

$CK < OPE$

$C < K < OPE$

$K < C < OPE$

$SK < OPE$

$CSK < OPE$

C means XC or YC.

S means XS or YS.

CS means XCS or YCS.

OPE means each operation of $f(B)$ and $f(C[S][K])$.

K means XK or YK.

CK means XCK or YCK.

SK means XSK or YSK.

CSK means XCSK or YCSK.

OPE means the operation instruction. This operation can use the operation instructions $f(B)$ or $f(C[S][K])$ of "[C\[S\]\[K\] < f\(B\)](#)" on page 2-113 and "[C\[S\]\[K\] < f\(C\[S\]\[K\]\)](#)" on page 2-120.

- C<S (XC<XS, YC<YS)
This instruction loads into XC and YC registers the addresses saved in the save registers.
- S<C (XS<XC, YS<YC)
This instruction saves the addresses of XC and YC registers into save registers.
- C<S (XC<XS, YC<YS)
This instruction exchanges the addresses stored in the save registers of XS and YS with the addresses stored in the registers of XC and YC.
- CS<OPE (XCS<OPE, YCS<OPE)
This instruction stores an operation result in XS and YS simultaneously in XC and YC operation instructions.
- C<S<OPE (XC<XS<OPE, YC<YS<OPE)
This instruction loads the addresses stored in each save register of XS and YS into each register of XC and YC, and stores the operation result only in each save register of XS and YS.
Before-loading, addresses of XC(XS) and YC(YS) are used for operation.
- S<C<OPE (XS<XC<OPE, YS<YC<OPE)
This instruction loads the addresses stored in each register of XC and YC into each save register of XS and YS, and stores the operation result in each register of XC and YC.
Before-loading the addresses of XC(XS) and YC(YS) are used for operation.
- C<K(XC<XK, YC<YK)
This instruction loads into XC and YC registers the addresses saved in the save registers.
- K<C(XK<XC, YK<YC)
This instruction saves the addresses of XC and YC registers into save registers.
- C<K(XC<XK, YC<YK)
This instruction exchanges the addresses stored in the save registers of XK and YK with the addresses stored in the registers of XC and YC.
- CK<OPE(XCK<OPE, YCK<OPE)
This instruction stores an operation result in XK and YK simultaneously in XC and YC operation instructions.
- C<K<OPE(XC<XK<OPE, YC<YK<OPE)
This instruction loads the addresses stored in each save register of XK and YK into each register of XC and YC, and stores the operation result only in each save register of XK and YK.
Before-loading, addresses of XC(XK) and YC(YK) are used for operation.
- K<C<OPE(XK<XC<OPE, YK<YC<OPE)
This instruction loads the addresses stored in each register of XC and YC into each save register of XK and YK, and stores the operation result in each register of XC and YC.
Before-loading the addresses of XC(XK) and YC(YK) are used for operation.
- SK<OPE(XSK<OPE, YSK<OPE)

This instruction stores an operation result in XS, XK, YS, and YK simultaneously in XC and YC operation instructions.

- CSK<OPE(XCSK<OPE, YCSK<OPE)

This instruction stores an operation result in XC, XS, XK, YC, YS, and YK simultaneously in XC and YC operation instructions.

2. 4. 6 Source Address Field

This instruction selects the addresses to be applied to DUT. The following addresses can be selected and output as each address of X and Y.

X<XB	Y<YB
X<XC	Y<YC
X<XS	Y<YS
X<XK	Y<YK
X<YB	Y<XB
X<YC	Y<XC
X<YS	Y<XS
X<YK	Y<XK
X<XB'Z	Y<XB'Z
X<XC'Z	Y<XC'Z
X<XS'Z	Y<XS'Z
X<XK'Z	Y<XK'Z
X<Z	Y<Z
X<RF	

RF means REFRESH counter.

The description of the following instructions may be omitted. (when such instructions are not described, the following instructions are executed.)

X<XB

Y<YB

This instruction cannot select different registers in the same group as a combination of the output selection of X and Y in the groups of X address generation registers (XB, XC, XS, XK) and Y address generation registers (YB, YC, YS, YK).

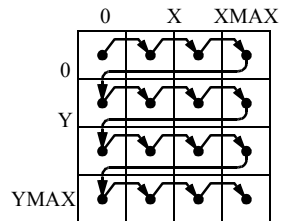
Specify the effective bits of X and Y addresses to be applied to DUT by XMAX register and YMAX register respectively. Specify XMAX and YMAX by $2n-1$ when the number of effective bits is n .

Examples of the use of this instruction are given below.

2. 4 Address Pattern Generation Instructions

$X < X_B \quad Y < Y_B$
 $X < X_C \quad Y < Y_C$
 $X < X_S \quad Y < Y_S$
 $X < X_K \quad Y < Y_K$

Use this instruction mainly to scan the address generation sequence in the X direction as follows:

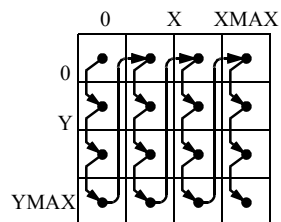


```

NOP      :
          XB<0      YB<0
ST0: JNII ST0  XB<XB+1  YB<YB+1^BX  [X<XB  Y<YB]
          :
```

$X < Y_B \quad Y < X_B$
 $X < Y_C \quad Y < X_C$
 $X < Y_S \quad Y < X_S$
 $X < Y_K \quad Y < X_K$

Use this instruction mainly to scan the address generation sequence in the Y direction as follows:



```

NOP      :
          XB<0      YB<0
ST0: JNII ST0  XB<XB+1  YB<YB+1^BX  [X<YB  Y<XB]
          :
```

2. 4 Address Pattern Generation Instructions

$$X < X B'Z \quad Y < X B'Z$$

$$X < X C'Z \quad Y < X C'Z$$

$$X < X S'Z \quad Y < X S'Z$$

$$X < X K'Z \quad Y < X K'Z$$

Use this instruction mainly when X-carry controls the address generation by Y or when Y-carry controls the address generation by X like the moving inversions pattern.

MOVING INVERSIONS

```
MPAT      MOVI      ; MOVING INVERSIONS TEST PATTERN
```

REGISTER

```
XMAX=#1FF      ; 2n-1
YMAX=#1FF      ; 2m-1
IDX1=#3FFFE    ; 2(n+m) -2
IDX2=#7         ; n-2
IDX3=#7         ; m-2
D3B  =#1
D4B  =#0
```

```
START #0
```

```
NOP      XC<0   YC<0   Z<0   D3<D3B   D4<D4B   TP<0
```

```
ST0: JN11 ST0 W X<XC   Y<YC   XC<XC+1   YC<YC+1^CY
```

```
ST1: NOP      R X<XC'Z   Y<YC
      NOP      W X<XC'Z   Y<YC /D
      JN11 ST1 R X<XC'Z   Y<YC /D   XC<XC+D3   YC<YC+1^CY   Z<Z+1^CY
      JN12 ST1 Z<0      D3<D3*2L   D4<D4*2L   TP</TP
```

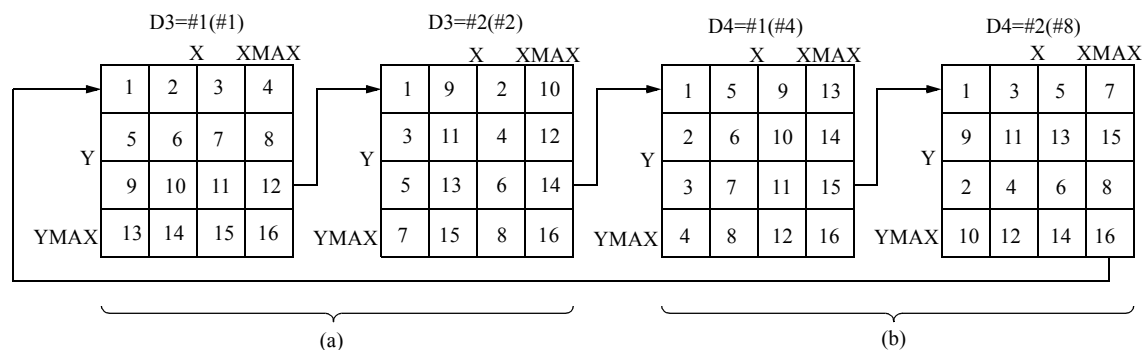
```
ST2: NOP      R X<YC   Y<XC'Z
      NOP      W X<YC   Y<XC'Z /D
      JN11 ST2 R X<YC   Y<XC'Z /D   XC<XC+D4   YC<YC+1^CY   Z<Z+1^CY
      JN13 ST2 Z<0      D4<D4*2   TP</TP
```

```
JZD ST0 D3<D3B D4<D4B
```

```
STPS
```

```
END
```

The address sequence of moving inversions is shown below (where $n=2$ and $m=2$).



Parenthesized figures are addends to moving inversions.

X<Z Y<Z

Use this instruction to set data in DUT other than addresses by mainly using the address line.

X<RF

This instruction outputs the contents of the refresh counter to the X address of DUT.

2. 4. 7 N Register Field

N Register Field is an operation instruction used to generate bank address, etc. and includes the following instructions:

N<N

N<N+1

N<N-1

N<NH

N<0

N</N

N<NDn

N<N+NDn

N<N-NDn

N<N

This instruction is the HOLD instruction of the N register. This instruction is used when no N register operation is carried out.

When omitting N register operation instructions which include this instruction, the result is the same as if this instruction had been specified.

N<N+1

Use this instruction to increment the N register data.

N<N-1

Use this instruction to decrement the N register data.

N<NH

This instruction is used to load the contents of the NH register to the N register. This instruction is used to initialize the N register.

N<0

Use this instruction to clear the N register data to 0. This instruction is used to initialize the N register.

N</N

Use this instruction to invert the contents of the N register and to store the inverted value to the N register.

N<NDn

This instruction is used to load the contents of the NDn register to the N register. This instruction is used to initialize the N register.

$N < N + NDn$

This instruction is used to add the contents of the NDn register to the N register.

$N < N - NDn$

This instruction is used to subtract the contents of the NDn register from the N register.

NDn represents ND1 to ND7.



Tip

ND1 can be described as ND.

2. 4. 8 B Register Field

The B Register Field is an operation instruction used to generate bank address, etc. and includes the following instructions:

$B < B$

$B < B + 1$

$B < B - 1$

$B < BH$

$B < 0$

$B < /B$

$B < BDn$

$B < B + BDn$

$B < B - BDn$

$B < B$

This instruction is the HOLD instruction of the B register. This instruction is used when no B register operation is carried out.

When omitting B register operation instructions which include this instruction, the result is the same as when this instruction has been specified.

$B < B + 1$

Use this instruction to increment the B register data.

$B < B - 1$

Use this instruction to decrement the B register data.

$B < BH$

This instruction is used to load the contents of the BH register to the B register. This instruction is used to initialize the B register.

$B < 0$

Use this instruction to clear the B register data to 0. This instruction is used to initialize the B register.

B</B
 Use this instruction to invert the contents of the B register and to store the inverted value to the B register.

B<BDn
 This instruction is used to load the contents of the BDn register to the B register. This instruction is used to initialize the B register.

B<B+BDn
 This instruction is used to add the contents of the BDn register to the B register.

B<B-BDn
 This instruction is used to subtract the contents of the BDn register from the B register.

BDn represents BD1 to BD7.


Tip

BD1 can be described as BD.

2. 4. 9 Address Inversion Field

The address inversion field instructions invert addresses to be applied to the DUT. When an INVERSION FIELD instruction is entered, the corresponding address, X or Y or Z, is inverted only in the current cycle.

/X

$XMAX_n \bullet \overline{X_n}$

→ X_n

/Y

$YMAX_n \bullet \overline{Y_n}$

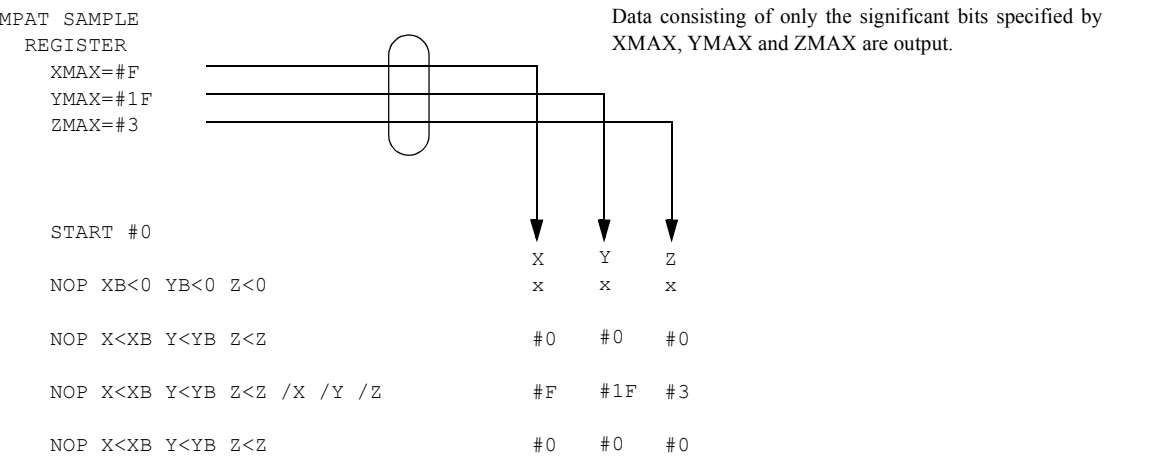
→ Y_n

/Z

$ZMAX_n \bullet \overline{Z_n}$

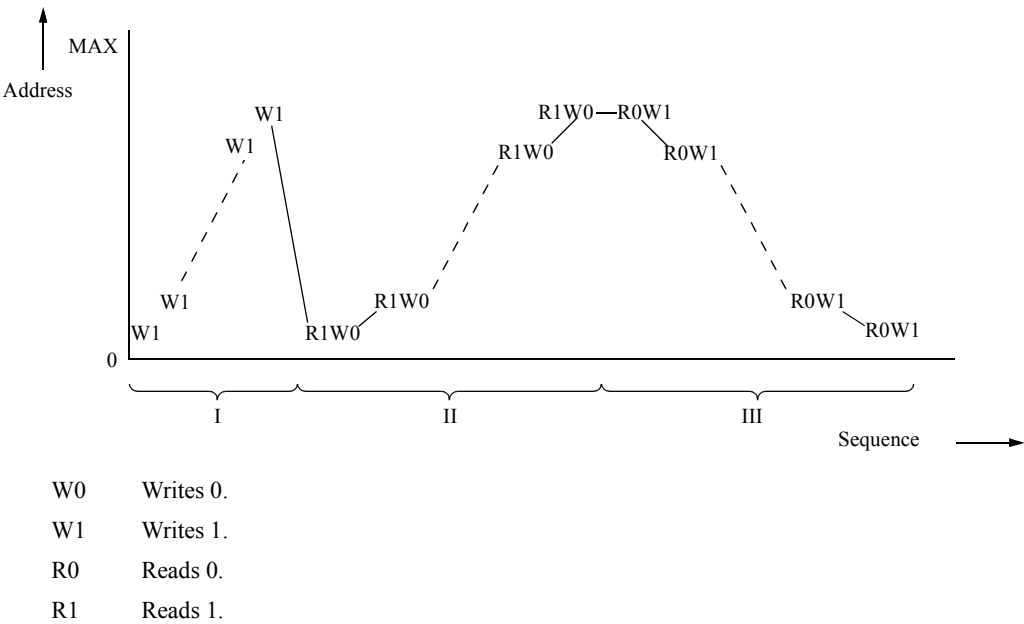
→ Z_n

n = 0 to 17(For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, n = 0 to 15)



Use the address inversion field instructions to generate inverted addresses for the generation of Marching or Complement marching pattern. Examples are shown below.

◆ Example of Marching Pattern



```
MPAT MARCH
REGISTER
XMAX=#7F
YMAX=#7F
TPH=#0
DCMR=#0
IDX1=#3FFE
PC=#0

START #00
NOP XB<0 YB<0 TP<TPH
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB /D
ST1: NOP R X<XB Y<YB /D
JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST2: NOP R X<XB Y<YB /X /Y
JN11 ST2 W XB<XB+1 YB<YB+1^BX X<XB Y<YB /X /Y /D
NOP
STPS

END
```

2. 4.10 Refresh Counter Field

The ALPG incorporates two refresh counters, RL and RH, to generate addresses to be refreshed when conducting a refresh test.

The $RF<RF+1$ is used to control the refresh counters. This instruction increments the counter.

$RF<RF$ is an instruction to hold the operation of a REFRESH counter; its description may be omitted.

An example of use of the REFRESH COUNTER FIELD instruction is described next.

```

MPAT REF      ;DISTRIBUTED REFRESH
MODE REFRESHB 128
REGISTER
  XMAX=#7F
  YMAX=#7F
  TPH=#0
  DCMR=#0
  TIMER=16US
  ISP=#10
  IDX1=#3FFE
  IDX2=#3FFD
  PC=#00

  START #00

  NOP T XB<0 YB<0 TP<TPH
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP      W XC<XB+1 YC<YB+1^BX X<XB Y<YB /D
ST2: NOP      R              X<XC Y<YC
      NOP      R              X<XB Y<YB /D
      JN12 ST2 R XC<XC+1 YC<YC+1^CY X<XC Y<YC
      JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
      NOP
      STPS

  START #10

RTN  RF<RF+1  X<RF  →  This instruction is accessed every 16 ms.
                          The moment the REFRESH counter is incremented,
                          the REFRESH counter value is output
                          as the address to be applied to DUT.

END

```

2. 4.11 D3, D4 Register Field

The D3, D4 FIELD instructions comprise the following operation instructions to be executed against the D3, D4 register that stores the operand and its initial value needed when generating disturb cell addresses which are stored into XC and YC.

D3<D3	D4<D4
D3<D3*2	D4<D4*2
D4<D4/2	
D3<D3B	D4<D4B

$D3 < D3 * 2L$ $D4 < D4 * 2L$

$D3 < D3 + 1$

$D3 < D3 - 1$

$D3 < 0$

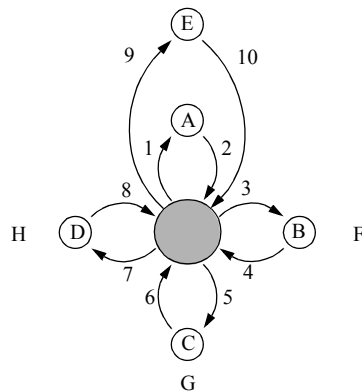
The operation of each instruction is described in the following.

$D3 < D3B$, $D4 < D4B$

This instruction is used to initialize registers D3 and D4 and to load the data of registers D3B and D4B respectively.

$D3 < D3 + 1$

This instruction increments the contents of the D3 register, for example, during butterfly pattern generation. An example of use of this instruction for butterfly pattern generation is shown in the following.



After accessing in the order of 1 through 8, incrementing D3 register to access cell (E).

```

MPAT BUTTER
REGISTER
:
D3B=#1      Initialization of the D3B register
:
START #0

NOP XB<0 YB<0 TP<TPH D3<D3B
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP W YC<YB-D3 X<XB Y<YB /D
ST2: NOP R          X<XB Y<YC
      NOP R XC<XB+D3 X<XB Y<YB /D
      NOP R          X<XC Y<YB
      NOP R YC<YB+D3 X<XB Y<YB /D
      NOP R          X<XB Y<YC
      NOP R XC<XB-D3 X<XB Y<YB /D
      NOP R          X<XC Y<YB D3<D3+1
JN12 ST2 R YC<YB-D3 X<YB Y<YB /D
JN13 ST1 W XB<XB+1 YB<YB+1 X<XB Y<YB D3<D3B
NOP
STPS
END

```

D3<D3-1

This instruction decrements the D3 register data.

D3<D3*2, D4<D4*2

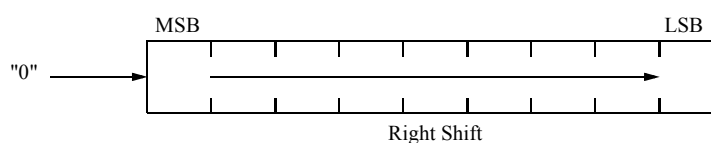
This instruction shifts the contents of the D3, D4 register. It operates as illustrated below.



The LSB becomes "0".

D4<D4/2

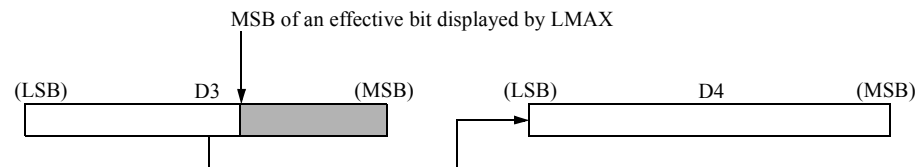
This instruction shifts the D4 register data as illustrated below.



The MSB becomes "0".

D3<D3*2L D4<D4*2L

This instruction concatenates and shifts the D3 register and D4 register. The data to be shifted out from MSB of the effective bits in the D3 register shown by LMAX are loaded as LSB data in D4 register.



D3<0

This instruction initializes the D3 register, 0-clear the D3 register.

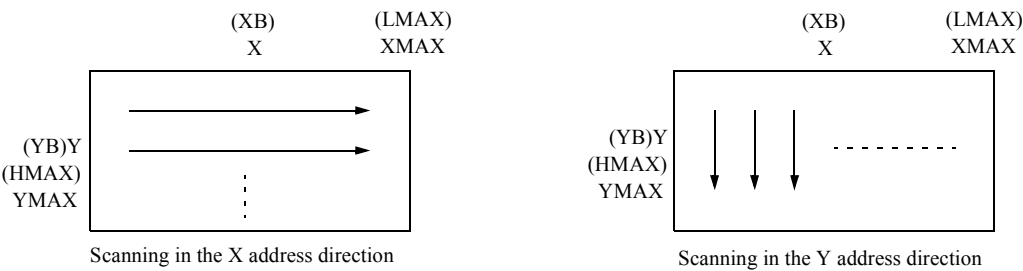
2. 4.12 Address Swap Field

Generally, LMAX controls the effective bits of the operation of XB, XC, XS, XK, and D3 register, and HMAX controls the effective bits of YB, YC, YS, and YK. The values of the registers controlled by LMAX and HMAX are exchanged in the cycle immediately after any of the following instructions.

LMAX<>HMAX

This instruction is a toggle instruction; an odd number of execution of the instruction exchanges LMAX with HMAX, and an even number of executions restores the original state of LMAX and HMAX.

Use this instruction to switch in real time from the scanning in the X address direction to scanning in the Y address direction in a DUT test with different bit lengths between X address and Y address.



◆ Example

```

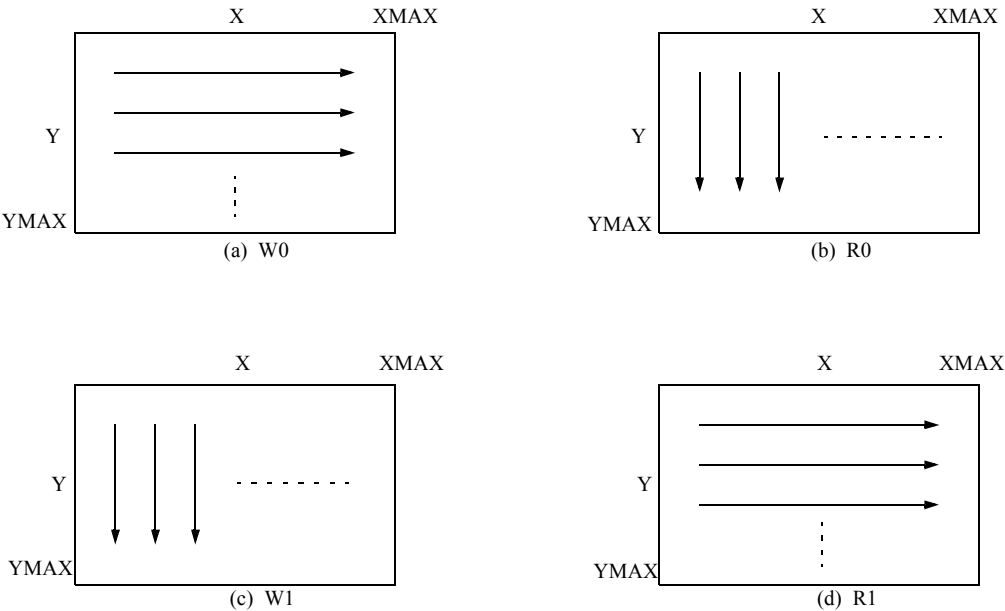
MPAT SWAP

REGISTER
XMAX=#3FF
YMAX=#FF
IDX1=#3FFFE
PC=#0

START #0
NOP      XC<0      YC<0      TP<0
(a) ST0: JN11 ST0  XC<XC+1  YC<YC+1^CY  X<XC  Y<YC  W
NOP      LMAX<>HMAX
(b) ST1: JN11 ST1  XC<XC+1  YC<YC+1^CY  X<YC Y<XC  R
(c) ST2: JN11 ST2  XC<XC+1  YC<YC+1^CY  X<YC Y<XC  W /D
NOP      LMAX<>HMAX
(d) ST3: JN11 ST3  XC<XC+1  YC<YC+1^CY  X<XC  Y<YC  R /D
STPS

END

```



2. 4.13 Instructions and Execution Cycle

Address operation instructions are classified into the following two groups, depending on the cycle of outputting the results of operation to X, Y and Z addresses.

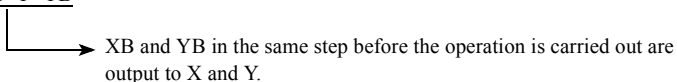
Execution in the step in which instructions are described (Event time 1)

The instructions below are valid as soon as they are entered.

- Source address field

 $X \leq n$
$$Y \leq n$$

NOP XB<XB+1 YB<YB+1^BX X<XB Y<YB



- Address inversion field

 $\backslash \mathbf{X}$

/Y

$$/Z$$

NOP XB<XB+1 YB<YB+1^BX X<XB Y<YB /X /Y



The address selected by $X < X_B$ and $Y < Y_B$ is inverted and output.

Execution in the step next to the one in which instructions are described (Event time 2)

The operation instructions of registers XB, YB, Z, N, B, XC, XS, XK, YC, YS, YK, D3, D4, and RF are the instructions of Event time 2.

The Event time 2 operation instruction is for registers; the results of the operation are output in the next step.

```

NOP XB<0      YB<0
NOP XB<XB+1   YB<YB+1 X<XB Y<YB
NOP XB<XB+1   YB<YB+1 X<XB Y<YB

```

	XB	YB	X	Y
; x	x	x	x	x
; 0	0	0	0	0
; 1	1	1	1	1
; 2	2	2	2	2

The results of the operation by the D3 and D4 operation instructions are used for the operation of the current address field in the next step.

	(D3=#1)	D3	XC
NOP	XC<0	; 1	x
NOP	XC<XC+D3	; 1	0
NOP	<u>D3<D3+1</u>	; 2	1
NOP		; 2	3
NOP		; 2	5

2. 4.14 Notes on Program Generation

SET instruction

This SET instruction can be used only in the normal mode (1WAY).

Use the DSET instruction in the 2WAY/4WAY interleaved mode.

XMAX, YMAX, ZMAX

Specify effective bits of X, Y and Z addresses to be applied to DUT by XMAX, YMAX and ZMAX.

```

REGISTER
  XMAX = #FF
  YMAX = #3FF

:

START #0
NOP      XB<0      YB<0      TP<0
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
ST1: JN11 ST1 R XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
JZD ST0
STPS

```

LMAX, HMAX

Specify LMAX and HMAX to use conditional operation instructions in the YB, YC, YS, YK, and Z operations. If LMAX and HMAX are not specified, the following are the default values:

LMAX=XMAX

HMAX=YMAX

Default Values of Address Operation

The default values of the address operation instructions are as follows:

XB<XB YB<YB XC<XC XS<XS XK<XK YC<YC YS<YS YK<YK

D3<D3 D4<D4 RF<RF N<N Z<Z X<XB Y<YB B<B

2. 5 Data Pattern Generating Instructions

2. 5. 1 Hardware Configuration for Data Pattern Generator Section

Figure 2-9 Data Pattern Generator Section Function Block Diagram (T5593)

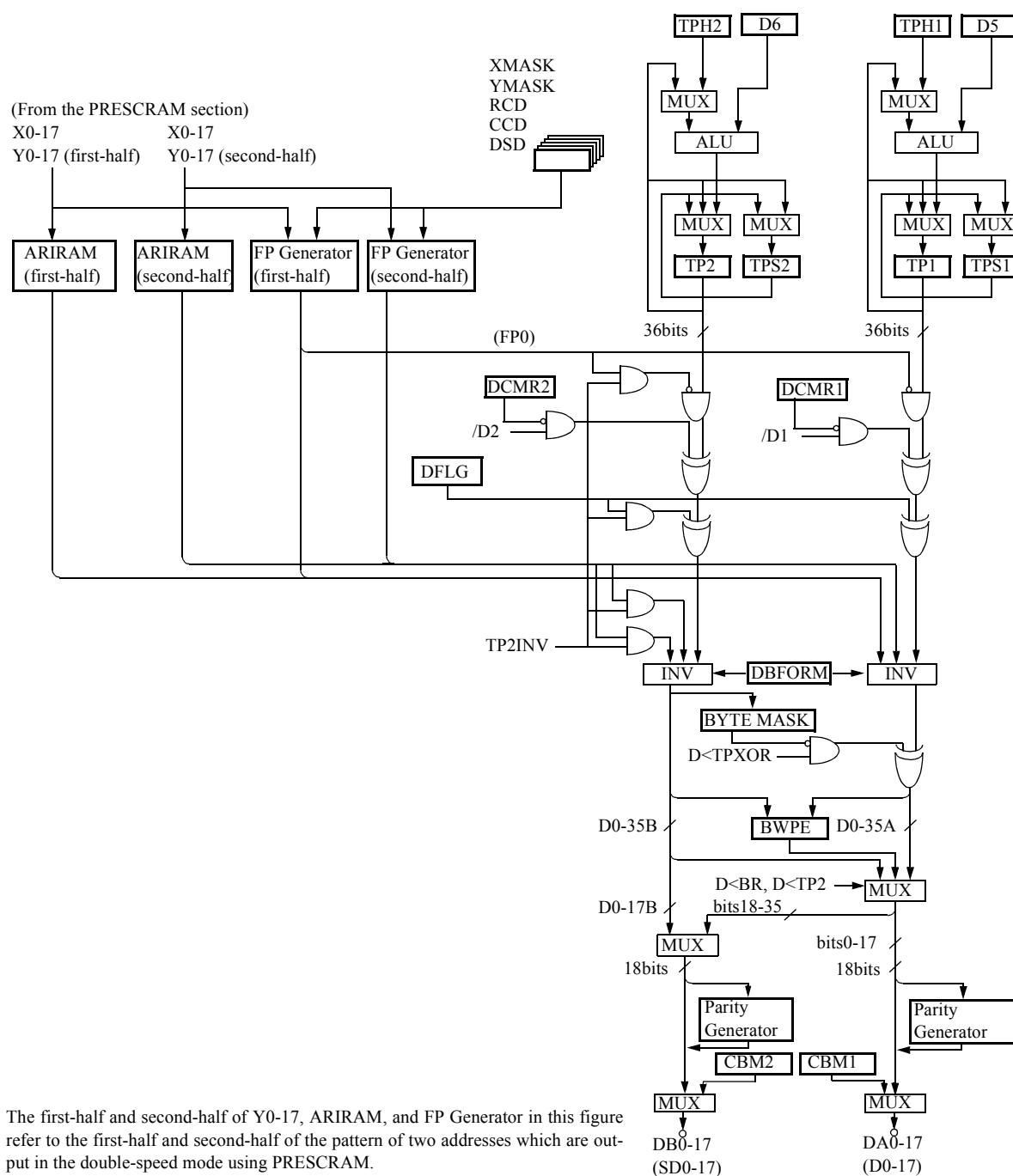
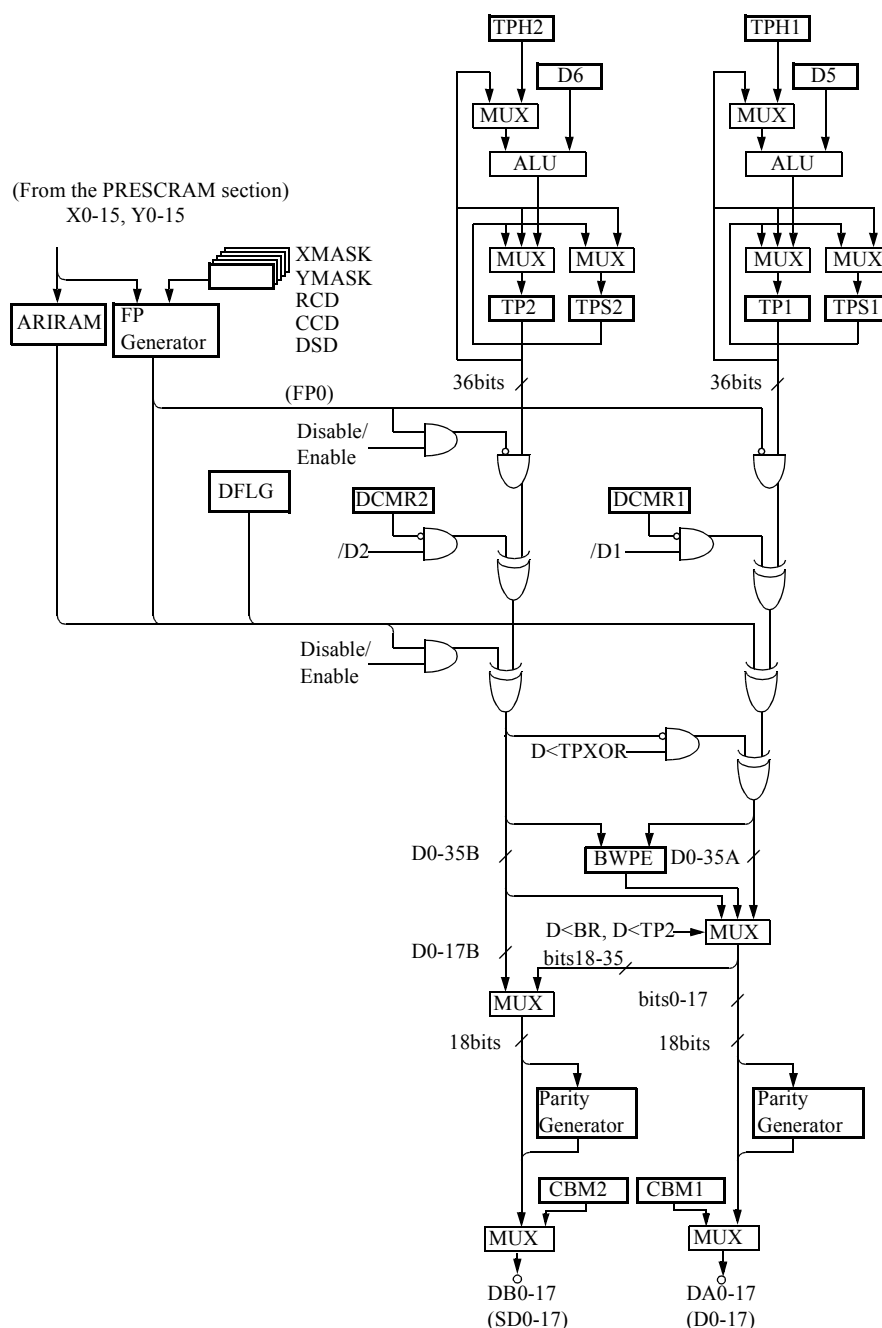


Figure 2-10 Data Pattern Generator Section Function Block Diagram (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)



TP, TP2

The TPs are used to generate comparison data (expected value) between the data to be applied to DUT and data to be output from DUT. TP2 is used to generate the mask data in the measurement of the DRAM that has the write-per-bit function.

TPH, TPH2

The TPH is a register which stores the initial value to be loaded into TP and TP2 registers. These can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program and by the REG statement in the test plan program.

DCMR, DCMR2

The DCMR is a register used to specify the mask of the bits to be reversed by the /D and /D2 of a data pattern generation instruction. Data will be logically reversed at a cycle describing the /D and /D2 of "0" bits in this register. This register can be set by the SET instruction, the DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test plan program. When this register is not set, DCMR=#0 and DCMR2=#0.

DFLG

When the DFLG register is "1," the value that the TP register inverts is output to DA0-17.

Data in register DFLG can be inverted by the JZD instruction.

When outputting the value which TP2 register reverses from DFLG register to DB0-17, specify TP2INV in the MODE statement.

FP Generator

The FP Generator is an address function using the addresses of X and Y. When the output of this address function is "1," the inverted value of the TP register is output to DA0-17.

When outputting the value which TP2 register reverses from FP Generator to DB0-17, specify TP2INV in the MODE statement.

When the double speed mode using prescrambler is specified, which Y address, in the first half or latter half of the double speed mode, should be used as the inverting condition of TP1 and TP2 data can be specified by using DBFORMn(n=1 to 4) in the MODE statement.

➔ For information on the double speed mode, refer to "8.19 Selecting operation mode of PRE-SCRAMBLER" in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

XMASK, YMASK

This register is used to specify the effective bits of X and Y to be used as an address function. These can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program and by the REG statement in the test plan program.

DSD, RCD, CCD

This register is used as offset values or comparison values of the address function. These can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program and by the REG statement in the test plan program.

D5, D6

D5 for TP and D6 for TP2 are used when data operation are executed with them. These can be set by the SET instruction, DSET instruction and REGISTER statement in the pattern program and by the REG statement in the test plan program.

ARIRAM (Area reverse)

This is used when a specific address described in ARIRAM inverts the pattern data DA0-17.

When outputting the value which TP2 register reverses from ARIRAM to DB0-17, specify TP2INV in the MODE statement.

When the double speed mode using prescrambler is specified, you can specify with DBFORMn(n=1 to 4) in the MODE statement which Y address, the first half or latter half of the double speed mode, should be used as the inverting condition of TP1 and TP2 data.

➔ For information on the double speed mode, refer to "8.19 Selecting operation mode of PRE-SCRAMBLER" in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Parity Generator

This is used to output DA0-7 odd parity to DA16 and DA8-15 odd parity to DA17 when PARITY is specified by the MODE statement.

The odd parity refers to the parity generation in which "1" is output to the parity bit (DA16 or DA17) when an even number of bits are flagged with "1" and "0" is output when an odd number of bits are flagged with "1".

The data parity generation on DB0-17 side is the same as above.

When TPM36 is specified with the MODE statement, no data parity is generated on DB0-17 side.

When using D16 or D17 as even parity for the input data or the expected data in DUT, specify </D16> or </D17> by the PIN statement.

PD1 = IN1</D16>

PC33 = OUT1</D16>

2. 5. 2 Configuration of Data Generation Instruction

The data generation part comprises the following instructions.

- TP, TP2 register field
- FP function field
- DSD, RCD, CCD field
- Data inversion field
- Data hold field
- Source data field
- BWPE field
- CBM address field
- DSC field

2. 5. 3 Outline of the Data Generation Instruction

The mnemonic instructions of the data generation part are shown below.

One instruction can be selected in the same cycle from among those in a group delimited by parentheses. The square in parentheses indicates that one instruction can be selected from among those in the group.

TP, TP2 register field

TP</TP	TP2<TP2
TP<TP*2	TP2<TP2*2
TP</TP	TP2</TP2
TP<0	TP2<0
TP<TP+1	TP2<TP2+1
TP<TP-1	TP2<TP2-1
TP<TPH	TP2<TPH2
TP<D5	TP2<D6
TP<TP'D5 (OR)	TP2<TP2'D6 (OR)
TP<TP&D5 (AND)	TP2<TP2&D6 (AND)
TP<TP"D5 (XOR)	TP2<TP2"D6 (XOR)
TP<TP+D5	TP2<TP2+D6
TP<TP-D5	TP2<TP2-D6
TP</D5	TP2</D6
TP<TPS	TP2<TPS2
TPS<TP	TPS2<TP2
TP<>TPS	TP2<>TPS2

It is possible to write TP1 for TP, TPS1 for TPS, and TPH1 for TPH.

FP function field

FP0	(Fix 0)
FP1	(Checker board)
FP3	(Diagonal)
FP4	(Invert diagonal)
FP5	(Diagonal2)
FP6	(Invert diagonal2)
FP7	(Mask parity)
FP8	(Row bar)
FP9	(Column bar)
FP10	(Row/column bar)
FPX	(Universal FP)

FP5, FP6, and FPX can be specified in T5593.

DSD, RCD, CCD field

DSD<DSD+1
RCD<RCD+1
CCD<CCD+1
RCD<RCD+1 CCD<CCD+1
DSD<0
RCD<0
CCD<0

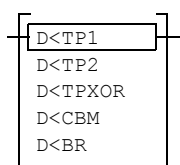
Data inversion field

[/D]
[/D2]

Data hold field

[PD]

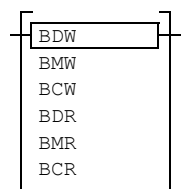
Source data field



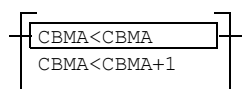
TPXOR represents TP1.XOR.and TP2.

BR is the output data from BWPE.

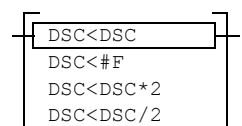
BWPE(Block Write Pseudo Emulator) field



CBM address field



DSC field



2. 5. 4 TP, TP2 Register Field

The TP, TP2 register field is the instruction to generate the pattern data and expected data to be written to DUT by the operation of the TP and TP2.

TPS and TPS2 registers are used as save-registers of TP and TP2 registers respectively. The values of TPS and TPS2 registers are not output to PDS (Pin Data Selector) directly. In order to output them, it is necessary to pass through TP and TP2 registers.

TP, TPS, TP2, and TPS2 registers are 36-bit registers, but when TPM 18 (default) is specified in MODE statement, only the lower 18 bits are output. In order to output all 36 bits, specify TPM 36 in the MODE statement.

When there is no specification about TPS and TPS2 registers, TPS and TPS2 registers operate the same as HOLD of TPS<TPS and TPS2<TPS2. (Because TPS<TPS and TPS2<TPS2 are not prepared as mnemonic, they cannot be specified.)

TP, TPH and TPS registers can be specified by TP1, TPH1, and TPS1 respectively.

TP<TP TP2<TP2 (Hold)

This instruction is the HOLD instruction of the TP, TP2 register. Specification of this instruction is omissible.

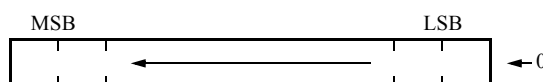
TP<TPH TP2<TPH2 (Load)

This instruction initializes the TP, TP2 registers, loading the contents of the TPH, TPH2 registers into the TP, TP2 registers.

TP<TP*2 TP2<TP2*2 (Shift left)

These are the instructions to shift 1 bit leftward and store the contents of TP and TP2 registers to TP and TP2 registers.

0 is always loaded on LSB side. (Not Rotate Shift)



TP<TP+1 TP2<TP2+1 (Increment)

These are the instructions to increment (+1) and store the contents of the TP and TP2 registers to TP and TP2 registers.

TP<TP-1 TP2<TP2-1 (Decrement)

These are the instructions to decrement (-1) and store the contents of the TP and TP2 registers to TP and TP2 registers.

TP<0 TP2<0 (Clear)

This instruction zeros (0) the contents of the TP, TP2 registers.

TP</TP TP2</TP2 (Compliment)

These are the instructions to invert and store the contents of the TP and TP2 to TP and TP2 registers.

TP<D5 TP2<D6

This instruction loads each contents of the D5 and D6 registers to the TP and TP2 registers.

TP<TP'D5 TP2<TP2'D6

This instruction stores the OR of the TP(TP2) register and the D5(D6) register into the TP(TP2) register.

TP<TP&D5 TP2<TP2&D6

This instruction stores the AND of the TP(TP2) register and the D5(D6) register to the TP(TP2) register.

TP<TP"D5 TP2<TP2"D6

This instruction stores the exclusive OR (XOR) of the TP(TP2) register and the D5(D6) register to the TP(TP2) register.

TP<TP+D5 TP2<TP2+D6

This instruction stores the arithmetic addition of the TP(TP2) register and the D5(D6) register to the TP(TP2) register.

TP<TP-D5 TP2<TP2-D6

This instruction stores the arithmetic subtraction of the TP(TP2) register and the D5(D6) register to the TP(TP2) register.

TP</D5 TP2</D6

This instruction stores each reverse data of the D5 register and D6 register to the TP and TP2 registers.

TPS<TP TPS2<TP2 (Save)

This instruction saves the contents of TP and TP2 registers to TPS and TPS2 registers respectively.

TP<TPS TP2<TPS2 (Recovery)

This instruction loads the contents from TPS and TPS2 registers into TP and TP2 registers respectively.

TP<>TPS TP2<>TPS2 (Swap)

These are the instructions to swap the contents of TP register and TPS register, and TP2 register and TPS2 register.

2. 5. 5 FP Function Field

The FP function field comprises the instructions used to generate the pattern data to be applied to the DUT and the corresponding reference data through logical operations on the X and Y addresses to be applied to the DUT. When the output of the function selected by this instruction is "1," the contents of the TP register are inverted. When inverting the contents of the TP2 register with the FP function, specify the TP2INV in the MODE statement.

When the double speed mode using prescrambler is specified, which Y address, in the first half or latter half of the double speed mode, should be used as the inverting condition of TP1 and TP2 data can be specified by using DBFORMn(n=1 to 4) in the MODE statement.

➔ For information on the double speed mode, refer to "8.19 Selecting Operation Mode of PRE-SCRAMBLER" in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

The functions and the instructions that can be selected are explained in the following.

FP0 (Fix 0)

When this instruction is selected, always "0" is output regardless of the contents of the TP (TP2) register and the values of X and Y addresses.

This instruction is used to generate background data for a test pattern like galloping or walking.

FP1(Checker board)

If this instruction is selected, the contents of the TP (TP2) register are inverted in accordance with the conditions of exclusive OR of the LSB of X and Y addresses applied to DUT.

		X				
		00	01	10	11	
Y	00	0	1	0	1	
	01	1	0	1	0	
	10	0	1	0	1	
	11	1	0	1	0	

$X0 \oplus Y0$

FP3 (Diagonal)

The contents of TP (TP2) register are inverted when the algebraic sum of the Y address to be applied to the DUT and the contents of the DSD register equals the X address.

In addition, comparison between the algebraic sum and the X address is performed within the effective bits shown by the XMAX register.

		DSD		X		
		00	01	10	11	
Y	00	0	1	0	0	
	01	0	0	1	0	
	10	0	0	0	1	
	11	1	0	0	0	

$(Y + DSD) = (X \text{ .AND. } XMAX)$
 (The figure on the left shows the case DSD = 1)

The data patterns output depending on the contents of the DSD register are illustrated in the following.

		X			
		00	01	10	11
Y	00	1			
	01		1		
	10			1	
	11				1

DSD=0

		X			
		00	01	10	11
Y	00		1		
	01			1	
	10				1
	11	1			

DSD=1

		X			
		00	01	10	11
Y	00			1	
	01				1
	10	1			
	11		1		

DSD=2

		X			
		00	01	10	11
Y	00				1
	01	1			
	10		1		
	11			1	

DSD=3

FP4 (Inversion diagonal)

This instruction generates the inversions of the diagonal patterns (generated by FP3). The contents of TP (TP2) register are inverted when the 1's complement of the algebraic sum of the Y address to be applied to the DUT and the contents of the DSD register equals the X address. In addition, comparison between the 1's complement and the X address is performed within the effective bits shown by the XMAX register.

		X			
		00	01	10	11
Y	00	0	0	1	0
	01	0	1	0	0
	10	1	0	0	0
	11	0	0	0	1

$$\overline{(Y + DSD).AND.XMAX} = (X .AND. XMAX)$$

(The figure on the left shows the case DSD = 1)

The patterns output depending on the contents of the DSD register are illustrated in the following.

		X			
		00	01	10	11
Y	00				1
	01			1	
	10		1		
	11	1			

DSD=0

		X			
		00	01	10	11
Y	00			1	
	01		1		
	10	1			
	11				1

DSD=1

		X			
		00	01	10	11
Y	00		1		
	01	1			
	10				1
	11			1	

DSD=2

		X			
		00	01	10	11
Y	00	1			
	01				1
	10			1	
	11		1		

DSD=3

FP5 (Diagonal 2)

The contents of TP (TP2) register are inverted when the algebraic sum of the Y address to be applied to the DUT and the contents of the DSD register equals the X address. In addition, comparison between the algebraic sum and the X address is performed within the effective bits shown by the AND of the XMAX register and the YMAX register.

2. 5 Data Pattern Generating Instructions

This instruction is similar to FP3 (Diagonal), except that the effective bits are different.

		DSD →				X				XMAX			
		000	001	010	011	100	101	110	111				
Y	000	0	1	0	0	0	1	0	0				
	001	0	0	1	0	0	0	1	0				
	010	0	0	0	1	0	0	0	1				
	YMAX 011	1	0	0	0	1	0	0	0				

$(Y + DSD).AND.(XMAX.AND.YMAX)$
 $= X.AND.(XMAX.AND.YMAX)$
 (The figure on the left shows the case DSD = 1)

The data patterns output depending on the contents of the DSD register are illustrated in the following:

		X				XMAX			
		000	001	010	011	100	101	110	111
Y	000	1				1			
	001		1				1		
	010			1				1	
	YMAX 011				1				1

DSD = 0

		X				XMAX			
		000	001	010	011	100	101	110	111
Y	000			1				1	
	001				1				1
	010	1				1			
	YMAX 011		1				1		

DSD = 2

FP6 (Inversion diagonal 2)

This instruction generates the inversions of the diagonal patterns generated by FP5 (Diagonal 2). The contents of TP (TP2) register is inverted when the 1's complement of the algebraic sum of the Y address to be applied to the DUT and the contents of the DSD register equals the X address. In addition, comparison between the 1's complement of the algebraic sum and the X address is performed within the effective bits shown by the AND of the XMAX register and the YMAX register.

This instruction is similar to FP4 (Inversion diagonal), except that the effective bits are different.

		X									
		000	001	010	011	100	101	110	111	XMAX	
Y	000	0	0	1	0	0	0	1	0		
	001	0	1	0	0	0	1	0	0		
	010	1	0	0	0	1	0	0	0		
	011	0	0	0	1	0	0	0	1		
YMAX											

$\xleftarrow{\text{DSD}}$

$$\overline{(Y + \text{DSD})} \cdot \text{AND}(\text{XMAX} \cdot \text{AND} \cdot \text{YMAX})$$

$$= \text{X} \cdot \text{AND}(\text{XMAX} \cdot \text{AND} \cdot \text{YMAX})$$

(The figure on the left shows the case DSD = 1)

The patterns output depending on the contents of the DSD register are illustrated in the following.

		X								XMAX	
		000	001	010	011	100	101	110	111		
Y	000				1				1		
	001			1				1			
	010		1				1				
	011	1				1					
YMAX											

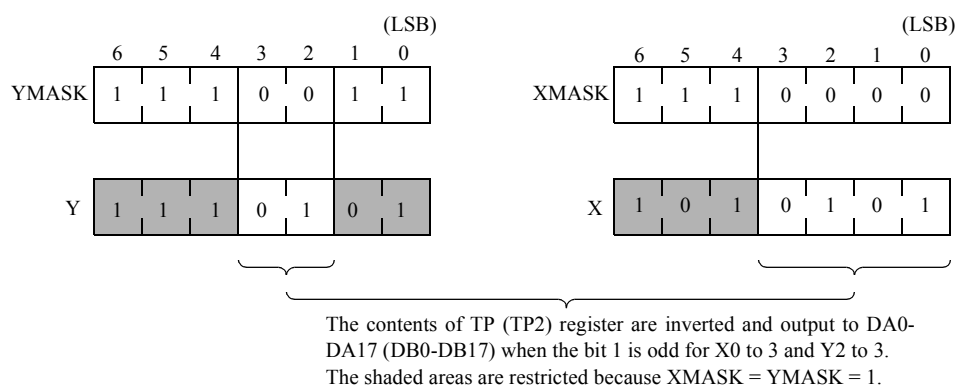
DSD = 0

		X								XMAX	
		000	001	010	011	100	101	110	111		
Y	000		1				1				
	001	1				1					
	010				1				1		
	011			1				1			
YMAX											

DSD = 2

FP7 (Mask parity)

The contents of TP (TP2) register are inverted when the number of "1" bits, which are not inhibited by the XMASK and YMASK registers, of the X and Y addresses to be applied to the DUT is odd.



Examples are shown below.

XMASK=#3FE YMASK=#3FE

$X0 \oplus Y0$ generates a checker board.

XMASK=#3FD YMASK=#3FD

$X1 \oplus Y1$ generates a double checker board.

XMASK=#0 YMASK=#3FF

X-parity results.

XMASK=#3FF YMASK=#0

Y-parity results.

FP8 (Row bar)

When this instruction is selected, the XMASK register compares the bit of "0" and the contents of the RCD at the X address to be applied to DUT and, if they coincide with each other, inverts the contents of the TP (TP2) register. When $(X.AND.XMASK) = RCD$, the contents of the TP (TP2) register are inverted.

Examples are shown below.

XMASK = #3FE RCD = #1

		X			
		00	01	10	11
Y	00	0	1	0	1
	01	0	1	0	1
	10	0	1	0	1
	11	0	1	0	1

Stripe pattern

XMASK = #3FD RCD = #2

		X			
		00	01	10	11
Y	00	0	0	1	1
	01	0	0	1	1
	10	0	0	1	1
	11	0	0	1	1

If X1 of the X address is "1," the contents of TP (TP2) register are inverted and output.

FP9 (Column bar)

When this instruction is selected, the YMASK register compares the bit of "0" and the contents of the CCD at the Y address to be applied to DUT and, if they coincide with each other, inverts the contents of the TP (TP2) register. When (Y.AND.YMASK), the contents of the TP(TP2) register are inverted.

Examples are shown below.

YMASK = #3FE CCD = #1

		X			
		00	01	10	11
Y	00	0	0	0	0
	01	1	1	1	1
	10	0	0	0	0
	11	1	1	1	1

Stripe pattern

FP10 (Row/column bar)

The contents of the TP (TP2) register are converted by the AND of the row and column bar.

XMASK = #3FE RCD = #1

YMASK = #3FE CCD = #1

		X			
		00	01	10	11
Y	00	0	0	0	0
	01	0	1	0	1
	10	0	0	0	0
	11	0	1	0	1

FPX (Universal FP)

Select the address function (FP0, FP1, FP3, FP4, FP5, FP6, FP7, FP8, FP9, and FP10) specified by the FPSEL register for the cycle which specifies this instruction. If DISABLE is specified in the FPSEL register, this instruction is the same as when not specified because the address function is not selected.

The FPSEL register can be specified in the REGISTER statement in the pattern program or the REG MPAT statement in the test plan program.

By specifying the FPSEL register in the REG MPAT statement, the address function can be selected from the test plan program.

Examples are shown below.

```

PRO pro_file_name
:
TEST 100
REG MPAT PC = #0
REG MPAT FPSEL = FP3      ; Selects FP3 for FPX.
MEAS MPAT mpat_file_name
:
TEST 200
REG MPAT PC = #0
REG MPAT FPSEL = FP4      ; Selects FP4 for FPX.
MEAS MPAT mpat_file_name
:
END

```

```

MPAT mpat_file_name
:
START #0
:
NOP FPX      ; FP3 is selected in TEST100 and FP4 is selected in TEST200.
:
STPS
END

```

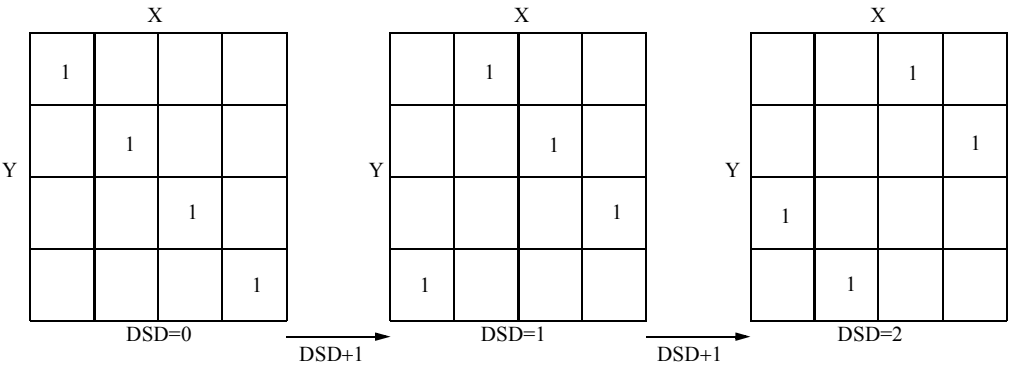
2. 5. 6 DSD Field

The DSD field comprises the following instructions that are used to modify the contents of the DSD, RCD, and CCD registers needed when pattern data are generated by means of address functions.

DSD<DSD+1

This instruction increments the values of the DSD register for pattern generation by FP3, FP4, FP5 or FP6.

This instruction is used for shifted-diagonal pattern generation.



RCD<RCD+1

This instruction increments the contents the RCD register used for generating FP8 and FP10.

CCD<CCD+1

This instruction increments the contents the CCD register used for generating FP9 and FP10.

RCD<RCD+1 CCD<CCD+1

This instruction increments the contents the RCD register and the CCD register in the same cycle.

DSD<0

This instruction is 0-clear the contents the DSD register.

RCD<0

This instruction is 0-clear the contents the RCD register.

CCD<0

This instruction is 0-clear the contents the CCD register.

 Tip

For the operating instructions of DSD, RCD, and CCD, only RCD<RCD+1 CCD<CCD+1 can be specified in the same step simultaneously. Otherwise, simultaneous specification is not allowed.

2. 5. 7 Temporary Inversion of Data (/D, /D2)(Data Inversion Field)

/D instruction inverts the data of the TP register.

The bits inverted with the /D instruction are specified by the DCMR.

The data of the TP register that corresponds to the bit of "0" of the DCMR register is inverted.

If DCMR is not specified, it results in DCMR=#0.

/D2 instruction inverts the data of the TP2 register.

The bits inverted with the /D2 instruction are specified by the DCMR2 register.

If DCMR2 is not specified, it results in DCMR2=#0.

REGISTER

DCMR = #3FF00

:

START #0

; DA0-17

NOP TP<0

; X

NOP

; #0

NOP /D

; #FF

DA8 to DA17 are not inverted.

2. 5. 8 Inversion with Data Flag (DFLG)

When DFLG = 1, the contents of the TP register are inverted.
Data in register DFLG can be inverted by the JZD instruction.

```

START  #0
NOP          XB<0  YB<0  TP<0

[
  ST0:JNI1    ST0  W  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB
  ST1:JNI1    ST1  R  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB

  JZD        ST0
  STPS
]
Processing in this step is executed twice.
The first time, the DFLG set to 0.
The second time, the data in TP register are inverted and output to DA0 to DA17 when the DFLG is set to 1.

```


Tip

When inverting the contents of the TP2 register with the DFLG, specify the TP2INV in the MODE statement.

2. 5. 9 Regional Inversion

➔ For more information on regional inversion, refer to [4. 3. 4 "Regional Inversion Test \(ARIRAM\)" on page 4-41](#).

2. 5.10 Saving Output Data (PD)(Data Hold Field)

Data in DA0 to DA17, DB0 to DB17 in the previous cycle output to DA0 to DA17, DB0 to DB17 unchanged in the cycle in which PD is entered.

◆ Example

		; TP0-35	DA0-17
NOP	TP<0	; X	X
NOP	TP</TP	; #0	#0
NOP	TP<0	; #FFFFFFFF	#3FFFF
NOP	<u>PD</u>	; #0	#3FFFF
NOP	<u>PD</u>	; #0	#3FFFF
NOP		; #0	#0

2. 5.11 Source Data Field

Source data field instructions are used to select in realtime the data to be output to PDS (Programmable Data Selector).

If D is specified for the test data in the DATA DEFINE statement in the pattern program, or the SELECT PDS DATA statement in the test plan program, data in DA0 to 17 is output to D0 to 17 of the pin statement, and data in DB0 to 17 is output to SD0 to 17.

In this case, the data to be output to DA0 to 17 and DB0 to 17 are selected as shown in [Table 2-15](#) by the Source data field instruction and specification by the MODE TPMn.

If FDA or FDB is specified as the test data, the data to be output from the PM is output to DA0 to 17, and DB0 to 17, without depending on the Source data field instruction.

If CBM1 or CBM2 is specified as the test data, the data to be output from the CBM is output to DA0 to 17, and DB0 to 17, without depending on the Source data field instruction.

If the Source data field instruction is omitted, the results are the same as when D<TP1 has been specified.

Table 2-15 Output Data to DA0 to 17 and DB0 to 17 when D is Specified for the Test Data

MODE TPM18		MODE TPM36		OUTPUT DATA	
35.....0		35.....0			
17....DB.....0	17.....DA.....0	17.....DB.....0	17.....DA.....0		
17....TP2.....0	17.....TP1.....0	35.....TP1.....0		D<TP1 (TP1 data, Default)	
17....TP2.....0	17.....TP2.....0	35.....TP2.....0		D<TP2 (TP2 data)	
17....TP2.....0	17...TPXOR...0	35.....TPXOR.....0		BYTEMASK DEFINE statement Not specified	D<TPXOR (TP XOR data)
35.....TPXOR.....0		35.....TPXOR.....0		BYTEMASK DEFINE statement Specified	
17....TP2.....0	17...BWPE.....0	35.....BWPE.....0		D<BR (BWPE data)	
35....CBM...18	17.....CBM.....0	35....CBM.....18	17.....CBM.....0	D<CBM (CBM data)	

TPXOR represents the results of operation of TP1.XOR.and $\overline{\text{TP2}}$.

BR means the output of Block write pseudo emulator. BWPE bits 32-35 are Fix "0."

CBM means the data stored in CBM memory.

CBM0-17 means bits 0-17 of CBM1 and CBM18-35 means bits 0-17 of CBM2.

2. 5.12 BWPE(Block Write Pseudo Emulator) Field

BWPE artificially emulates the operation of VRAM and SGRAM 8 Column × 8 I/O × 4 Byte Block Write and generates expected values. BWPE is composed of the following registers.

CD REG-FILE

4 words register file which stores Color Data (Block Write Data) under block writing.

BD REG-FILE

4 words register file which stores Data (Column Select Data) under block writing.

MD REG-FILE

4 words register file which stores Mask Data under block writing.

RF ADDRESS SEL(Register File ADDRESS SElect)

The register which selects 2 bits address input to the above each register file for each register file. It is selected by the BWPE ADDRESS statement.

Instructions to read/write the data to each register file are as follows.

BDW

BDW is the instruction to write TP1 register data into BD REG-FILE.

Write in 1 word specified by 2 bits address which is selected by BWPE ADDRESS statement in 4 words register.

BMW

BMW is the instruction to write TP2 register data into MD REG-FILE.

Write in 1 word specified by 2 bits address which is selected by BWPE ADDRESS statement in 4 words register.

BCW

BCW is the instruction to write TP1 register data into CD REG-FILE.

Write in 1 word specified by 2 bits address which is selected by BWPE ADDRESS statement in 4 words register.

BDR

BDR is the instruction to read register data, which is specified by 2 bits address selected by BWPE ADDRESS statement, from BD REG-FILE. Specification of D<BR is not necessary here.

BMR

BMR is the instruction to read register data, which is specified by 2 bits address selected by BWPE ADDRESS statement, from MD REG-FILE. Specification of D<BR is not necessary here.

BCR

BCR is the instruction to read register data, which is specified by 2 bits address selected by BWPE ADDRESS statement, from CD REG-FILE. Specification of D<BR is not necessary here.

The following shows the outline and the block diagram of 8 Column × 8 I/O × 4 Byte Block Write specification which will be artificially emulated with BWPE.

C0-7

C0-7 shows the word specified when Column Address LSB 3 bit, which will be "Don't Care" in 8 Column Block Write, is decoded. (Common in 4 blocks)
(Pre-SCRAM part output Y0-2 is used in BWPE.)

CS0-31

CS0-31 shows Column Select (Address Mask) bit which will be input to Block Write Cycle from Data I/O (DUT Data Pin).

(BD register file data is used in BWPE.)

D0-31

D0-31 shows Data I/O bit.

(CD register file data is used in BWPE.)

M0-31

D0-31 shows Mask data bit.

(MD register file data or TP2 register data is used in BWPE.)

Figure 2-11 Outline of 8 Column × 8 I/O × 4 Byte Block Write Specification

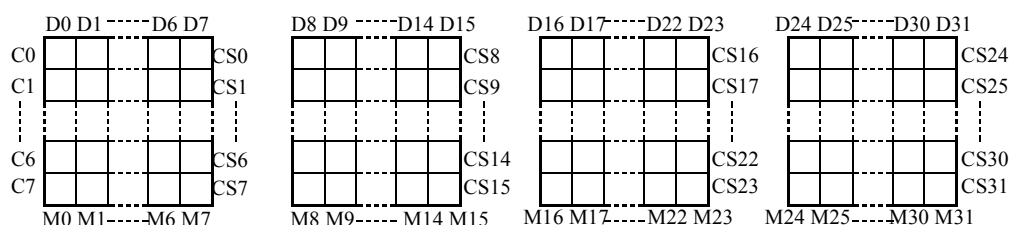
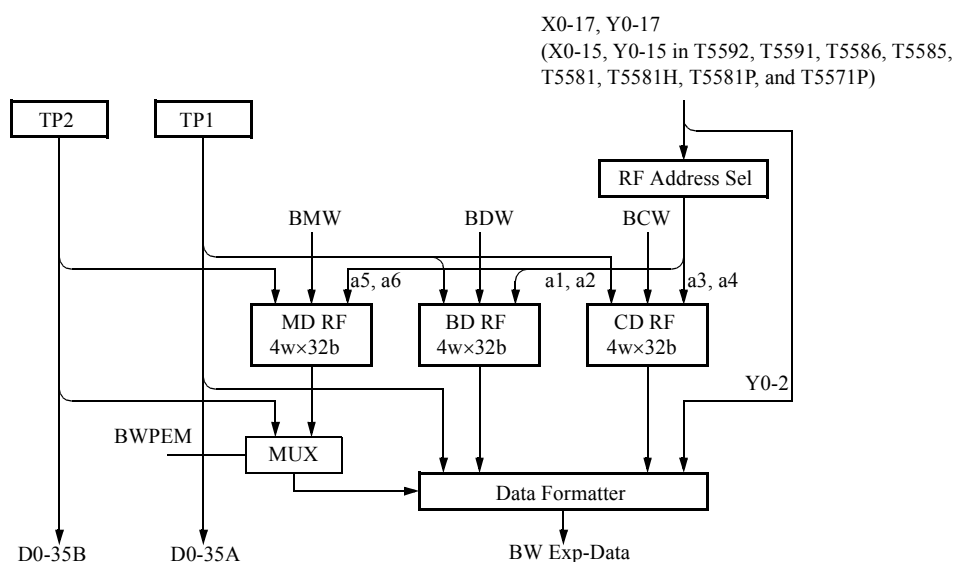


Figure 2-12 Block Diagram of BWPE Outline



Procedure and example when BWPE is used are shown below.

- Initial setting of BWPE
- Initialization of the test area of DUT
- Setting of DUT Color/Mask Reg. (Setting of BWPE CD/MD REG-FILE)
- Execution of block write to DUT (Setting of BWPE BD REG-FILE)

- Read of the block written data of DUT (Expected data generation by BWPE)
1. Initial setting of BWPE
Specify MODE TPM36 to use the output of TP/TP2 register at 36 bits. Specify the mask mode (MODE BWPEM) and RF ADDRESS SEL (BWPE ADDRESS statement) as necessary.
 2. Initialization of the test area of DUT
When the test area of DUT is initialized, no operation is executed for the BWPE.
 3. Setting of DUT Color/Mask Reg. (Setting of BWPE CD/MD REG-FILE)
Set the data to the Color/Mask Reg. inside the DUT.
For the BWPE, the same data is set to the CD/MD REG-FILE.
 4. Execution of block write to DUT (Setting of BWPE BD REG-FILE)
When the block write is executed to DUT, write the same data to the BD REG-FILE of BWPE.
Up to four data can be specified since the BD REG-FILE is the 4-word register file. The selection of four types data is done according to the address specified by the BWPE ADDRESS statement.
 5. Read of the block written data of DUT (Expected data generation by BWPE)
In the cycle which read the block written data from the DUT, the expected value from BWPE instead of TP data is output by the D<BR instruction. In this case, program the TP register so that the pattern when the DUT test area of (2) was initialized is to be generated.
Also, when the mask mode BWPEM was specified in (1), program the TP2 register so that the pattern when the block write was executed to the DUT of (4) is to be generated.
BWPE uses the data generated in the TP register for the data of masked bit at the DUT block write, and the data of the CD register for the data of not masked bit. Then BWPE prepares the expected data based on above data by using BD, MD and TP2 registers.

◆ Example

```

(1) MODE TPM36
    BWPE ADDRESS Y3,0,0,0,0,0

SDEF COUT = X<YC Y<XC

REGISTER
    IDX1=#mm
    IDX2=#nn
    D1=#8*1
    TPH =#FFFFFFFF
    TPH2=#33333333

    START #0
(2) NOP      XC<0      YC<0      TP<0
    JN11 .    W      XC<XC+1  YC<YC+1^CY

(3) NOP      XC<0      YC<0      TP<TPH  TP2<TPH2
    NOP      *2          BCW
    NOP      *3          BMW  TP<#55555555  D<TP2

(4) NOP      COUT XC<D1      BDW
    NOP      COUT XC<0      YC<0      BDW /D
    NOP      W COUT XC<XC+D1 YC<YC+1^CY
    JN12 .-1 W COUT XC<XC+D1 YC<YC+1^CY /D

(5) NOP      COUT XC<0      YC<0      TP<0*4
    JN11 .    R COUT XC<XC+1 YC<YC+1^CY D<BR

:

```

*1 The number of column addresses at 1 block write (8 addresses)

*2 Write the Color Reg. of DUT.

*3 Write the Mask Reg. of DUT.

*4 Write data at the initialization is occurred again.

2. 5.13 CBM Address Field

This is the instruction to generate CBM address, CBMA.

The data of CBM stored in the address of CBMA can be output to DA0-17 and DB0-17.

CBMA<CBMA(Hold)

This instruction is the HOLD instruction of the CBMA. Specification of this instruction is omisable.

CBMA<CBMA+1(Increment)

This instruction is the instruction to increment CBMA.

2. 5.14 DSC Field

The DSC register is used to control data when the 72-bit data mode is used.

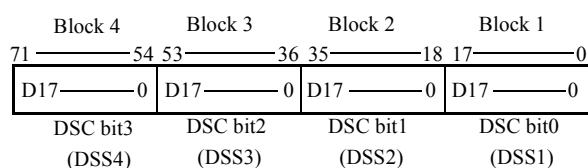
In the measurement of DUT having 36 bits or less data length, the 36-bit data of the TP register is output to 36 bits of DA0-17 and DB0-17 by specifying TPM36 in the MODE statement. Therefore, it is easy to apply bit-independent patterns to DUT.

In the measurement of DUT having more than 36 bits data length, use the 72-bit data mode to apply the bit-independent patterns to DUT.

The 72-bit data mode operates depending on specifications of DSS1-4 in the PIN statement, specifications of DFCAn DFCBn in the MODE statement and output of the control data from the DSC register.

The 72-bit data mode expands 18-bit data of DA0-17 and DB0-17 each to four blocks of 72-bit data at the PDS section and then controls each block in bit-correspondence manner through the 4-bit DSC register. (See the illustration below.)

The 18-bit data of DA0-17 and DB0-17 each is output as it is to the data of the block corresponding to the bit where the DSC register value is 1. Data of the block corresponding to the bit where the DSC register value is 0 is fixed to all 0 (#0) or all 1 (#3FFFF). (The selection of Data Format between all 0 and all 1 is made by the MODE DFCAn DFCBn statement.)



Data Format-1					Data Format-2				
71	54	53	36	35	18	17	0		
all 0	all 0	all 0	D17----0	DSC=#1	all 1	all 1	all 1	D17----0	
all 0	all 0	D17----0	all 0	DSC=#2	all 1	all 1	D17----0	all 1	
all 0	D17----0	all 0	all 0	DSC=#4	all 1	D17----0	all 1	all 1	
D17----0	all 0	all 0	all 0	DSC=#8	D17----0	all 1	all 1	all 1	

An example of the program is shown below. In this example, the TP register output data #1, #2, #4, #8, #10, ...#20000 four times and the DSC register output data #1, #2, #4, #8. It allows the applying of one set of 72-bit bit-independent patterns (#1, #2, #4, #8, #10, ...#8000000000000000000) to DUT.

Note that the TP register does not perform data rotation due to the shift instruction.

◆ **Pattern Program**

```

:
MODE DFCA 1
REGISTER
  IDX2=#2
  TPH1=#1
:

START #0
:

NOP          TP<TPH  DSC<#1
ST0: IDX11 #F  W TP<TP*2
JNI2 ST0 W TP<TPH  DSC<DSC*2
:

```

◆ **Test Plan Program**

```

:
P33-50  =IN1,...OUT1,STRB1,DSS1,...<D0-17>
P97-114 =IN1,...OUT1,STRB1,DSS2,...<D0-17>
P161-178=IN1,...OUT1,STRB1,DSS3,...<D0-17>
P225-242=IN1,...OUT1,STRB1,DSS4,...<D0-17>
:

```

DSC<DSC

This instruction is the HOLD instruction of the DSC register. Specification of this instruction is omissible.

DSC<#F

This instruction is the instruction to initialize the data of the DSC register to #F.

The specification and the meaning of this instruction are equivalent to those of the DSET instruction. DSC#F can be used together with the DSET instruction in the same register group (such as TP<#1) because DSC#F is independent as an instruction code. However, DSC<0 cannot be used together with a DSET instruction in the same register group (such as TP<#1) because it is a DSET instruction.

DSC<DSC*2

This instruction is the Rotate Shift Left instruction of the DSC register.

This instruction makes rotation shift operation. Executing this instruction sends data from DSC bit3 to DSC bit0. Therefore, when the data of the DSC register is #8, executing this instruction makes the data #1.

DSC<DSC/2

This instruction is the Rotate Shift Right instruction of the DSC register.

This instruction makes rotation shift operation. Executing this instruction sends data from DSC bit0 to DSC bit3. Therefore, when the data of the DSC register is #1, executing this instruction makes the data #8.

Tip

- When the ALPG is started by using the statement other than the following, #F is automatically loaded to the DSC register.
MEAS MPAT(1) filename
START MPAT (1)
- In the T5592, if CYPn (cycle palette instruction) is written in the program, the 72-bit data mode cannot be used.
- When the MSM mode is specified in the MODE statement, the 72-bit data mode cannot be used.

2. 5.15 Instructions and Execution Cycle

There are two types of data generation instructions depending on the cycle in which the results of the operation output to DA0 to DA17, DB0 to DB17, as follows:

Execution in the step in which instructions are described (Event time 1)

The following instructions are valid as soon as they are entered:

- FP function field instruction
- Data inversion field instruction
- Data hold field instruction
- Source data field instruction
- BWPE field instruction

◆ Example

```
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
```

Data in DA0 to DA17, DB0 to DB17 is inverted, if X0.XOR. Y0 is set to 1 for the address selected by X<XB and Y<YB described in the same step.

Execution in the step next to the one in which instructions are described (Event time 2)

The results of the calculation for TP, TP2, DSD, RCD, CCD, CBMA and DSC register are valid in the next step.

◆ Example

```

                                TP0-35
NOP  TP < 0                    ;x
NOP  TP < TP+1                ;#0 ← Operation result of TP<0
NOP                                ;#1 ← Operation result of TP<TP+1

```

2. 5.16 Notes on Program Generation**SET instruction**

This SET instruction can be used only in the normal mode (1WAY).

Use the DSET instruction in the 2WAY/4WAY interleaved mode.

Duplicate conditions for inverting data bits

When the following conditions are executed an odd number of times, data bits (D0 to D17 or DB0 to DB17) are inverted. When the conditions are executed an even number of times, the data bits are not inverted.

The inversion condition for data is as follows:

/D, /D2, DFLG, FP function, ARIRAM (regional inversion)

Default Value for Data Operation

TP1<TP1, TP2<TP2, D<TP1, CBMA<CBMA, DSC<DSC

2. 6 Control Bit

2. 6. 1 Configuration of Control Instruction

The control section comprises of the following instructions.

TS field

TS1 to TS16

MUT field

T5593

R, W, M1-2, C0-23, MMA, MMB

T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

R, W, M1-2, C0-15, MMA, MMB

Address control field

XT, YT

SCRAM control field

SCROFF

PRESCRAM control field

PSCROFF

ARIRAM control field

ARIOFF

PM control field

PM/2, PM/4

Cycle palette field (Available for the T5593 and T5592)

T5593

CYPA1 to 16 (CYP1 to 16), CYPB1 to 16

T5592

CYPA1 to 16 (CYP1 to 16)

DBM control field (Available for the T5593 and T5592)

INHDBM

X Address save field (Available for T5593).

XASV, XALD

2. 6. 2 TS Field

The period and phase of the clock, used to generate the address, data, and clock to be applied to DUT, can be changed at each cycle by the following statement in the pattern program.

```

:
NOP TSn(or /Tn)   n=1 to 16
:

```

When nothing is described, TS1 will be allocated.

```

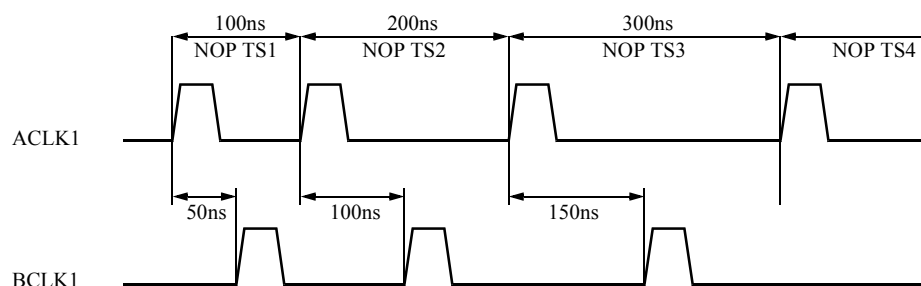
PRO program-name
:
RATE  = 100NS, 200NS, 300NS, 400NS
ACLK1 =  0NS:4
BCLK1 =  50NS, 100NS, 150NS, 200NS
:
END

```

```

MPAT program-name
:
NOP TS1
NOP TS2
NOP TS3
NOP TS4
:
END

```



2. 6. 3 MUT Field

In the MUT field, the following control bits can be described.

T5593

R, W, M1, M2, C0-23, MMA, MMB

T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

R, W, M1, M2, C0-15, MMA, MMB

R, W, M1, M2 and C0-23 are used in the following applications.

- Data patterns on the control terminals of $\overline{WE}$, $\overline{CS}$, and $\overline{OE}$ of DUT.
- Control of Enable/disable of logical comparison. (CPE1 to 4)
- Control of Driver Enable/Disable of I/O pins. (DRE1 to 2)
- Real time inversion of driver time. (RINV)
- Timing set switching and inhibition of address multiplex in page mode test and nibble mode test. (R, W, M1, M2)
- Specifying the cycles of real time data swapping (RDSM). (C0)
- Specifying the cycle to perform Hi-Z comparison in the realtime Hi-Z mode.
- Specifying the cycle to control STRB and clock in auto-scan mode.

2. 6. 4 Data Pattern on the DUT Control Terminals

Describing the control bits (R, W, M1-2, C0-23) in pattern programs generates the data patterns on control terminals of WE, CS, and OE of DUT.

Specify the data to be applied to each pin by the PIN statement in a test plan program.

```

:
:
PD1=IN1,/RZO,BCLK2,(CCLK2),<WT>;/WE
:
:

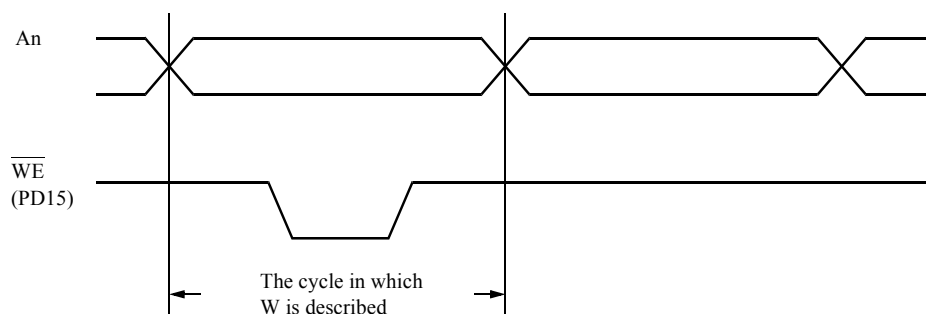
```

↓
Indicates W.

```

MPAT M16K
:
:
START #0
NOP          XB<0  YB<0  TP<0
ST0: JN11  ST0  W  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
ST1: JN11  ST1  R  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
JZD  ST0
STPS
END

```



2. 6. 5 Specifying the Cycles for Performing Logical Comparison

The data output from DUT is logically compared with the expected value in the pattern program at a cycle describing the control bits (R, W, M1, M2, C0-23, PREF) specified by CPE1 to 4.

PREF is output according to the following conditions.

$R_t .OR. (R_{t-1} .AND. M1t)$

```

MPAT M16K
REGISTER
XMAX = #7F
YMAX = #7F
CPE1 = R ; FIXL, FIXH, R, W, M1 to M2, CO to C23, PREF can be selected.
:
:
START #0
NOP          XB<0 YB<0 TP<0
ST0: JN11  ST0 W   XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
ST1: JN11  ST1 R*1 XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
JZD  ST0
STPS
END

```

**1 The output from the DUT is logically compared with the expected data in the cycle specified by CPE_n in the REGISTER statement.*

Tip

- Unless the control bit is specified by CPE1, CPE2, CPE3 or CPE4, R is selected.
- Logical comparison cannot be performed during the STPS or STFL cycle.

2. 6. 6 Specifying to Turn Driver ON or OFF in I/O Operation

The control bit turns the driver on or off to test the DUT whose DIN and DOUT are in the I/O structure. The I/O mode for pins is specified by the pin condition statement of the test plan program. The driver operates in the cycle during which the control bit specified by the DRE1, DRE2 is entered in the pattern program, and it is placed in the Hi-Z state during other cycles.

```

DREL1 = 100NS
DRET1 = 200NS
:
:
P33-40=IN1, OUT1, IOC*1, XOR, ACLK1 BCLK1, STRB1, (DRE1), (DRECLK1), DRERZ, <D0-7>
:
:

```

**1 Specifying I/O*


```

MPAT SAMPLE
REGISTER
:
:
DRE=W ; FIXL, FIXH, R, W, M1 to M2, C0 to C23 can be selected.
:
:
START #0
NOP
ST0: JN11 ST0 W*1 XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
ST1: JN11 ST1 R XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
JZD ST0
STPS
END

```

**1 Only the drivers describing "W" which is specified in the DRE cycle are enabled.*

Tip

- When selecting DRE patterns by the pin conditional statement of I/O pin, specify the pattern by DRE1 (pattern of DRE1) and DRE2 (pattern of DRE2). If no pattern is specified, DRE1 will be specified.
- When selecting DREL/T clocks by the pin conditional statement of I/O pin, specify the pattern by DRECLK1(DREL/T1) to DRECLK256(DREL/T256). If no pattern is specified, DRECLK1 will be specified.

2. 6. 7 Specifying Real Time Inversion of the Driver

The driver pattern from ALPG is output inverted at a cycle describing the control bits (R, W, M1-2, C0-23), specified by RINV in pattern program. Specify RINV by the pin conditional statement to designate pins for real time inversion.

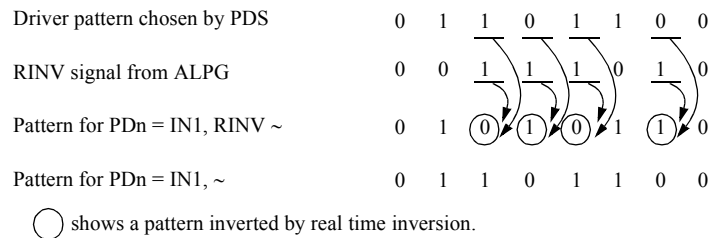
PD1 = IN1, NRZA, ACLK1, RINV, <D0>

```

MPAT SAMPLE
REGISTER
:
:
RINV=C0 ; FIXL, FIXH, R, W, M1-2, and C0-23 can be selected.
:
START #0
NOP
NOP C0*1
NOP
:

```

*1 The driver pattern from ALPG is output inverted only at a cycle describing the control bits "C0" specified by RINV.

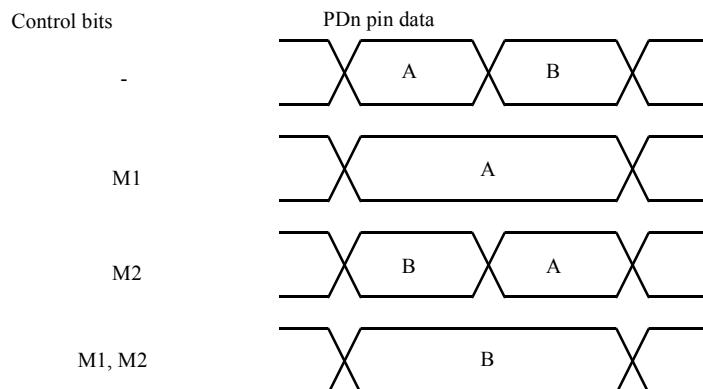


2. 6. 8 Address Multiplexing Control

The control bits R, W, M1, and M2 provide the following controls.

When SDM (Swap data mode) is specified by the PIN conditional statement, pin data are controlled as follows:

PDn = IN1,, SDM, <A, B>



The function used to switch the timing set by the specification of SDM is not provided.

To specify the cycle which carries out Hi-Z comparison in the realtime Hi-Z mode, specify the control bit (FIXL, FIXH, R, W, M1, M2, and C0 to C23) specified to the RHZA and RHZB registers in the REGISTER statement to the cycle for which Hi-Z comparison should be performed.

For STRB to carry out specification of the realtime Hi-Z mode to the pin and Hi-Z comparison, the following connecting conditions are set in the PIN statement of the test plan program.

Hi-Z comparison of the cycle specified by the RHZA register

RHZA1, RHZA2, RHZA129, and RHZA130

Hi-Z comparison of the cycle specified by the RHZB register

RHZB1, RHZB2, RHZB129, and RHZB130

Tip

- The realtime Hi-Z mode can be executed in the T5593.
- If DATA, TP1, or TP2 is selected for the delay data in the SET PATDELAY statement in the test plan program, realtime Hi-Z mode cannot be used.

◆ Example Usage of the Data Strobe Signal (DQS)

```

:
DQS=IN1, DNRZ, BCLK1, CCLK1, IOC, OUTL1, [STRB3, CPE3], RHZA1*1, EXPA1, @
[STRB131, CPE3], RHZB129*2, EXPB129, <D0, SD0>
:

```

*1 In odd-phase STRB on the STRB1 to 128 side, Hi-Z comparison is carried out in the cycle in which the control bit specified for the RHZA register is described.

*2 In odd-phase STRB on the STRB129 to 256 side, Hi-Z comparison is carried out in the cycle in which the control bit specified for the RHZB register is described.

```

MPAT SAMPLE <2WAY>

REGISTER
  CPE1 = R      ; DQ
  CPE3 = C3     ; DQS
  RHZA = C20    ; STRB3
  RHZB = C21    ; STRB131
  :

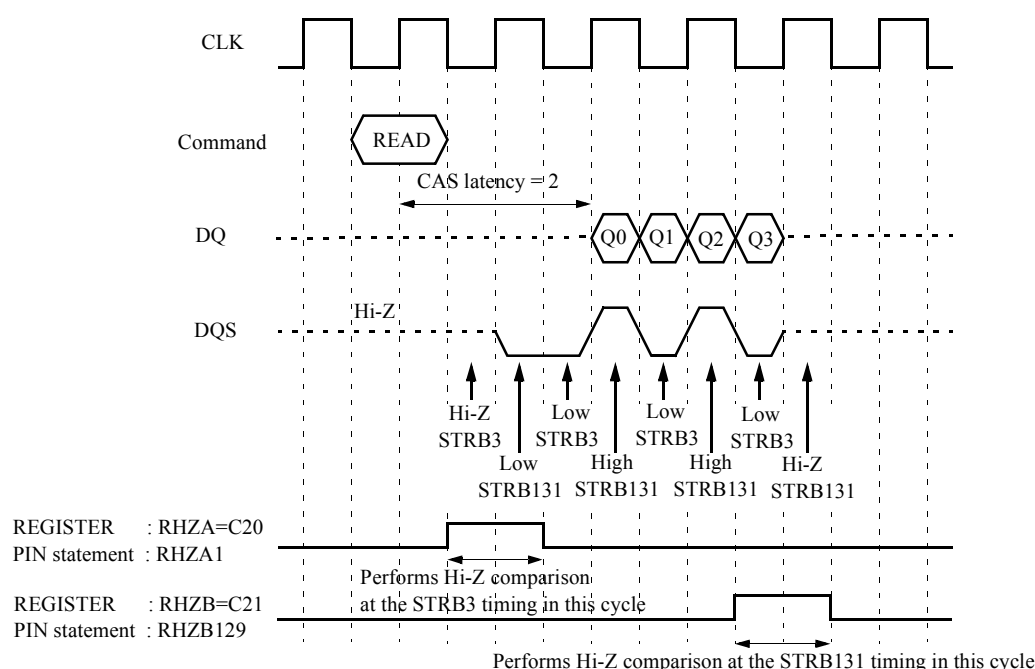
START      #0
  NOP : TP<#00  TP2<#00
  :
  :                                     ; STRB3 STRB131
  NOP :   C3  C20*1                       ; Hi-Z  Low
  : R C3                               /D2      ; Low  High
  NOP : R C3                               /D2      ; Low  High
  :   C3  C21*2                       ; Low  Hi-Z
  :
  STPS

END

```

**1 Hi-Z comparison is carried out in the cycle in which the control bit specified for the RHZA of the REGISTER statement is described.*

**2 Hi-Z comparison is carried out in the cycle in which the control bit specified for the RHZB of the REGISTER statement is described.*



2. 6.11 Specifying the Cycle which Controls the STRB and Clock in Auto-scan Mode

To specify the cycle which controls the STRB and clock in the auto-scan mode, specify the control bit (FIXL, FIXH, R, W, M1, M2, and C0 to C23) specified to the ASCNT and ASINI registers in the REGISTER statement to the cycle to be controlled.

In the specified cycle, control the STRB and clock as follows:

Cycle specified by the ASCNT register

Increments/decrements the values of the STRB and clock.

Cycle specified by the ASINI register

Initializes the values for STRB and clock.

Tip

The auto-scan mode can be executed in the T5593.

◆ Example

```

MPAT SAMPLE

REGISTER
  ASINI=C0 ; FIXL, FIXH, R, W, M1, M2, and C0 to C23 can be selected.
  ASCNT=C1 ; FIXL, FIXH, R, W, M1, M2, and C0 to C23 can be selected.
  :
START      #0
  :
  NOP C0*1
  NOP C1*2
  NOP C1*2
  NOP C1*2
  :
STPS
END

```

**1 The values for the STRB and clock are initialized in the cycle in which the control bit specified by ASINI in the REGISTER statement is described.*

**2 The values of the STRB and clock are incremented or decremented in the cycle in which the control bit specified by ASCNT in the REGISTER statement is described.*

2. 6.12 Address Control Field

XT

Outputs the XT register value to the X address temporarily.

YT

Outputs the YT register value to the Y address temporarily.

Address control field is used to output logical data temporally for the address line on the DUT measurement which executes the command setting from the address line.

2. 6.13 SCRAM Control Field

SCROFF

Descramble of the address is temporally off.

When converting the address generated in the ALPG with the address descramble memory, SCROFF is used to stop the next conversion temporally.

It is used to execute the command setting from the address line together with XT, YT.

```
MPAT filenameA

REGISTER
  XT=n
  YT=n

SCRAM filenameB
  :
  :
  NOP  X<XB Y<YB
  NOP  X<XB Y<YB      XT YT SCROFF*1
  NOP  X<XB Y<YB
  :
  :
```

**1 The XT register value is output to the X address and the YT register value is output to the Y address without descrambling using only this cycle.*

2. 6.14 PRESCRAM Control Field

PSCROFF

The address conversion operation of the PRE-SCRAM section is set to OFF temporarily.

2. 6.15 ARIRAM Control Field

ARIOFF

To reverse the data with the regional inversion memory is temporally off.

It is used to stop the regional inversion by ARIRAM temporally when measuring the device which executes the command and mode setting from the data line.

2. 6.16 PM Control Field

PM control field is used for the ROM test with byte/word converted function.

When using PM in the EXP mode, the data length specified in the AFM BITSIZE statement can be changed in the pattern program with PM/2 to 1/2 of the originally specified data length and with PM/4 to 1/4 of the originally specified data length. (Refer to the table below.)

Specification by AFM BITSIZE statement	PM/2	PM/4
8, 9	-	-
16, 18	8, 9	-
32, 36	16, 18	8, 9

The line shows to forbid the use.

The data in PM is accessed as below.

(a) AFM BITSIZE=32		(b) AFM BITSIZE=16		(c) AFM BITSIZE=8	
↓		↓		↓	
Ad d r e s s		Ad d r e s s		Ad d r e s s	
D33...D26 D25...D18 D15...D8 D7...D0		D33.....D18 D15...D8 D7...D0		D33.....D18 D15...D8 D7...D0	
0	D C	0	"0" B A	0	"0" "0" A
1	H G	1	"0" D C	1	"0" "0" B
2	L K	2	"0" F E	2	"0" "0" C
3	P O	3	"0" H G	3	"0" "0" D
⋮		⋮		⋮	E
					F
					G
(d) AFM BITSIZE=36		(e) AFM BITSIZE=18		(f) AFM BITSIZE=9	
↓		↓		↓	
Ad d r e s s		Ad d r e s s		Ad d r e s s	
D35 D34 D33...D26 D25...D18 D17 D16 D15...D8 D7...D0		D35.....D18		D35.....D18 D17 D16 D15...D0	
0	d c D C b a B A	0	"0" b a B A	0	"0" "0" a "0" A
1	h g H G f e F E	1	"0" d c D C	1	"0" "0" b "0" B
2	l k L K j i J I	2	"0" f e F E	2	"0" "0" c "0" C
3	p o P O n m N M	3	"0" h g H G	3	"0" "0" d "0" D
⋮		⋮		⋮	

2. 6.17 Cycle Palette Field

CYPAn(CYPn)

CYPBm

CYPAn and CYPBm is the specification of cycle palette A and B respectively.

The specification ranges for n and m are as follows:

Table 2-16 Specification Ranges for n and m of CYPAn and CYPBm

Test system	CYPAn	CYPBm
T5593	1 to 16	1 to 16
T5592 with the cycle palette 16 expansion option	1 to 16 ^{*1}	Cannot be specified
T5592 without the cycle palette 16 expansion option	1 to 8	Cannot be specified

^{*1} When specifying n=9 to 16, specify CYPC15 in the MODE statement.

The cycle palette instruction is used to switch the ALPG output data assignment and pin data for each cycle in real-time.

Pin data can be switched in each cycle by specifying CYPAn and CYPBm in the pattern program. (The CYPAn and CYPBm are valid only for the cycle specified in the pattern program.)

The correspondence of CYPAn and CYPBm to pin data is specified by the data selector of the PIN statement and CYPA and CYPB.

Tip

- CYPAn can be described as CYPn.
- For the cycle in which the description of CYPAn and CYPBm is omitted, CYPA1 and CYPB1 are allocated.
- If the pin data for the cycle palette is not specified in the pin statement, the previously specified data remains.
 - ➔ For more information, refer to "Chapter 1 Pin Condition Setting Statement" in [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).
- The settings for CYPAn and CYPBm, which is described in field C(Sub PG instruction) are invalid. CYPAn and CYPBm, which are described in field B(Main PG instruction), are set for both the main PG and sub PG.
- The Cycle palette is available in the T5593 and T5592.

Limitation

- In the T5592, the cycle palette and 72 bit data mode cannot be used at the same time.
- If the cycle palettes CYPAn to CYPAn are specified in T5592, MUT signal C15 cannot be used.
- Two or more CYPAn cannot be described in the same step.
Two or more CYPBm cannot be described in the same step.
CYPAn and CYPBm can be described in the same step.

◆ Example

```

PRO   program-name
      :
      P33=...,CYPA,CYP1,<D0>,<D1>,<D2>,<D3> ; *1
      P34=...,CYPB,CYP1,<SD0>,<SD1>,<SD2>,<SD3> ; *2
END

```

*1 CYP1=D0, CYP2=D1, CYP3=D2, and CYP4=D3 are specified.

*2 CYPB1=SD0, CYPB2=SD1, CYPB3=SD2, and CYPB4=SD3 are specified.

```

MPAT program-name
      :
      NOP CYPA1 CYPB2 ; D0 is output to P33 and SD1 is output to P34.
      NOP CYPA2 CYPB3 ; D1 is output to P33 and SD2 is output to P34.
      NOP CYPA3 CYPB4 ; D2 is output to P33 and SD3 is output to P34.
      NOP CYPA4 CYPB1 ; D3 is output to P33 and SD0 is output to P34.
      :
END

```

2. 6.18 DBM Control Field

INHDBM

If PPAT, PDRE or PCPE is specified in the PIN statement and the DBM pattern data is output, INHDBM outputs the ALPG pattern data in the cycle where INHDBM is written.

Specifying INHDBM allows the DBM pattern data output and ALPG pattern data output to be switched in real-time.

Used when you want to use the temporary ALPG pattern output during the DBM pattern output.

```

MPAT   program-name
      :
START  #0
      IDX11 #6 DBMINC
      NOP    DBMINC R INHDBM *1
      NOP    DBMINC
      NOP
      :

```

*1 Only in this cycle, the DBM pattern data output is prohibited and the ALPG pattern data is output.



Tip

- For pins, for which DBMFX is specified, in the PIN statement, the specification of INHDBM is invalid.

-
- *INHDBM is available for the T5593 and T5592.*
-

2. 6.19 X Address Save Field

In a memory device which has two or more banks, each bank can be controlled individually (multi-bank operation).

By interleaving each bank, such operations as after giving a row address to bank A, giving a row address to another bank B and coming back to and giving a column address to bank A and reading data become possible.

When testing such operations, row address which are given to the FM (X address from ALPG) and row address which are used for the inversion conditions of the FP function and ARIRAM are row addresses of the bank which is not used for reading data.

In order to solve such a problem, the X Address Save File instruction is used.

For example, the row address of the bank which is used for reading data can be used in the FM, FP function and ARIRAM by saving the row address of each bank and loading the saved row addresses in the cycle in which data is read.

XASV

XASV saves the row address of a specified cycle (X address from ALPG).

Input an active command to the bank and specify the cycle to which the row address is given.

A row address can be saved for each bank by specifying the bank address on the row side with the BANK SAVE ADDRESS statement. (Up to 64 row addresses can be saved.)

XALD

XALD loads the row address saved by XASV (X address from ALPG) in the cycle in which XALD is specified.

Specify for all cycles, from the cycle, which inputs a read command that gives a column address to the bank, to the cycle that reads data.

Row addresses can be loaded for each bank by specifying the bank address on the column side by using the BANK LOAD ADDRESS statement.

Tip

- *The X Address Save Field can be executed in the T5593.*
- *XASV and XALD can be described in the same step. In this case, XALD is executed after XASV is executed.*
- *The X addresses that are loaded by the XALD instruction are output according to the specification in the SELECT PRESCRAM ADDRESS statement.*

- When XASV is used in the 2WAY interleave mode, there are certain restrictions.

➔ For more information, refer to [6. 5. 19 "Restrictions when Specifying the XASV Instruction" on page 6-57.](#)

◆ Example Usage

The following example generates addresses for the FM, FP function and ARIRAM by reading the multi-bank operations of the following conditions.

Burst type = sequential

Burst length = 4

CAS latency = 2

```
MPAT SAMPLE    <2WAY>

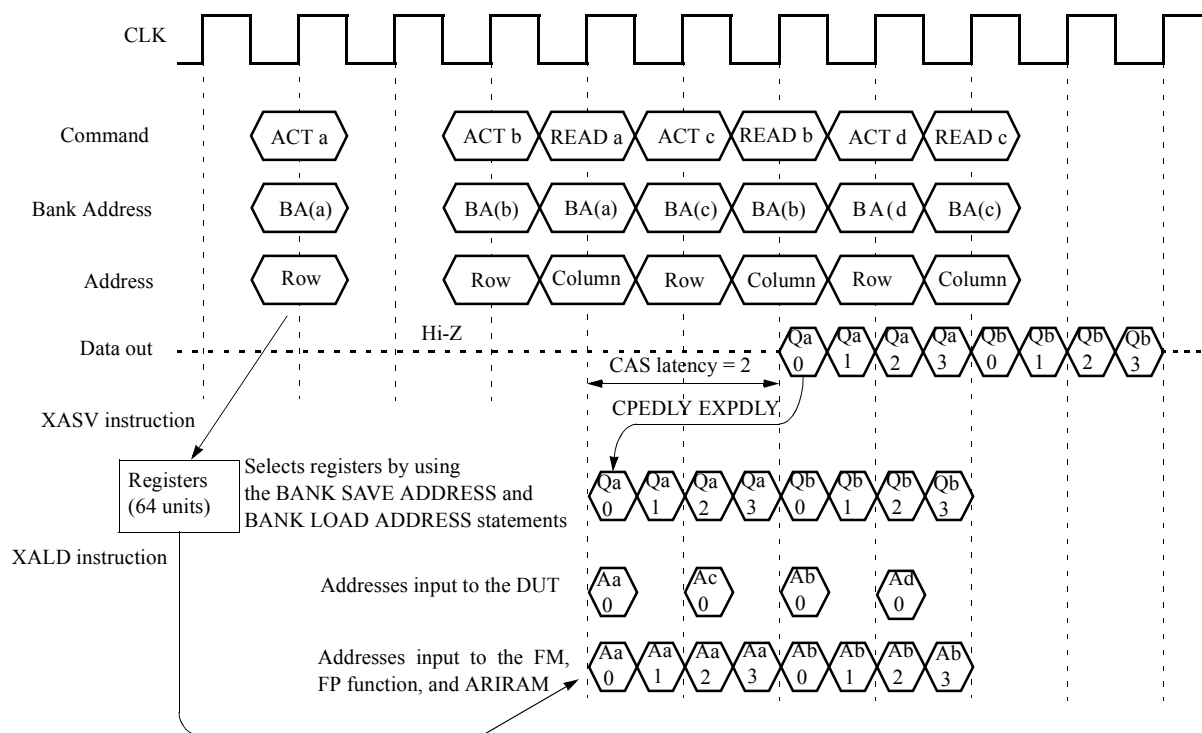
SDEF ACT  = C0  ; Activate command
SDEF READ = C1  ; Read command

ADDRESS DEFINE
  X13-14 = B0-1          ; Row Bank address
  Y13-14 = N0-1          ; Column Bank address
  XY=ZN

BANK SAVE ADDRESS X13,X14      ; Row Bank address selection
BANK LOAD ADDRESS Y13,Y14     ; Column Bank address selection

START #0
  NOP :                               ; Command      Row address
  :                               ;
  :                               ;
  NOP : ACT    XASV                B<#0      ; [BANK A] Activate, Save
  :                               ;
  :                               ;
  NOP : ACT    XASV                B<B+1      ; [BANK B] Activate, Save
  :                               ;
  :                               ;
  : READ      XALD R    B<B+1      ; [BANK A] Read,      Load
  NOP : ACT    XASV XALD R    N<N+1      ; [BANK C] Activate, Save, [BANK A] Load
  :                               ;
  : READ      XALD R    B<B+1      ; [BANK B] Read,      Load
  NOP : ACT    XASV XALD R    N<N+1      ; [BANK D] Activate, Save, [BANK B] Load
  :                               ;
  : READ      XALD R                ; [BANK C] Read,      Load
  :                               ;
  :                               ;
STPS

END
```

Figure 2-13 Read Operation of Multi-bank Operation

2. 7 ROM Test Pattern Data Program Section

This section describes the syntax of the ROM test pattern data program section. The ROM test pattern data program section begins with AFM PATTERN statement and ends with an END statement, a data declaration statement, or a test pattern program.

2. 7. 1 Programming Format

Describe a ROM test pattern in the following format:

```
MPAT program-name
:
AFM PATTERN                ---- (1)
  BITSIZE = m,n            ---- (2)
  AMAX   = x1,y1          ---- (3)
  AMIN   = x2,y2          ---- (4)
  FMA    = x3,y3          ---- (5)
  XX
  XXXX                ---- (6)
  XX!a
  XXXX!b
  XX__XXXX, XX!c__XXXX!d
    <Tab>      <Space>
:
END
```

Statements (2) to (6) must come after the AFM PATTERN statement.

2. 7. 1. 1 AFM PATTERN Statement

Function: This is a ROM test pattern start declaration statement; it can be entered in the common area and each module area.

A ROM test pattern must be defined after the MODE statement and REGISTER statement.

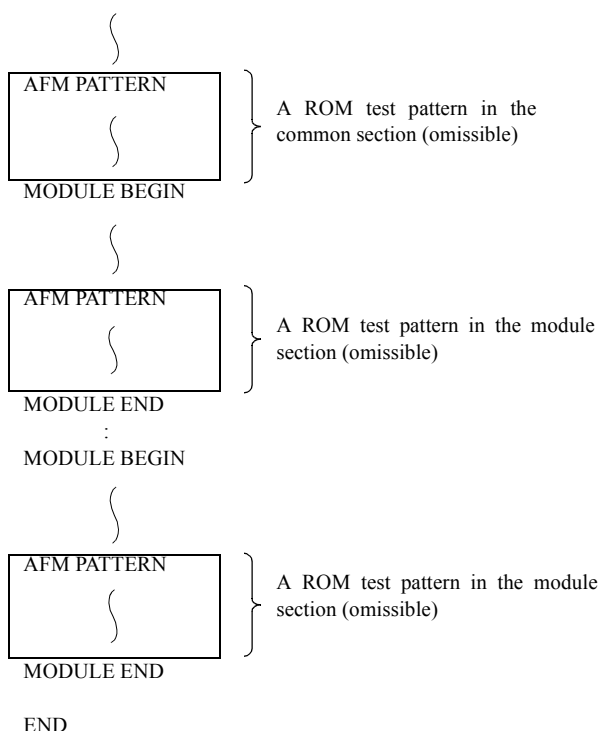
Syntax

```
AFM PATTERN
```

Example

(1) A Description Example of the AFM PATTERN Statement

MPAT program-name



2. 7. 1. 2 BITSIZE Statement

Function: This statement is a compilation control instruction. This is a compiler-controlling statement to define the transfer mode of ROM test pattern data.

This statement can be defined only once before ROM test pattern data.

If the BITSIZE statement is omitted, m=8 and n=1 by default.

m and n are compiled with a current radix.

Syntax

```
BITSIZE = m[,n]
```


Argument: m

Specifies the configuration of a ROM test pattern in the memory.

m=8	8bit
m=9	9bit
m=16	16bit
m=18	18bit
m=32	32bit
m=36	36bit

n

Specifies the number of patterns in a single piece of data.

When n is not specified, n=1 by default.

m=8	$1 \leq n \leq 4$
m=9	$1 \leq n \leq 4$
m=16	$1 \leq n \leq 2$
m=18	$1 \leq n \leq 2$
m=32, 36	$1 \leq n \leq 1$

2. 7. 1. 3 AMAX Statement

Function: This statement is a compilation control instruction. This statement is used to define the maximum X and Y address values.

This statement can be entered only once before ROM test pattern data.

Syntax

```
AMAX = x1[, y1]
```

Description

x₁ and y₁ are restricted as follows:

$0 \leq x_1 \leq 65535$

$0 \leq y_1 \leq 65535$

X and Y must not exceed the maximum X and Y determined by the AFM ADDRESS statement of a test plan program.

If the AMAX statement is not specified, operation is performed at the maximum X and Y determined by the AFM ADDRESS statement in a test plan program. Any one of X and Y can be specified by the following method:

AMAX = x_1 ; Specify X only.

AMAX = 0, y_1 ; Specify Y only.

2. 7. 1. 4 AMIN Statement

Function: This statement is a compilation control instruction. This statement is used to define the minimum X and Y address values.

This statement can be entered only once before ROM test pattern data.

Syntax

```
AMIN =  $x_2$  [,  $y_2$ ]
```

Description

x_2 and y_2 are restricted as follows:

$0 \leq x_2 \leq 65535$ and $x_2 \leq x_1$

$0 \leq y_2 \leq 65535$ and $y_2 \leq y_1$

If this statement is omitted, $x=0$ and $y_2=0$.

2. 7. 1. 5 FMA Statement

Function: This statement is a compilation control instruction. Data coming after this statement is compiled as ROM test pattern data until another compilation control instruction appears. This statement also specifies the start address of PM which transfers ROM test pattern data.

If this statement is not specified, it assumes the same value as that specified by the AMIN statement.

Syntax

```
FMA =  $x_3$  [,  $y_3$ ] or ROMPAT =  $x_3$  [,  $y_3$ ]
```

Argument: x_3

Specifies the start address of X.

y_3

Specifies the start address of Y.

Description

Up to 500 FMA statements can be entered in the same program.

 x_3 and y_3 are compiled with a current radix. x_3 and y_3 must fall in the range specified by the AMAX and AMIN statements.

Example

(1) Usage of the FMA Statement

```

AFM PATTERN
:
:
FMA =  $x_1$ ,  $y_1$  ----- (1)
#4
#6
:

FMA =  $x_2$ ,  $y_2$  ----- (2)
#7
#8
:
:
(500 pcs maximum)

```

2. 7. 1. 6 ROM Test Pattern Data

Function: This generates the ROM test pattern data repeatedly.

Syntax

```

p1[!n1][,p2[!n2]]-----

```

Argument: p₁, p₂

Specify ROM test pattern data.

n₁, n₂Specify how many times p₁, p₂ should be repeated. When this is omitted or n₁=0, n₁=1 is assumed.

Description

p_1 , p_2 , n_1 , and n_2 are compiled with a current radix.

They can assume the following characters:

- 0 Numbers from 0 to 9
- Alphabetical characters from A to F
- # (Compilation in hexadecimal regardless of the current radix)
- \$ (Compilation in octal regardless of the current radix)

However, even though the current radix is 16, if the starting character is an alphabetical character, an error will result. In this case, begin with 0 or #. The maximum number of characters one line can accommodate is 132.

Entries of ROM test patterns are explained below in conjunction with PM.

Pattern Data Delimiter

Tabs, commas (,) and spaces can be used as delimiters to separate pattern data and to handle each as a unit data.

◆ X, Y Address only

```
MPAT SAMPLE1

AFM PATTERN
AMAX      = #F, #F
FMA       = #0, #0

2
#AA
#55!2
#11, #22 #33!2 #44

END
```

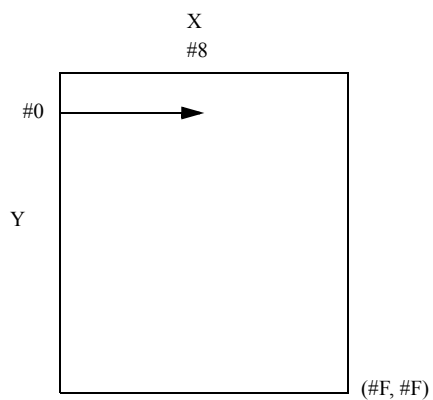
The following pattern is obtained.

Address		Data	
X	Y	Bit 0	Bit 7
		↓	↓
#0	#0	00000010	(2)
#1	#0	10101010	(#AA)
#2	#0	01010101	(#55)
#3	#0	01010101	(#55)
#4	#0	00010001	(#11)
#5	#0	00100010	(#22)
#6	#0	00110011	(#33)
#7	#0	00110011	(#33)
#8	#0	01000100	(#44)
#9	#0		
:	:		
#F	#F		

↑ No data
 ↓

("!2" causes the same data to be repeated twice.)
 ("!2" causes the same data to be repeated twice.)

Correspondence with PM is as shown below.



2. 7. 2 Example of Programming

A program to transfer a 64 K ROM tester pattern to PM is exemplified below.

```
PRO program-name
:
:
RESET AFM ALL
AFM ADDRESS=X0-7, Y0-7
AFM BITSIZE=8
AFM ACTION=PM>EXP
SEND FM SAMPLE1
MEAS MPAT SAMPLE1
:
:
:
END

MPAT SAMPLE1

REGISTER
XMAX=#FF
YMAX=#FF
IDX1=#FFFE
TPH =#0
START #0
NOP      XB<0 YB<0 TP<0
:
:
:
AFM PATTERN
BITSIZE=8
AMAX=#FF,#FF
AMIN=#0,#0
FMA =#0,#0
#AA!#10000

END
```

The following pattern is obtained.

Address		Data	
X	Y	Bit 0	Bit 7
#0	#0	↓	↓
#1	#0	10101010	10101010
#2	#0	10101010	
#3	#0		
⋮	⋮		
#FF	#0		
#0	#1		
#1	#1		
⋮	⋮		
#FF	#FF	10101010	

Pattern 10101010 is transferred to PM. This pattern can be used as a driver pattern or expectation pattern.

Tip

In order to generate the expected value pattern from PM, specify "SELECT PDS DATA FDA/FDB" in the test pattern program or specify "PDS DATA = FDA/FDB" with DATA DEFINE statement in the pattern program.

In the above format, the ALPG and DBM patterns are entered together in a single pattern program. It is also possible for only the DBM pattern to be entered.

Tip

- *Individual specification for MAIN or SUB is allowed only in the following cases.*
In the 1WAY mode used with the T5581
In the 1WAY or 2WAY mode with the T5591
In the 1WAY mode used with the T5593
- *In the common section or the same module section, DBM PATTERN statement and MAIN DBM PATTERN statement, or DBM PATTERN statement and SUB DBM PATTERN statement cannot be described at the same time.*
- *When PCPE/PCPE1/PCPE129 is specified in pin statement, set the CPE signal by DBM so that it becomes "0" in STPS/STFL cycles.*

2. 8. 2 DBM CHANNEL Statement

Function: This statement specifies a DBM data bit number.

Syntax

```
DBM CHANNEL i
DBM CHANNEL i,j
DBM CHANNEL i-j
```

Description

Specify this statement only once between the [MAIN/SUB] DBM PATTERN statement and the corresponding DBMA statement. (The above specification cannot be omitted if the DBM pattern is entered.)

Ascending and descending continuous specification and random specification can be used together in the range of 0 to 143.

A channel number is processed in decimal regardless of the current radix.

Example

(1) DBM CHANNEL Specification

```
DBM CHANNEL 0-35
DBM CHANNEL 0,18-35,54-72
DBM CHANNEL 0-17,36-53
DBM CHANNEL 0,1,2,3,8-11
DBM CHANNEL 35-0
DBM CHANNEL 53-36,17-0
```

(2) Entry for Each Module

```
MODULE BEGIN
DBM PATTERN
  DBM CHANNEL 0-7
  DBMA #0
    00001100
    11111111
    :
    START #0
    NOP
    :
    STPS
MODULE BEGIN
MODULE END
DBM PATTERN
  DBM CHANNEL 8-15
  DBMA #100
    HHHHLLLL
    0000LLLL
    :
    START #100
    NOP
    :
    STPS
MODULE END
```

2. 8. 3 DBM PINSEL Statement

Function: This statement is the declaration to define tester pin number corresponding to bit number of DBM specified in DBM CHANNEL statement. Specify this statement directly following the DBM CHANNEL statement.

Syntax

```
DBM PINSEL (CHTYPEt, BITMODEn)
    Dn=Pm
    :
```

Argument: CHTYPEt

Declares the channel number type of using test head.

CHTYPE3

This is described when the 352 CH test head is used.

CHTYPE4

Description when the 1600 (576IO) CH or 800 (288IO) CH test head is used.

CHTYPE5

Description when the 1600 (288IO) CH or 800 (144IO) CH test head is used.

CHTYPE9

This is described when the 2880 CH test head is used.

CHTYPE10

This is described when the 1344 CH test head is used.

CHTYPE12

This is described when the 3072 CH test head is used.

CHTYPE13

This is described when the 3072 CH test head with the high-speed clock option is used.

When the DBM PINSEL statement is described, it cannot be omitted.

BITMODEn

Declares the bit mode of the test head to be used.

BITMODE1

This can be specified for CHTYPE3, CHTYPE4, CHTYPE5, CHTYPE9, CHTYPE10, CHTYPE12, or CHTYPE13.

BITMODE2

This can be specified for CHTYPE3, CHTYPE 10, CHTYPE12 and CHTYPE13.

BITMODE3

This can be specified for CHTYPE3 and CHTYPE10.

BITMODE4

This can be specified for CHTYPE3 and CHTYPE10.

The BITMODE specification can be omitted. The default is BITMODE1.

Dn

Logical DBM bit number. The bit defined by the DBM CHANNEL statement can be specified.

Pm

Tester pin number. Specify within the following range:

CHTYPE3	P1 to P512
CHTYPE4	P1 to P256
CHTYPE5	P1 to P128
CHTYPE9	P1 to P256
CHTYPE10	P1 to P256
CHTYPE12	P1 to P256
CHTYPE13	P1 to P256

Description

Since the pattern program is not linked with the socket program, simultaneous measurement is not expanded for the pin direction (P1 through P512). (Hardware performs the expansion to CHILDA through CHILDH.) Therefore, the expansion of the pin direction must be defined to conform to the socket program.

Refer to ["Correspondence between DBM bit and tester pin numbers" on page 2-209](#) and define the expansion so that the parallel test can be performed. The table shows physical pin connections between the DBM bit and PIN. Logical connections can be made by using the DBM CHANNEL and DBM PINSEL statements.

Tip

The DBM PINSEL statement cannot be omitted for the T5593 and T5592 test systems.

For test systems other than the T5593 and T5592, the DBM PINSEL statement can be omitted. If omitted, the DBM channel numbers will be equal to the DBM bit numbers.

Specifying the tester pin number (Pm)

There are two ways to specify the tester pin number as follows:

1. When A or B is specified for the tester pin number:

This specification is used when BITMODE2 or BITMODE4 is specified.

In this case, assign port A and port B to different Dn's even if the tester pin numbers are the same.

- When assigning tester pin numbers to port A:

Pm_1A

Individual specification.

Pm_1A , Pm_2A or Pm_1, m_2A

Multiple specifications. No limit is placed on the number of specifications.

Pm_1-m_2A

Continuous specification. Either ascending or descending order of pin numbers can be specified.

Pm_1A , Pm_2A , Pm_3-Pm_4A

A combination of multiple and continuous specifications can be specified.

- When assigning tester pin numbers to port B

Pm_1B

Individual specification.

Pm_1B , Pm_2B or Pm_1, m_2B

Multiple specifications. No limit is placed on the number of specifications.

Pm_1-m_2B

Continuous specification. Either ascending or descending order of pin numbers can be specified.

Pm_1B , Pm_2B , Pm_3-Pm_4B

Multiple specification and continuous specification can be combined.

2. When A or B is not specified for the tester pin number:

This specification is used when BITMODE1 or BITMODE3 is specified.

In this case, assign port A and port B to the same Dn if the tester pin numbers are the same.

Pm_1

Individual specification.

Pm_1 , Pm_2 or Pm_1, m_2

Multiple specifications. No limit is placed on the number of specifications.

Pm_1-Pm_2 or Pm_1-m_2

Continuous specification. Either ascending or descending order of pin numbers can be specified.

Pm_1 , Pm_2 , Pm_3-Pm_4

Multiple specification and continuous specification can be combined.

Pin number must be defined for all the bit specified by the DBM CHANNEL statement. A pin number which does not exist in ["Correspondence between DBM bit and tester pin numbers" on page 2-209](#) cannot be specified.



Tip

- The pin number has A or B specification when BITMODE2 or BITMODE4 is specified. In other cases, the pin number has no A or B specification.

Examples where those parameters cannot be specified, are as follows:

```
DBM PINSEL(CHTYPE3, BITMODE1)
    D1 = P33A, P97A ;A or B cannot be specified for the pin number.
```

```
DBM PINSEL(CHTYPE3, BITMODE2)
    D1 = P33, P97 ; A or B must be specified for the pin number.
```

```
DBM PINSEL(CHTYPE4)
    D2 = P21A, P161A ; A or B cannot be specified for the pin number.
```

- *MAIN or SUB of the DBM PINSEL statement is determined according to the MAIN or SUB specification of the DBM PATTERN statement.*
- *Pin numbers that are assigned to the same DBM number cannot be defined to multiple Dn's. Examples where those parameters cannot be specified, are as follows: P1 or P225 cannot be specified because both of them use DBM60.*

```
DBM PINSEL(CHTYPE4)
    D1 = P1, P2
    D2 = P225
```

- *Also, the following examples cannot be specified.*

```
DBM PINSEL(CHTYPE3, BITMODE2)
    D1 = P33A, P97B ;A and B cannot be mixed in one line.
```

```
DBM PINSEL(CHTYPE4)
    D2 = P21, P21 ;The same pin number cannot be specified.
```

Correspondence between DBM bit and tester pin numbers

- T5581, T5581H, T5581P, and T5571P (CHTYPE4/5)

Pin No.	DR PIN		--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
	I/O PIN		33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	--
DBM	PPAT/PCPE/PDRE	PortA/PortB	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Pin No.	DR PIN		75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93
	I/O PIN		97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	--
DBM	PPAT/PCPE/PDRE	PortA/PortB	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38
Pin No.	DR PIN		21	22	23	24	25	26	27	28	29	30	65	66	67	68	69	70	71	72	73
	I/O PIN		161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	--
DBM	PPAT/PCPE/PDRE	PortA/PortB	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58
Pin No.	DR PIN		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	I/O PIN		225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	--
DBM	PPAT/PCPE/PDRE	PortA/PortB	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78

--" indicates that the pin number or the DBM bit is not assigned.

• T5591 (CHTYPE3, BITMODE1)

			DR PIN											I/O PIN														
Pin No.			1	2	3	4	5	6	7	8	9	10	11	33	34	35	36	37	38	39	40	41	42	43				
			257	258	259	260	261	262	263	264	265	266	267	289	290	291	292	293	294	295	296	297	298	299				
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	--	0	1	2	3	4	5	6	7	8	9	--				
		PortB	0	1	2	3	4	5	6	7	8	9	--	0	1	2	3	4	5	6	7	8	9	--				
			--	--	--	--	--	--	--	--	--	--	--	0	1	2	3	4	5	6	7	8	9	--				
Pin No.			17	18	19	20	21	22	23	24	25	26	27	49	50	51	52	53	54	55	56	57	58	59				
			273	274	275	276	277	278	279	280	281	282	283	305	306	307	308	309	310	311	312	313	314	315				
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	--	10	11	12	13	14	15	16	17	18	19	--				
		PortB	10	11	12	13	14	15	16	17	18	19	--	10	11	12	13	14	15	16	17	18	19	--				
			--	--	--	--	--	--	--	--	--	--	--	10	11	12	13	14	15	16	17	18	19	--				
Pin No.			65	66	67	68	69	70	71	72	73	74	75	97	98	99	100	101	102	103	104	105	106	107				
			321	322	323	324	325	326	327	328	329	330	331	353	354	355	356	357	358	359	360	361	362	363				
DBM	PPAT/PCPE	PortA	20	21	22	23	24	25	26	27	28	29	--	20	21	22	23	24	25	26	27	28	29	--				
		PortB	20	21	22	23	24	25	26	27	28	29	--	20	21	22	23	24	25	26	27	28	29	--				
			--	--	--	--	--	--	--	--	--	--	--	20	21	22	23	24	25	26	27	28	29	--				
Pin No.			81	82	83	84	85	86	87	88	89	90	91	113	114	115	116	117	118	119	120	121	122	123				
			337	338	339	340	341	342	343	344	345	346	347	369	370	371	372	373	374	375	376	377	378	379				
DBM	PPAT/PCPE	PortA	30	31	32	33	34	35	36	37	38	39	--	30	31	32	33	34	35	36	37	38	39	--				
		PortB	30	31	32	33	34	35	36	37	38	39	--	30	31	32	33	34	35	36	37	38	39	--				
			--	--	--	--	--	--	--	--	--	--	--	30	31	32	33	34	35	36	37	38	39	--				
Pin No.			129	130	131	132	133	134	135	136	137	138	139	161	162	163	164	165	166	167	168	169	170	171				
			385	386	387	388	389	390	391	392	393	394	395	417	418	419	420	421	422	423	424	425	426	427				
DBM	PPAT/PCPE	PortA	40	41	42	43	44	45	46	47	48	49	--	40	41	42	43	44	45	46	47	48	49	--				
		PortB	40	41	42	43	44	45	46	47	48	49	--	40	41	42	43	44	45	46	47	48	49	--				
			--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	--				
Pin No.			145	146	147	148	149	150	151	152	153	154	155	177	178	179	180	181	182	183	184	185	186	187				
			401	402	403	404	405	406	407	408	409	410	411	433	434	435	436	437	438	439	440	441	442	443				
DBM	PPAT/PCPE	PortA	50	51	52	53	54	55	56	57	58	59	--	50	51	52	53	54	55	56	57	58	59	--				
		PortB	50	51	52	53	54	55	56	57	58	59	--	50	51	52	53	54	55	56	57	58	59	--				
			--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	--				
Pin No.			193	194	195	196	197	198	199	200	201	202	203	225	226	227	228	229	230	231	232	233	234	235				
			449	450	451	452	453	454	455	456	457	458	459	481	482	483	484	485	486	487	488	489	490	491				
DBM	PPAT/PCPE	PortA	60	61	62	63	64	65	66	67	68	69	--	60	61	62	63	64	65	66	67	68	69	--				
		PortB	60	61	62	63	64	65	66	67	68	69	--	60	61	62	63	64	65	66	67	68	69	--				
			--	--	--	--	--	--	--	--	--	--	--	60	61	62	63	64	65	66	67	68	69	--				
Pin No.			209	210	211	212	213	214	215	216	217	218	219	241	242	243	244	245	246	247	248	249	250	251				
			465	466	467	468	469	470	471	472	473	474	475	497	498	499	500	501	502	503	504	505	506	507				
DBM	PPAT/PCPE	PortA	70	71	72	73	74	75	76	77	78	79	--	70	71	72	73	74	75	76	77	78	79	--				
		PortB	70	71	72	73	74	75	76	77	78	79	--	70	71	72	73	74	75	76	77	78	79	--				
			--	--	--	--	--	--	--	--	--	--	--	70	71	72	73	74	75	76	77	78	79	--				

--" indicates that the pin number or the DBM bit is not assigned.

• T5591 (CHTYPE3, BITMODE2)

			DR PIN											I/O PIN										
Pin No.			1	2	3	4	5	6	7	8	9	10	11	33	34	35	36	37	38	39	40	41	42	43
			257	258	259	260	261	262	263	264	265	266	267	289	290	291	292	293	294	295	296	297	298	299
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	--	0	1	2	3	4	5	6	7	8	9	--
		PortB	40	41	42	43	44	45	46	47	48	49	--	40	41	42	43	44	45	46	47	48	49	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	0	1	2	3	4	5	6	7	8	9	--
Pin No.			17	18	19	20	21	22	23	24	25	26	27	49	50	51	52	53	54	55	56	57	58	59
			273	274	275	276	277	278	279	280	281	282	283	305	306	307	308	309	310	311	312	313	314	315
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	--	10	11	12	13	14	15	16	17	18	19	--
		PortB	50	51	52	53	54	55	56	57	58	59	--	50	51	52	53	54	55	56	57	58	59	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	10	11	12	13	14	15	16	17	18	19	--
Pin No.			65	66	67	68	69	70	71	72	73	74	75	97	98	99	100	101	102	103	104	105	106	107
			321	322	323	324	325	326	327	328	329	330	331	353	354	355	356	357	358	359	360	361	362	363
DBM	PPAT/PCPE	PortA	20	21	22	23	24	25	26	27	28	29	--	20	21	22	23	24	25	26	27	28	29	--
		PortB	60	61	62	63	64	65	66	67	68	69	--	60	61	62	63	64	65	66	67	68	69	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	20	21	22	23	24	25	26	27	28	29	--
Pin No.			81	82	83	84	85	86	87	88	89	90	91	113	114	115	116	117	118	119	120	121	122	123
			337	338	339	340	341	342	343	344	345	346	347	369	370	371	372	373	374	375	376	377	378	379
DBM	PPAT/PCPE	PortA	30	31	32	33	34	35	36	37	38	39	--	30	31	32	33	34	35	36	37	38	39	--
		PortB	70	71	72	73	74	75	76	77	78	79	--	70	71	72	73	74	75	76	77	78	79	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	30	31	32	33	34	35	36	37	38	39	--
Pin No.			129	130	131	132	133	134	135	136	137	138	139	161	162	163	164	165	166	167	168	169	170	171
			385	386	387	388	389	390	391	392	393	394	395	417	418	419	420	421	422	423	424	425	426	427
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	--	0	1	2	3	4	5	6	7	8	9	--
		PortB	40	41	42	43	44	45	46	47	48	49	--	40	41	42	43	44	45	46	47	48	49	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	0	1	2	3	4	5	6	7	8	9	--
Pin No.			145	146	147	148	149	150	151	152	153	154	155	177	178	179	180	181	182	183	184	185	186	187
			401	402	403	404	405	406	407	408	409	410	411	433	434	435	436	437	438	439	440	441	442	443
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	--	10	11	12	13	14	15	16	17	18	19	--
		PortB	50	51	52	53	54	55	56	57	58	59	--	50	51	52	53	54	55	56	57	58	59	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	10	11	12	13	14	15	16	17	18	19	--
Pin No.			193	194	195	196	197	198	199	200	201	202	203	225	226	227	228	229	230	231	232	233	234	235
			449	450	451	452	453	454	455	456	457	458	459	481	482	483	484	485	486	487	488	489	490	491
DBM	PPAT/PCPE	PortA	20	21	22	23	24	25	26	27	28	29	--	20	21	22	23	24	25	26	27	28	29	--
		PortB	60	61	62	63	64	65	66	67	68	69	--	60	61	62	63	64	65	66	67	68	69	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	20	21	22	23	24	25	26	27	28	29	--
Pin No.			209	210	211	212	213	214	215	216	217	218	219	241	242	243	244	245	246	247	248	249	250	251
			465	466	467	468	469	470	471	472	473	474	475	497	498	499	500	501	502	503	504	505	506	507
DBM	PPAT/PCPE	PortA	30	31	32	33	34	35	36	37	38	39	--	30	31	32	33	34	35	36	37	38	39	--
		PortB	70	71	72	73	74	75	76	77	78	79	--	70	71	72	73	74	75	76	77	78	79	--
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	30	31	32	33	34	35	36	37	38	39	--

--" indicates that the pin number or the DBM bit is not assigned.

• T5591 (CHTYPE3, BITMODE3)

			DR PIN											I/O PIN															
Pin No.			1	2	3	4	5	6	7	8	9	10	11	33	34	35	36	37	38	39	40	41	42	43					
			257	258	259	260	261	262	263	264	265	266	267	289	290	291	292	293	294	295	296	297	298	299					
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	--	40	41	42	43	44	45	46	47	48	49	--					
		PortB	0	1	2	3	4	5	6	7	8	9	--	40	41	42	43	44	45	46	47	48	49	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	--					
Pin No.			17	18	19	20	21	22	23	24	25	26	27	49	50	51	52	53	54	55	56	57	58	59					
			273	274	275	276	277	278	279	280	281	282	283	305	306	307	308	309	310	311	312	313	314	315					
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	--	50	51	52	53	54	55	56	57	58	59	--					
		PortB	10	11	12	13	14	15	16	17	18	19	--	50	51	52	53	54	55	56	57	58	59	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	--					
Pin No.			65	66	67	68	69	70	71	72	73	74	75	97	98	99	100	101	102	103	104	105	106	107					
			321	322	323	324	325	326	327	328	329	330	331	353	354	355	356	357	358	359	360	361	362	363					
DBM	PPAT/PCPE	PortA	20	21	22	23	24	25	26	27	28	29	--	60	61	62	63	64	65	66	67	68	69	--					
		PortB	20	21	22	23	24	25	26	27	28	29	--	60	61	62	63	64	65	66	67	68	69	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	60	61	62	63	64	65	66	67	68	69	--					
Pin No.			81	82	83	84	85	86	87	88	89	90	91	113	114	115	116	117	118	119	120	121	122	123					
			337	338	339	340	341	342	343	344	345	346	347	369	370	371	372	373	374	375	376	377	378	379					
DBM	PPAT/PCPE	PortA	30	31	32	33	34	35	36	37	38	39	--	70	71	72	73	74	75	76	77	78	79	--					
		PortB	30	31	32	33	34	35	36	37	38	39	--	70	71	72	73	74	75	76	77	78	79	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	70	71	72	73	74	75	76	77	78	79	--					
Pin No.			129	130	131	132	133	134	135	136	137	138	139	161	162	163	164	165	166	167	168	169	170	171					
			385	386	387	388	389	390	391	392	393	394	395	417	418	419	420	421	422	423	424	425	426	427					
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	--	40	41	42	43	44	45	46	47	48	49	--					
		PortB	0	1	2	3	4	5	6	7	8	9	--	40	41	42	43	44	45	46	47	48	49	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	--					
Pin No.			145	146	147	148	149	150	151	152	153	154	155	177	178	179	180	181	182	183	184	185	186	187					
			401	402	403	404	405	406	407	408	409	410	411	433	434	435	436	437	438	439	440	441	442	443					
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	--	50	51	52	53	54	55	56	57	58	59	--					
		PortB	10	11	12	13	14	15	16	17	18	19	--	50	51	52	53	54	55	56	57	58	59	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	--					
Pin No.			193	194	195	196	197	198	199	200	201	202	203	225	226	227	228	229	230	231	232	233	234	235					
			449	450	451	452	453	454	455	456	457	458	459	481	482	483	484	485	486	487	488	489	490	491					
DBM	PPAT/PCPE	PortA	20	21	22	23	24	25	26	27	28	29	--	60	61	62	63	64	65	66	67	68	69	--					
		PortB	20	21	22	23	24	25	26	27	28	29	--	60	61	62	63	64	65	66	67	68	69	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	60	61	62	63	64	65	66	67	68	69	--					
Pin No.			209	210	211	212	213	214	215	216	217	218	219	241	242	243	244	245	246	247	248	249	250	251					
			465	466	467	468	469	470	471	472	473	474	475	497	498	499	500	501	502	503	504	505	506	507					
DBM	PPAT/PCPE	PortA	30	31	32	33	34	35	36	37	38	39	--	70	71	72	73	74	75	76	77	78	79	--					
		PortB	30	31	32	33	34	35	36	37	38	39	--	70	71	72	73	74	75	76	77	78	79	--					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	70	71	72	73	74	75	76	77	78	79	--					

--" indicates that the pin number or the DBM bit is not assigned.

• T5591 (CHTYPE3, BITMODE4)

			DR PIN											I/O PIN															
Pin No.			1	2	3	4	5	6	7	8	9	10	11	33	34	35	36	37	38	39	40	41	42	43					
DBM	PPAT/PCPE	PortA	257	258	259	260	261	262	263	264	265	266	267	289	290	291	292	293	294	295	296	297	298	299					
		PortB	0	1	2	3	4	5	6	7	8	9	49	40	41	42	43	44	45	46	47	48	49	9					
	PDRE	PortA/PortB	20	21	22	23	24	25	26	27	28	29	69	60	61	62	63	64	65	66	67	68	69	29					
			--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	9					
Pin No.			17	18	19	20	21	22	23	24	25	26	27	49	50	51	52	53	54	55	56	57	58	59					
			273	274	275	276	277	278	279	280	281	282	283	305	306	307	308	309	310	311	312	313	314	315					
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	59	50	51	52	53	54	55	56	57	58	59	19					
		PortB	30	31	32	33	34	35	36	37	38	39	79	70	71	72	73	74	75	76	77	78	79	39					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	19					
Pin No.			65	66	67	68	69	70	71	72	73	74	75	97	98	99	100	101	102	103	104	105	106	107					
			321	322	323	324	325	326	327	328	329	330	331	353	354	355	356	357	358	359	360	361	362	363					
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	49	40	41	42	43	44	45	46	47	48	49	9					
		PortB	20	21	22	23	24	25	26	27	28	29	69	60	61	62	63	64	65	66	67	68	69	29					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	9					
Pin No.			81	82	83	84	85	86	87	88	89	90	91	113	114	115	116	117	118	119	120	121	122	123					
			337	338	339	340	341	342	343	344	345	346	347	369	370	371	372	373	374	375	376	377	378	379					
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	59	50	51	52	53	54	55	56	57	58	59	19					
		PortB	30	31	32	33	34	35	36	37	38	39	79	70	71	72	73	74	75	76	77	78	79	39					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	19					
Pin No.			129	130	131	132	133	134	135	136	137	138	139	161	162	163	164	165	166	167	168	169	170	171					
			385	386	387	388	389	390	391	392	393	394	395	417	418	419	420	421	422	423	424	425	426	427					
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	49	40	41	42	43	44	45	46	47	48	49	9					
		PortB	20	21	22	23	24	25	26	27	28	29	69	60	61	62	63	64	65	66	67	68	69	29					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	9					
Pin No.			145	146	147	148	149	150	151	152	153	154	155	177	178	179	180	181	182	183	184	185	186	187					
			401	402	403	404	405	406	407	408	409	410	411	433	434	435	436	437	438	439	440	441	442	443					
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	59	50	51	52	53	54	55	56	57	58	59	19					
		PortB	30	31	32	33	34	35	36	37	38	39	79	70	71	72	73	74	75	76	77	78	79	39					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	19					
Pin No.			193	194	195	196	197	198	199	200	201	202	203	225	226	227	228	229	230	231	232	233	234	235					
			449	450	451	452	453	454	455	456	457	458	459	481	482	483	484	485	486	487	488	489	490	491					
DBM	PPAT/PCPE	PortA	0	1	2	3	4	5	6	7	8	9	49	40	41	42	43	44	45	46	47	48	49	9					
		PortB	20	21	22	23	24	25	26	27	28	29	69	60	61	62	63	64	65	66	67	68	69	29					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	40	41	42	43	44	45	46	47	48	49	9					
Pin No.			209	210	211	212	213	214	215	216	217	218	219	241	242	243	244	245	246	247	248	249	250	251					
			465	466	467	468	469	470	471	472	473	474	475	497	498	499	500	501	502	503	504	505	506	507					
DBM	PPAT/PCPE	PortA	10	11	12	13	14	15	16	17	18	19	59	50	51	52	53	54	55	56	57	58	59	19					
		PortB	30	31	32	33	34	35	36	37	38	39	79	70	71	72	73	74	75	76	77	78	79	39					
	PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	50	51	52	53	54	55	56	57	58	59	19					

"--" indicates that the pin number or the DBM bit is not assigned.

• T5586 and T5585 (CHTYPE9)

Pin No.	DR PIN	-- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 157 --
	I/O PIN	33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 -- -- 51 53 55 -- -- -- -- -- -- 52 54 56 -- -- -- -- -- -- -- --
DBM	PPAT/PCPE/PDRE	PortA/PortB
Pin No.	DR PIN	75 76 77 78 79 80 81 82 83 84 85 86 87 88 -- -- -- -- 159 -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 160 --
	I/O PIN	97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 -- -- 115 117 119 -- -- -- -- -- -- 116 118 120 -- -- -- -- -- -- -- --
DBM	PPAT/PCPE/PDRE	PortA/PortB
Pin No.	DR PIN	21 22 23 24 -- -- -- -- -- -- 65 66 67 68 69 70 71 72 73 74 -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 221 -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 222 --
	I/O PIN	161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 -- -- 179 181 183 -- -- -- -- -- -- 180 182 184 -- -- -- -- -- -- -- --
DBM	PPAT/PCPE/PDRE	PortA/PortB
Pin No.	DR PIN	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 223 -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- 224 --
	I/O PIN	225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 -- -- 243 245 247 -- -- -- -- -- -- 244 246 248 -- -- -- -- -- -- -- --
DBM	PPAT/PCPE/PDRE	PortA/PortB
		60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79

--" indicates that the pin number or the DBM bit is not assigned.

• T5592 (CHTYPE10, BITMODE1)

Pin No.	DR PIN	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
DBM	PPAT	PortA
	PCPE/PDRE	PortA/PortB
Pin No.	DR PIN	65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85
DBM	PPAT	PortA
	PCPE/PDRE	PortA/PortB
Pin No.	DR PIN	129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149
DBM	PPAT	PortA
	PCPE/PDRE	PortA/PortB
Pin No.	DR PIN	193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213
DBM	PPAT	PortA
	PCPE/PDRE	PortA/PortB

Pin No.	I/O PIN	33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56
DBM	PPAT/PCPE/PDRE	PortA
		PortB
Pin No.	I/O PIN	97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120
DBM	PPAT/PCPE/PDRE	PortA
		PortB
Pin No.	I/O PIN	161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184
DBM	PPAT/PCPE/PDRE	PortA
		PortB
Pin No.	I/O PIN	225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248
DBM	PPAT/PCPE/PDRE	PortA
		PortB

--" indicates that the pin number or the DBM bit is not assigned.

• T5592 (CHTYPE10, BITMODE2)

Pin No.	DR PIN		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
			129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149
DBM	PPAT	PortA	0	1	2	16	17	18	32	33	34	48	49	50	64	65	66	80	81	82	--	--	--
		PortB	6	7	8	22	23	24	38	39	40	54	55	56	70	71	72	86	87	88	--	--	--
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85
			193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213
DBM	PPAT	PortA	3	4	5	19	20	21	35	36	37	51	52	53	67	68	69	83	84	85	--	--	--
		PortB	9	10	11	25	26	27	41	42	43	57	58	59	73	74	75	89	90	91	--	--	--
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Pin No.	I/O PIN		33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56
			161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184
DBM	PPAT/PCPE/PDRE	PortA	0	1	2	16	17	18	32	33	34	48	49	50	64	65	66	80	81	82	12	28	44	60	76	92
		PortB	6	7	8	22	23	24	38	39	40	54	55	56	70	71	72	86	87	88	14	30	46	62	78	94
Pin No.	I/O PIN		97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
			225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248
DBM	PPAT/PCPE/PDRE	PortA	3	4	5	19	20	21	35	36	37	51	52	53	67	68	69	83	84	85	13	29	45	61	77	93
		PortB	9	10	11	25	26	27	41	42	43	57	58	59	73	74	75	89	90	91	15	31	47	63	79	95

-- indicates that the pin number or the DBM bit is not assigned.

• T5592 (CHTYPE10, BITMODE3)

Pin No.	DR PIN		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
			129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149
DBM	PPAT	PortA	0	1	2	16	17	18	32	33	34	48	49	50	64	65	66	80	81	82	--	--	--
		PortB	0	1	2	16	17	18	32	33	34	48	49	50	64	65	66	80	81	82	--	--	--
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85
			193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213
DBM	PPAT	PortA	3	4	5	19	20	21	35	36	37	51	52	53	67	68	69	83	84	85	--	--	--
		PortB	3	4	5	19	20	21	35	36	37	51	52	53	67	68	69	83	84	85	--	--	--
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Pin No.	I/O PIN		33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56
			161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184
DBM	PPAT/PCPE/PDRE	PortA	6	7	8	22	23	24	38	39	40	54	55	56	70	71	72	86	87	88	14	30	46	62	78	94
		PortB	6	7	8	22	23	24	38	39	40	54	55	56	70	71	72	86	87	88	14	30	46	62	78	94
Pin No.	I/O PIN		97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
			225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248
DBM	PPAT/PCPE/PDRE	PortA	9	10	11	25	26	27	41	42	43	57	58	59	73	74	75	89	90	91	15	31	47	63	79	95
		PortB	9	10	11	25	26	27	41	42	43	57	58	59	73	74	75	89	90	91	15	31	47	63	79	95

-- indicates that the pin number or the DBM bit is not assigned.

• T5592 (CHTYPE10, BITMODE4)

Pin No.	DR PIN		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
			65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85
			129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149
			193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213
DBM	PPAT	PortA	0	1	2	16	17	18	32	33	34	48	49	50	64	65	66	80	81	82	--	--	--
		PortB	3	4	5	19	20	21	35	36	37	51	52	53	67	68	69	83	84	85	--	--	--
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Pin No.	I/O PIN		33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56
			97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
			161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184
			225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248
DBM	PPAT/PCPE/PDRE	PortA	6	7	8	22	23	24	38	39	40	54	55	56	70	71	72	86	87	88	14	30	46	62	78	94
		PortB	9	10	11	25	26	27	41	42	43	57	58	59	73	74	75	89	90	91	15	31	47	63	79	95

"--" indicates that the pin number or the DBM bit is not assigned.

• T5593 (CHTYPE12/CHTYPE13, BITMODE1)

Pin No.	DR PIN		1	2	3	4	5	6	7	8	9	10	11	12
DBM	PPAT	PortA	96	97	98	100	101	102	104	105	106	108	109	110
		PortB	96	97	98	100	101	102	104	105	106	108	109	110
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		13	14	15	16	17	18	19	20	21	22	23	24
DBM	PPAT	PortA	112	113	114	116	117	118	120	121	122	124	125	126
		PortB	112	113	114	116	117	118	120	121	122	124	125	126
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		65	66	67	68	69	70	71	72	73	74	75	76
DBM	PPAT	PortA	99	128	129	103	130	131	107	132	133	111	135	134
		PortB	99	128	129	103	130	131	107	132	133	111	135	134
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		77	78	79	80	81	82	83	84	85	86	87	88
DBM	PPAT	PortA	115	136	137	119	138	139	123	140	141	127	142	143
		PortB	115	136	137	119	138	139	123	140	141	127	142	143
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--

Pin No.	I/O PIN		33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56
DBM	PPAT/PCPE/PDRE	PortA	0	1	2	8	9	10	16	17	18	24	25	26	32	33	34	40	41	42	48	49	50	56	57	58
		PortB	0	1	2	8	9	10	16	17	18	24	25	26	32	33	34	40	41	42	48	49	50	56	57	58
Pin No.	I/O PIN		97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
DBM	PPAT/PCPE/PDRE	PortA	3	4	5	11	12	13	19	20	21	27	28	29	35	36	37	43	44	45	51	52	53	59	60	61
		PortB	3	4	5	11	12	13	19	20	21	27	28	29	35	36	37	43	44	45	51	52	53	59	60	61
Pin No.	I/O PIN		161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184
DBM	PPAT/PCPE/PDRE	PortA	6	7	64	14	15	68	22	23	72	30	31	76	38	39	80	46	47	84	54	55	88	62	63	92
		PortB	6	7	64	14	15	68	22	23	72	30	31	76	38	39	80	46	47	84	54	55	88	62	63	92
Pin No.	I/O PIN		225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248
DBM	PPAT/PCPE/PDRE	PortA	65	66	67	69	70	71	73	74	75	77	78	79	81	82	83	85	86	87	89	90	91	93	94	95
		PortB	65	66	67	69	70	71	73	74	75	77	78	79	81	82	83	85	86	87	89	90	91	93	94	95

"--" indicates that the pin number or the DBM bit is not assigned.

When CHTYPE13 is used, do not specify 22, 23, 24, 86, 87 and 88 for the pin number. If specified, a compilation error occurs.

• T5593 (CHTYPE12/CHTYPE13, BITMODE2)

Pin No.	DR PIN		1	2	3	4	5	6	7	8	9	10	11	12
DBM	PPAT	PortA	96	97	98	100	101	102	104	105	106	108	109	110
		PortB	99	128	129	103	130	131	107	132	133	111	135	134
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		13	14	15	16	17	18	19	20	21	22	23	24
DBM	PPAT	PortA	112	113	114	116	117	118	120	121	122	124	125	126
		PortB	115	136	137	119	138	139	123	140	141	127	142	143
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		65	66	67	68	69	70	71	72	73	74	75	76
DBM	PPAT	PortA	96	97	98	100	101	102	104	105	106	108	109	110
		PortB	99	128	129	103	130	131	107	132	133	111	135	134
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--
Pin No.	DR PIN		77	78	79	80	81	82	83	84	85	86	87	88
DBM	PPAT	PortA	112	113	114	116	117	118	120	121	122	124	125	126
		PortB	115	136	137	119	138	139	123	140	141	127	142	143
	PCPE/PDRE	PortA/PortB	--	--	--	--	--	--	--	--	--	--	--	--

Pin No.	I/O PIN		33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56
DBM	PPAT/PCPE/PDRE	PortA	0	1	2	8	9	10	16	17	18	24	25	26	32	33	34	40	41	42	48	49	50	56	57	58
		PortB	6	7	64	14	15	68	22	23	72	30	31	76	38	39	80	46	47	84	54	55	88	62	63	92
Pin No.	I/O PIN		97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
DBM	PPAT/PCPE/PDRE	PortA	3	4	5	11	12	13	19	20	21	27	28	29	35	36	37	43	44	45	51	52	53	59	60	61
		PortB	65	66	67	69	70	71	73	74	75	77	78	79	81	82	83	85	86	87	89	90	91	93	94	95
Pin No.	I/O PIN		161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184
DBM	PPAT/PCPE/PDRE	PortA	0	1	2	8	9	10	16	17	18	24	25	26	32	33	34	40	41	42	48	49	50	56	57	58
		PortB	6	7	64	14	15	68	22	23	72	30	31	76	38	39	80	46	47	84	54	55	88	62	63	92
Pin No.	I/O PIN		225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248
DBM	PPAT/PCPE/PDRE	PortA	3	4	5	11	12	13	19	20	21	27	28	29	35	36	37	43	44	45	51	52	53	59	60	61
		PortB	65	66	67	69	70	71	73	74	75	77	78	79	81	82	83	85	86	87	89	90	91	93	94	95

"--" indicates that the pin number or the DBM bit is not assigned.

When CHTYPE13 is used, do not specify 22, 23, 24, 86, 87 and 88 for the pin number. If specified, a compilation error occurs.

Example

(1) When CHTYPE3 and BITMODE1 are Specified:

```
DBM PINSEL(CHTYPE3, BITMODE1)
    D1 = P33, P97          ;The data defined to D1 is set to DBM0 and DBM20.
    D2 = P34, P98          ;The data defined to D2 is set to DBM1 and DBM21.
```

(2) When CHTYPE3 and BITMODE2 are Specified:

```
DBM PINSEL(CHTYPE3, BITMODE2)
    D1 = P33A, P97A        ;The data defined to D1 is set to DBM0 and DBM20.
    D2 = P34A, P98A        ;The data defined to D2 is set to DBM1 and DBM21.
    D3 = P33B, P97B        ;The data defined to D3 is set to DBM40 and DBM60.
    D4 = P34B, P98B        ;The data defined to D4 is set to DBM41 and DBM61.
```

(3) When CHTYPE4 or CHTYPE5 is Specified:

```

DBM PINSEL(CHTYPE4)
    D0 = P1, P11          ;The data defined to D0 is set to DBM60 and DBM70.
    D1 = P34, P42         ;The data defined to D1 is set to DBM1 and DBM9.
DBM PINSEL(CHTYPE4)
    D2 = P21, P161        ;The data defined to D2 is set to DBM40.
                           ;Because P21 and P161 are allocated to the same DBM40,
                           they are set to only DBM40.

```

2. 8. 4 DBMA Statement

Function: This statement specifies a DBM starting address from which DBM pattern data is to be transferred.

Multiple DBMA statements can be described to the [MAIN/SUB] DBM PATTERN statement. However, an address larger than the starting address of a previously described DBMA statement should be specified. A DBM starting address should not be identical to the DBM data address that has already been entered.

Be sure to specify this statement before entering the DBM pattern data. (This cannot be omitted.)

Syntax

```
DBMA n
```


Argument: n

Start address

DBM board	Interleave mode	Bit size
64 K board	1WAY	#0 ≤ n ≤ #FFFF
	2WAY	#0 ≤ n ≤ #1FFFE and is a multiple of 2
	4WAY	#0 ≤ n ≤ #3FFFC and is a multiple of 4
256 K board	1WAY	#0 ≤ n ≤ #3FFFF
	2WAY	#0 ≤ n ≤ #7FFFE and is a multiple of 2
	4WAY	#0 ≤ n ≤ #FFFC and is a multiple of 4
1 M board	1WAY	#0 ≤ n ≤ #FFFF
	2WAY	#0 ≤ n ≤ #1FFFE and is a multiple of 2
	4WAY	#0 ≤ n ≤ #3FFFC and is a multiple of 4
2M board ^{*1}	1WAY	#0 ≤ n ≤ #1FFFF
	2WAY	#0 ≤ n ≤ #3FFFF and is a multiple of 2
	4WAY	#0 ≤ n ≤ #7FFFF and is a multiple of 4

^{*1} When transferring the DBM test pattern in which the MODE RDBMA statement is described, the maximum address is the same as 1M board.

2. 8. 5 DBM Data Field

The DBM pattern data (logic pattern) is entered. The DBM CHANNEL statement and DBMA statement are required before the DBM pattern data is entered.

DBM pattern data

A test pattern is entered for the DBM data bit selected by the DBM CHANNEL statement.

The data which can be written in the DBM data field are five kind: 0, 1, L, H, and X.

Each character has 3 meanings.

Data	Applied pattern	DRE signal	CPE signal
0	0	1	0
1	1	1	0
L	0	0	1
H	1	0	1
X	1	0	0

- Applied pattern

The data applied to DUT or the data with expected value

Only the pins with PPAT specified in PIN statement are effective.

For the pins with no PPAT specified, the data selected in PDS is output.

- DRE signal

This is driver enable signal.

Only the pins with PDRE specified in PIN statement can be controlled by DRE signal from DBM.

For the pins with no PDRE specified, the control is performed by DRE1 and DRE2.

- CPE signal

This is comparator enable signal.

Only the pins with PCPE/PCPE1/PCPE129 specified in PIN statement can be controlled by CPE signal from DBM.

The comparator can be controlled by using CPE signal.

About the pins with ACPE/ACPE1/ACPE129 and CCPE/CCPE1/CCPE129 specified, this signal is used to control the control by CPE1-4.

For the pins with no PCPE/PCPE1/PCPE129, ACPE/ACPE1/ACPE129 and CCPE/CCPE1/CCPE129 specified, the control is performed by CPE1-4.

Data allocation

Data is entered from left to right, according to the order specified by the DBM CHANNEL statement. An address is assigned to a single line.

◆ Example

```
DBM CHANNEL 0,2,5,8
DBMA #0      ;0 2 5 8 ← bit number
01LH         0 1 L H ← Address #0 data
11HH         1 1 H H ← Address #1 data
HH11         H H 1 1 ← Address #2 data

DBM CHANNEL 0-7,8
DBMA #10     ;0 1 2 3 4 5 6 7 8 ← bit number
HHHHLLLLH   H H H H L L L L H ← Address #10 data
1100HHLLH   1 1 0 0 H H L L H ← Address #11 data
100000001   1 0 0 0 0 0 0 0 1 ← Address #12 data
```

◆ Settings in the Program

```

PRO SAM3
:
PD1-12  = IN1,NRZA,ACLK1,PPAT,PDRE ... Selects DBM data and DRE signal for PAT
                                         and DRE.
PD13-24 = IN1,NRZA,ACLK1,PPAT,PDRE ... Selects DBM data and DRE signal for PAT
                                         and DRE.
PC33-40 = OUT1,PCPE ..... Selects CPE signal of DBM for CPE.
PC41-48 = OUT1,PCPE ..... Selects CPE signal of DBM for CPE.
:
SEND DBM PATDB ..... Transfers the DBM pattern.
SEND MPAT PATM ..... Transfers the ALPG pattern.
MEAS MPAT ..... Performs measurement.

```

Field division

A pattern can be field-divided by using delimiters.

Field division enables the pattern field immediately after the DBMA statement to be effective for the later patterns. (This field on the first line is called base field.)

Specifying "-" takes effect when setting the same data as that of the preceding address in the same field.

◆ Setting the Same Data as the Preceding One

```

DBM CHANNEL 0-35
DBMA #0
00000000 00000000 00 00000000 00000000 00
11111111 -          11 11111111 -          -
00000000 11111111 - -          -          11
11111111 -          - 00000000 -          -

```

Data is set as follows:

```

00000000 00000000 00 00000000 00000000 00
11111111 00000000 11 11111111 00000000 00
00000000 11111111 11 11111111 00000000 11
11111111 11111111 11 00000000 00000000 11

```



Tip

The start line of the DBM data field serves as a base for determining later data format, so "-" cannot be specified at that position. And, if data is omitted from the start line, all the omitted bits are processed as a single field.

◆ When Data is Omitted in the Start Line:

```
DBM CHANNEL 0-35
DBMA #0
00000000 00000000 00 00000000
11111111 -      11 11111111 0000000000
00000000 11111111 - -      0000000011
11111111 -      - 00000000 -

This data is identical with that in (Example 1).
```

2. 8. 5. 1 REPEAT Statement

Function: This is a compilation control statement for generating the same pattern repeatedly.

Syntax

```
REPEAT [n],m
      DBM pattern
```

Argument: n

Number of repetitions (default 1)

$1 \leq n \leq 16777216$ (#1000000)

m

The number of data items repeated (cannot be omitted).

$1 \leq m \leq 16777216$ (#1000000)

Description

The above n and m are processed as decimal numbers regardless of the current radix, unless they are marked with # to indicate use of hexadecimal.

Example

(1) When REPEAT is Used

```
DBM CHANNEL 0-7
DBMA #0
0000LLLL          ← Address #0 data
REPEAT 2,3
1111HHHH
0101HHLL
1100LLHH
1111HHHH          ← Address #7 data
```

Repeat 1111HHHH to 1100LLHH twice.

In the DBM pattern specified by the REPEAT statement, field division and data omission function can be used in the same way as for regular DBM patterns.

(2) Logic Test

```
PRO T244          ; SAMPLE PROGRAM
VS1 = 5V
IN1 = 3.0V,0V
OUT1= 1.3V,1.3V

RATE =100NS
BCLK1= 0NS
STRB1= 90NS

PD1-4 = IN1,NRZB,BCLK1,PPAT*1
PD5-8 = IN1,NRZB,BCLK1,PPAT*1
PD9   = IN1,/NRZB,BCLK1,PPAT*1
PD10  = IN1,NRZB,BCLK1,PPAT*1
P33-36= OUT1,STRB1,PPAT,PCPE*2
P37-40= OUT1,STRB1,PPAT,PCPE*2

CALL CALB("C244",NORMAL)

REG MPAT PC = #0
SEND DBM P244*3
MEAS MPAT P244
END
```

^{*1} The DBM pattern data is used as a driver pattern.

^{*2} The DBM pattern data is used as a expected value pattern and a comparator enable signal.

^{*3} The DBM pattern is transferred to the pattern memory.

```

MPAT P244
REGISTER
  DBMA=#0*1
  IDX1=#7
START #0
IDX11 #8 DBMINC      ; DBMA 0-9*2
STPS

DBM PATTERN
DBM CHANNEL 0-17
DBM PINSEL (CHTYPE4)*3
  D0 =P1
  D1 =P2
  D2 =P3
  D3 =P4
  D4 =P33
  D5 =P34
  D6 =P35
  D7 =P36
  D8 =P5
  D9 =P6
  D10=P7
  D11=P8
  D12=P37
  D13=P38
  D14=P39
  D15=P40
  D16=P9
  D17=P10
DBMA #0
;11111111 22222222 12
;AAAAYYYY AAAYYYYY GG
;12341234 12341234
;----- DBMA
0000LLLL 00000000 10 ; 0
1000HLLL 00000000 10 ; 1
0100LHLL 00000000 10 ; 2
0010LLHL 00000000 10 ; 3
0001LLLH 00000000 10 ; 4
00000000 0000LLLL 01 ; 5
00000000 1000HLLL 01 ; 6
00000000 0100LHLL 01 ; 7
00000000 0010LLHL 01 ; 8
00000000 0001LLLH 01 ; 9
END

```

**1 Initialize DBM address pointer with #0.*

**2 This is DBM address generating instruction. DBM memory data is output at this address.*

**3 Define the channel number specified in DBM CHANNEL statement and the pin number to apply data actually.*

2. 8. 6 Control of Driver Enable/Comparator Enable

Driver enable control

Specifying PDRE in the PIN statement allows the driver to be controlled by the DRE (PDRE signal) output signal from the DBM.

By specifying PDRE in the PIN statement, control of driver enable is as shown below.

Table 2-17 PIN Statement Specifications and Driver Enable Control

PIN statement specification	Driver enable control
When PDRE is specified	Driver enable is controlled by the PDRE signal. ^{*1}
When PDRE is not specified	Driver enable is controlled by the DRE 1, DRE2 and C12 through C15.

**1 For the T5593 and T5592 systems, driver enable is controlled by DRE1, DRE2 and C12 through C15 in the cycles for which INHDBM is written in the pattern program.*

For the T5593 system, driver enable is controlled by PDRE signal when DBMFIIX is written in the PIN statement.

Comparator enable control

The comparator is controlled by the CPE signal (PCPE signal) output from DBM and one of the following modes in the PIN statement: PCPE, PCPE1, PCPE129, ACPE, ACPE1, ACPE129, CCPE, CCPE1, or CCPE129 mode.

Control of comparator enable is controlled according to the PIN statement specification as shown below.

Table 2-18 PIN Statement Specifications and Comparator Enable Control

Strobe to be used	PIN statement specification	Comparator enable control
STRB1-128, WSTRB1-64	When PCPE and PCPE1 are specified	Comparator enable is controlled by the PCPE signal. ^{*1}
	When ACPE and ACPE1 are specified	When PCPE signal is: "0": prohibits CPE1-4, "1": enables CPE1-4.
	When CCPE and CCPE1 are specified	When PCPE signal is: "0": enables CPE of odd-phase STRB, "1": enables CPE of even-phase STRB. (This operation is used for switching 0/1 comparison and HI-Z detection in real time when 0/1 is judged by using an odd-phase STRB and HI-Z is detected by using an even-phase STRB.)
	When none of PCPE, PCPE1, ACPE, ACPE1, CCPE, or CCPE1 is specified	Comparator enable is controlled by CPE1-4.
STRB129-256, WSTRB65-128	When PCPE129 is specified	Comparator enable is controlled by the PCPE signal. ^{*1}
	When ACPE129 is specified	When PCPE signal is: "0": prohibits CPE1-4, "1": enables CPE1-4.
	When CCPE129 is specified	When PCPE signal is: "0": enables CPE of odd-phase STRB, "1": enables CPE of even-phase STRB. (This operation is used for switching 0/1 comparison and HI-Z detection in real time when 0/1 is judged by using an odd-phase STRB and HI-Z is detected by using an even-phase STRB.)
	When none of PCPE129, ACPE129, or CCPE129 is specified	Comparator enable is controlled by CPE1-4.

^{*1} For the T5593 and T5592 system, comparator enable is controlled by CPE1 through CPE4 in the cycles for which INHDBM is written in the pattern program.

For the T5593 system, comparator enable is controlled by PCPE signal when DBMFIx is written in the PIN statement.

◆ Logic Test Performing the HI-Z Comparison

```

PRO T245          ; SAMPLE PROGRAM
VS1 = 5V
IN1 = 3.0V,0V
OUT1= 1.3V,1.3V

RATE =100NS
ACLK1= 0NS
STRB1= 90NS
STRB2= 90NS

PD1-4 = IN1,NRZA,ACLK1,PPAT*1
PD5-8 = IN1,NRZA,ACLK1,PPAT*1
PD9   = IN1,/NRZA,ACLK1,PPAT*1
PD10  = IN1,NRZA,ACLK1,PPAT*1
P33-36= OUT1,[STRB1,CPE1],[STRB2,CPE2],HZ1,PPAT,CCPE*2
P37-40= OUT1,STRB1,PPAT,PCPE*3

CALL CALB("C245",NORMAL)

REG MPAT PC = #0
SEND DBM P245*4
MEAS MPAT P245
END

```

*1 The DBM pattern data is used as a driver pattern.

*2 The DBM pattern data is used as an expected value pattern. CPE1 and CPE2 in the pattern program are used as the comparator enable signal. The HI-Z detection and I/O judgment are exchanged by the PCPE signal from the DBM. When the PCPE signal is "0," HI-Z detection is performed on STRB1 and when the PCPE signal is "1," I/O judgment is used for STRB2.

*3 The DBM pattern data is used as a expected value pattern and a comparator enable signal.

*4 The DBM pattern is transferred to the pattern memory.

```

MPAT P245
REGISTER
  DBMA=#0*1
  IDX1=#7
  CPE1=R*2
  CPE2=C0*3
START      #0
IDX11 #8 DBMINC R C0 ;DBMA 0-9*4
STPS

DBM PATTERN
DBM CHANNEL 0-17
DBM PINSEL (CHTYPE4)*5
  D0 =P1
  D1 =P2
  D2 =P3
  D3 =P4
  D4 =P33
  D5 =P34
  D6 =P35
  D7 =P36
  D8 =P5
  D9 =P6
  D10=P7
  D11=P8
  D12=P37
  D13=P38
  D14=P39
  D15=P40
  D16=P9
  D17=P10
DBMA #0
;11111111 22222222 12
;AAAAYYYY AAAYYYYY GG
;12341234 12341234
;----- DBMA
0000LLLL 00000000 10 ; 0
1000HLLL 00000000 10 ; 1
0100LHLL 00000000 10 ; 2
0010LLHL 00000000 10 ; 3
0001LLLH 00000000 10 ; 4
0000LLLL 0000LLLL 01 ; 5
0000XXXX 1000HLLL 01 ; 6*6
0000XXXX 0100LHLL 01 ; 7*6
0000XXXX 0010LLHL 01 ; 8*6
0000LLLL 0001LLLH 01 ; 9
END

```

- *1 Initialize DBM address pointer with #0.
- *2 Select "R" for the cycle used to compare the HI-Z detection.
- *3 Select "C0" for the cycle used to compare the I/O judgment.
- *4 This is DBM address generating instruction. DBM memory data is output at this address.
- *5 Define the channel number specified in DBM CHANNEL statement and the pin number to apply data actually.
- *6 The HI-Z expected value is set to "X" for a pin with the CCPE mode.

Tip

- The CCPE mode can be selected for comparator pins only, using the Pin statement. This mode cannot be selected for I/O common pins.
- Set the HI-Z expected value to "X" in CCPE mode. "Z" cannot be set.
- When performing the HI-Z comparison in CCPE mode, use HZ1. HZ2 cannot be used.

2. 8. 7 DBM STORAGE Statement

Function: The DBM STORAGE statement is used to secure the area for debugging at the end of DBM pattern transfer.

Specify this statement only once between the [MAIN/SUB] DBM PATTERN statement and the DBMA statement.

The body text can be omitted. When omitted, same as n=0

Syntax

```
DBM STORAGE n
```

Argument: n

The number of patterns of the area to be secured
n can be set within the following range.

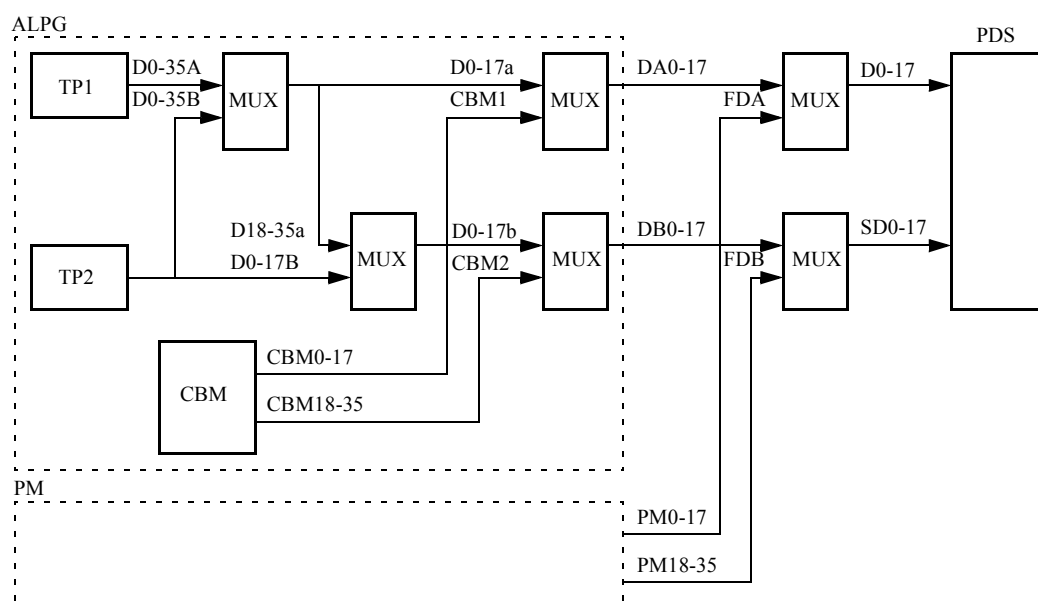
Interleave mode	Bit size
1WAY	#0 ≤ n ≤ #400000
2WAY	#0 ≤ n ≤ #800000 and is a multiple of 2
4WAY	#0 ≤ n ≤ #1000000 and is a multiple of 4

2. 9 CBM Pattern

CBM is a buffer memory to store random data specified in the pattern program and all PG UNIT are equipped with the CBM. Random data which is difficult to be created by TP register can be created by storing the data in CBM previously.

CBM is the same as DBM in generating random data, but CBM cannot control DRE signal and CPE signal. Also the memory capacities are different.

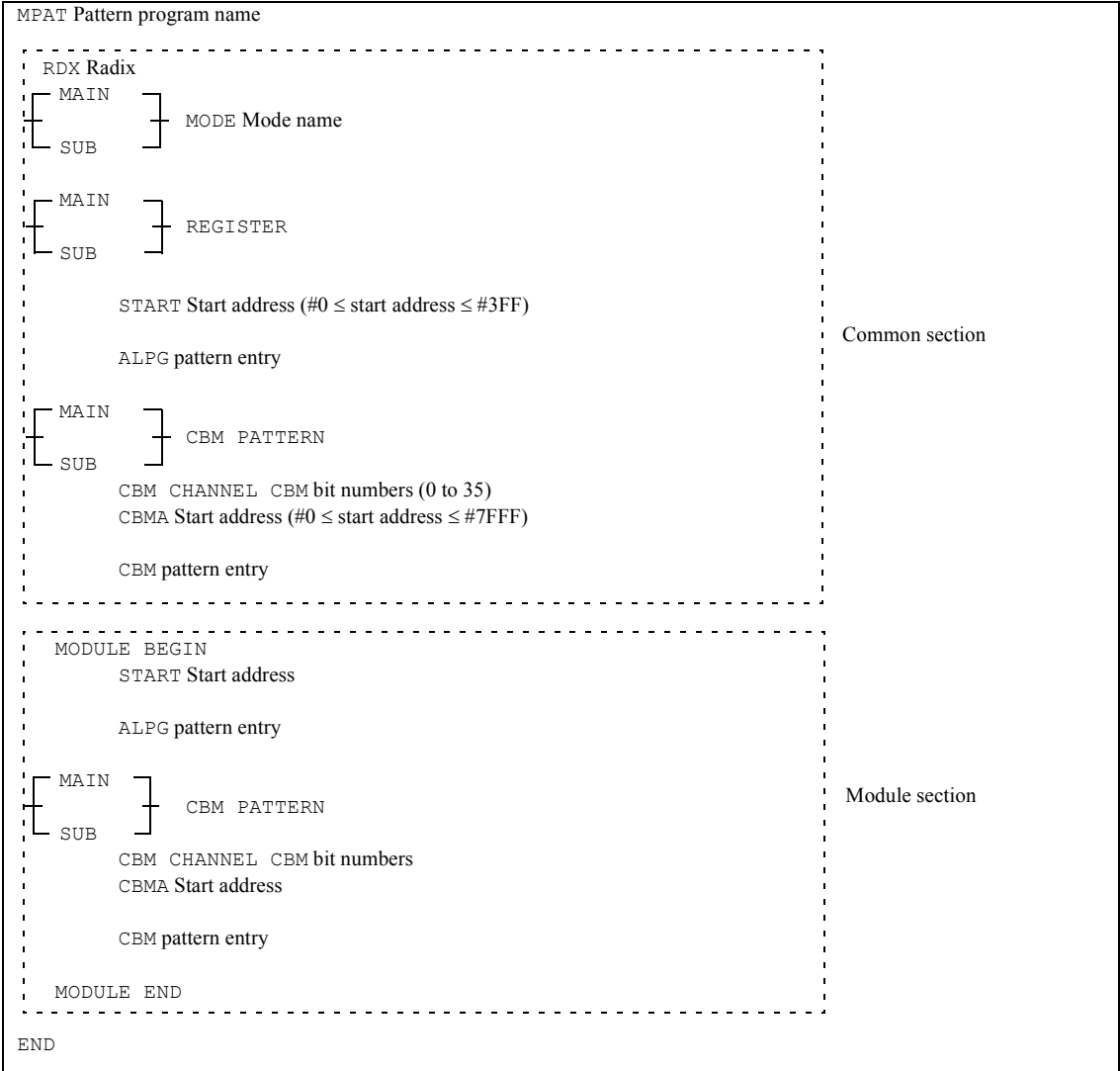
The following figure shows the relationship between the data generated by TP1 & TP2 and CBM data & PM data.



The data from TP1 and TP2 can be switched to real time by selecting mode specification (CBM1 and CBM2 can be specified individually) or by specifying a source data field instruction in the pattern program.

2. 9. 1 Programming Format

Syntax



MODULE BEGIN

START Start address

ALPG pattern entry

MAIN

SUB

CBM PATTERN

CBM CHANNEL CBM bit numbers

CBMA Start address

CBM pattern entry

MODULE END

Module section

END

-  Tip
- MAIN or SUB can be specified individually only in the following cases.

T5581 1WAY mode

T5591 1WAY or 2WAY mode

T5593 1WAY mode, 2WAY mode when SUBPG option is equipped
 - In the same common section/module section, CBM PATTERN statement and MAIN CBM PATTERN statement, or CBM PATTERN statement and SUB CBM PATTERN statement cannot be described simultaneously.

- The CBM starting address specified by the CBMA statement is: up to #7FFF when MODE ECBM is not specified, up to #FFFE in the 2WAY mode when MODE ECBM is specified, or up to #1FFFC in the 4WAY mode when MODE ECBM is specified.
MODE ECBM can be specified in the 2WAY or 4WAY mode.

2. 9. 2 CBM CHANNEL Statement

Function: This statement specifies a CBM data bit number. Specify this statement only once between the [MAIN/SUB] CBM PATTERN statement and the corresponding CBMA statement. (The above specification cannot be omitted if the CBM pattern is entered.)

Syntax

```
CBM CHANNEL i
CBM CHANNEL i,j
CBM CHANNEL i-j
```

Description

Ascending and descending continuous specification and random specification can be used together in the range of 0 to 35.

A channel number is processed in decimal regardless of the current radix.

Data with the bit number which was not specified in this statement becomes undefined.

Example

(1) CBM CHANNEL Specification

```
CBM CHANNEL 0-17
CBM CHANNEL 0,10-17,20
CBM CHANNEL 0-7,18-25
CBM CHANNEL 0,1,2,3,8-11
```

(2) Entry for Each Module

```

MODULE BEGIN
CBM PATTERN
  CBM CHANNEL 0-7
  CBMA #0
    00001100
    11111111
    :
  START #0
  NOP
  :
  STPS
MODULE END
MODULE BEGIN
CBM PATTERN
  CBM CHANNEL 8-15
  CBMA #100
    11110000
    00001111
    :
  START #100
  NOP
  :
  STPS
MODULE END

```

2. 9. 3 CBMA Statement

Function: This statement specifies a CBM start address from which CBM pattern data is to be transferred.

Multiple CBMA statement can be described to the [MAIN/SUB] CBM PATTERN statement. However, an address larger than the start address of a previously described CBMA statement should be specified. A CBM start address should not be identical to the CBM data address that has already been entered.

Be sure to specify this statement before entering the CBM pattern data. (This cannot be omitted.)

Syntax

CBMA n

Argument: n

Start address

Specify this address in the following range according to the interleaved mode and MODE ECBM specified or not.

Mode	ECBM not specified	ECBM specified
1WAY	$\#0 \leq n \leq \#7FFF$	Impossible to specify the ECBM mode
2WAY	$\#0 \leq n \leq \#7FFF$	$\#0 \leq n \leq \#FFFE$ and is a multiple of 2
4WAY	$\#0 \leq n \leq \#7FFF$	$\#0 \leq n \leq \#1FFFC$ and is a multiple of 4

Description

In the ECBM mode, the maximum starting address that can be specified by in the CBMA statement is #FFFE in the 2WAY mode, but data can be set up to #FFFF. In the same way, the maximum starting address is #1FFFC in the 4WAY mode, but data can be set up to #1FFFF.

When the ECBM mode is specified in the 2WAY mode, the specified data corresponding to CBMA = Even Address is stored in CBM #1 (PG UNIT1); and the data corresponding to CBMA = Odd Address is stored in CBM#2 (PG UNIT2). This data is accessed with the same address as in CBM #1 or CBM #2.

This operation doubles the CBM capacity.

(If ECBM mode is not specified, the same data is stored in CBM #1 and #2. This enables the data to be output without any limitation on CBMA operation.)

When ECBM is specified in the 4WAY mode, the specified data is stored in CBM#1(PG UNIT1), CBM#2(PG UNIT2), CBM#3(PG UNIT3), and CBM#4(PG UNIT4) in this order. This data is accessed with the same address as in CBM #1, CBM #2, CBM #3, or CBM #4.

This operation increases the CBM capacity by four.

(If ECBM mode is not specified, the same data is stored in CBM #1, #2, #3, and #4. This enables the data to be output without any limitation on CBMA operation.)

Tip

- In the 2WAY interleaved mode besides in the ECBM mode, the system can process double amount data than the normal mode. In this operation mode, however, the following restrictions exist with the use of the CBM address field instruction:

The CBMA operation instruction can be written only once in the second step only between the two interleaving steps.

There, even address data of CBM is output to the first step of 2WAY and odd address data is output to the second step. The value of the CBM address pointer obtained by the CBMA operation instruction is multiplied by 2 in the first step, and multiplied by 2 + 1 in the second step and two CBM data are output per one sequence instruction.

- In the 4WAY interleaved mode, besides in the ECBM mode, the system can process 4 times more data than the normal mode. In this operation mode, however, the following restrictions exist with the use of the CBM address field instruction:

The CBMA operation instruction can be written only once in the fourth step only between the four interleaving steps.

The CBM address pointer value obtained by the CBMA operation instruction is always multiplied by 4 in the first step, by 4 + 1 in the second step, by 4 + 2 in the third step, and by 4 + 3 in the fourth step. Four sets of CBM data are output for each sequence instruction.

Example

(1) In 2WAY Interleaved Mode

```
MPAT xxxx <2WAY>
MODE ECBM
CBM PATTERN
  CBM CHANNEL 0-15
  CBMA #0          ; CBMA
  0000 0000 0000 0000 ; #0
  0000 0000 0000 0001 ; #1
  0000 0000 0000 0010 ; #2
  0000 0000 0000 0011 ; #3
  :
  CBMA #FFFE
  1111 1111 1111 1110 ; #FFFE
  1111 1111 1111 1111 ; #FFFF
END
```

2. 9. 4 CBM Data Field

The CBM pattern data is entered. The CBMA statement is required before the CBM pattern data is entered. Data 0 and 1 can be specified.

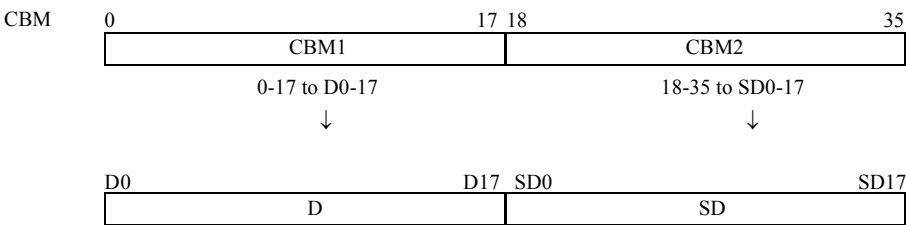
Data allocation

Data is entered from left to right, according to the order specified by the CBM CHANNEL statement. An address is assigned to a single line.

```
CBM CHANNEL 0,2,5,8
CBMA #0      ; 0 2 5 8 ← bit number
0101         ; 0 1 0 1 ← Address #0 data
1111         ; 1 1 1 1 ← Address #1 data
1111         ; 1 1 1 1 ← Address #2 data

CBM CHANNEL 0-7,8
CBMA #10     ; 0 1 2 3 4 5 6 7 8 ← bit number
111100001   ; 1 1 1 1 0 0 0 0 1 ← Address #10 data
110011001   ; 1 1 0 0 1 1 0 0 1 ← Address #11 data
100000001   ; 1 0 0 0 0 0 0 0 1 ← Address #12 data
```

CBM bits 0 to 35 output to D0-17, SD0-17 of PDS.



What data is output to D and SD of PDS is specified by the DATA DEFINE statement in the pattern program and by the SELECT PDS DATA statement in the test plan program. (However, entry in the test program is preferred.) In addition, a pin to which selected PDS data is output is specified by the PIN statement.

The following example may be helpful.

◆ Settings in the Program

```
PRO SAM3
:
SELECT PDS DATA CBM1/CBM2 ..... Selects data to be output to PDS.
:
PD1-12 =IN1,NRZA,ACLK1,<D0-11>.... Selects CBM0-11 for PD1-12.
PD13-15=IN1,RZO,BCLK1, <D12-14>... Selects CBM12-14 for PD13-15.
P33-40 =IN1,NRZA,ACLK2,<SD0-7>..... Selects CBM18-25 for P33-40.
P41-48 =IN1,NRZA,ACLK2,<SD8-15>.... Selects CBM26-33 for P41-48.
:
SEND CBM PATCB ..... Transfers the CBM pattern.
SEND MPAT PATM ..... Transfers the ALPG pattern.
MEAS MPAT ..... Performs measurement.
```

Data omission

When the number of bits of the CBM pattern is smaller than that selected by the CBM CHANNEL statement, data is assumed to be omitted and "0" is set.

◆ Data Omission

```
CBM CHANNEL 0-35
CBMA #0
1111 1111 1111 ← Bits 0 to 11 data, bits 12 to 35 are omitted.
1111 1111      ← Bits 0 to 7 data, bits 8 to 35 are omitted.
0000 1111 0000 ← Bits 0 to 11 data, bits 12 to 35 are omitted.
0000          ← Bits 0 to 3 data, bits 4 to 35 are omitted.
```

Tip

- Data with the bit number which was not specified by the CBM CHANNEL statement is undefined.

- When field division is used, data cannot be omitted for a part of the field.
→ For information on the field division, refer to ["Field division" on page 2-237](#).

Field division

A pattern can be field-divided by using delimiters.

Field division enables the pattern field immediately after the CBMA statement to be effective for the later patterns. (This field on the first line is called base field.)

Specifying "-" takes effect when setting the same data as that of the preceding address in the same field.

◆ Setting the Same Data as the Preceding One

```
CBM CHANNEL 0-35
CBMA #0
00000000 00000000 00 00000000 00000000 00
11111111 -          11 11111111 -          -
00000000 11111111 - -          -          11
11111111 -          - 00000000 -          -
```

Data is set as follows:

```
00000000 00000000 00 00000000 00000000 00
11111111 00000000 11 11111111 00000000 00
00000000 11111111 11 11111111 00000000 11
11111111 11111111 11 00000000 00000000 11
```



Tip

The start line of the CBM data field serves as a base for determining later data format, so "-" cannot be specified at that position. And, if data is omitted from the start line, all the omitted bits are processed as a single field.

◆ When Data is Omitted in the Start Line:

```
CBM CHANNEL 0-35
CBMA #0
00000000 00000000 00 00000000
11111111 -          11 11111111 0000000000
00000000 11111111 - -          0000000011
11111111 -          - 00000000 -
```

This data is identical with that in (Example 1).

The case to create CBM data

CBM memory is accessed by CBMA (CBM address pointer).

Use the following CBMA operation instructions to control the address pointer.

CBMA<CBMA (Hold)

CBMA<CBMA+1 (Increment)

CBMA<m (Set data)

#0 ≤ m ≤ #7FFF (However in ECBM mode, 2WAY: #0 ≤ m ≤ #FFFE, 4WAY: #0 ≤ m ≤ #1FFFC)

◆ CBMA Operating Instructions

```
START    #0
          NOP      CBMA<#0
ST0:     NOP      CBMA<CBMA+1  D<CBM  W
          JN11 ST0 CBMA<CBMA    D<CBM  R  C0
          STPS
```

2. 9. 4. 1 REPEAT Statement

Function: This is a compilation control statement for generating the same pattern repeatedly.

Syntax

```
REPEAT [n],m
      CBM pattern
```

Argument: n

Number of repetitions (default 1)

1 ≤ n ≤ 131072 (#20000)

m

The number of data items repeated. (This cannot be omitted.)

1 ≤ m ≤ 131072 (#20000)

Description

The above n and m are processed as decimal numbers regardless of the current radix, unless they are marked with # to indicate use of hexadecimal.

2.10 STISP, STIn Burst Test Condition Statement

In test condition statements which can be described in the burst block, STISP statement and STIn statement are used for the sequence control of patterns. These take the place of sequence control instruction STISP and STIn which exist in the pattern program function of T5300 series.

When executing the pattern program to transfer the STISP or STIn burst test condition statement using the OUT command in the BURST mode(MEAS/START MPAT(4)), the Pause Timer setting is as follows. The pause time necessary for the BURST mode ALPG Pause Timer should be set approximately 15 μ s longer than when the OUT command does not transfer the statements.



Tip

In the T5593, the sequence control instruction, STISP and STIn instructions can be used.

2.10. 1 STISP Statement

Function: This statement sets the value to the ISP (Interrupt Save Pointer) register. For the ISP register, specify the address which PC is branched at the interrupt generation from the inside timer. This is used for the refresh test etc.

STISP statement cannot be specified in the burst test condition statement of the test plan program.

Syntax

```
STISP m
```

Argument: m

Specify the value set to the ISP register by the address directory specification (PC address) or the label name.

When it is specified by a PC address, the specification range is #0 to #3FF.

When it is specified by a label name, note the following:

- Label specification in the common section burst block

All the label of common section/module section are the object.

The label which is used in multiple modules cannot be specified.

- Label specification in the module section burst block

Only the label in the module described the STISP statement is the object.

When the same label is used in multiple modules, the label in the module, in which the STISP statement is described, is the object.

The label which does not exist in the module cannot be specified.

Example

(1) Specifying a Label Name for the ISP Register Using the STISP Statement

```

MPAT    SAMPLE

        MODE REFRESHA

        REGISTER
            ISP=BMP1
            IDX1=#FFFE

        BURST BLOCK BEGIN(BMPX1)
            STISP BMP2
        BURST BLOCK END

        BURST BLOCK BEGIN(BMPX2)
            STISP BMP1
        BURST BLOCK END

        START #0
        NOP T
A:      NOP
        NOP
        JN11 A
        STPS

BMP1:   NOP
        OUT BMPX1      ← Set BMP2 to ISP register.
        RTN

BMP2:   NOP
        OUT BMPX2      ← Set BMP1 to ISP register.
        RTN

END

```

2.10. 2 STIn Statement

Function: Sets the count value to the IDXn register. IDXn register specifies the number of loops of the index loop.

STIn statement cannot be specified in the burst test condition statement of the test plan program.

Syntax

```
STIn m
```

Argument: n

IDX register number. Specify with a number from 1 to 8.

m

Count value which is set to the IDXn

Specification range

When the specification range is n=1,2

#0 to #FFFFFFF

When the specification range is n=3-8

#0 to #FFFFFF

Example

(1) When Setting #FF and #FFF in the IDX1 Register

```
MPAT    SAMPLE

REGISTER
    IDX1=#F

BURST BLOCK BEGIN(SET1)
    STI1 #FF
BURST BLOCK END

BURST BLOCK BEGIN(SET2)
    STI1 #FFF
BURST BLOCK END

START #0
NOP
A:      JN11 A
        OUT SET1      ← Set #FF to IDX1 register.
B:      JN11 B
        OUT SET2      ← Set #FFF to IDX1 register.
        STPS
C:      JN11 C

END
```

2.11 Packet Instruction

The packet instruction specifies the start address of packets data in the packet transmission memory. The start address of packets is stored in an object file as packet information and is used on a pattern program debugging tool (MPATsim).

➔ For more information, refer to [T5500 Series MPATsim User's Manual\(No. : 8311103\)](#).

2.11. 1 PKT Statement

Function: The PKT statement is used to store packet information (packet names, the PC address of the PKT statement, and the start bit number) in an object file.

The PKT statement is specified in field B and field C in test pattern programming format (it cannot be specified in field A). More than one PKT statement can be specified in one step.

Packet information is independent of test pattern generation.

Syntax

```
PKT (name, bit)
```

Argument: name

Packet name. Specify characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

Up to 32 names can be specified.

bit

Start bit number of packet information. Specify one bit number of the ALPG signals shown below.

X0, X1, ..., X17

Y0, Y1, ..., Y17

D0, D1, ..., D17

SD0, SD1, ..., SD17

C0, C1, ..., C23

R(RD), W(WT), M1, M2

In the T5592, T5591, T5586, T5585, T5581, T5581H, T5581P and T5571P, X16-X17, Y16-Y17 and C16-C23 cannot be specified.

Example

(1) Usage of the PKT Statement

```

MPAT    SAMPLE1<4WAY>

      :
NOP : X<XC Y<YC C6 C7 M2 W PKT(ROWR,C6) PKT(COLCX,M2) PKT(DQA,D7)
      : X<XC Y<YC TP1<#30155 TP2<#00155
      : X<XC Y<YC
      : X<XC Y<YC TP1<#30155 TP2<#00155
      :
END

```

(2) When Defining the PKT Statement Using the SDEF Statement

```

MPAT    SAMPLE2<4WAY>

SDEF    XYLOAD = X<XC Y<YC
SDEF    ALLDEV = C6 C7 PKT(ROWR,C6)
SDEF    PRER01 = TP1<#30155 TP2<#00155
SDEF    DQAPKT = PKT(DQA,D7)

      :
NOP : XYLOAD ALLDEV M2 W PKT(COLCX,M2) DQAPKT
      : XYLOAD PRER01
      : XYLOAD
      : XYLOAD PRER01
      :
END

```

3. Pattern Generation Instructions for Test Plan Programs

This chapter describes test plan program instructions used to generate patterns and control the pattern program.

3. 1 Instructions

The test plan program statements related to the pattern file transfer are listed below:

```

MEAS MPAT filename
MEAS MPAT(1) filename
MEAS MPAT(3) filename
MEAS MPAT(4, block-name) filename
REG MPAT register-name=data
REG MPAT MAIN register-name=data
REG MPAT SUB register-name=data
RESET REG
RESET REG MAIN
RESET REG SUB
RESET REG ALL
RESET REG MAIN ALL
RESET REG SUB ALL
RESET REG MPAT
RESET REG MAIN MPAT
RESET REG SUB MPAT

SEND MPAT filename

START MPAT
START MPAT *
START MPAT(1)
START MPAT(1) *
START MPAT(3)
START MPAT(3) *
START MPAT(4,block-name)
STOP MPAT
RESET MPAT
SELECT SCRDATA NORMAL
SELECT SCRDATA ENABLE
SELECT SCRDATA DISABLE
SELECT SCRDATA MAIN NORMAL
SELECT SCRDATA MAIN ENABLE
SELECT SCRDATA MAIN DISABLE
SELECT SCRDATA SUB NORMAL
SELECT SCRDATA SUB ENABLE
SELECT SCRDATA SUB DISABLE
SELECT SCRAM MODE NORMAL
SELECT SCRAM MODE DIVIDE
SELECT SCRAM MODE XDIVIDE
SELECT SCRAM MODE YDIVIDE
SELECT SCRAM MODE DIVIDE2
SELECT SCRAM MODE YDIVIDE2

```

```

SELECT SCRAM MODE MAIN NORMAL
SELECT SCRAM MODE MAIN DIVIDE
SELECT SCRAM MODE MAIN XDIVIDE
SELECT SCRAM MODE MAIN YDIVIDE
SELECT SCRAM MODE MAIN DIVIDE2
SELECT SCRAM MODE MAIN YDIVIDE2
SELECT SCRAM MODE SUB NORMAL
SELECT SCRAM MODE SUB DIVIDE
SELECT SCRAM MODE SUB XDIVIDE
SELECT SCRAM MODE SUB YDIVIDE
SELECT SCRAM MODE SUB DIVIDE2
SELECT SCRAM MODE SUB YDIVIDE2
SEND SCRDATA filename
SEND SCRDATA MAIN filename
SEND SCRDATA SUB filename
RESET SCRDATA
RESET SCRDATA MAIN
RESET SCRDATA SUB

```

```

SELECT PRESCRAM NORMAL
SELECT PRESCRAM ENABLE
SELECT PRESCRAM DISABLE
SELECT PRESCRAM MAIN NORMAL
SELECT PRESCRAM MAIN ENABLE
SELECT PRESCRAM MAIN DISABLE
SELECT PRESCRAM SUB NORMAL
SELECT PRESCRAM SUB ENABLE
SELECT PRESCRAM SUB DISABLE
SELECT PRESCRAM MODE NORMAL
SELECT PRESCRAM MODE OUTY
SELECT PRESCRAM MODE EORZ
SELECT PRESCRAM MODE ADDZ
SELECT PRESCRAM MODE EORZ2
SELECT PRESCRAM MODE ADDZ2
SELECT PRESCRAM MODE YBURST
SELECT PRESCRAM MODE MAIN NORMAL
SELECT PRESCRAM MODE MAIN OUTY
SELECT PRESCRAM MODE MAIN EORZ
SELECT PRESCRAM MODE MAIN ADDZ
SELECT PRESCRAM MODE MAIN EORZ2
SELECT PRESCRAM MODE MAIN ADDZ2
SELECT PRESCRAM MODE MAIN YBURST
SELECT PRESCRAM MODE SUB NORMAL
SELECT PRESCRAM MODE SUB OUTY
SELECT PRESCRAM MODE SUB EORZ
SELECT PRESCRAM MODE SUB ADDZ
SELECT PRESCRAM MODE SUB EORZ2
SELECT PRESCRAM MODE SUB ADDZ2
SELECT PRESCRAM MODE SUB YBURST

```

```
SELECT PRESCRAM ADDRESS DISABLE
SELECT PRESCRAM ADDRESS DUT
SELECT PRESCRAM ADDRESS FM
SELECT PRESCRAM ADDRESS FP
SELECT PRESCRAM ADDRESS DUT FM
SELECT PRESCRAM ADDRESS DUT FP
SELECT PRESCRAM ADDRESS FM FP
SELECT PRESCRAM ADDRESS DUT FM FP
SELECT PRESCRAM ADDRESS MAIN DISABLE
SELECT PRESCRAM ADDRESS MAIN DUT
SELECT PRESCRAM ADDRESS MAIN FM
SELECT PRESCRAM ADDRESS MAIN FP
SELECT PRESCRAM ADDRESS MAIN DUT FM
SELECT PRESCRAM ADDRESS MAIN DUT FP
SELECT PRESCRAM ADDRESS MAIN FM FP
SELECT PRESCRAM ADDRESS MAIN DUT FM FP
SELECT PRESCRAM ADDRESS SUB DISABLE
SELECT PRESCRAM ADDRESS SUB DUT
SELECT PRESCRAM ADDRESS SUB FM
SELECT PRESCRAM ADDRESS SUB FP
SELECT PRESCRAM ADDRESS SUB DUT FM
SELECT PRESCRAM ADDRESS SUB DUT FP
SELECT PRESCRAM ADDRESS SUB FM FP
SELECT PRESCRAM ADDRESS SUB DUT FM FP
SET WRAP ADDRESS=Xi-j,Ym-n
SET WRAP ADDRESS MAIN=Xi-j,Ym-n
SET WRAP ADDRESS SUB=Xi-j,Ym-n
SEND PRESCRAM filename
SEND PRESCRAM MAIN filename
SEND PRESCRAM SUB filename
RESET PRESCRAM
RESET PRESCRAM MAIN
RESET PRESCRAM SUB
```

```
SELECT ARIRAM NORMAL
SELECT ARIRAM ENABLE
SELECT ARIRAM DISABLE
SELECT ARIRAM MAIN NORMAL
SELECT ARIRAM MAIN ENABLE
SELECT ARIRAM MAIN DISABLE
SELECT ARIRAM SUB NORMAL
SELECT ARIRAM SUB ENABLE
SELECT ARIRAM SUB DISABLE
```

```
SELECT PDS DATA D/D
SELECT PDS DATA CBM1/CBM2
SELECT PDS DATA FDA/FDB
SELECT PDS DATA MAIN D/D
SELECT PDS DATA MAIN CBM1/CBM2
```

```

SELECT PDS DATA MAIN FDA/FDB
SELECT PDS DATA SUB D/D
SELECT PDS DATA SUB CBM1/CBM2

SELECT MPAT NORMAL
SELECT MPAT INHIBIT
SELECT MPAT REPEAT
SELECT MPAT INHIBIT REPEAT

TIME LIMIT t:label
WAIT MPAT t:label

BURST BLOCK BEGIN (block-name)
condistion(VS,WAIT TIME,SEND EXT)
SET BURST(v,n)
BURST BLOCK END
WINDOW MODE=DISABLE
WINDOW MODE MAIN=DISABLE
WINDOW MODE SUB=DISABLE
WINDOW MODE=X
WINDOW MODE MAIN=X
WINDOW MODE SUB=X
WINDOW MODE=Y
WINDOW MODE MAIN=Y
WINDOW MODE SUB=Y
WINDOW MODE=X,Y
WINDOW MODE MAIN=X,Y
WINDOW MODE SUB=X,Y
WINDOW XADRS=a
WINDOW XADRS MAIN=a
WINDOW XADRS SUB=a
WINDOW YADRS=a
WINDOW YADRS MAIN=a
WINDOW YADRS SUB=a

SELECT REFRESH NORMAL
SELECT REFRESH A
SELECT REFRESH B
SELECT REFRESH DISABLE

SELECT PAUSE NORMAL
SELECT PAUSE DISABLE
SET PATDELAY X,Y,DATA,TP1,TP2
RESET PATDELAY X,Y,DATA,TP1,TP2

```

3. 2 Functional Test Execution

The functional test is executed with a MEAS statement.

```
      :  
MEAS MPAT object filename  
      :
```

A pattern program normally consists of two or more test patterns as shown in ["Example of Pattern Program when Modules are not Used" on page 3-7](#) and ["Example of Pattern Program when Modules are Used" on page 3-8](#). A test pattern can be selected by specifying PC with a REG MPAT statement in the test plan program.

```
      :  
REG MPAT PC = #10 (or REG MPAT MAIN PC=#10)  
MEAS MPAT object filename  
      :
```

If the pattern program is divided by the MODULE statement (["Example of Pattern Program when Modules are Used" on page 3-8](#)), the value specified by the START statement described at the beginning of each module is set for PC.(["Test Pattern Selection" on page 3-9](#))

If no value is specified in the REG MPAT PC statement, the test pattern specified by the REGISTER statement in the pattern program is executed.(["Test Pattern Selection" on page 3-9](#))

The REG MPAT statement specification is valid only for the first test, as shown in (a) in ["REG MPAT Statement Valid Range" on page 3-10](#).

To repeat execution of a test pattern such as Shmoo plotting, generate the test program as shown in (b) and (c) in ["REG MPAT Statement Valid Range" on page 3-10](#).

◆ Example of Pattern Program when Modules are not Used

```

MPAT M256K                ;sample program for 256K dynamic RAMs.

MODE MUX
REGISTER
    XMAX=#FF
    YMAX=#3FF
    IDX1=#3FFFE
SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX   ;increment base

START #0                    ;checker board
IDXI2 #6                    INIT
ST0:  JNII ST0 W             INC      FP1
ST1:  JNII ST1 R             INC      FP1
      JZD ST0
      STPS

START #10                   ;marching
IDXI1 #6                    INIT
ST2:  JNII ST2 W             INC
ST3:  NOP                   R
      JNII ST3 W             INC      /D
ST4:  NOP                   R      /X /Y /D
      JNII ST4 W             INC      /X /Y
      JZD ST2
      STPS
END

```


◆ Example of Pattern Program when Modules are Used

```

MPAT M256K                ;sample program for 256K dynamic RAMs.

MODE MUX
REGISTER
    XMAX=#FF
    YMAX=#3FF
    IDX1=#3FFFE
SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX    ;increment base

MODULE BEGIN                ;checker board
START #0
    IDXI1 #6      INIT
ST0:  JNII ST0 W    INC      FP1
ST1:  JNII ST1 R    INC      FP1
    JZD ST0
    STPS
MODULE END

MODULE BEGIN                ;marching
START #10
    IDXI2 #6      INIT
ST1:  NOP      R
ST0:  JNII ST0 W    INC
    JNII ST1 W    INC      /D
ST2:  NOP      R      /X /Y /D
    JNII ST2 W    INC      /X /Y
    JZD ST0
    STPS
MODULE END

END

```

◆ Test Pattern Selection

```

PRO program-name
:
:
MEAS MPAT M256K
:
:
REG MPAT PC=#10 (or REG MPAT MAIN PC=#10)
MEAS MPAT M256K
:
:
END

MPAT M256K ;sample program for 256K dynamic RAMs.

MODE MUX
REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE
SDEF INIT=XB<0 YB<0 TP<0 ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base

MODULE BEGIN ;checker board
START #0
IDX12 #6 INIT
ST0: JN11 ST0 W INC FP1
ST1: JN11 ST1 R INC FP1
JZD ST0
STPS
MODULE END

MODULE BEGIN ;marching
START #10
IDX12 #6 INIT
ST0: JN11 ST0 W INC
ST1: NOP R
JN11 ST1 W INC /D
ST2: NOP R /X /Y
JN11 ST2 W INC /X /Y
JZD ST0
STPS
MODULE END

END

```

The diagram illustrates the flow of the test pattern selection process. It starts with the 'MEAS MPAT M256K' command in the 'PRO' block. A line from this command branches to the 'checker board' module (the first 'MODULE BEGIN' block) and the 'marching' module (the second 'MODULE BEGIN' block). Both modules have arrows pointing back to the 'MEAS MPAT M256K' command, indicating a loop or selection process.

◆ REG MPAT Statement Valid Range

```

PRO program-name

    MEAS MPAT filename
        :
        :
    REG MPAT PC=#10      ---- (1)
    MEAS MPAT filename
        :
        :
    MEAS MPAT filename    ;REG MPAT statement in (1) is invalid

END

(a)

PRO program-name
:
    REG MPAT PC=#10      ---- (2)
    DO LP01 I=3V, 5V, 0.2V
        VS1=I
        MEAS MPAT(3) filename
            :
            :
LP01:    CONT
        :
        :
        MEAS MPAT filename ;REG MPAT statement specified in (2) becomes valid
            :
            :
END

(b) MEAS MPAT(3)

PRO program-name
:
    DO LP01 I=3V, 5V, 0.2V
        VS1=I
        REG MPAT PC=#10    ;Code REG MPAT statement in the loop.
        MEAS MPAT filename
            :
            :
LP01:    CONT
:
:
END

(c) Shmoo

```

3. 3 Register Setting and Modification

When the same pattern program is used to test devices whose memory capacity is different, or when the execution sequence is changed for the device testing, use a REG MPAT statement for the test plan program to change the register.

```

      :
REG MPAT register-name = data (or REG MPAT MAIN register-name=data
MEAS MPAT filename           REG MPAT SUB register-name=data)
      :

```

Each of the registers listed in [3. 1 "Instructions" on page 3-2](#) can be set. If a value is specified in the REG MPAT statement, the value specified by the REGISTER statement in the pattern program becomes invalid. (["REG MPAT Statement Valid Range" on page 3-10](#))

The data specified in the REG MPAT statement is valid for only the first MEAS statement; the value specified in the pattern program is valid for subsequent MEAS statements. (["REG MPAT Statement Valid Range" on page 3-10](#))

To repeat execution of a test pattern such as Shmoo plotting, generate the test program as shown in (b) and (c) in ["REG MPAT Statement Valid Range" on page 3-10](#).

3. 4 Using Failure Analysis Memory

To reduce the test time, test pattern generation by ALPG is terminated if a failure is detected (if FAIL is determined).

To execute the test pattern to the end and store the results in the AFM, test pattern generation termination must be inhibited by specifying the following statement:

```
      :  
SELECT MPAT INHIBIT  
MEAS MPAT filename  
      :
```

The INHIBIT mode specified by this statement remains valid until it is released by the following statement:

```
SELECT MPAT NORMAL
```

If the INHIBIT mode is specified, only test termination is inhibited; GO/NOGO can be determined by the following statements:

```
IF PASS ~  
IF FAIL ~
```

◆ Example of Specifying SELECT MPAT INHIBIT

```

PRO program-name

TEST    1          ;functional test
      :
      MEAS MPAT filename
      :
TEST    2          ;fail data display

      RESET AFM ALL
      AFM MULTIMODE=SINGLE
      AFM DUTSEL=1
      AFM ADDRESS=X0-7, Y0-9
      AFM RATE=LS
      AFM BITSIZE=1
      AFM ACTION=FT>FMX

      SELECT MPAT INHIBIT ;Inhibits termination if FAIL is detected
      MEAS MPAT filename
      IF FAIL THEN ENTRY
        CALL FBMAP(...)
      EXIT

TEST    3          ;functional test
      :
      SELECT MPAT NORMAL ;Mode release
      MEAS MPAT filename
      :
      :

END

```

3. 5 Shmoo Test

The T5500 series memory test system obtains a Shmoo plot by either of the following two methods:

- Specify MANUAL and HOLD in the MEAS MPAT statement for obtaining a Shmoo plot, and call the subroutine (SHM2 or SHM3) for Shmoo plotting.
- Include a CALL statement that calls the subroutine for Shmoo plotting in the test plan program.

This section explains how to include the CALL statement that calls the subroutine for Shmoo plotting in the test plan program.

An example of the program is shown in ["Example of Shmoo Test Program" on page 3-15](#).

The subroutines for Shmoo plotting include subroutines for two- and three-dimensional Shmoo plotting.

The subroutines for Shmoo plotting are called by the following statements:

CALL SHM2 ("XXX, n") : Two-dimensional Shmoo plotting

CALL SHM3 ("XXX, n") : Three-dimensional Shmoo plotting

XXX indicates the name of the file in which the conditions for the X, Y, and Z axes for Shmoo plotting are stored. This file must be created with the controller commands before using this statement, as follows:

```
## TEST.┐
:
/SHM2.┐ (/SHM3.┐) ; Execute Shmoo plot
:
Enter conditions for X, Y, and Z axes with the CHANGE command.
:
Write the set conditions into file XXX with the WRITE command.
:
```

A test mode is specified for n.

➔ For more information, refer to [T5500 Series Tester Utility Program Reference Manual\(No. : 8256295\)](#).

The test pattern transferred before calling the Shmoo plot is executed during Shmoo plotting.

If conditions are set by the pattern program REG MPAT statement, the REG MPAT statement must be written before the CALL statement.

◆ Example of Shmoo Test Program

```
PRO Sample

TIME1=1MS:VS1
TIME2=1MS:IN

IN1=2.4V, 0.8V
OUT1=2.4V, 0.4V

RATE=270NS
ACLK1=0NS
:
:

PD1-8=IN1, XOR, ACLK1, BCLK1, CCLK1, SDM, <X0-7,Y0-7>
:
:
PC33=OUT1, STRB1, <D0>

SEND MPAT filename
REG MPAT PC=#10

CALL SHM2("xxxx, 1")
:
:
END
```


3. 6 Scramble Program Transfer

The T5500 series memory test system has an address descrambler to compensate for DUT address topology and decoder topology.

The address data for X and Y generated by the ALPG is converted by the address descrambler and applied to the DUT.

The conversion table stored in the address descrambler is generated by the scrambler program.

➔ For more information, refer to [ATL-51 Scramble Program Reference Manual\(No. : 8256294\)](#).

The following instructions transfer the data in the conversion table generated by the scramble program to the address descrambler.

SEND SCRDATA filename (or SEND SCRDATA MAIN(SUB) filename)

Specify this statement in the test plan program to transfer the data specified by the file name to the address descrambler.

SCRAM filename (or MAIN(SUB) SCRAM filename)

Specify this statement in the pattern program to transfer the data specified by the file name to the address descrambler.

Tip

If file names are specified in both the test plan program and pattern program, only the file name specified in the test plan program is valid.

Independently of the above data transfer, the following statements determine whether address conversion is performed by the address descrambler. MAIN PG and SUB PG can be specified independently.

SELECT SCRDATA NORMAL (or SELECT SCRDATA MAIN(SUB) NORMAL)

The following statements perform address conversion using the data transferred to the address descrambler:

SEND SCRDATA filename (Test plan program)

SCRAM filename (Pattern program)

SELECT SCRDATA DISABLE (or SELECT SCRDATA MAIN(SUB) DISABLE)

Address conversion by the address descrambler is unconditionally disabled.

SELECT SCRDATA ENABLE (or SELECT SCRDATA MAIN(SUB) ENABLE)

The address conversion disabled by SELECT SCRDATA DISABLE is enabled.

If the above statements are not specified, SELECT SCRDATA NORMAL is the default.

An example of SCRAM file specification in MPAT is shown below:

```
MPAT M256K

MODE MUX
REGISTER
    XMAX=0xFF

MODULE BEGIN
START    #0
IDX12    #6
    :
STPS
MODULE END

MODULE BEGIN
START    #10
IDX12    #6
    :
STPS
MODULE END

MODULE BEGIN
SCRAM S256K2 ;Specification of file name
START    #20
IDX12    #2
    :
STPS
MODULE END

END
```

If different conversion data is stored in the address descrambler in each pattern program, a file name can be specified by the statement shown in the example.

```
SCRAM filename
```

An example of SCRAM specification in a test plan program is shown below:

```

PRO P256K      ;Sample program for 256K dynamic RAMs.

    TIME=1MS : VS1
    :
    VS1=5V
    :
    PD1-8=IN1, XOR, ACLK1, BCLK1, CCLK1, SDM,<X0-7, Y0-7> ;A0-7
    :
    PC33=OUT1, STRB1,<D0>                                ;DOUT
    :

    RATE=260NS
    :

TEST  1
    REG MPAT PC=#0
    MEAS MPAT M256K

TEST  2
    SEND SCRDATA S256K1*1
    REG MPAT PC=#10
    MEAS MPAT M256K

TEST  3
    SELECT SCRDATA DISABLE*2
    REG MPAT PC=#10
    MEAS MPAT M256K

TEST  4
    SELECT SCRDATA ENABLE*3
    REG MPAT PC=#10
    MEAS MPAT M256K

TEST  5
    REG MPAT PC=#20
    MEAS MPAT M256K

END

```

*1 If no SCRAM filename is specified in the pattern program, the test is performed by transferring the address descrambler data with the SEND SCRDATA statement.

*2 To disable address conversion with the address descrambler after transferring the address descrambler data with SEND SCRDATA filename, use the following statement:

```
SELECT SCRDATA DISABLE (or SELECT SCRDATA MAIN(SUB) DISABLE)
```

**3 To release the address conversion disabled by `SELECT SCRDATA DISABLE`, use the following statement:*

```
SELECT SCRDATA ENABLE (or SELECT SCRDATA MAIN(SUB) DISABLE)
```

3. 7 Specifying Window Addresses

The memory test system for T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P has a window function that performs logical comparison only within a specific address range. MAIN PG and SUB PG can be specified independently.

The following statement specifies an address range:

WINDOW XADRS = a	] Address specification a, b = #0 - #FFFF
WINDOW XADRS MAIN	
WINDOW XADRS SUB	
WINDOW YADRS = b	
WINDOW YADRS MAIN	
WINDOW YADRS SUB	
WINDOW MODE = X	] Operation mode specification
WINDOW MODE = Y	
WINDOW MODE = X, Y	
WINDOW MODE MAIN	
WINDOW MODE SUB	

WINDOW MODE = DISABLE → Operation mode release

Data output from the DUT and the expected value pattern are logically compared in the cycle during which the control bits set in CPE1 to CPE4 are specified.

If the window mode is specified, the above logical comparison is performed only in the following cycle:

WINDOW MODE =	Condition for valid logical comparison
X	$X0 - 15 = XA0 - 15$
Y	$Y0 - 15 = YA0 - 15$
X, Y	$(X0 - 15 = XA0 - 15) \text{ .AND. } (Y0 - 15 = YA0 - 15)$

➔ For more information about the WINDOW statement, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

An example of a test plan program using the window function is shown below:

```

PRO P16K           ;Sample program for 16K static RAMs.

    TIME1=1MS :VS1                ;time sequence
    TIME2=1MS :IN

    VS1=5V                    ;power
    IN1=2.2V, 0.8V            ;driver level
    OUT1=2.4V, 0.4V           ;reference level

    PD1-7=IN1, NRZB, BCLK1, <X0-6>      ;A0-6
    PD8-14=IN1, NRZB, BCLK1, <Y0-6>     ;A7-13
    PD15=IN1, /NRZA, ACLK1, <C0>        ;/CS
    PD16=IN1, /RZO, BCLK2, CCLK2, <WT>   ;/WE
    PD34=IN1, XOR, ACLK3, BCLK3, CCLK3, <D0>;Din
    PC33=OUT1, STRB1, <D0>              ;Dout

    RATE=70NS
    ACLK1=0NS                    ;/CS
    BCLK1=0NS                    ;An
    BCLK2=10NS                   ;/WE leading
    CCLK2=55NS                   ;/WE trailing
    ACLK3=0NS                    ;Din
    BCLK3=10NS                   ;Din leading
    CCLK3=55NS                   ;Din trailing
    STRB1=70NS                   ;Dout

    CALL CALB("C16K", NORMAL)        ;calibration

TEST    1

    REG MPAT PC=#0
    MEAS MPAT M16K

TEST    2

    WINDOW MODE=X *1
    WINDOW XADRS=#1F *2

    REG MPAT PC=#10
    MEAS MPAT M16K

    WINDOW MODE=DISABLE *3

END

```

**1 Operation Mode Setting*

In the example shown above, logical comparison is made only when the X address applied to the DUT matches the address (#1F) set by WINDOW XADRS.

**2 Address Specification*

**3 Operation Mode Release*

The operation mode is valid until this instruction is executed.

4. Programming I

This chapter describes how to write pattern programs and test plan programs for power supply current measurement, output voltage measurement, DRAM tests, bump tests and partial tests.

4. 1 Power Supply Current Measurement

The current measurement function in the bias power supply measures the average power supply current. The current is measured by executing the pattern program as shown in ["Example of a Power Supply Current Measurement Program" on page 4-2](#), and by using the MEAS statement after making the device under test operational.

Pattern program

Code the pattern program whereby an endless loop is formed.

Since pattern generation is stopped if the logical comparison result is a failure, inhibit pattern generation termination during power supply current measurement, as follows: (["Example of a Power Supply Current Measurement Program" on page 4-2](#))

- Mask logical comparison by disabling CPE1 and CPE2
Select "FIXL" (write FL in the test plan program) for CPE1 and CPE2 in the pattern program or test plan program.
- Inhibit termination if a failure occurs in the test plan program.
Inhibit termination if a failure occurs, by specifying the following statement in the test plan program:

```
SELECT MPAT INHIBIT
```

Once this statement is specified, it inhibits termination until the following statement is executed. Therefore, if a functional test or other tests are to be performed next, clear the mode by specifying the following statement:

```
SELECT MPAT NORMAL
```


◆ Example of a Power Supply Current Measurement Program

```

MPAT  program-name

      MODE MUX
      REGISTER
      CPE1=FIXL *1
      CPE2=FIXL *1

      :
      :

      MODULE BEGIN
      REGISTER
      CPE1=FIXL
      CPE2=FIXL
      :
      : Power supply current measurement           ;Forms an endless loop

      MODULE END
      :
      :
END

PRO  program-name
      :
      :

      SEND MPAT filename
      REG MPAT PC=#10
      SELECT MPAT INHIBIT
      SRON
      START MPAT *
      WAIT TIME t
      MEAS VS1 ;IDD
      STOP MPAT
      :
      :
      SELECT MPAT NORMAL
END

```

**1 If specified by the common section, these statements are valid for all modules.*

Test plan program

The power supply current is measured by the current measurement function in the bias power supply after pattern generation is started, by performing reference-on for the bias power supply and driver voltage supply as shown below.

```

PRO program-name

    IN1=2.4V, 0.8V
    :
    RATE=260NS
    ACLK1=0NS
    :
    :
    VS1=5V, R5V, M800MA, 100MA, -100MA *1
    SEND MPAT filename *2
    TIME1=t1:VS1
    TIME2=t2:IN
    SELECT MPAT INHIBIT
    REG MPAT PC=#10
    SRON *3
    START MPAT * *4
    WAIT TIME t3 *5
    MEAS VS1 *6
    STOP MPAT *7

END

```

**1 Setting the bias power supply conditions*

**2 SEND MPAT filename*

Transfer the pattern program to the ALPG.

If a pattern program with the same file name has already been transferred to the ALPG, the pattern program is not transferred.

**3 Performing reference-on*

Perform reference-on for the bias power supply and driver voltage supply.

The order for turning on the powers is specified by the TIME statement.

This setting can be omitted. (If the SRON statement is omitted, the next START MPAT statement performs reference-on.)*

4. 1 Power Supply Current Measurement

***4 START MPAT ***

Start pattern generation and proceed to the next statement.

When pattern generation is performed with this instruction, it must be terminated by specifying the following statement before starting the next pattern program transfer and pattern generation:

STOP MPAT

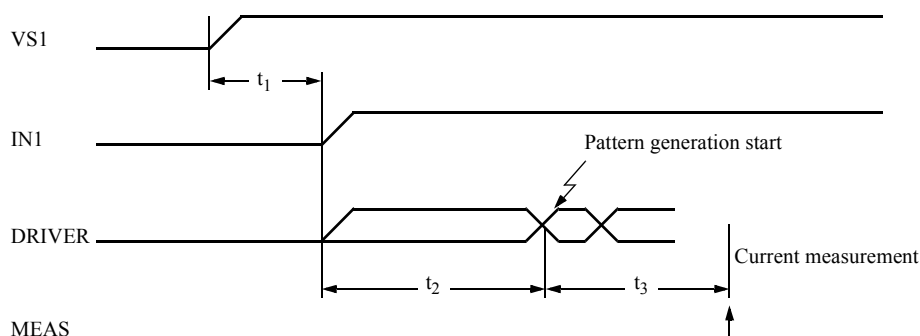
The register data in pattern program is modified with the REG statement.

The data specified by the REG statement is valid only for the first START MPAT after the specification.*

***5 WAIT TIME t**

This statement adjusts the time from application of the pattern to the DUT until the power supply current measurement.

The above time is shown below.

***6 MEAS VS1**

Current is measured by the current measurement function in the bias power supply.

***7 STOP MPAT**

Terminates pattern generation.

Clears the REG MPAT statement specification.

Use this statement with START MPAT.*

4. 2 Output Voltage Measurement (When output voltage is maintained)

When the output voltage from the DUT is maintained, as in a static memory, the output voltage is measured by the DC parametric test unit ISVM function after pattern generation for DUT initialization.

Pattern program

The pattern program is created so that the read data is held after data (0 or 1) to be measured is written into the DUT.

This program is created by the following three methods:

1. Form a loop in the read cycle.

If clock pulses are required to maintain the output voltage, the pattern program is created so that the read cycle forms an endless loop as shown in ["Example of VO Measurement Pattern Program \(endless loop\)" on page 4-6.](#)

2. Stop pattern generation with the STPS instruction.

Stop pattern generation with the STPS instruction. (Refer to ["Example of VO Measurement Pattern Program \(stop with STPS\)" on page 4-6.](#))

Create the program so that the STPS instruction cycle is a read cycle.

When pattern generation is stopped by the STPS instruction, two read cycles are applied to the DUT.

3. Temporarily stop pattern generation with the WAIT instruction.

Temporarily stop pattern generation with the WAIT instruction. (Refer to ["Example of VO Measurement Pattern Program with WAIT Instruction" on page 4-7.](#))

Create the program so that the WAIT instruction cycle is a read cycle.

The WAIT instruction temporarily stops pattern generation. Restart by specifying the following instruction:

```

:
START MPAT (1)
:
```

4. 2 Output Voltage Measurement (When output voltage is maintained)

◆ Example of VO Measurement Pattern Program (endless loop)

```

MPAT  program-name

      REGISTER
      XMAX=#7F
      XMAX=#F

      START  #0
      NOP                    XB<0 YB<0 TP<0
      NOP                    W   X<XB Y<YB
ST0:   JMP      ST0          X<XB Y<YB

      END

```

◆ Example of VO Measurement Pattern Program (stop with STPS)

```

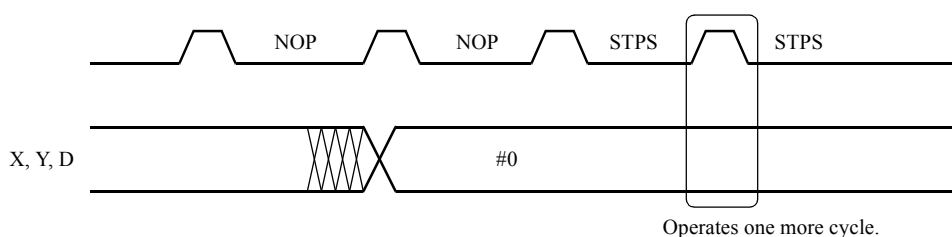
MPAT  program-name

      REGISTER
      XMAX=#7F
      XMAX=#F

      START  #0
      NOP                    XB<0 YB<0 TP<0
      NOP                    W   X<XB Y<YB
      STPS                                ;Data is applied to MUT in two cycles.

      END

```



4. 2 Output Voltage Measurement (When output voltage is maintained)

◆ Example of VO Measurement Pattern Program with WAIT Instruction

```

MPAT  program-name

    REGISTER
        XMAX=#7F
        XMAX=#F

    START  #0
    NOP                    XB<0 YB<0 TP<0
    NOP      W            X<XB Y<YB          ;write 0
    WAIT     X<XB Y<YB          ;read 0
    NOP      W            X<XB Y<YB /D       ;write 1
    WAIT     X<XB Y<YB          ;read 1
    STPS

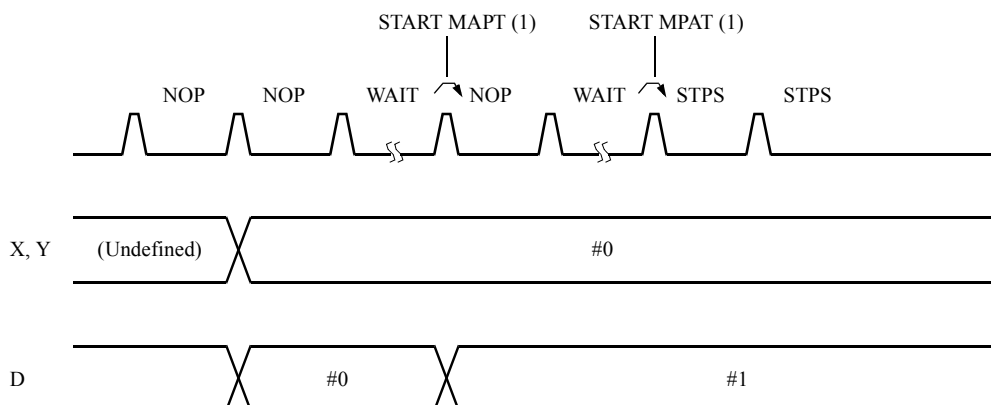
END

PRO  program-name

    SEND MPAT filename
    :
    :
    START MPAT
    :
    VOL measurement
    :
    START MPAT(1)
    :
    VOH measurement
    :
    STOP MPAT

END

```



4. 2 Output Voltage Measurement (When output voltage is maintained)

Test plan program

An example of a test plan program is shown in ["Example of VO Measurement Test Plan Program" on page 4-8](#). The program is created using the procedure below.

After the performing of reference-on for the bias power and driver voltage supply, pattern generation is started, then the output voltage is measured by the DC ISVM function.

◆ Example of VO Measurement Test Plan Program

```

PRO program-name          ; 8 bits I/O

    TIME1=1MS:VS1          ] *1
    TIME2=1MS:IN, DC
    SELECT DCOFF

    IN1=2.4V, 0.8V
    VS1=5V
    :
    RATE=200NS:16
    :
    P1-7=IN1, XOR, ACLK1, BCLK1, <X0-6>
    P8-11=IN1, XOR, ACLK1, BCLK1, <Y0-3>
    :
    P33-40=IN1, OUT1, IOC,.....,<D0-7>
    :
    SEND MPAT filename
    SELECT MPAT INHIBIT
    :
    ISVM=4MA, R8MA, M8V, 6V, -1V
    SETTLING TIME=1MS, 1MS
    SRON
    :
    REG MPAT PC=#0
    START MPAT *
    WAIT TIME 1MS
    :
    MEAS DC (PC33-40)
    :
    STOP MPAT

    ISVM=-1MA, R8MA, M8V, 6V, -1V
    SRON
    :
    REG MPAT PC=#10
    START MPAT *
    WAIT TIME 1MS
    :
    MEAS DC (PC33-40)
    :
    STOP MPAT
    :
    :
END

```

*1 Set the reference-on time sequence. (See [Figure 4-1](#).)

4. 2 Output Voltage Measurement (When output voltage is maintained)

**2 Set test conditions.*

Set conditions for writing data into the DUT. These conditions must be the same as those for the functional test. During the output voltage measurement, inhibit pattern generation termination when a failure is detected, without performing comparison with the comparator.

**3 Set the ISVM mode.*

To measure V_{OL} , set I_{OL} to 4mA (this value depends on the DUT used).

**4 Set the time from application of current to the MUT until measurement.*

This depends on the current value and range. (See [Figure 4-1.](#))

**5 Perform reference-on. (See [Figure 4-1.](#))*

This setting can be omitted. (If the SRON statement is omitted, the next START MPAT statement performs reference-on.)*

**6 Start the ALPG and wait until V_{OH} or V_{OL} has been valid.*

This depends on the type of ALPG pattern program.

- *When the pattern loops in the read cycle*

```
REG MPAT PC = #0
START MPAT*           ; Starts the PG
WAIT TIME 1MS         ; Waits until  $V_{OH}$  /  $V_{OL}$  is determined
MEAS DC(PC33 - 40)    ; Measurement
STOP MPAT             ; Stops the PG.
```

- *When the pattern is stopped by STPS*

```
REG MPAT PC = #0
START MPAT
MEAS DC(PC33 - 40)
```

- *When the pattern is stopped by WAIT*

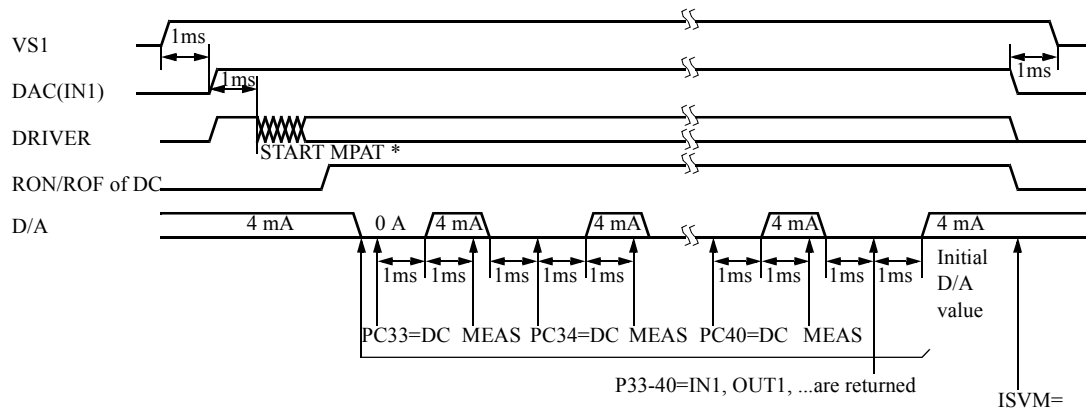
```
REG MPAT PC = #0
START MPAT
MEAS DC(PC33 - 40)
STOP MPAT           ; Required when restart is not performed
:
START MPAT(1)       ; Required when restart is performed
MEAS DC(PC33 - 40)
STOP MPAT
```

To measure V_{OH} and V_{OL} continuously, stop pattern generation with WAIT, and restart it with START MPAT (1).

**7 Measure the output voltage of output pins using ISVM.*

Output pins are switched by the timing shown in [Figure 4-1.](#)

4. 2 Output Voltage Measurement (When output voltage is maintained)

Figure 4-1 Example of VO Measurement Timing

4. 3 DRAM Test

4. 3. 1 Address Multiplexing

["Example of a Pattern Program Using the Address Multiplex Method" on page 4-12](#) shows a pattern program for testing a DUT that uses the address multiplex method for address input.

["Example of a Test Plan Program Using the Address Multiplex Method" on page 4-15](#) shows a test plan program used for this purpose. The program is created using the procedure below.



DUTs that use the address multiplex method for address input can be tested only in normal mode (1WAY).

Pattern program

◆ Example of a Pattern Program Using the Address Multiplex Method

```

MPAT M256K

MODE MUX      ;sample program for 256K dynamic RAMs.

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

  SDEF INIT=XB<0 YB<0 TP<0      ;clear
  SDEF INC=XB<XB+1 YB<YB+1^BX   ;increment base

  MODULE BEGIN                  ;checker board
  START #0

  IDXI2 #6      INIT
ST0:  JNII ST0 W      INC      FP1
ST1:  JNII ST1 R      INC      FP1
      JZD ST0
      STPS

  MODULE END

  MODULE BEGIN                  ;marching
  START #10

  IDXI2 #6      INIT
ST2:  JNII ST2 W      INC
ST3:  NOP      R
      JNII ST3 W      INC      /D
ST4:  NOP      R      /X /Y /D
      JNII ST4 W      INC      /X /Y
      JZD ST2
      STPS

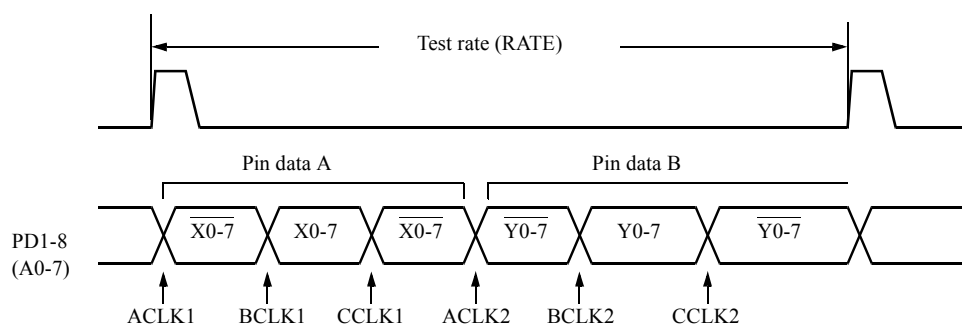
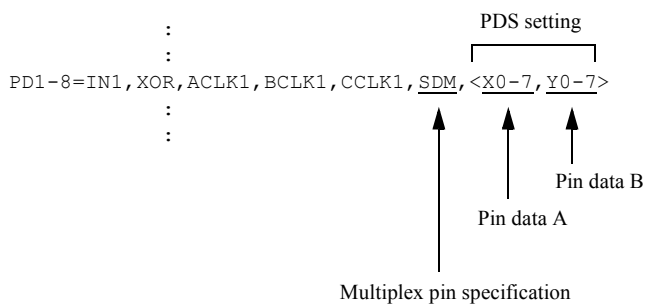
  MODULE END

END

```

When testing DUTs that use the address multiplex method for address input, specify the multiplex mode in the pattern program.

When the multiplex mode is specified, data is applied to the pins specified in the test plan program as follows:



When the multiplex mode is specified in the pattern program, the cycle for which control bits M1 and M2 are specified operates as follows:

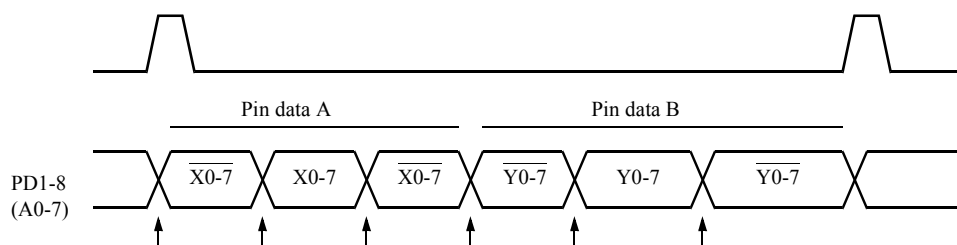
Use this mode for the refresh test and page mode test to be explained later.

```

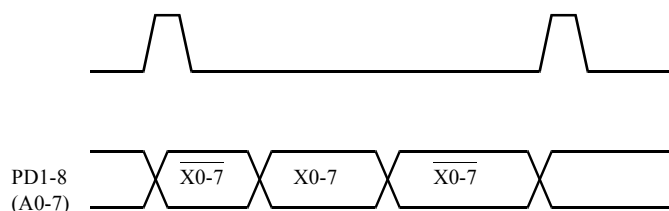
:
PD1 - 8 = IN1, XOR, ACLK1, BCLK1, CCLK1, SDM, < X0 - 7, Y0 - 7 >
:

```

- When neither M1 nor M2 is specified



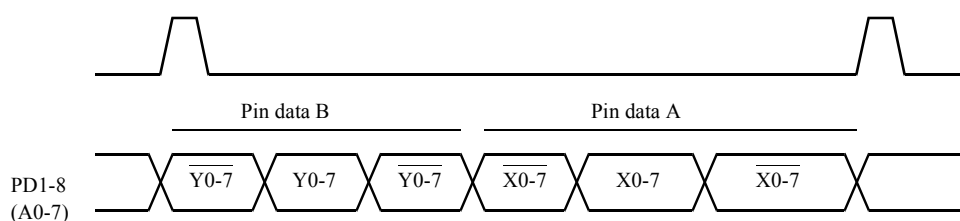
- M1 (used for RAS only refresh)



Tip

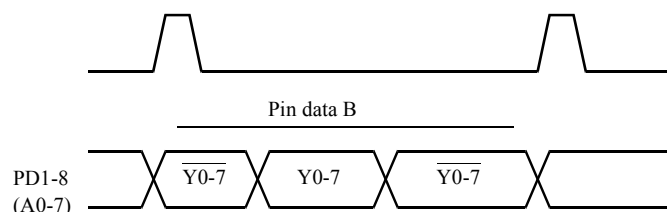
In this cycle, the clocks $ACLK2$, $BCLK2$, and $CCLK2$ for the output of pin data B ($Y0-7$) are suppressed. To disable the output, switch the timing set and then use the clock OPEN.

- M2



Pin data A and pin data B are switched with each other.

- M1, M2 (used for generation of page cycles)



Tip

In this cycle, output of the clocks $ACLK1$, $BCLK1$, and $CCLK1$ to output pin data A ($X0-7$) is disabled. To disable the output, switch the timing set and then use the clock OPEN.

Test plan program

◆ Example of a Test Plan Program Using the Address Multiplex Method

```

PRO P256K ;Sample program from 256K dynamic RAMs.
TIME1=1MS :VS1 *1
TIME2=1MS :IN ]

VS1=5V
IN1=2.4V,0.8V *2
OUT1=2.4V,0.4V ]

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK1,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

TR=1NS & TF=1NS ;Timing set
RATE=260NS
ACLK1=0NS
BCLK1=10NS
CCLK1=10NS+15NS+TF+TR
ACLK2=35NS
BCLK2=45NS
CCLK2=45NS+25NS+TF+TR
ACLK3=0NS
BCLK3=10NS
CCLK3=10NS+TF+150NS
BCLK4=45NS
CCLK4=45NS+TF+75NS
BCLK5=45NS
CCLK5=45NS+TF+25NS
BCLK6=45NS
CCLK6=45NS+TF+25NS+TR & STRB1=45NS+TF+75NS

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET) @
TCHANGE PD11(RESET)?
CALL CALB("C256K",TCAL)

TEST 1

REG MPAT PC=#0 *8
MEAS MPAT M256K *9

TEST 2

REG MPAT PC=#10
MEAS MPAT M256K

END

```

*1 Set the power-on sequence.

*2 Set the power supply (VS1), driver level (IN1), and comparison voltages (OUT1).

*3 Set the pin data.

*4 Set the pins for multiplexing.

- *5 Set the pin data (PDS).
- *6 Set the timing.
- *7 Timing calibration
- *8 Select a test pattern with the REG statement.
- *9 Test

4. 3. 2 Pause Test

An example of a pause test pattern program is shown below.

◆ Example of Pause Test Pattern Program

```

MPAT M256K                      ;sample program for 256K dynamic RAMs.

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX   ;increment base

MODULE BEGIN                   ;PAUSE

MODE PAUSE    *1

REGISTER
  TIMER=4MS    *2

START  #20

  IDXI2 #6      INIT
ST0:   JN11 ST0 W      INC
      NOP      T    *3      ; PAUSE
ST1:   JN11 ST1 R      INC
      JZD ST0
      STPS

MODULE END

END

```

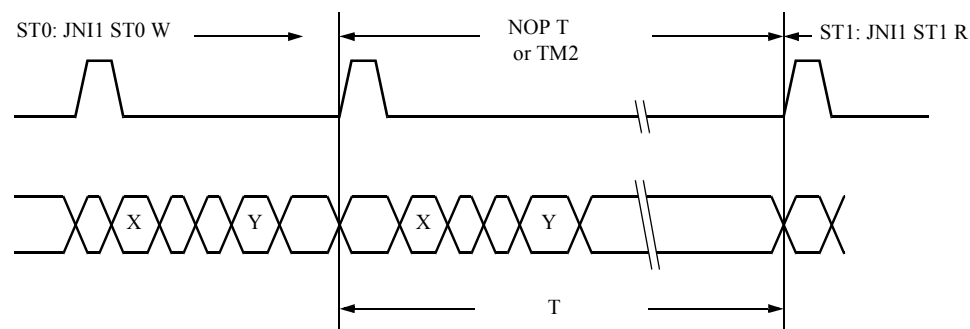
- *1 Set the pause mode.

*2 Set the timer data.

$TIMER = 100NS \text{ to } 409.5S$

$TPAUSE = 100NS \text{ to } 409.5S$

*3 Pattern generation is stopped during the following period in the cycle in which control bit T or $TM2$ is specified.



$$T = \text{setting} + 4 \times \text{RATE} + \text{error}$$

Period of the cycle in which T or $TM2$ is specified.

The table below shows the stop time errors during pattern generation:

Test system	Error
T5593	9.0 μs to 9.5 μs
For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P	4.1 μs to 4.6 μs

4. 3. 3 Refresh Test

◆ Specifying the Auto Refresh Mode

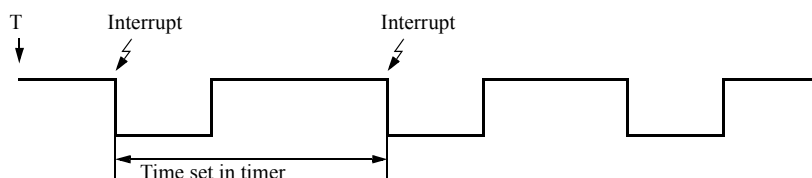
The refresh test can be performed with the internal timer and interrupt mechanism.

In the pattern program in which the refresh mode is specified, the internal timer is started in the cycle during which control bit T or $TM1$ is specified.

There are two timer operation modes:

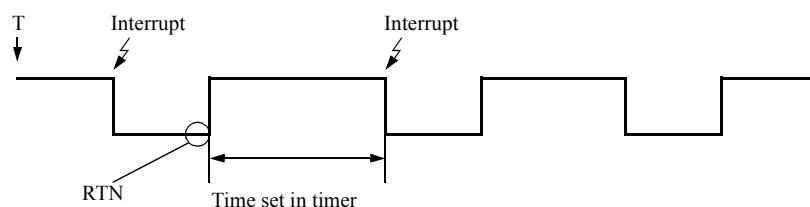
- REFRESHA n

When the timer operation is started with T or TM1, an internal interrupt occurs during the time interval specified in the internal timer.



- REFRESHB n

When the timer operation is started with T or TM1, an internal interrupt occurs after the time specified in the internal timer, and the internal timer stops. The internal timer is restarted by specifying RTN in the interrupt processing program.



When an internal interrupt occurs, ALPG operates as shown in [Figure 4-2](#). Refresh addresses are generated by the refresh counter (RF) during refresh test.

The refresh counter is incremented by the following instruction:

$$RF < RF + 1$$

The refresh counter is cleared at the beginning of the test.

The refresh address stored in the refresh counter is output by the X<RF instruction to X0 to X17 (X0 to X15 for T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P).

```
MODE REFRESHA n
:
X<RF
:
```

n specifies the bit which outputs the contents of the refresh counter to address X.

Table 4-1 Output Bits of the Refresh Counter (T5593)

n	X0 to X17	Y0 to Y17
64	(RF0-5)→ X0-5, (X6-17)t-1→ X6-17	(Y0-17)t-1
128	(RF0-6)→ X0-6, (X7-17)t-1→ X7-17	(Y0-17)t-1
256	(RF0-7)→ X0-7, (X8-17)t-1→ X8-17	(Y0-17)t-1
512	(RF0-8)→ X0-8, (X9-17)t-1→ X9-17	(Y0-17)t-1
1024	(RF0-9)→ X0-9, (X10-17)t-1→ X10-17	(Y0-17)t-1

n	X0 to X17	Y0 to Y17
2048	(RF0-10)→ X0-10, (X11-17)t-1→ X11-17	(Y0-17)t-1
4096	(RF0-11)→ X0-11, (X12-17)t-1→ X12-17	(Y0-17)t-1
8192	(RF0-12)→ X0-12, (X13-17)t-1→ X13-17	(Y0-17)t-1
16384	(RF0-13)→ X0-13, (X14-17)t-1→ X14-17	(Y0-17)t-1
32768	(RF0-14)→ X0-14, (X15-17)t-1→ X15-17	(Y0-17)t-1
65536	(RF0-15)→ X0-15, (X16-17)t-1→ X16-17	(Y0-17)t-1
131072	(RF0-16)→ X0-16, (X17)t-1→ X17	(Y0-17)t-1
262144	(RF0-17)→ X0-17	(Y0-17)t-1

(Xn-17) t-1 and (Y0-17) t-1 display the address data which was output to X and Y in the previous cycle.

Table 4-2 Output Bits of the Refresh Counter (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)

n	X0 to X15	Y0 to Y15
64	(RF0-5)→ X0-5, (X6-15)t-1→ X6-15	(Y0-15)t-1
128	(RF0-6)→ X0-6, (X7-15)t-1→ X7-15	(Y0-15)t-1
256	(RF0-7)→ X0-7, (X8-15)t-1→ X8-15	(Y0-15)t-1
512	(RF0-8)→ X0-8, (X9-15)t-1→ X9-15	(Y0-15)t-1
1024	(RF0-9)→ X0-9, (X10-15)t-1→ X10-15	(Y0-15)t-1
2048	(RF0-10)→ X0-10, (X11-15)t-1→ X11-15	(Y0-15)t-1
4096	(RF0-11)→ X0-11, (X12-15)t-1→ X12-15	(Y0-15)t-1
8192	(RF0-12)→ X0-12, (X13-15)t-1→ X13-15	(Y0-15)t-1
16384	(RF0-13)→ X0-13, (X14-15)t-1→ X14-15	(Y0-15)t-1
32768	(RF0-14)→ X0-14, (X15)t-1→ X15	(Y0-15)t-1
65536	(RF0-15)→ X0-15	(Y0-15)t-1

(Xn-15) t-1 and (Y0-15) t-1 are address data output to X and Y in the previous cycle.



*RLMAX in the register statement may be used instead of the refresh counter n specification.
The following two specifications are the same:*

◆ **Example**

```
MODE REFRESHA n
```

```
RLMAX=n-1
MODE REFRESHA
```

Interrupt mask by the control bit "I"

• **MODE REFRESHA n**

When an interrupt occurs in a cycle where control bit "I" is set, the interrupt is processed (PC<ISP) in the next cycle in which control bits are not set. ((2) in [Figure 4-2](#))

The timer restarts at the same time as the interrupt whether or not the bit "I" was set. ((4) in [Figure 4-2](#))

As a result, intervals between interrupts depends on how many control bits are set.

The time from interrupt processing (1) to (2) in [Figure 4-2](#) is longer than t_2 (timer setting value + setting error) and the time from interrupt processing (2) to (3) is shorter than t_2 (timer setting value + setting error). Calculate the error caused by setting the control bit as shown below.

The maximum time between two consecutive interrupts =

{maximum time of t_2 } + {(the maximum execution number of "I") * (maximum RATE)}

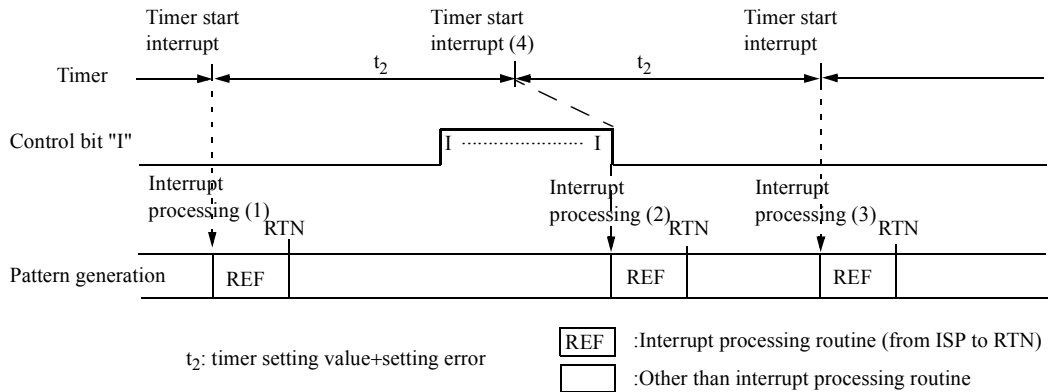
The minimum time between two consecutive interrupts =

{maximum time of t_2 } - {(the maximum execution number of "I") * (maximum RATE)}

"Maximum RATE" is the longest RATE used for one test. For the minimum and maximum times of t_2 , see t_2 in [Figure 4-4](#). The same applies to the time between the control bit "T (TM1)" and interrupt processing.

The interrupt processing routine cannot be executed correctly if an interrupt occurs in this routine. It should be avoided with the following methods:

1. Set the timer and RATE so that the minimum time between two interrupts is smaller than {(the number of patterns in an interrupt processing routine) * (maximum RATE)}.
2. If 1 cannot be satisfied, set the control bit "I" for all interrupt processing routines.

Figure 4-2 Operation by Control Bit "I" in MODE REFRESH_A n

- **MODE REFRESH_B n**

When an interrupt occurs in a cycle where control bit "I" is set, the interrupt is processed (PC<ISP) in the next cycle in which control bits are not set. ((6) in [Figure 4-3](#))

As a result, the interval between RTN and an interrupt depends on the number of cycles in which "I" is repeated in succession.

The time from RTN (5) to interrupt processing (6) shown in [Figure 4-3](#) is longer than t_3 (timer setting value + setting error).

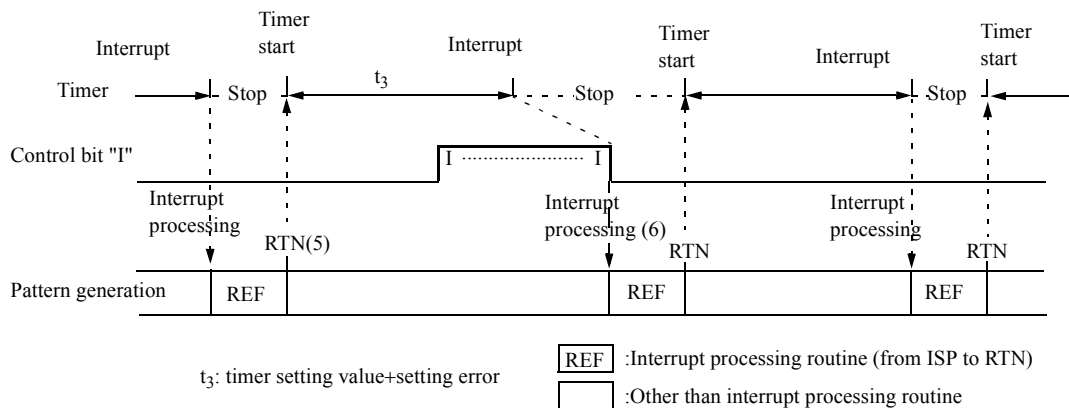
Calculate the error caused by setting the control bit as shown below.

The maximum time between RTN and an interrupt =

{maximum time of t_3 } + {(the maximum execution number of "I") * (maximum RATE)}

The minimum time between RTN and an interrupt = {minimum time of t_3 }

"Maximum RATE" is the longest RATE used for one test. For the minimum and maximum times of t_3 , see t_3 in [Figure 4-4](#). The same applies to the time between the control bit "T (TM1)" and interrupt processing.

Figure 4-3 Operation by Control Bit "I" in MODE REFRESH_B n

◆ Distributed Refresh and Burst Refresh

Examples of distributed refresh and burst refresh programs are shown below.

◆ Distributed Refresh and Burst Refresh Pattern Program (in 1WAY mode) (1/2)

```

MPAT REFRESH      ;Sample program for refresh test.

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  LMAX=#3FF
  HMAX=#FF
  IDX1=#3FFFE

SDEF  INIT=XB<0 YB<0 TP<0
SDEF  YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB
SDEF  REF =RF<RF+1 X<RF
           *5          *6

MODULE BEGIN      ;distributed refresh

MODE REFRESHA 256 *1

REGISTER
  TIMER=16US      *2
  ISP=ST2          *3

START  #0
NOP
IDX12  #5
T*7 INIT
ST0:   JN11  ST0    W      YINC  FP1
ST1:   JN11  ST1    R      YINC  FP1
      JZD    ST0
      STPS
ST2:   RTN                      REF *4

MODULE END

```

◆ Distributed Refresh and Burst Refresh Pattern Program (in 1WAY mode) (2/2)

```

MODULE BEGIN      ;burst refresh

MODE REFRESHA 256      *1

REGISTER
  TIMER=4MS          *2
  ISP=ST4             *3

START #10
NOP      T*7  INIT
IDX12 #5

ST2:  JN11  ST2  W      YINC  FP1
ST3:  JN11  ST3  R      YINC  FP1
      JZD   ST3
      STPS

ST4:  NOP                      REF
      IDX11 #FC                REF
      RTN                      REF      *4

MODULE END

END

```

◆ Distributed Refresh and Burst Refresh Pattern Program (in 2 WAY mode)

```

MPAT REFRESH <2WAY>

MODULE BEGIN      ;burst refresh(2way)

MODE REFRESHA 256      *1

REGISTER
  TIMER=4MS          *2
  ISP=ST2             *3
  IDX1=#1FFFE          ;normal #3FFFE

START #10
NOP      T*7  INIT
IDX12 #1

ST0:  JN11 ST0  W      YINC  FP1
ST1:  JN11 ST1  R      YINC  FP1
      JZD   ST0
      STPS

ST2:  NOP                      REF
      IDX11 #FC                REF
      RTN                      REF      *4

MODULE END

END

```

RF<RF+1 is expanded in the first step of interleaving two steps by the MPAT compiler as shown below.

Therefore, the IDX11 #FC value does not change even in the 2WAY mode.
In the same way, RF<RF+1 is expanded in the first step of interleaving four steps in the 4 WAY mode.

```
ST2:      NOP      : REF
          :          ; dummy
          IDX11 #FC : REF
          :          ; dummy
          RTN       : REF
          :          ; dummy
```

- *1 Refresh mode specification
 - REFRESHA n(n = 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, or 262144)
The interval during which interrupts occur is shown in [Table 4-3](#).
 - REFRESHA n(n = 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, or 262144)
The interval during which interrupts occur is shown in [Table 4-4](#).
- *2 Set the timer data
- *3 Branch destination specification if an interrupt occurs
- *4 Interrupt processing cycle description
- *5 Instruction that increments the refresh counter (RF)
- *6 Instruction that outputs the refresh address stored in the refresh counter
- *7 Timer Start Instruction (T or TM1)

Figure 4-4 Refresh Cycle

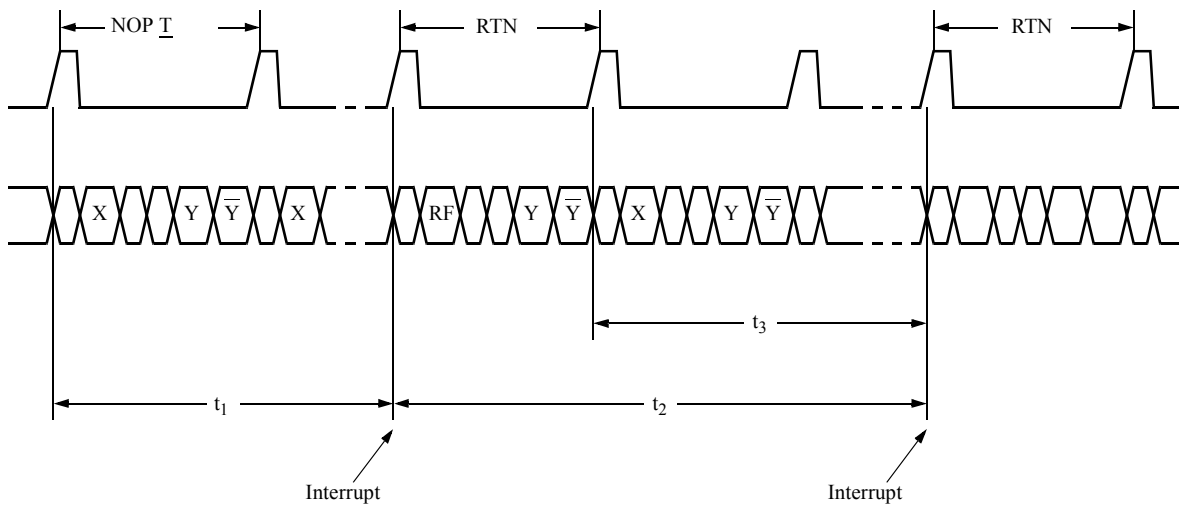


Table 4-3 Refresh Period (for REFRESH A)

Test system	Interleave mode	Time/ number of times	t_1^{*1}	t_2^{*1}	t_3^{*1}
T5593	In 1WAY mode	Time	Set value + (195 to 220 cycles) + (400 to 500 ns)	Set value + (-13 to 13 cycles)	-----
		Number of times	Set value + (199 cycles)	Set value	-----
	In 2WAY mode	Time	Set value + (390 to 440 cycles) + (400 to 500 ns)	Set value + (-26 to 26 cycles)	-----
		Number of times	Set value × 2 + (398 cycles)	Set value × 2	-----
For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P	In 1WAY mode	Time	Set value + (120 to 146 cycles) + (250 to 350 ns)	Set value + (-13 to 13 cycles)	-----
		Number of times	Set value + (125 cycles)	Set value	-----
	In 2WAY mode	Time	Set value + (240 to 292 cycles) + (250 to 350 ns)	Set value + (26 to 26 cycles)	-----
		Number of times	Set value × 2 + (250 cycles)	Set value × 2	-----
	In 4WAY mode	Time	Set value + (480 to 584 cycles) + (250 to 350 ns)	Set value + (-52 to 52 cycles)	-----
		Number of times	Set value × 4 + (500 cycles)	Set value × 4	-----
	In PCC 2 mode	Time	Set value + (120 to 148 cycles) + (250 to 350 ns)	Set value + (0 to 15 cycles)	-----
		Number of times	Set value + (125 to 127 cycles)	Set value + (0 to 2 cycles)	-----

Table 4-4 Refresh Period (for REFRESH B)

Test system	Interleave mode	Time/number of times	t_1^{*1}	t_2^{*1}	t_3^{*1}
T5593	In 1WAY mode	Time	Set value + (195 to 220 cycles) + (400 to 500 ns)	----	Set value + (15 to 41 cycles) + (400 to 500 ns)
		Number of times	Set value + (199 cycles)	----	Set value + (20 cycles)
	In 2WAY mode	Time	Set value + (390 to 440 cycles) + (400 to 500 ns)	----	Set value + (30 to 82 cycles) + (400 to 500 ns)
		Number of times	Set value $\times$ 2 + (398 cycles)	----	Set value $\times$ 2 + (40 cycles)
For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P	In 1WAY mode	Time	Set value + (120 to 146 cycles) + (250 to 350 ns)	----	Set value + (10 to 36 cycles) + (250 to 350 ns)
		Number of times	Set value + (125 cycles)	----	Set value + (15 cycles)
	In 2WAY mode	Time	Set value + (240 to 292 cycles) + (250 to 350 ns)	----	Set value + (20 to 72 cycles) + (250 to 350 ns)
		Number of times	Set value $\times$ 2 + (250 cycles)	----	Set value $\times$ 2 + (30 cycles)
	In 4WAY mode	Time	Set value + (480 to 584 cycles) + (250 to 350 ns)	----	Set value + (40 to 144 cycles) + (250 to 350 ns)
		Number of times	Set value $\times$ 4 + (500 cycles)	----	Set value $\times$ 4 + (60 cycles)
	In PCC 2 mode	Time	Set value + (120 to 148 cycles) + (250 to 350 ns)	----	Set value + (10 to 38 cycles) + (250 to 350 ns)
		Number of times	Set value + (125 to 127 cycles)	----	Set value + (15 to 17 cycles)

**1 t_1 , t_2 , and t_3 indicate t_1 , t_2 , and t_3 respectively in [Figure 4-4](#).*

TIMER set value must be within the following range:

TIMER= (time) 200 ns to 409.5 s or

TREFRESH (number of times) 2 to 4095

When TIMER is set in time notation for the RTTC test that changes RATE values, please note the following.

- t_1 , t_2 , and t_3 are calculated using the formula $\{(\text{set value}) + (\text{number of cycles}) + (\text{time})\}$. The number of cycles indicates cycle errors and time indicates synchronization errors. However, t_2 does not include synchronization errors. Minimum and maximum cycle errors of t_1 , t_2 and t_3 are as shown below.

t_1 minimum cycle error: The minimum RATE of patterns not used for the Refresh routine.

t_1 maximum cycle error: The maximum RATE of patterns not used for the Refresh routine.

t_2 and t_3 minimum cycle error: The minimum RATE of patterns including patterns to be used for the Refresh routine.

t_2 and t_3 maximum cycle error: The maximum RATE of patterns including patterns to be used for the Refresh routine.

- t_2 and t_3 includes the following error type in addition to the cycle and synchronization error types.

$$(\text{RATE error}) = \pm \{(\text{Maximum RATE of all patterns}) - (\text{Minimum RATE of all patterns})\} * N$$

Here, patterns to be used for the refresh routine are also included with all patterns.

N should be as follows:

T5593

N = 186 in 1WAY mode, N = 372 in 2WAY mode, or N = 744 in 4WAY mode

T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

N = 115 in 1WAY mode, N = 230 in 2WAY mode, or N = 460 in 4WAY mode

Accordingly, t_2 and $t_3 = (\text{Set value}) + (\text{Cycle error}) + (\text{Synchronization error}) \pm (\text{RATE error})$

When the RATE error is larger than the value of {(Set value) + (Minimum cycle error) + (Minimum synchronization error)}, set the number of cycles for the timer.

```

MPAT SAMPLE
MODE MUX REFRESHB 256
REGISTER
  XMAX=#FF
  YMAX=#3FF
  TIMER=1MS
  ISP=ST2
  IDX1=#3FFFE

  START #0
  NOP      XB<0      YB<0      TP<0 /T1
ST0: JN11 ST0 XB<XB+1 YB<YB+1^BX W FP1 /T1
ST1: JN11 ST1 XB<XB+1 YB<YB+1^BX R FP1 /T2
      JZD ST0 /T1
      STPS

ST2 :NOP      RF<RF+1 X<RF /T3
      IDXI2 #FC RF<RF+1 X<RF /T4
      RTN      RF<RF+1 X<RF /T3
END

```

When running the pattern program shown above with RATE = 50 NS, 100 NS, 150 NS, 1 US, t₁ and t₃ shown in [Figure 4-4](#) are given as described below. (This is an example for T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P.)

In this case, the maximum RATE and minimum RATE produced by the pattern program are listed below.

Pattern	Maximum RATE	Minimum RATE
Patterns which are not used for the refresh routine	100 ns(TS2)	50 ns(TS1)
All Patterns	1 μs(TS4)	50 ns(TS1)

The pattern to be used for the refresh routine is created by the ST2 label to RTN instruction in the above pattern program.

The 1WAY mode described in [Table 4-3](#) and [Table 4-4](#) has a t₁ cycle error of 120 to 146 cycles and a t₃ cycle error of 10 to 36 cycles.

The t₁ cycle error is calculated with the RATE of patterns which are not used for the refresh routine below.

t₁ cycle error = 120 * 50 ns (minimum RATE) to 146 * 100 ns (maximum RATE) = 6.0 μs to 14.6 μs
...(1)

The t₃ cycle error is calculated with all pattern RATEs shown below.

t_3 cycle error = $10 * 50 \text{ ns}$ (minimum RATE) to $36 * 1 \mu\text{s}$ (maximum RATE) = $0.50 \mu\text{s}$ to $36.0 \mu\text{s}$... (2)

Also in 1WAY mode from Table 4-3 and Table 4-4, the synchronization errors for t_1 and t_3 are as follows:

t_1 synchronization error = 250 to 350 ns ... (3)

t_3 synchronization error = 250 to 350 ns ... (4)

Then, the t_3 RATE error is calculated using the equation of (RATE error) = $\pm\{(\text{Maximum RATE of all patterns}) - (\text{Minimum RATE of all patterns})\} * N$ below. In 1WAY mode, the following formula can be used assuming $N=115$.

t_3 RATE error = $\pm\{1 \text{ ms (maximum RATE)} - 50 \text{ ns (minimum RATE)}\} * 115 = \pm 109.25 \text{ ms}$... (5)

Because the TIMER set value is 1 ms in the above pattern program, t_1 is as follows considering the synchronization errors for t_1 shown in (1) and (3).

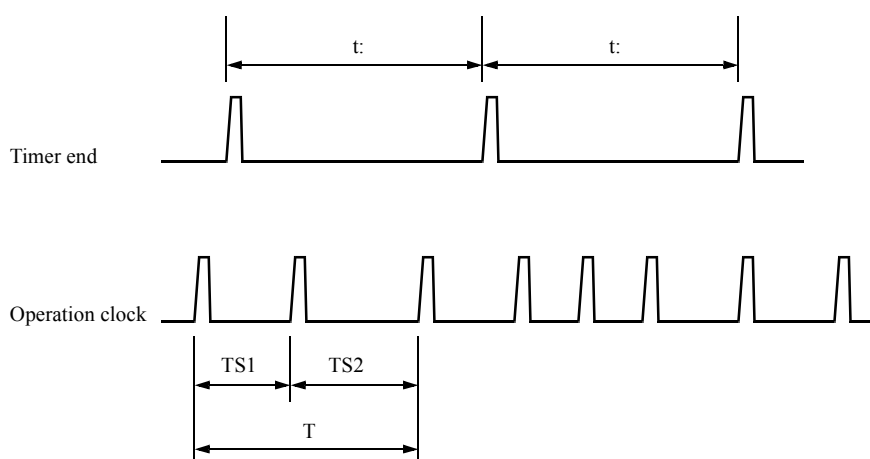
$t_1 = (1 \text{ ms} + 6.0 \mu\text{s} + 250 \text{ ns})$ to $(1 \text{ ms} + 14.6 \mu\text{s} + 350 \text{ ns}) = 1.00625 \text{ ms}$ to 1.01495 ms

When the cycle error (2) of t_3 , synchronous error (4), and RATE error (5) are considered, t_3 should be as follows:

$t_3 = (1 \text{ ms} + 0.50 \mu\text{s} + 250 \text{ ns} - 109.25 \mu\text{s})$ to $(1 \text{ ms} + 36.0 \mu\text{s} + 350 \text{ ns} + 109.25 \mu\text{s}) = 0.8915 \text{ ms}$ to 1.1456 ms

In the REFRESH A mode, t_1 and t_2 are calculated in the same way by substituting t_2 for t_3 shown above.

Limit of TIMER set value when REFRESHA MODE is set



$$t + t \times 50 \times 10^{-6} > T$$

t :

Time set value

$$50 \times 10^{-6}$$

Time clock error

T

In 1WAY mode, the total of T values of any continuous two cycles ($TS1 + TS2$ in this example)

In 2WAY mode, the total of T values of any continuous four cycles

In 4WAY mode, the total of T values of any continuous eight cycles

If the total of RATEs of any continuous two cycles in the pattern program in 1WAY mode is greater than the timer set value, no timer interrupts occur. Therefore, no refresh is executed.

If the total of RATEs of any continuous four cycles in the pattern program in 2WAY mode is greater than the timer set value, no timer interrupts occur. Therefore, no refresh is executed.

If the total of RATEs of any continuous eight cycles in the pattern program in 4WAY mode is greater than the timer set value, no timer interrupts occur. Therefore, no refresh is executed.

If (Refresh routine execution time) > t_2 , the refresh routine is interrupted and fails to operate correctly.

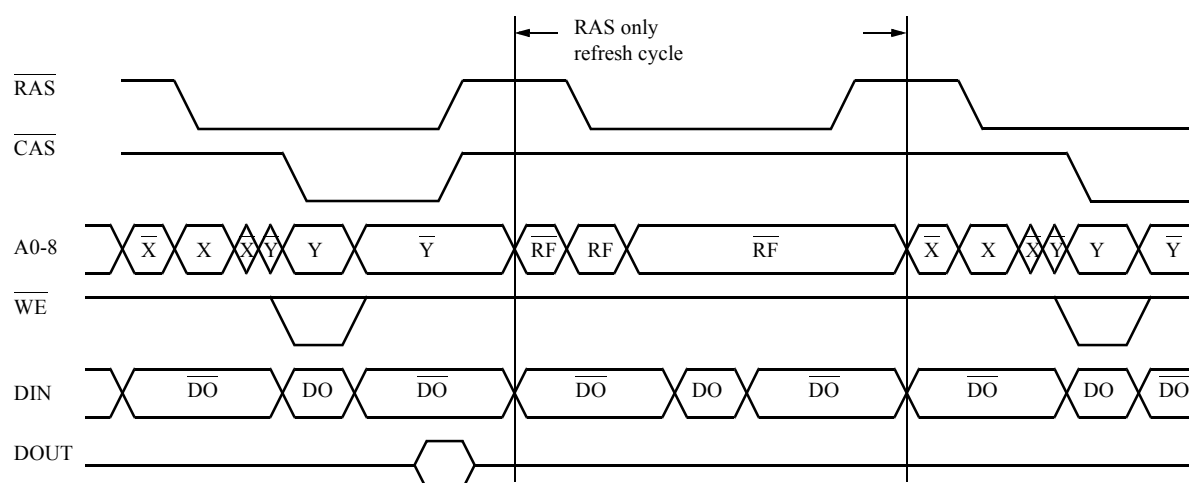
t_2 is the time shown in [Figure 4-4](#) and [Table 4-3](#).

t_2 should be calculated taking the cycle error and RATE error into account.

◆ RAS Only Refresh

[Figure 4-5](#) shows waveforms in a RAS only refresh cycle.

Figure 4-5 RAS Only Refresh Cycle



Pattern program

An example of an RAS-only-refresh test pattern program is shown below.

```
MPAT M256K ;Sample program for refresh test.
;RAS only refresh
MODE MUX

REGISTER
XMAX=#FF
YMAX=#3FF
LMAX=#3FF
HMAX=#FF
IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0
SDEF YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB
SDEF REF =RF<RF+1 X<RF
*5 *6

MODULE BEGIN ;distributed refresh

MODE REFRESHA 256 *1

REGISTER
TIMER=16US *2
ISP=ST2 *3
CPE1=R *11
CPE2=C0

START #0
NOP T INIT
IDX12 #5 *7

ST0: JN11 ST0 W YINC FP1
ST1: JN11 ST1 R YINC FP1
JZD ST0
STPS
ST2: RTN C0 M1 TS2 REF *4
*10 *8 *9

MODULE END

END
```

**1 Setting of the refresh mode*

REFRESHA 256
 |
 Refresh mode

Specification that outputs RF0-7 to X0-7

**2 Setting of the timer data*

TIMER = 200 ns to 409.5 s (time)

(TRAS) 2 to 4095 (the number of times)

**3 Specification of the branch destination if an interrupt occurs*

**4 Generation of a refresh address*

**5 Instruction that increments the refresh counter*

**6 Instruction that outputs the refresh counter contents to the X address*

**7 Start of the internal timer (T or TM1)*

**8 Cancellation of the multiplex mode by control bit M1*

Since only the refresh address is applied in the RAS only refresh cycle, the multiplex mode is cancelled.

**9 Switching of the timing set*

*Cancel the multiplex mode by control bit M1 and apply only the refresh address (row address).
Switch the timing set to suppress the clock that outputs the column address.*

Suppress the clock pulses with the OPEN instruction as shown in Example 4-11.

**10 Suppress $\overline{CAS}$ by control bit C0.*

Suppress $\overline{CAS}$ in the refresh cycle by selecting C0 as the pin data for the $\overline{CAS}$ pin.

**11 DOUT is Hi-Z during the RAS only refresh cycle. Determine Hi-Z by STRB2 to check for Hi-Z.*

Set CPE2 to determine by STRB2 for RAS only refresh.

Test plan program

The example below shows a test plan program which is used for a RAS only refresh test.

```

PRO P256K ;Sample program for 256K dynamic RAMs.

TIME1=1MS :VS1 ] *1
TIME2=1MS :IN ]

VS1=5V
IN1=2.4V, 0.8V ] *2
OUT1=2.4V, 0.4V ]

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZZ,BCLK4,CCLK4,<C0> *4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,STRB2,<D0> *6 ;DOUT
SELECT HZSENSE ENABLE *5

TR=1NS & TF=1NS ;Timing set
RATE=260NS:2
ACLK1=0NS:2
BCLK1=10NS:2
CCLK1=10NS+15NS+TF+TR:2
ACLK2=35NS,OPEN *8
BCLK2=45NS,OPEN *8
CCLK2=45NS+25NS+TF+TR,OPEN *8
ACLK3=0NS:2
BCLK3=10NS:2
CCLK3=10NS+TF+150NS:2
BCLK4=45NS:2
CCLK4=45NS+TF+75NS:2
BCLK5=45NS:2
CCLK5=45NS+TF+25NS:2
BCLK6=45NS:2
CCLK6=45NS+TF+25NS+TR:2
STRB1=45NS+TF+75NS:2
STRB2=45NS+TF

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET)@
TCHANGE PD11(RESET) ?
CALL CALB("C256K", TCAL)

TEST 1

REG MPAT PC=#0 *10
MEAS MPAT M256K *11

END

```

*1 Specification of the power-up sequence

*2 Setting of the power supply voltage (VS1), driver level (IN1), and comparison level (OUT1)

*3 Setting of the pin data

*4 Select control bit C0 as the pin data to suppress $\overline{\text{CAS}}$ in the RAS only refresh cycle.

*5 Set the Hi-Z detection mode for determination by STRB2.

**6 Select STRB2 to determine the DOUT state in the RAS only refresh cycle.*

**7 Setting of the timing*

**8 Apply only the row address in the RAS only refresh cycle. Open the clock to suppress the column address clock.*

**9 Calibration of the timing*

**10 Select a test pattern with the REG statement.*

**11 Test*

◆ CAS-before-RAS-refresh

The CAS-before-RAS-refresh test is executed with the ALPG timing switching function (TSn) by changing the phases of the clock pulses that generate CAS in the refresh cycle only.

Pattern program

An example of a CAS-before-RAS-refresh pattern program is shown below.

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  LMAX=#3FF
  HMAX=#FF
  IDX1=#3FFFE

SDEF   INIT=XB<0 YB<0 TP<0
SDEF   YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN

MODE REFRESHA 256 *1

REGISTER
  TIMER=16US *2
  ISP=ST2 *3

START   #0
NOP      T *4                               INIT
IDX12   #5
ST0:    JN11 ST0 W                           YINC FP1
ST1:    JN11 ST1 R                           YINC FP1
JZD ST0
STPS
ST2:    RTN      TS2 *5

MODULE END

END

```

**1 Setting of the refresh mode*

**2 Setting of the timer data*

**3 Branch destination specification if an interrupt occurs*

**4 Internal timer operation start*

**5 Change the timing set in the refresh cycle only.*

Test plan program

An example of a CAS-before-RAS-refresh test plan program is shown below.

```

PRO P256K

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.5V, 0.8V
OUT1=2.4V, 0.4V

PD1=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK1,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

TR=1NS & TF=1NS ;Timing set
RATE=260NS:2
ACLK1=0NS:2
BCLK1=40NS:2
CCLK1=40NS+15NS+TF+TR:2
ACLK2=65NS:2
BCLK2=75NS:2
CCLK2=75NS+25NS+TF+TR:2
ACLK3=0NS:2
BCLK3=40NS:2
CCLK3=40NS+TF+15NS:2
BCLK4=75NS,40NS-30NS-TF *1
CCLK4=75NS+TF+75NS,40NS+TF+30NS *1
BCLK5=75NS:2
CCLK5=75NS+TF+25NS:2
BCLK6=75NS:2
CCLK6=75NS+TF+25NS+TR:2
STRB1=75NS+TF+76NS:2

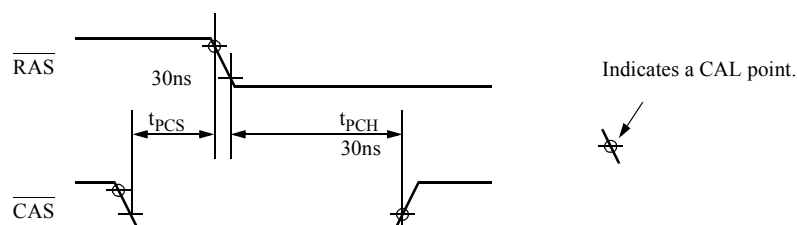
STRING TCAL(200) ;Calibration
TCAL=?DRREF PF1-12,33=2V,1.2V,TCHANGE PD10(RESET)@
TCHANGE PD11(RESET)?
CALL CALB("C256K",TCAL)

TEST 1
REG MPAT PC=#0
MEAS MPAT M256K

END

```

*1 Change the setting of the clock pulses that generates the CAS clock in the refresh cycle.

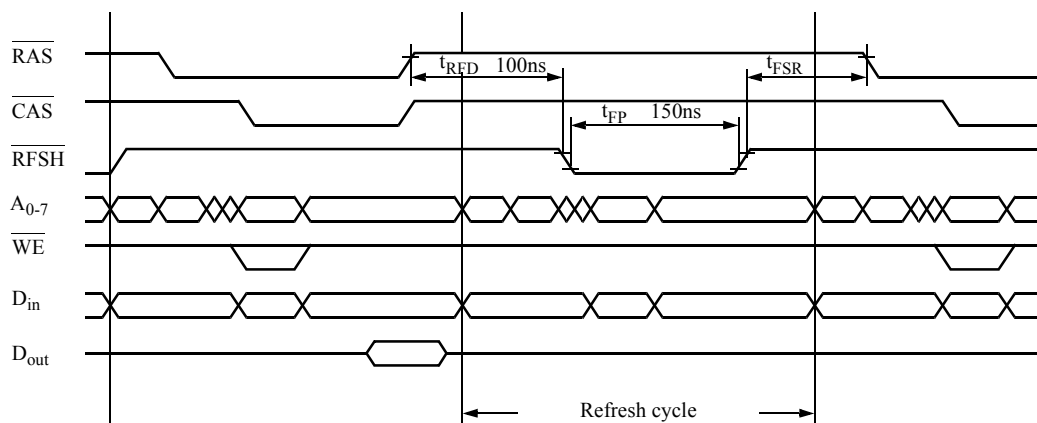


◆ Memory with a Refresh Control Terminal

Figure 4-6 shows the refresh cycle for the memory with a refresh control terminal. This figure shows a distributed refresh cycle. Clock pulses $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ in this cycle are suppressed by the control bits in the pattern program.

The clock pulses applied to terminal $\overline{\text{RFSH}}$ are controlled by the control bit in the pattern program.

Figure 4-6 Refresh Cycle for Memory with a Refresh Control Terminal



Pattern program

The pattern program example shown below is used for memory devices with a refresh control terminal.

```
MPAT M64K

MODE MUX

REGISTER
  XMAX=#7F
  YMAX=#1FF
  LMAX=#1FF
  HMAX=#7F
  IDX1=#FFFE

SDEF    INIT=XB<0 YB<0 TP<0
SDEF    YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN

MODE REFRESHA 128 *1

REGISTER
  TIMER=16US *2
  ISP=ST2 *3

START   #0
NOP     T *4                               INIT
IDX12   #5
ST0:    JN11    ST0    W                      YINC    FP1
ST1:    JN11    ST1    R                      YINC    FP1
        JZD     ST0
        STPS
ST2:    RTN     C0 *5

MODULE END

END
```

*1 Specification of the refresh mode

*2 Setting of the timer data

*3 Branch destination setting if an internal interrupt occurs.

*4 Timer operation start

*5 *Specify the control bit that indicates the refresh cycle.*

When creating the program, note that if M1 and M2 are described, the multiplex mode operation changes.

➔ *For more information, refer to [4. 3. 1 "Address Multiplexing" on page 4-11](#).*

Test plan program

An example of a test plan program for memory with a refresh control terminal is shown below.

```

PRO P64K
    TIME1=1MS :VS1
    TIME2=1MS :IN

    VS1=5V
    IN1=2.4V, 0.8V
    OUT1=2.4V, 0.4V
    PD1-7=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-6,Y0-6> ;A0-6
    PD8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y7,Y8> ;A7
    PD9=IN1,/RZO,BCLK7,CCLK7,<C0> *1 ;/RFSH
    PD10=IN1,/RZZ,BCLK3,CCLK3,<C0> *2 ;/RAS
    PD11=IN1,/RZZ,BCLK4,CCLK4,<C0> *2 ;/CAS
    PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
    PD34=IN1,XOR,ACLK1,BCLK6,CCLK6,<D0> ;DIN
    PC33=OUT1,STRB1,<D0> ;DOUT

    TR=1NS & TF=1NS
    RATE=270NS
    ACLK1=0NS
    BCLK1=40NS
    CCLK1=40NS+15NS+TF+TR
    ACLK2=65NS
    BCLK2=75NS
    CCLK2=75NS+45NS+TF+TR
    BCLK3=40NS
    CCLK3=40NS+TF+150NS
    BCLK4=75NS
    CCLK4=75NS+TF+100NS
    BCLK5=75NS
    CCLK5=75NS+TF+45NS
    BCLK6=75NS
    CCLK6=75NS+TF+45NS+TR
    BCLK7=20NS+TR+TF
    CCLK7=20NS+TR+TF+150NS
    STRB1=40NS+TF+150NS

    STRING TCAL(200)
    TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET)@
    TCHANGE PD11(RESET)@
    TCHANGE PD9(RESET)?
    CALL CALB("C64K",TCAL)

TEST 1
    REG MPAT PC=#0
    MEAS MPAT M64K

END

```

*1 Apply clock pulses to terminal $\overline{RFSH}$ in the refresh cycle only.

*2 Suppress $\overline{RAS}$ and $\overline{CAS}$ in the refresh cycle.

4. 3. 4 Regional Inversion Test (ARIRAM)

In most dynamic RAMs, I/O data and data stored in internal cells is inverted at specific addresses as listed in [Table 4-5](#).

Therefore, if a test is to be performed on cell data, it is necessary to invert the expected values of I/O data applied to the DUT at specific addresses. A regional inversion function is provided to facilitate these tests.

Table 4-5 Example of DRAM Data Topology

R7*1	R0*1	Input data	Cell data*2	Output data
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

*1 Rn indicates a row address.

*2 Cell data 0: Discharging state
 1: Charging state

Regional Inversion Mechanism

DA0 to DA17, DB0 to DB17 used as expected value data for input data applied to the DUT and output data from the DUT are inverted and output under the inversion conditions set in the regional inversion circuit.

Use the address conditions of the ARIRAM DEFINE statement to specify the data inversion conditions.

➔ For more information on the ARIRAM DEFINE statement, refer to [2. 2. 6 "ARIRAM DEFINE Statement" on page 2-32](#).

When the double speed mode which uses PRESCRAM is specified in the T5593, the Y addresses are output for the first and second halves. Use the MODE DBFORM statement to specify which of the first and latter halves of the Y address should be used as the inversion conditions for the TP1 and TP2 data.

Programming

An example of a pattern program using the regional inversion mechanism is shown below.

```
MPAT M64K

MODE MUX

REGISTER
  XMAX=#7F
  YMAX=#1FF
  IDX1=#FFFE

ARIRAM DEFINE ] *2
  D0-17=X0"Y0

SDEF INIT=XB<0 YB<0 TP<0 ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base

MODULE BEGIN ;checker board

ARIRAM DEFINE ] *1
  D0-17=X0"Y0

START #10

  IDX12 #6      INIT
ST0:  JN11 ST0 W  INC   FP1
ST1:  JN11 ST1 R  INC   FP1
      JZD ST0
      STPS

MODULE END

END
```

**1 Inversion Condition Setting*

DA0 to DA17, and DB0 to DB17 applied to DUT are inverted for output at the address satisfying the following condition:

$X0 \text{ " } Y0 = 1$

" indicates exclusive OR.

**2 When the inversion conditions are specified in the common section of the pattern program, they are valid for all the pattern programs specified in the pattern program.*

4. 4 Bump Test (BURST BLOCK)

4. 4. 1 How to Specify Conditions in the Test Plan Program

When conditions are specified in the test plan program, the power supply voltage is changed during the bump test when pattern generation is stopped by the PAUSE instruction in the pattern program. The voltage to be changed is specified in the test plan program.

The following section explains how to create a pattern program and a test plan program for the bump test shown in [Figure 4-7](#).

Pattern program

A pattern program for the bump test is shown below:

```
MPAT BUMP

MODE MUX PAUSE *1

REGISTER
  XMAX=#FF
  YMAX=#3FF
  TIMER=t *2
  IDX2=#3FFFE
  IDX3=#0

START #0
  IDX11 #6      XB<0 YB<0 TP<0
ST0:  JN12  ST0  W      XB<XB+1 YB<YB+1^BX *4
      NOP    T *3      ; PAUSE
ST1:  JN12  ST1  R      XB<XB+1 YB<YB+1^BX *4
      JN13  ST0
      JZD   ST0
      STPS

END
```

*1 Set the pause mode.

*2 Set the timer time.

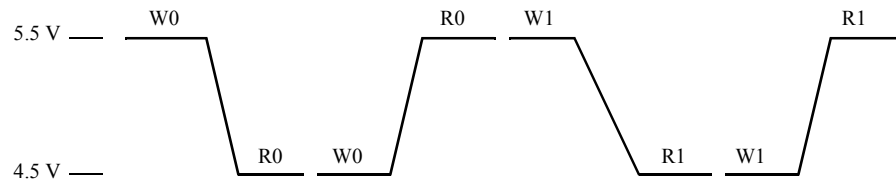
Set the following value for the timer:

$t = \text{Data transfer time} + \text{Settling time}$

➔ For more information, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

*3 Pause instruction

*4 Create a pattern program so that the minimum time from the pattern start to the first pause instruction is 5 μ s or more and between pause instructions (NOP T) are 25 μ s or more.

Figure 4-7 VS Change during Bump Test

W0 :All cell Write "0"

R0 :All cell Read "0"

W1 :All cell Write "1"

R1 :All cell Read "1"

Test plan program

A test plan program for the bump test is shown below:

```

PRO BUMP

DIM(1) A(200)=0 *1

TIME1=1MS :VS1
TIME2=1MS :IN
VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V
RATE=260NS
ACLK1=0NS
:
:
STRB1=200NS
PD1-8=IN1,XOR,ACLK1,BCLK1,SDM,<X0-7,Y0-7>
PD9=IN1
:
:
PC33=OUT1,STRB1,<D0>

BURST BLOCK BEGIN(A) *2

VS1=5.5V & BURST WAIT TIME 1MS *3
VS1=4.5V *4
VS1=5.5V *4
VS1=4.5V *4
VS1=5.5V *4

BURST BLOCK END *2
:
:
MEAS MPAT(4,A) BUMP *5
:
:

END

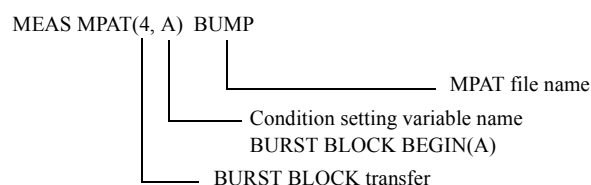
```

**1 Secure the variable area which stores data (data specified in BURST BLOCK) transferred during the bump test.*

**2 Define the data whose conditions are changed during the bump test.*

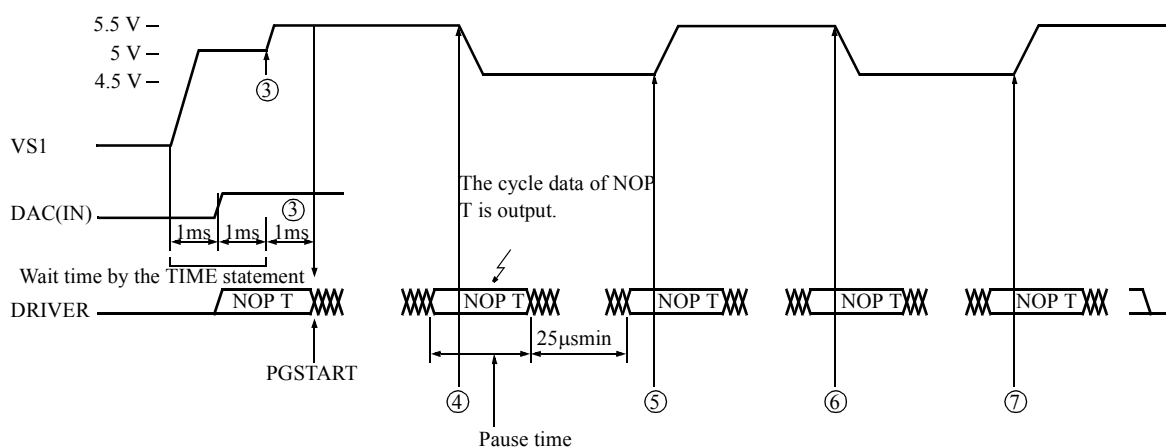
**3 Specify the voltage set before starting pattern generation and the time to start pattern generation in the MEAS statement.*

**4 Voltage set for each pause during pattern generation*

**5 Test Execution*

The power supply voltage changes in the time sequence shown in [Figure 4-8](#).

Figure 4-8 Burst Data Transfer and VS Change



4. 4. 2 How to Specify Conditions in the Pattern Program

When conditions are specified in the pattern program, the power supply voltage is changed by the OUT instruction in the pattern program during the bump test. Specify the voltage to be changed in the pattern program.

The following section explains how to create a pattern program and a test plan program for the bump test shown in [Figure 4-7](#).

Pattern program

A pattern program for the bump test is shown below:

```

MPAT BUMP

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX2=#3FFFE
  DPUTYPE2 *1
  BURST BLOCK BEGIN(A) *2
    VS1=4.5V *3
    WAIT TIME 1MS *4
  BURST BLOCK END (3)

  BURST BLOCK BEGIN(B)
    VS1=5.5V
    WAIT TIME 1MS
  BURST BLOCK END (4)

START #0
IDX11 #6 XB<0 YB<0 TP<0
ST0: JN12 ST0 W XB<XB+1 YB<YB+1^BX
      OUT A (1) *5
ST1: JN12 ST1 R XB<XB+1 YB<YB+1^BX
ST2: JN12 ST2 W XB<XB+1 YB<YB+1^BX
      OUT B (2) *5
ST3: JN12 ST3 R XB<XB+1 YB<YB+1^BX
      JZD ST0
      STPS

END

```

*1 To specify VS in the burst test condition, specify the DPUTYPE statement according to the test system.

*2 Specify the variable (burst block name) shown in parentheses. If the burst block name is specified for the OUT instruction, the data enclosed by BURST BLOCK BEGIN and BURST BLOCK END is set when the OUT instruction is executed.

*3 Set the power supply voltage.

*4 Waiting Time after Setting the Power Supply

*5 Change the voltage with the OUT instruction.

Set the data sets ((3) and (4) in the example) that are specified by the OUT instruction operands ((1) and (2) in the example).

While the above data is set, data with the cycle specified by the OUT instruction is applied to the DUT.

When the OUT instruction is executed, the timing generator stops with the pattern applied by the OUT instruction, and stops clock pulse generation.

When the data set selected by the OUT instruction has been transferred, pattern generation restarts.

Tip

- Set the voltage so that the bias power supply source range to be applied to the DUT for setting the power supply voltage during the bump test is not changed.
- When T5593 is used, use care not to change the polarity of the power supply voltage to be applied to DUT.

Test plan program

A test plan program for the bump test is shown below:

```

PRO BUMP

TIME1=1MS:VS1      *1
TIME2=1MS:IN      ]

VS1=5.5V           *2
IN1=2.4V,0.8V
OUT1=2.4V,0.4V

RATE=260NS
ACLK1=0NS
:
:
STRB1=200NS
PD1-8=IN1,XOR,ACLK1,BCLK1,SDM,<X0-7,Y0-7>
PD9=IN1
:
:
PC33=OUT1,STRB1,<D0>
:
:
MEAS MPAT BUMP     *4
:
:
END
  
```

*1 Reference-on time sequence

*2 Set the initial power supply voltage.

In this example, since the test starts with a power supply voltage of 5.5 V, set the initial voltage to 5.5 V.

*3 Set test conditions.

*4 Test

The power supply voltage changes in the time sequence shown in [Figure 4-9](#). The execution time (t) of the OUT instruction is shown in [Table 4-6](#). The transfer data and transfer time are shown in [Table 4-7](#).

Figure 4-9 Out Instruction and VS Change

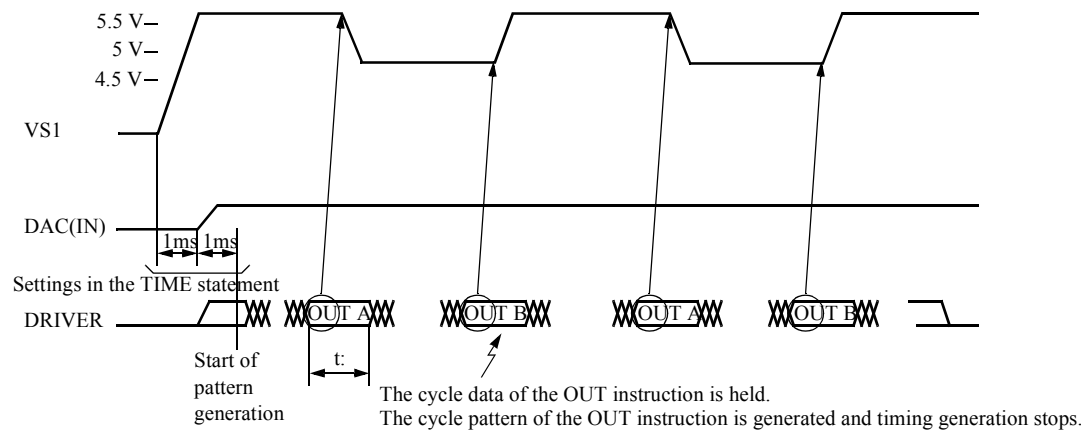


Table 4-6 Execution Time (t) of OUT Instruction

Test system	Execution time (t) of OUT instruction
T5593	$t = \text{RATE of OUT cycle} * n + (9.8 \mu\text{s to } 10.3 \mu\text{s}) + \text{data transfer time}$
For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P	$t = \text{RATE of TS1} * n + (3.9 \mu\text{s to } 4.1 \mu\text{s}) + \text{data transfer time}$

t:

t shows the total execution time for data transfer caused by the OUT instruction.

Data transfer is executed after patterns are generated in the cycle where the OUT instruction is specified.

Data transfer time

Data transfer time is the sum of individual data transfer times which are transferred by the OUT instruction.

For more information on individual data transfer times, refer to [Table 4-7](#).

n

Specify n as shown below (according to the interleave mode to be specified):

1 WAY	n=1
2 WAY	n=4
4 WAY	n=8

◆ Example

OUT A : TS2
: TS3

Execution time for OUT operation (t) = RATE of TS1 * 4 + (3.9 to 4.1 μs) + data transfer time (when T5592 is used)

Execution time of entire cycle where OUT instruction is specified = RATE of TS2 + RATE of TS3 + t

Table 4-7 OUT Instruction Transfer Time

Data to be transferred	Transfer time
VS _n = vs, R (vr)	4 μs
VS _n = vs	
WAIT TIME t	1.6 μs + t

4. 5 Partial Test (Z Address)

4. 5. 1 Partial Test Using Z Address

The pattern program for the partial test can be easily performed using the Z address. To split a DUT into memory blocks for testing, use Z0 to Z17 (Z0 to Z15 for T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, T5571P) for individual blocks. Z0 to Z17 are applied to the DUT as follows:

ADDRESS DEFINE

The ADDRESS DEFINE statement specified in the pattern program has Z0 to Z17 output to X0 to X17 and Y0 to Y17 (output Z0 to Z15 to X0 to X15 and Y0 to Y15 for T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P).

Addresses are converted by the address descrambler after the Z address has been multiplexed.

```
ADDRESS DEFINE
  Xl - m = Zn - o      0 ≤ l, m ≤ 17(15)      1 ≤ m
  Yl - m = Zn - o      0 ≤ n, o ≤ 17(15)      n ≤ o

MPAT  PART
:
ADDRESS DEFINE
  Y6 - 9 = Z0 - 3
:    256 K DRAM is divided into 16 parts.
END
```

4. 5. 2 Programming

An example of a partial test pattern program is shown below:

◆ Partial Test Pattern Program (1/2)

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 Z<0 TP<0
SDEF BINC=XB<XB+1 YB<YB+1^BX
SDEF PINC=XB<XB+1 YB<YB+1^BX Z<Z+1^BX *3
SDEF CINC=XC<XC+1 YC<YC+1^CY
SDEF CLAD=XC<XB+1 YC<YB+1^BX
SDEF ZINC=Z<Z+1
SDEF BASE=X<XB Y<YB
SDEF DIST=X<XC Y<YC

MODULE BEGIN ;checker board
START #0

  IDXI2 #6 INIT
ST0: JN11 ST0 W FP1 BINC
ST1: JN11 ST1 R FP1 BINC
JZD ST0
STPS

MODULE END

```

◆ Partial Test Pattern Program (2/2)

```

MODULE BEGIN                                ;partial

REGISTER
  XMAX=#FF
  YMAX=#3F *2
  ZMAX=#F
  IDX1=#3FFFE
  IDX2=#3FFE
  IDX3=#3FFD
  IDX4=#E

ADDRESS DEFINE
  Y6-9=Z0-3 *1
  XY=ZN *2

START      #10
IDXI5      #6
ST0: JN11   ST0      W      BASE      INIT
ST1: NOP                    W      BASE      /D      CLAD
ST2: NOP                    R      DIST
      NOP                    R      BASE      /D
      JN13   ST2      R      DIST      CINC
      JN12   ST1      W      BASE      BINC
      JN14   ST1                        ZINC *4
      JZD    ST0
      STPS

MODULE END

END

```

*1 Output Z0 to Z3 to Y6 to Y9 with the ADDRESS DEFINE statement.

*2 Because Z0 to Z3 are ORed to Y6 to Y9, mask Y6 to Y9 with YMAX.
If they are not masked, the high-order bits (YB6-9) of YB and YC are output.
Or, specify XY=ZN in ADDRESS DEFINE statement.

*3 To scan all cells, use the link instruction.

*4 Increment the Z address.

5. *Programming II*

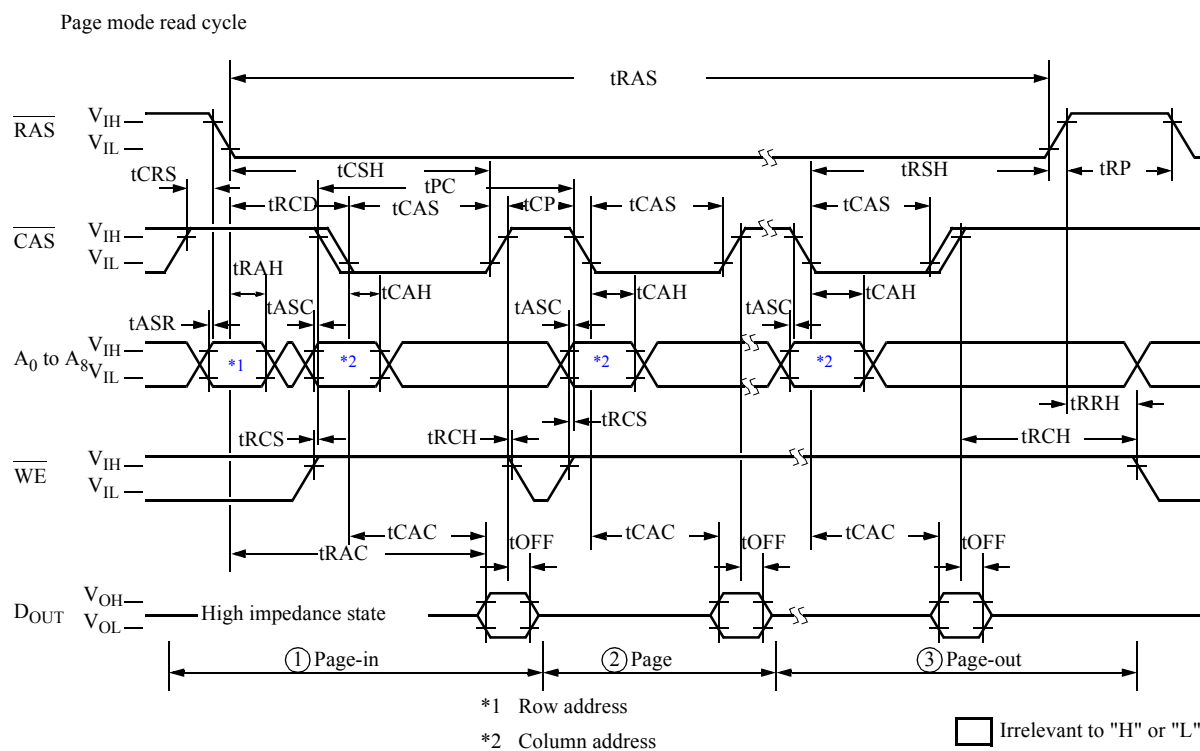
This chapter describes how to write pattern programs and test plan programs for page mode tests, tests for multi-bit memory chips, read modify write tests, EPROM tests, and dual port memory tests.

5. 1 Page Mode Test

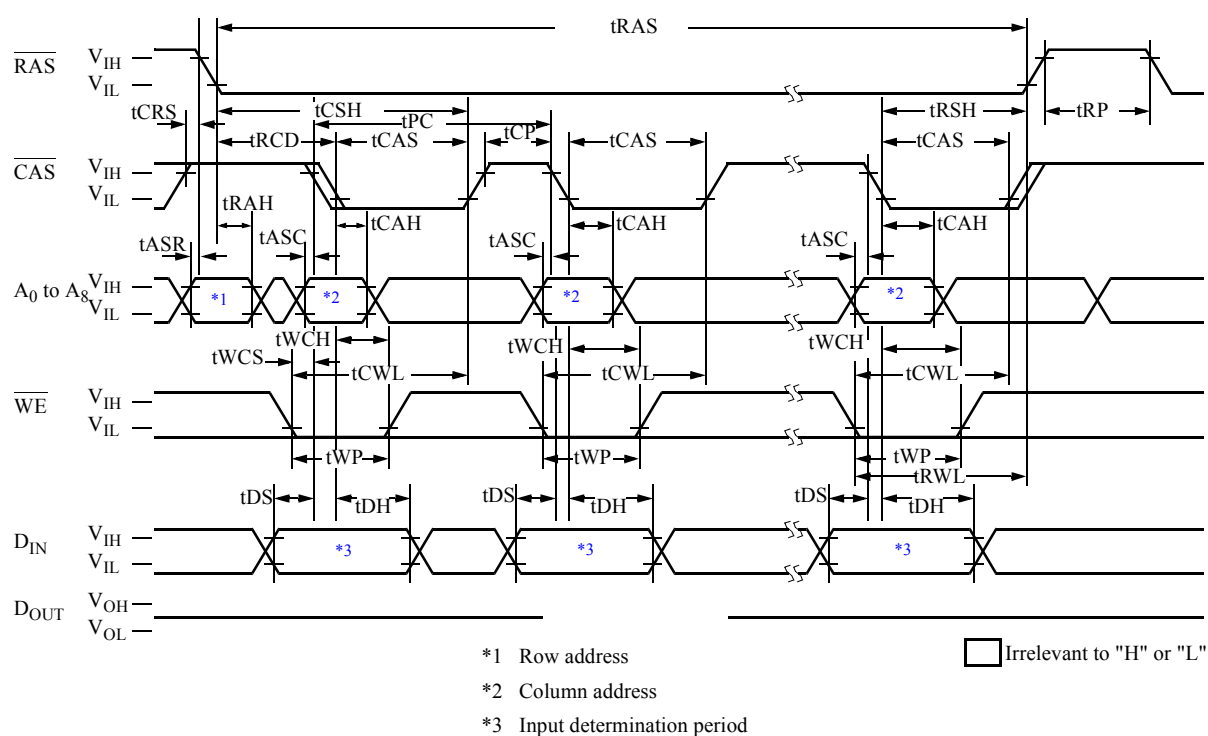
5. 1. 1 Page Mode Test

Most dynamic RAMs (DRAMs) have an access mode called page mode which shortens the cycle time.

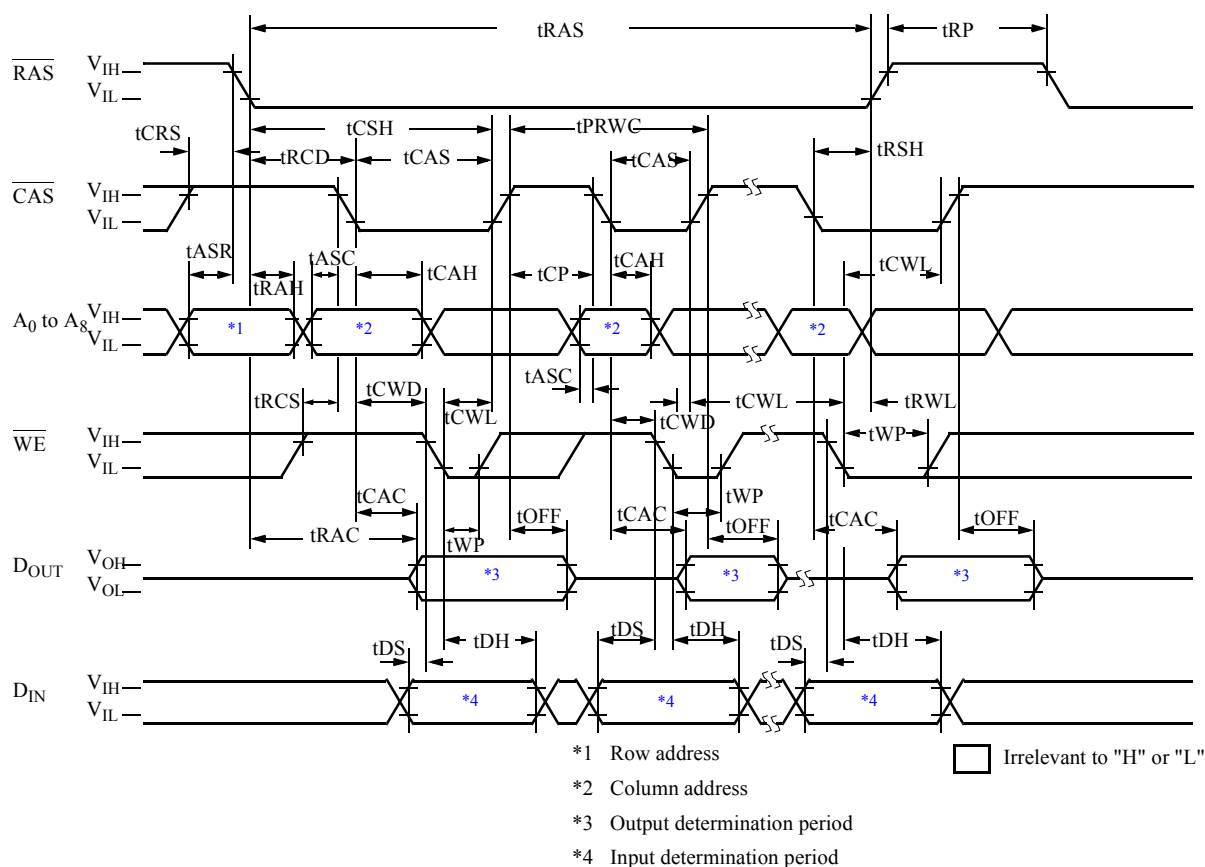
[Figure 5-1](#) shows page mode storage: a row address is latched internally by an $\overline{\text{RAS}}$ clock pulse, and a $\overline{\text{CAS}}$ clock pulse is applied to change a column address with $\overline{\text{RAS}}$ held low. To create a page mode test program, address multiplexing and $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ clock pulses are controlled with control bits coded in the pattern program.

Figure 5-1 Page Mode Time Chart

Page mode write cycle (early write)



Page mode read/write cycle



5. 1. 2 Waveform Control by Control Bits

An example of a pattern program is shown in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#) and an example of a test plan program is shown in ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#). The program is created using the procedure below.

Generation of addresses and data patterns

In the program for generating addresses and data patterns, address multiplexing and $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ clock pulses are controlled by control bits coded in the pattern program. Therefore, steps for generating page-in, page, and page-out addresses must be coded separately. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

The page-in cycle is the cycle in which the page mode operation begins. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

The page cycle indicates the cycle in which the page mode operation is executed. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

Use the OPEN instruction to inhibit clock pulses. ((f) in ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#))

In ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#), clock pulses ACLK2, BCLK2, and CCLK2 are inhibited and ACLK1, BCLK1, and CCLK1 are used to generate column address.

◆ Pattern Program for Page Mode Test Using Control Bits

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0 ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base
SDEF YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN ;checker board
START #0

  IDXI2 #6 INIT
ST0: JN11 ST0 W INC FP1
ST1: JN11 ST1 R INC FP1
  JZD ST0
  STPS

MODULE END

MODULE BEGIN ;page mode

REGISTER
  LMAX=#3FF
  HMAX=#FF
  IDX1=#1FC
  IDX3=#1FE

START #10

  IDXI2 #6
ST0: NOP W TS2 INIT YINC FP1 ;page-in
ST1: JN11 ST1 W M1 M2 TS3 (b) YINC FP1 ;page
      JN13 ST0 W M1 M2 TS4 YINC FP1 ;page-out (a)
ST2: NOP R TS2 YINC FP1 ;page-in
ST3: JN11 ST3 R M1 M2 TS3 YINC FP1 ;page
      JN13 ST2 R M1 M2 TS4 YINC FP1 ;page-out
  JZD ST0 (c)
  STPS

MODULE END

END

```

◆ Test Plan Program for Page Mode Test Using Control Bits

```

PRO P256K      ;Sample program for 256K dynamic RAMs.
               ;page mode

TIME1=1MS :VS1
TIME2=1MS :IN
VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V
PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y9,Y8> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

;Timing set.
TR=2NS & TF=2NS
;      TS1          TS2          TS3          TS4
;      normal      page-in      page      page-out
RATE=   260NS      229NS      150NS      179NS      (d)
ACLK1=   0NS:4
BCLK1=   10NS:4
CCLK1=   10NS+TF+15NS+TR:2      10NS+TF+25NS+TR:2
ACLK2=   57NS:2
BCLK2=   10NS+TF+75NS-TR:2
CCLK2=   10NS+TF+75NS-TR+TF+25NS+TR:2
ACLK3=   0NS:4
BCLK3=   10NS:2
CCLK3=   10NS+TF+150NS
BCLK4=   10NS+TF+75NS-TR:2
CCLK4=   10NS+TF+75NS-TR+TF+75NS:2
BCLK5=   10NS+TF+75NS-TR:2
CCLK5=   10NS+TF+75NS-TR+TF+25NS:2
BCLK6=   20NS+TF+75NS-TR:2
CCLK6=   10NS+TF+75NS-TR+TF+25NS:2
STRB1=   10NS+TF+75NS-TR+TF+75NS:2

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10-11(RESET)?
CALL CALB("C256K",TCAL)

TEST 1
REG MPAT PC=#10
MEAS MPAT M256K

END

```

(d)

(f)

(e)

RAS clock

5. 2 Test for Multi-bit Memory Chips

5. 2. 1 Data Generation

To generate data for multiple-bit memory tests, the TP register and the inversion function are used.
An example of a pattern program for a multiple-bit memory chip test is shown below:

```
MPAT M16KS

REGISTER
  XMAX=#7F
  YMAX=#F
  TPH=#55 *2
  IDX1=#7FE

SDEF INIT=XB<0 YB<0 TP<TPH *1 ;initialize reg.
SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base

MODULE BEGIN ;checker board
START #0

NOP          INIT
ST0: JN11 ST0 W INC      FP1 ] *3
ST1: JN11 ST1 R INC      FP1 ]
      JZD ST0
      STPS

MODULE END

MODULE BEGIN ;marching
START #10

NOP          INIT
ST0: JN11 ST0 W INC
ST1: NOP      R
      JN11 ST1 W INC      /D
ST2: NOP      R      /X /Y /D ] *4
      JN11 ST2 W INC      /X /Y ]
      JZD ST0
      STPS

MODULE END

END
```

***1 Register TP initialization**

Initialize register TP with the following instructions:

$TP < 0$: Clear

$TP < TPH$: Loads the contents of register TPH.

$TP < D5$: Load the content of D5 register.

2 Register TRH initialization for register TP initializationRegister TPH can also be initialized with the following instruction in the test plan program:*

```

      :
REG MPAT TPH = #55
      :

```

3 Data inversion with the address functionInvert the contents of register TP according to the condition indicated by the FPN address function to output it to DA0 to DA17 (DB0 to DB17).***4 Data inversion with the /D instruction**Invert the contents of register TP with the /D instruction to output it to DA0 to DA17 (DB0 to DB17). Bits to be inverted can be specified by the DCMR.**Data for testing multiple-bit memory chips can be generated by combining operation instructions in register TP (below) and the above inversion function.**TP < TP + 1**TP < TP - 1**TP < / TP**TP < TP*2 (LSB = 0)**TP < TP'D5**TP < TP&D5**TP < TP"D5**TP < TP+D5**TP < TP-D5**TP < /D5**TP < TPS**TPS < TP**TP <> TPS*

5. 2. 2 I/O Control

Most of multiple-bit memory outputs data from the I/O pins.

An example of a test plan program for I/O chip testing is shown below:

```

PRO P16KS                      ;sample program for 2K*8 static RAMs.
    TIME=1MS:VS1
    TIME=1MS:IN

    VS1=5V
    IN1=2.4V,0.6V
    OUT1=2.2V, 0.8V

    PD1-4=IN1,NRZA,ACLK1,<Y0-3>      ;A0-3
    PD5-11=IN1,NRZA,ACLK1,<X0-6>     ;A4-11
    PD12=IN1,/RZO,BCLK1,CCLK1,<WT>   ;/WE
    PD13=IN1,/NRZA,ACLK1,<FH>        ;/OE
    PD14=IN1,/NRZA,ACLK1,<FH>        ;/CS
    P33-40=IN1,OUT1,IOC,XOR,ACLK2,BCLK2,DRE1,DREC1STRB1,<D0-7>;I/O
                                           *2          *1

    RATE=200NS
    ACLK1=0NS
    BCLK1=20NS
    CCLK1=160NS
    ACLK2=0NS
    BCLK2=100NS
    CCLK2=164NS
    DREL1=90NS *2
    DRET1=170NS *2
    STRB1=200NS

    STRING TCAL(100)
    TCAL=?DRREF PD1-14,33-40=2.2V,0.8V TSELECT PD1-11(A-SET,A-RESET)?
    CALL CALB("C16KS1",TCAL)

TEST    1
        REG MPAT          PC=#0
        MEAS MPAT M16KS

END

```

*1 Specify I/O pins.

*2 Specify timing in which write data is input to the chip.

The procedure for cycle specification is as follows:

- Use the REG statement in the pattern program.

```
MPAT filename
REGISTER
:
DRE=n    (n = FIXH, FIXL, R, W, M1-2, C0-23)
:
```

n is a control bit.

FIXL, *FIXH*, *R* and *W* can also be written with *FL*, *FH*, *RD* and *WT*, respectively.

Write data is input to the chip in the cycle for which a control bit is specified as *n*.

The default value is *W*.

- Use the REG statement in the test plan program.

```
:
REG MPAT DRE = n    (n = FL, FH, RD, WT, M1-2, C0-23)
:
```

The function has a 2 WAY I/O control of *DRE1* and *DRE2*.

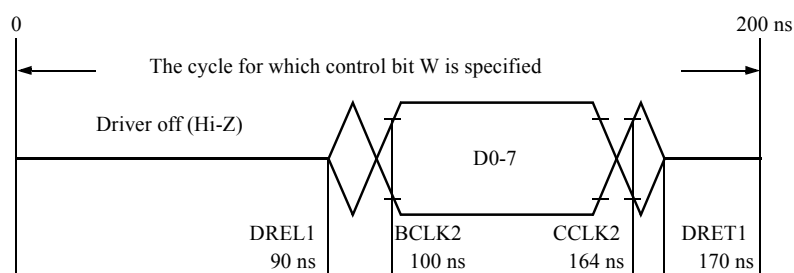
Specify *DRE1* or *DRE2* as a register name. For default, *DRE1* is selected.



Tip

DRE and *DRE1* have a same meaning.

In this example, the following data is applied to the DUT in the cycle for which control bit *W* is specified in the pattern program:



5. 2. 3 Generation of Parity Data

The memory of the X9 configuration is convenient for the use of the parity function.

Pattern program

An example of a multi-bit configuration memory test pattern program using parity generating function is shown below:

```
MPAT      P72K      ; SAMPLE PROGRAM FOR 8K*9 SRAM

MODE PARITY *1
REGISTER
  XMAX=#FF
  YMAX=#1F
  TPH=#FF
  IDX1=#1FFE
  IDX2=#7
  DRE1=W
  CPE1=R

SDEF ACLR=XB<0 YB<0
SDEF AINC=X<XB Y<YB XB<XB+1 YB<YB+1^BX
SDEF AHLD=X<XB Y<YB

START #0
NOP      ACLR  TP<TPH
ST0:     JN11  ST0  W  AINC
ST1:     NOP      R  AHLD
          JN11  .   W  AINC  /D
          JN12  ST1  ACLR  TP<TP*2 *2
STPS

END
```

**1 Setting a Mode for Generation Parity*

Parity of D0 to D7 interrupts D16.

Parity of D8 to D15 interrupts D17.

The parity referred to is odd parity. Data for D16 (or D17) is generated in such a way that the sum of the bits in the 1 state in D0 to D7 and D16 (or D8 to D15, D17) is odd.

**2 In this example, tests are made by matching pattern with D0 to D7 changed as #FF → #1FE(#FE) → #3FC(#FC) → #7F8(#F8) → #FF0(#F0) → #1FE0(#E0) → #3FC0(#C0) → #7F80(#80).*

Test plan program

The example below shows a test plan program which uses the parity generation function for a multi-bit configuration memory test.

```

PRO      M72K      ;   SAMPLE PROGRAM FOR 8K*9 SRAM

      TIME1=1MS      :VS1
      TIME2=1MS      :IN

      VS1=5V
      IN1=2.2V,0.8V
      OUT1=2.4V,0.4V

      PD1-8 =IN1,XOR,ACLK1,BCLK1,CCLK1,<X0-7>      ;ADDRESS
      PD9-13=IN1,XOR,ACLK1,BCLK1,CCLK1,<Y0-4>      ;ADDRESS
      PD14 =IN1,/RZO,BCLK2,CCLK2,<WT>      ;/WE
      PD15 =IN1,/NRZA,ACLK2,<RD>      ;/OE
      PD16 =IN1,FIXL      ;/CS1
      PD17 =IN1,FIXH      ; CS2
      P33-40=IN1,XOR,ACLK3,BCLK3,CCLK3,OUT1,STRB1,IOC,DRERZ,<D0-7>
      P41 =IN1,XOR,ACLK3,BCLK3,CCLK3,OUT1,STRB1,IOC,DRERZ,<D16> *1
      RATE =65NS
      ACLK1= 0NS & BCLK1=20NS & CCLK1=55NS      ;ADDRESS
      ACLK2= 0NS      ;OE
      BCLK2=32NS & CCLK2=52NS      ;/WE
      ACLK3= 0NS & BCLK3=35NS & CCLK3=52NS      ;IO1-9
      STRB1=55NS
      DREL1=25NS
      DRET1=62NS

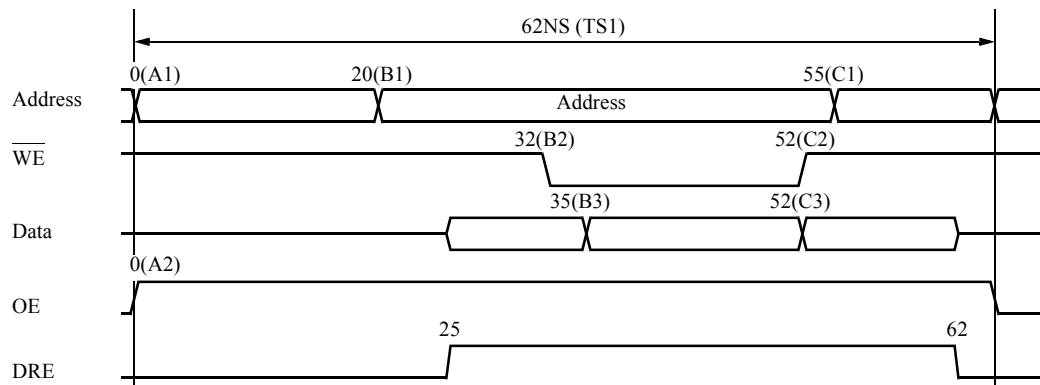
      CALL CALB ("C72K",NORMAL)
TEST    1
      REG MPAT PC=#0
      MEAS MPAT P72K

      END

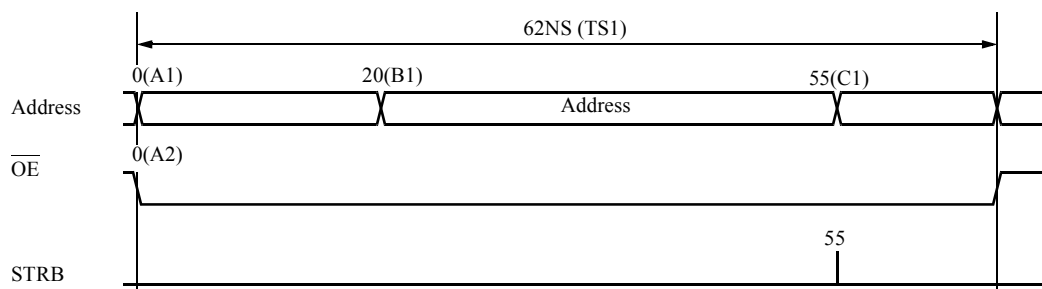
```

**1 Allocate parity bit D16 to the data bit of the device.*

Write cycle



Read cycle



5. 3 Read Modify Write Test

5. 3. 1 Read/Write Data Inversion

Since both read and write operations must be completed in one memory cycle in a read modify write test, write data and expected data must be generated separately. The memory test system enables inversion of write data and expected data, with the following data inversion function:

RINV

Pattern data of a driver pin for which RINV is specified in the pin statement is inverted in the cycle for which the control bit, selected by RINV in the pattern program, is specified.

```
MPAT  program-name
REGISTER
  RINV = C0      ; Select a control bit
  :              FIXH, FIXL, R, W, M1, M2, or C0-23 can be specified.
  NOP  C0        ; In the cycle where the RINV-specified control bit is
  :              specified, D0 for PD34 is inverted.
```

```
  :
  PD34 = IN1, XOR, ACLK1, BCLK2, RINV,< D0 >
  :
```

5. 3. 2 Programming

In the examples as shown below, a read modify write test is performed with a marching pattern:

◆ Pattern Program for Read Modify Write Test

```
MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX    ;increment base

MODULE BEGIN      ;RMW march.

REGISTER
  RINV=C0 *1

START      #0

  IDXI2 #6      INIT
ST0:  JNII ST0 W      INC
ST1:  JNII ST1 R W C0 *1  INC
ST2:  JNII ST2 R W C0  INC      /X /Y /D
  JZD ST0
  STPS

MODULE END

END
```

*1 Specify a cycle in which a driver pattern is inverted.

In the example shown above, the driver pattern is inverted in the cycle where control bit C0 is specified.

◆ Test Plan Program for Read Modify Write Test

```

PRO P256K                ;Sample program for 256K dynamic RAMs.

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,RINV*1,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

TR=2NS & TF=2NS ;Timing set
RATE=260NS
ACLK1=0NS
BCLK1=10NS
CCLK1=10NS+15NS+TF+TR
ACLK2=35NS
BCLK2=45NS
CCLK2=45NS+25NS+TF+TR
ACLK3=0NS
BCLK3=10NS
CCLK3=10NS+TF+150NS
BCLK4=45NS
CCLK4=45NS+TF+85NS
BCLK5=45NS+TF+25NS
CCLK5=45NS+TF+25NS+TF+25NS *2
BCLK6=45NS+TF+25NS *2
CCLK6=45NS+TF+25NS+TF+25NS+TR *2
STRB1=45NS+TF+75NS

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10-12(RESET)?
CALL CALB("C256K",TCAL)

TEST 1

REG MPAT PC=#0
MEAS MPAT M256K

END

```

*1 Specify RINV in the pin statement.

**2 Set the timing for read modify write cycle.*

5. 4 EPROM Test

5. 4. 1 Using the Match Flag Sense Instruction

The memory test system for T5586, T5585, T5581, T5581H, T5581P, and T5571P provides flag sense instructions which change the program sequence according to the results of logical comparison (pass or fail) to facilitate the verify operation in EPROM testing. [Table 5-1](#) lists flag sense instructions.

FLG1 indicates the result of logical comparison. It is set under the following conditions:

```

:
PC33-40=OUT1, STRB1, MATCH, <D0 - 7>
:

```

The output data is compared at the STRB1 pulse, in the cycle in which a flag sense instruction is specified. If the result for a pin for which MATCH is specified in the pin statement is pass, FLG1 = 1.

If MATCH is specified for several pins, FLG1 = 1 if the results for all the applicable pins are pass.

Operation period for a flag sense instruction is as follows:

RATE ≥ set the value of STRB1 + 5.0 μs

Table 5-1 Match Flag Sense Instruction

Mnemonic	Action
FLGLI1 m	1st (IDXR8) → DX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC 2nd If (IDXW8) = 0 then (BAR) → PC else (IDXW8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC

5. 5 Test for Dual Port Memory

5. 5. 1 Test for Dual Port Memory

The T5500 Series memory test systems (excluding T5592 and T5571P) can generate two types of patterns from the pattern generator so that they can support dual port memory tests.

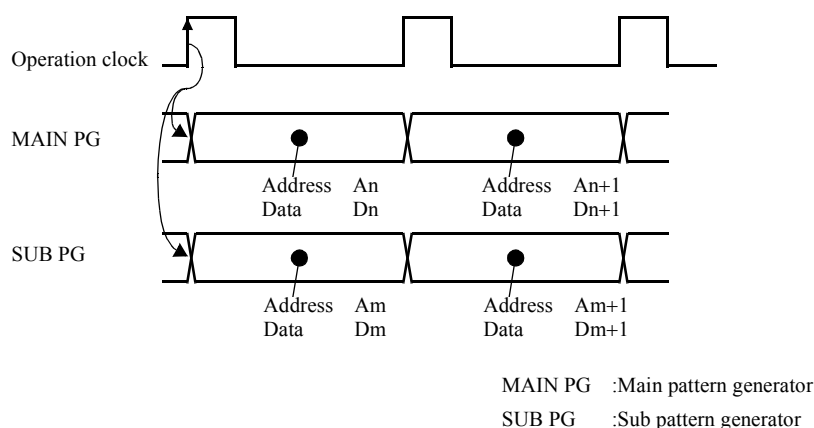
The system can thus generate independent patterns for each port.

The following operation modes are available.

Synchronous Mode

This mode is used to get the main pattern generator and sub-pattern generator to operate with the same operation clock.

The main- and sub-generators can generate the same patterns as well as different patterns.

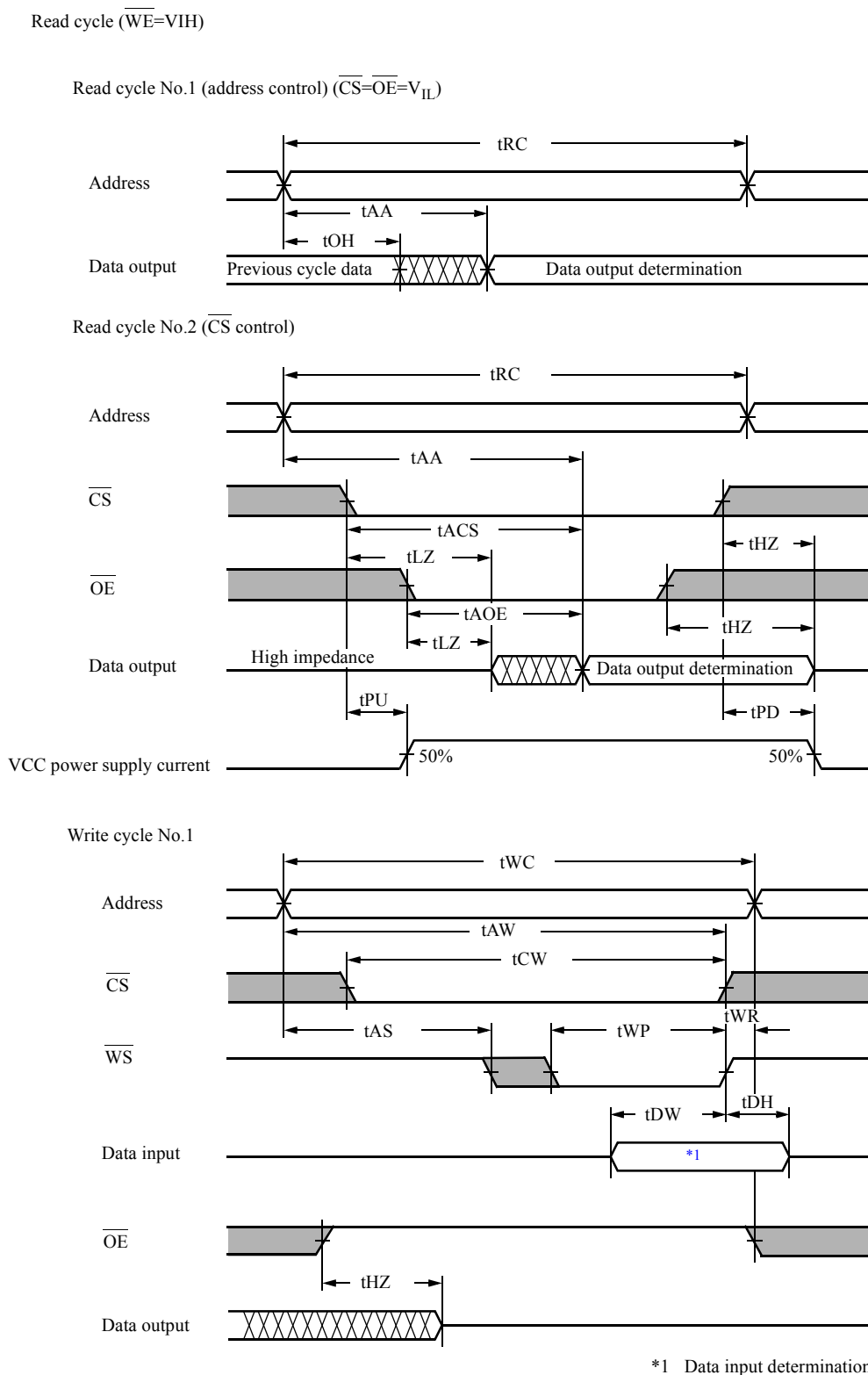


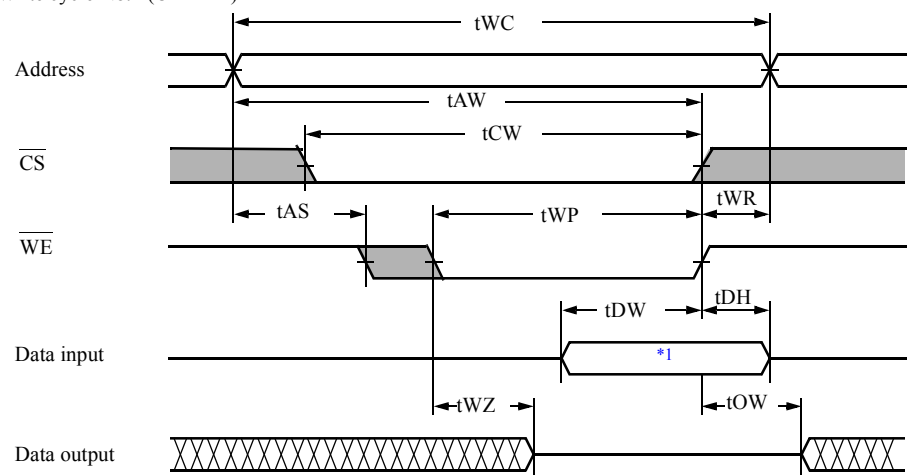
$A_n = A_m$ as well as $A_n \neq A_m$ are possible.

Examples of device test programs in synchronous mode are shown in [5. 5. 2 "MAIN/SUB PG Synchronous Mode Test" on page 5-19](#), with attention being paid to the $2\text{ K} \times 8\text{-bit}$ dual port static memory and the video dual port memory.

5. 5. 2 MAIN/SUB PG Synchronous Mode Test

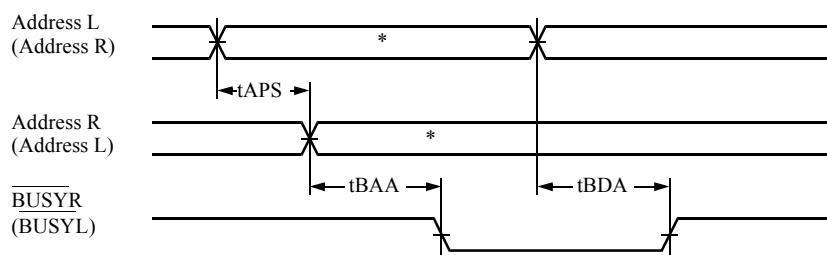
Examples of device test programs using MAIN/SUB PG synchronous mode are shown below with attention being paid to the $2\text{ K} \times 8\text{-bit}$ dual port static memory (see [Figure 5-2](#)).

Figure 5-2 Timing Chart for 2 K × 8-bit Dual Port Static Memory

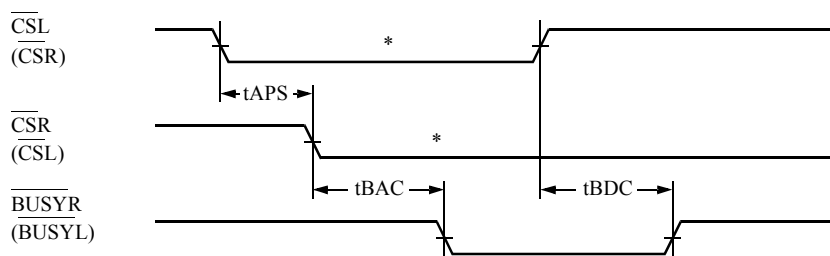
Write cycle No.2 ($\overline{OE} = "L"$)

*1 Data input determination

Contention cycle No.1 (address control)



*Address L = Address R

Contention cycle No.2 ($\overline{CS}$ control)

*Address L = Address R

Contents of Test

1. The write to memory operation is performed in the contention cycle with the left port as the first arriving port.

At this time, the inversion data of the left port is written and also the output of $\overline{\text{BUSY}}$ is verified through the right port.

2. Memory read operation is performed in the contention cycle with the left port as the first arriving port.

At this time, it is confirmed that the data written through the left port are read from the left and right ports.

3. The write to memory operation is performed in the contention cycle with the right port as the first arriving port.

At this time, the inversion data of the right port is written and also the output of $\overline{\text{BUSY}}$ is verified through the left port.

4. The memory read operation is performed in the contention cycle with the right port as the first arriving port.

At this time, it is confirmed that the data written through the right port are read from the left and right ports.

Figure 5-3 shows the timing chart for 1 and 2 and Figure 5-4 shows the timing chart for 3 and 4.

An example of a test plan program is shown in "MAIN/SUB PG Synchronous Mode Test Plan Program" on page 5-25 and an example of a pattern program is shown in "MAIN/SUB PG Synchronous Mode Test Pattern Program" on page 5-27.

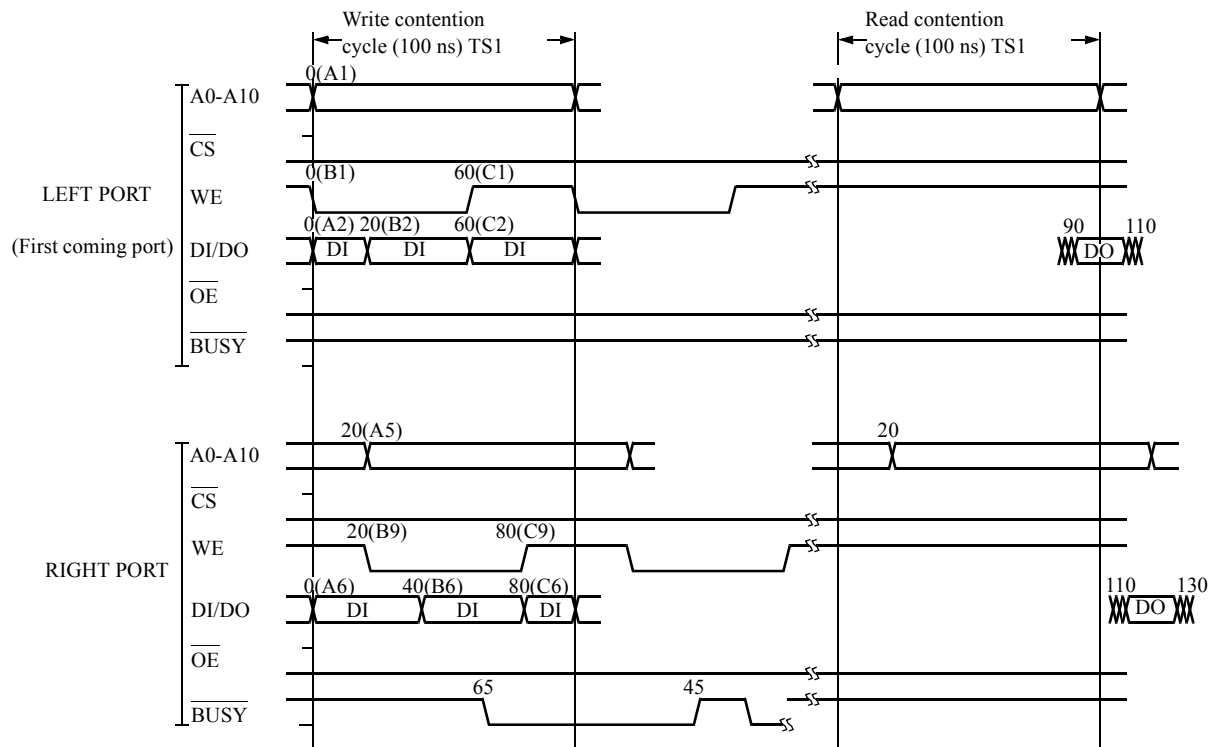
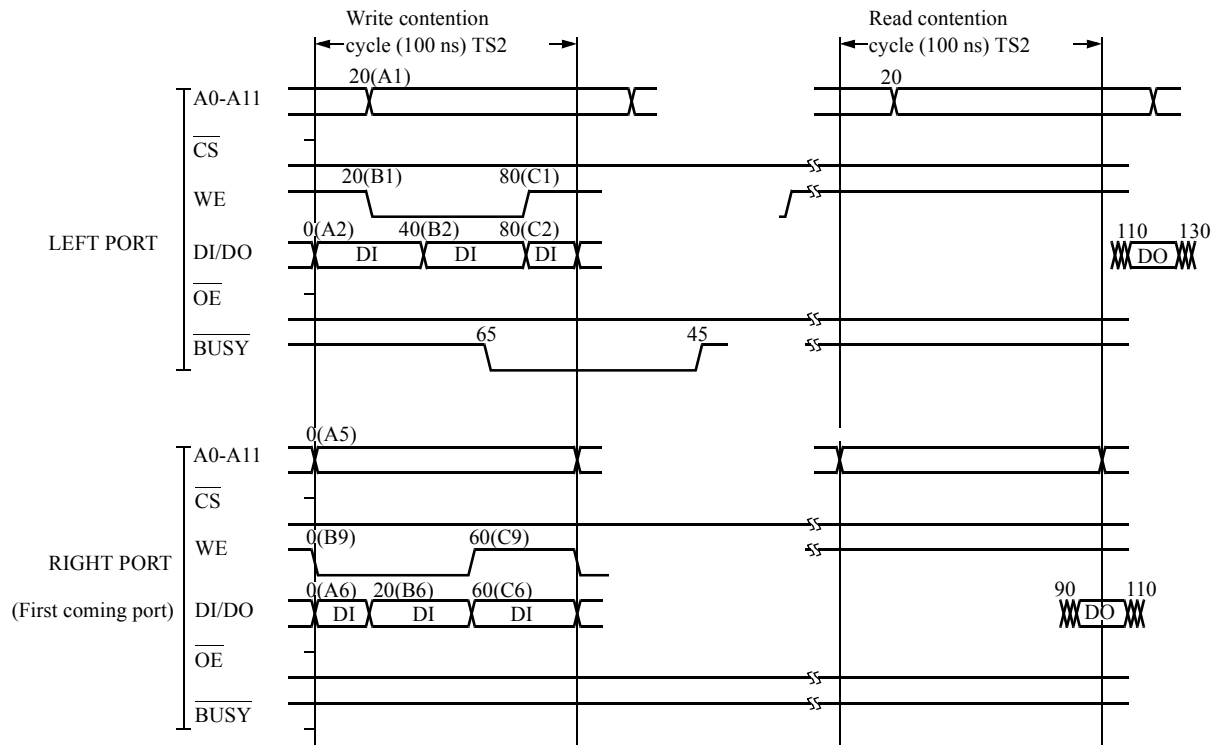
Figure 5-3 Timing Chart for Test Program (1)

Figure 5-4 Timing Chart for Test Program (2)

◆ MAIN/SUB PG Synchronous Mode Test Plan Program

```

PRO  DPSRAM      ;SAMPLE PROGRAM FOR 2K*8 DUAL PORT SRAM
      TIME1=1MS :VS1
      TIME2=1MS :IN
      VS1 =5.0V
      IN1 =2.2V,0.8V
      OUT1=2.4V,0.4V

;LEFT-PORT= (L) = (MAIN-PG)
      PD1-7  =IN1,NRZA,ACLK1,<X0-6>          ;A0-6 (L)
      PD8-11 =IN1,NRZA,ACLK1,<Y0-3>          ;A7-10 (L)
      PD25   =IN1,FIXL                        ; /CS (L)
      PD26   =IN1,FIXL                        ; /OE (L)
      PD27   =IN1,/RZO,BCLK1,CCLK1,<WT>      ; /WE (L)
      PC49   =OUT1,STRB3,CPE3,<C0>           ; /BUSY (L)
      P33-40 =IN1,XOR,ACLK2,BCLK2,CCLK2,DRE1,DREC1,DREZ,@
              OUT1,STRB1,IOC,CPE1,<D0-7>      ;DI/O0-7 (L)

;RIGHT-PORT= (R) (SUB-PG)
      PD13-19=IN1,NRZA,ACLK5,BSUB,<X0-6>      ;A0-6 (R)
      PD20-23=IN1,NRZA,ACLK5,BSUB,<Y0-3>      ;A7-10 (R)
      PD77   =IN1,FIXL                        ; /CS (R)
      PD78   =IN1,FIXL                        ; /OE (R)
      PD79   =IN1,/RZO,BCLK9,CCLK9,BSUB,<WT>  ; /WE (R)
      PC50   =OUT1,STRB4,CPE4,BSUB,<C0>       ; /BUSY (R)
      P41-48 =IN1,XOR,ACLK6,BCLK6,CCLK6,DRE2,DREC2,DREZ,@
              OUT1,STRB2,IOC,CPE2,BSUB,<D0-7> ;DI/O0-7 (R)

```

```

;LEFT-PORT (MAIN-PG)
;
      TS1   TS2
RATE = 100NS, 100NS
ACLK1=  0NS,  20NS
BCLK1=  0NS,  20NS
CCLK1= 60NS,  80NS
ACLK2=  0NS,  20NS
BCLK2= 20NS,  40NS
CCLK2= 60NS,  80NS
STRB1= 110NS, 130NS
STRB3= 145NS, 145NS
DREL1=  10NS,  30NS
DRET1=  70NS,  90NS

;RIGHT-PORT (SUB-PG)
ACLK5=  20NS,   0NS
BCLK9=  20NS,   0NS
CCLK9=  80NS,  60NS
ACLK6=  0NS,   0NS
BCLK6=  40NS,  20NS
CCLK6=  80NS,  60NS
STRB2= 130NS, 110NS
STRB4= 145NS, 145NS
DREL2=  30NS,  10NS
DRET2=  90NS,  70NS

      CALL CALB ("DP16KS",NORMAL)

TEST      1
      REG MPAT      PC=#0
      MEAS MPAT     DM16KS

END

```

◆ MAIN/SUB PG Synchronous Mode Test Pattern Program

```

MPAT DM16KS                                ;SAMPLE PROGRAM FOR 2K*8 DUAL PORT SRAM

REGISTER                                     ] *1
  XMAX=#7F
  YMAX=#F
  IDX1=#7FE
MAIN REGISTER                               ] *2
  CPE1=R                                   ;READ DATA COMP.
  CPE2=C1                                  ;/BUSY COMP.
  DRE1=W                                   ;DI/O DRIVER-ENABLE
SUB REGISTER                               ] *3
  CPE3=R                                   ;READ DATA COMP.
  CPE4=C1                                  ;/BUSY COMP.
  DRE2=W                                   ;DI/O DRIVER-ENABLE

DATA DEFINE                                ] *4
  PDS DATA=D/D

SDEF INIT=XB<0 YB<0 TP<0                  ;INITIALIZE REG.
SDEF INC =XB<XB+1 YB<YB+1^BX              ;INCREMENT BASE

MODULE BEGIN
START #0

NOP      INIT                               TS1
ST0:     JN11  ST0 W INC FP1      C1 C0 TS1 ! W INC FP1 /D C1      TS1
ST1:     JN11  ST1 R INC FP1      C1 C0 TS1 ! R INC FP1      C1 C0 TS1
        JZD    ST0                               TS1
ST2:     JN11  ST2 W INC FP1 /D C1      TS2 ! W INC FP1      C1 C0 TS2
ST3:     JN11  ST3 R INC FP1      C1 C0 TS2 ! R INC FP1      C1 C0 TS2
        JZD    ST2 ] *5 ] *6
STPS

MODULE END

END

```

*1 Setting the registers in MAIN/SUB PG.

*2 Setting the registers in MAIN PG.

*3 Setting the registers in SUB PG.

*4 Select the data to be output to PDS.

*5 Control of pattern generation at the MAIN PG side (In the example shown above, the program is used to generate patterns for the left port.)

*6 Control of pattern generation at the SUB PG side (in the example shown above, the program is used to generate patterns for the right port.)

6. *Programming for Interleave Mode*

In order to generate high-speed patterns over 125 MHz in the T5500 series memory test system, it is necessary to specify the interleave mode or BMUX mode in the pattern program. This chapter describes how to write a pattern program using the interleave mode. It also describes the programming restrictions.

Instruction mnemonics used for conventional T5300 series memory test systems are also used for pattern programs.

➔ For more information about the changes of conventional instructions and the instructions that require no considerations for the interleave mode, refer to [2. "Pattern Program Instructions" on page 2-1](#).

6. 1 Specification of Interleave Mode

The interleave mode (ILMODE) can be selected from among 1WAY, 2WAY, and 4WAY using any of the following methods:

Mode specification can be omitted. The default mode is 1WAY (normal).

The 1WAY normal mode has no bearing on the description given in this chapter.

If more than one method is used to specify a mode, priority is assigned in the order of Title statement < Common section < Module section.

- Specification in the title statement

```
MPAT program-name <2WAY>
```

If ILMODE is specified in the title statement, be sure to enclose the mode name with < and >.

- Specification in the common section

```
ILMODE 2WAY
```

- Specification in the module section

```
MODULE BEGIN
ILMODE 2WAY
```


6. 2 Operation in Interleave Mode

Patterns can be generated at high speed if continuous pattern generation is performed using two or more ALPGs (PG UNIT1 to 4).

If two or more ALPG are assigned for MAIN PG or SUB PG, two types of pattern for one cycle can be generated.

The following table shows the relationship among PG UNIT1 to 4, interleave mode, and operation of MAIN and SUB.

Test system	Interleave mode	MAIN PG	SUB PG
T5593 Without SUBPG option	2WAY 1WAY	1, 2 1	None 2
T5593 With SUBPG option	2WAY 1WAY	1, 2 1	3, 4 2
T5592	4WAY 2WAY 1WAY	1, 2, 3, 4 1, 3 1	None None None
T5591	4WAY 2WAY 1WAY	1, 2, 3, 4 1, 3 1	None 2, 4 2
T5581	2WAY 1WAY	1, 2 1	None 2
T5571P	1WAY	1	None

If a PG UNIT is assigned to SUB PG, then MODE, REGISTER, ADDRESS DEFINE, DATA DEFINE, ARIRAM DEFINE, SCRAM, PRESCRAM, DBM PATTERN, CBM PATTERN, BWPE ADDRESS, BANK SAVE ADDRESS, BANK LOAD ADDRESS, and WCS pattern generating instructions can be specified for MAIN and SUB. (It is the same as the synchronous mode of Dual ALPG function with the T5300 series test systems.)

If a PG UNIT is not assigned to SUB PG, the above settings are not separated for MAIN and SUB.

Operation in normal mode (1WAY)

The PG UNIT operates without using the interleaved mode.

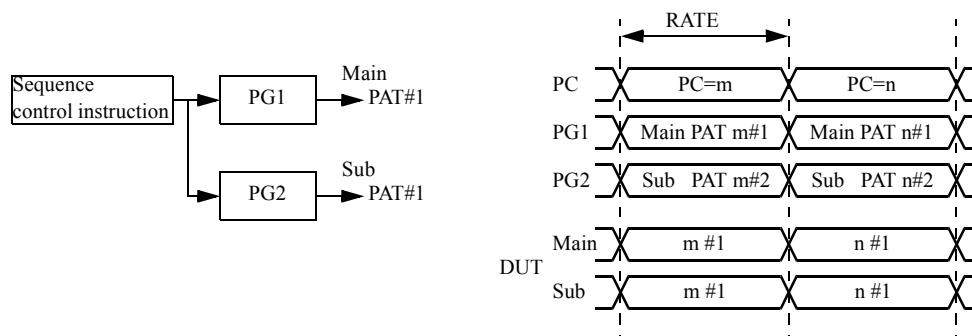
For T5593, T5591, and T5581

Two types of patterns can be generated in each cycle using the PG UNIT1 as MAIN PG and the PG UNIT2 as SUB PG. (In this case, the MAIN PG and the SUB PG must operate in synchronous mode.)

T5592 and T5571P

Only one pattern for each cycle is generated.

- Operation in normal mode on T5593, T5591, and T5581



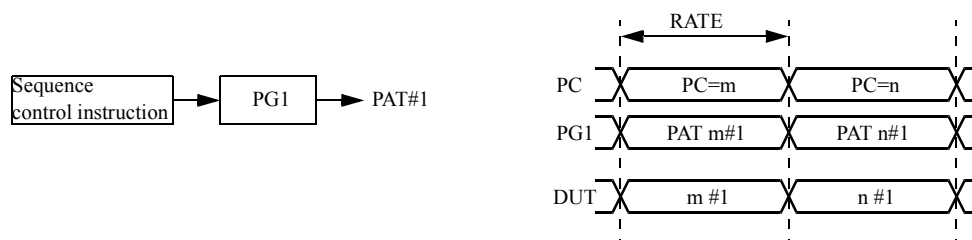
The MAIN PG pattern is output to the PDS of the main bank (bank 1).

The SUB PG pattern is output to the PDS of the sub bank (bank 2).

In a pattern program, only one address and one data generation instruction should be specified for the sequence control instruction.

Sequence control instruction Main PAT#1 ! Sub PAT#1

- Operation in normal mode on T5592 and T5571P



The PG pattern is output to the PDS of bank 1.

In a pattern program, only one address and one data generation instruction should be specified for the sequence control instruction.

Sequence control instruction PAT#1

The address and data generation instruction, and the sequence control instruction are valid at the cycle (RATE) specified in the TS Field.

Operation in 2WAY interleave mode

T5581

Using the 2WAY interleave mode allows PG UNIT1 and PG UNIT2 to generate patterns continuously.

In this mode, MAIN and SUB PG settings cannot be made separately in the pattern program.

T5591

Using the 2WAY interleave mode allows PG UNIT1 and PG UNIT3, or PG UNIT2 and PG UNIT4 to generate patterns continuously.

Two type of patterns can be generated in each cycle using PG UNIT1 and PG UNIT3 as MAIN PG, and PG UNIT2 and PG UNIT4 as SUB PG. (In this case, the MAIN PG and the SUB PG must operate in synchronous mode.)

T5592

Using the 2WAY interleave mode allows PG UNIT1 and PG UNIT3 to generate patterns continuously.

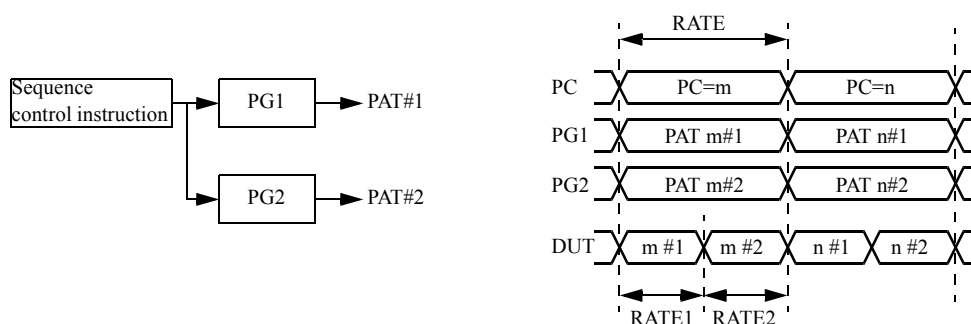
In this mode, MAIN and SUB PG settings cannot be made separately in the pattern program.

T5593

Using the 2WAY interleave mode allows PG UNIT1 and PG UNIT2, or PG UNIT3 and PG UNIT4 to generate patterns continuously.

In a system which has the SUBPG option installed, two type of patterns can be generated in each cycle by using PG UNIT1 and PG UNIT2 as the MAIN PG, and PG UNIT3 and PG UNIT4 as the SUB PG. (In this case, the MAIN PG and the SUB PG must operate in synchronous mode.)

- Operation in 2WAY interleave mode on T5581
Operating in the 2WAY interleave mode in the T5593 (without the SUBPG option)



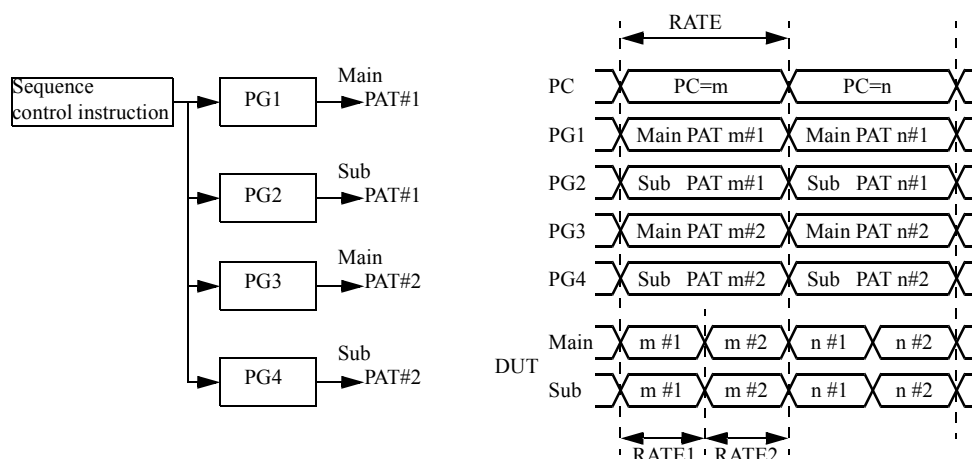
The PG1 pattern is output to the PDS of bank 1.

The PG2 pattern is output to the PDS of bank 2.

In a pattern program, two addresses and two data generation instructions should be specified for the sequence control instruction.

Sequence control instruction : PAT#1
: PAT#2

- Operation in 2WAY interleave mode on T5591



The PG1 pattern is output to the PDS of the main bank (bank 1).

The PG2 pattern is output to the PDS of the sub bank (bank 2).

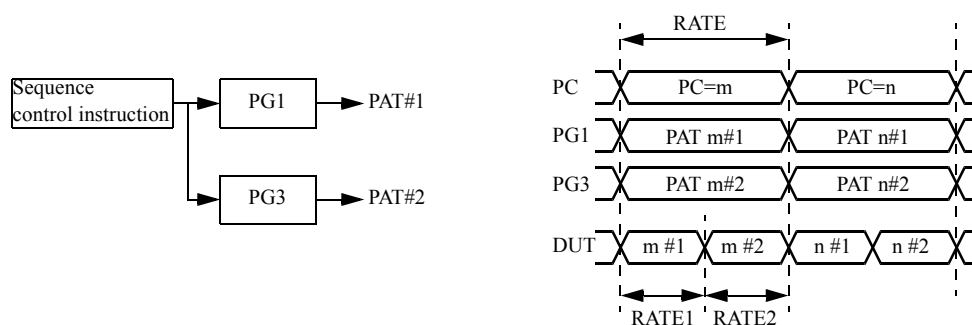
The PG3 pattern is output to the PDS of the main bank (bank 3).

The PG4 pattern is output to the PDS of the sub bank (bank 4).

In a pattern program, two addresses and two data generation instructions should be specified for the sequence control instruction.

Sequence control instruction : Main PAT#1 ! Sub PAT#1
: Main PAT#2 ! Sub PAT#2

- Operation in 2WAY interleave mode on T5592



The PG1 pattern is output to the PDS of bank 1.

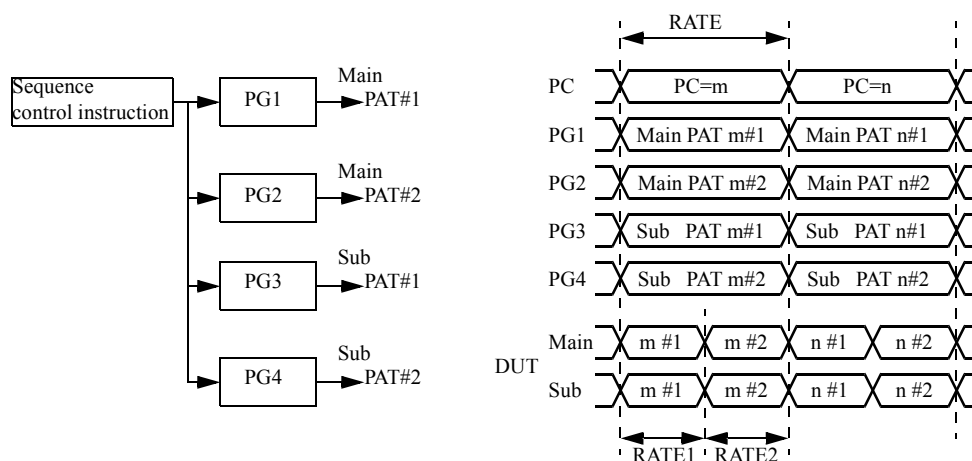
The PG3 pattern is output to the PDS of bank 3.

In a pattern program, two addresses and two data generation instructions should be specified for the sequence control instruction.

Sequence control instruction : PAT#1
: PAT#2

The address or data generation instruction for each step is valid in the cycle (RATE1 or RATE2) which was specified in the TS Field. The sequence control instruction is valid in the cycle (RATE) including RATE1 and RATE2.

- Operating in the 2WAY interleave mode in the T5593 (with the SUBPG option)



The PG1 pattern is output to the PDS of the main side of bank 1.

The PG2 pattern is output to the PDS of the main side of bank 2.

The PG3 pattern is output to the PDS of the sub side of bank 1.

The PG4 pattern is output to the PDS of the sub side of bank 2.

In a pattern program, two addresses and two data generation instructions should be specified for the sequence control instruction.

Sequence control instruction : Main PAT#1 ! Sub PAT#1
: Main PAT#2 ! Sub PAT#2

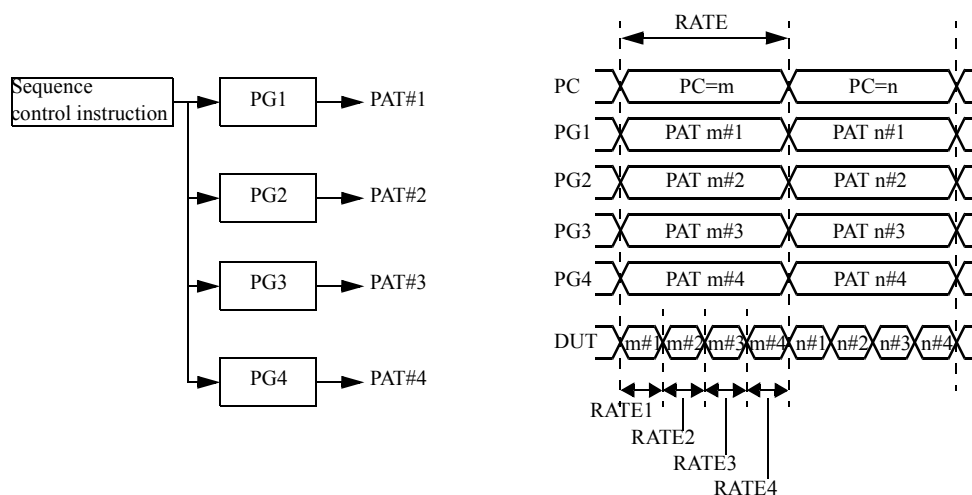
Operation in 4WAY interleave mode

T5592 and T5591

Using the 4WAY interleave mode allows PG UNIT1, PG UNIT2, PG UNIT3, and PG UNIT4 to generate patterns continuously.

In this mode, MAIN and SUB PG settings cannot be made separately in the pattern program.

- Operation in 4WAY interleave mode on T5592 or T5591



The PG1 pattern is output to the PDS of bank 1.

The PG2 pattern is output to the PDS of bank 2.

The PG3 pattern is output to the PDS of bank 3.

The PG4 pattern is output to the PDS of bank 4.

In a pattern program, four addresses and four data generation instructions should be specified for the sequence control instruction.

Sequence control instruction : PAT#1
 : PAT#2
 : PAT#3
 : PAT#4

The address or data generation instruction for each step is valid in the cycle (RATE1, RATE2, RATE3 or RATE4) which was specified in the TS Field. The sequence control instruction is valid in the cycle (RATE) including RATE1 to 4.

6. 3 Programming Format for Interleave Mode

Instruction mnemonic which is specified in the pattern program is classified as follows:

field A : PC Operation code, Operand, I, T, TM1, TM2, DBMINC

field B : MAIN ALPG Instruction

field C : SUB ALPG Instruction

Separate specification and normal specification shown below can be selected optionally in the pattern program.

Specification format in the 2WAY or 4WAY mode 1 (separate specification)

This is a format to describe the instructions using fields B and C in two or four steps for one step of the field A instruction. (The field C instruction cannot be described in for the 4 WAY mode.)

Use a colon ":" as a field separating character.

- <2WAY>

```

--- A --- : --- B --- ! --- C ---
           : --- B --- ! --- C ---

```

- <4WAY>

```

--- A --- : --- B ---
           : --- B ---
           : --- B ---
           : --- B ---

```



Tip

Description of two or four steps is always necessary for the separate specification. Even if there is no instruction to be specified in the second step or second to fourth steps, ":" cannot be omitted.

Specification format in the 2WAY or 4WAY mode 2 (normal specification)

This is a format to describe the instructions using fields B and C in one step for one step of the field A instruction.

If the field separating character ":" is omitted, it is judged to be a format of this specification.

- <2WAY>

```

--- A ---   --- B --- ! --- C ---

```

- <4WAY>

```

--- A ---   --- B ---

```

When the normal specification is recognized by the MPAT compiler, compilation is performed after the instructions using fields B and C are copied and they are expanded to a two- or four-step separate specification.

However, the following instructions are compiled with a special process.

- The SWAP(LMAX<>HMAX), RF(RF<RF+1 X<RF) or X<RF instruction is processed as the one specified in the first step of the separate specification without being copied.
- In the ECBM mode, the CBMA register operating instruction is processed as the one specified in the second or fourth step of the separate specification without being copied.
- DSD, RCD, and CCD register operating instructions or CLEAR instruction is processed as the one specified in the second or fourth step of the separate specification without being copied.

Mixed specification format

The separate specification and the normal specification can be optionally selected in the pattern program. Examples of the specification are shown in ["Pattern Program for Interleave Mode Test \(Mixed Specification Format\)" on page 6-10](#) and ["Pattern Program for Interleave Mode Test \(Specification after Processed to Separate Specification\)" on page 6-10](#).

["Pattern Program for Interleave Mode Test \(Mixed Specification Format\)" on page 6-10](#) has the mixed specification, which will be all processed to separate specifications as shown in ["Pattern Program for Interleave Mode Test \(Specification after Processed to Separate Specification\)" on page 6-10](#) and compiled.

As ["Pattern Program for Interleave Mode Test \(Specification after Processed to Separate Specification\)" on page 6-10](#) shows the specification after processed to separate specification (the source file is not changed), XB<0 YB<0 instructions are specified in two steps. If they are specified in the separate specification from the beginning, XB<0 YB<0 instructions as shown in (1) and (2) need not to be specified in two steps. (The patterns which are generated in ["Pattern Program for Interleave Mode Test \(Mixed Specification Format\)" on page 6-10](#) and ["Pattern Program for Interleave Mode Test \(Specification after Processed to Separate Specification\)" on page 6-10](#) are the same.)

◆ **Pattern Program for Interleave Mode Test (Mixed Specification Format)**

```

MPAT SAMPLE1 <2WAY>

REGISTER
  IDX1=#7FFE
  IDX2=#FFFE
  XMAX=#FF
  YMAX=#FF

START #0    ; Marching

      NOP      XB<0  YB<0      TP<TPH
ST0:  JN11 .    W XB<XB+1 YB<YB+1^BX
      JN12 .    : R
      : W XB<XB+1 YB<YB+1^BX      /D
      JN12 .    : R      /X /Y /D
      : W XB<XB+1 YB<YB+1^BX /X /Y
      JZD ST0   XB<0  YB<0
      STPS

```

◆ **Pattern Program for Interleave Mode Test (Specification after Processed to Separate Specification)**

```

MPAT SAMPLE1 <2WAY>

REGISTER
  IDX1=#7FFE
  IDX2=#FFFE
  XMAX=#FF
  YMAX=#FF

START #0    ; Marching

      NOP :    XB<0  YB<0      TP<TPH
      :    XB<0  YB<0      TP<TPH      (1)
ST0:  JN11 .    : W XB<XB+1 YB<YB+1^BX
      : W XB<XB+1 YB<YB+1^BX
      JN12 .    : R
      : W XB<XB+1 YB<YB+1^BX      /D
      JN12 .    : R      /X /Y /D
      : W XB<XB+1 YB<YB+1^BX /X /Y
      JZD ST0 :    XB<0  YB<0
      :    XB<0  YB<0      (2)
      STPS      :
      :

```

6. 4 Outline of Restrictions in Interleave Mode

When 2WAY or 4WAY mode is specified in the pattern program, some restrictions which do not matter in the normal (1WAY) mode arise newly. The outline is shown below.

(The restrictions are applied only when 2WAY/4WAY mode is specified.)

Restriction on specification of the register operation instruction (restriction on coexistence of different kinds of operation instructions)

In the interleaved mode, there is restriction on specification of the register operation instruction (restriction on the combination) in successive two or four steps of pattern generation. The restriction is imposed in the case where the following register operator codes are used.

- + (Addition)
- – (Subtraction)
- & (AND operation)
- ' (OR operation)
- " (Exclusive logical OR)
- / (Inverse operation)
- *2 (Left-shift operation)
- /2 (Right-shift operation)

These operator codes are classified into four groups. The following four kinds of operation instructions cannot be specified together in the successive two or four steps.

- Addition and subtraction instruction (Addition and subtraction)
- Logical operation instruction (Logical AND, logical OR and exclusive logical OR)
- Inversion operation instruction (Inverse operation)
- Shift operation instruction (Left-shift and right-shift)

Restriction on number of instruction specifications (once)

They can be specified only once in successive two or four steps of pattern generation. In the interleaved mode, the following instructions have restriction.

- SWAP instruction (example: LMAX<>HMAX)
- DSET instruction (example: XH<#FF)
- Logical operation instruction (example: XC<XC&D1)
- Inversion operation instruction (example: XC</XC)
- DSD register operation instruction (example: DSD<DSD+1)
- DSC register operation instruction (example: DSC<DSC*2)

Restriction on specification of pair instructions

In the interleaved mode, the following instructions should be specified in pairs in a step.

- Operation instruction with Carry (BX, CY) and Carry generation instruction (example: $XB < XB+1 \ YB < YB+1 \wedge BX$)
- $D3 < D3*2L$ instruction and $D4 < D4*2L$ instruction
- $RF < RF+1$ instruction and $X < RF$ instruction

Restriction on specification of the operation instruction in the RTN instruction line

In the interleave mode, there is a restraint that register operating instructions other than $RF < RF+1$ instruction cannot be specified in steps 1 to 2, or in steps 1 to 4 of the RTN instruction. (Instructions not executing register operation, such as MUT, TS and Control Bit, can be specified.)

Restriction that some instruction cannot be specified for the interleave mode

The following instructions cannot be specified for the interleave mode.

- FLGLI1 instruction (Match Flag Sense instruction)
- PD instruction (Data Hold instruction)
- SET instruction (Data Set instruction)

Instruction restrictions when using the SEPARATE format

There are three restrictions which apply to separate instructions used in the interleaved mode. These are shown below:

Instructions that can be specified only in the first step (cannot be specified in the second step or second to fourth steps).

- $RF < RF+1$ operation instruction
- $X < RF$ instruction

Instructions that can be specified only in the second or fourth step (cannot be specified in the first or first to third steps).

- The CBMA register operating instruction when in the ECBM mode
- DSD, RCD, and CCD register operating instructions or CLEAR instruction

Instructions which are unused as the separate instruction.

BDW, BCW, BMW (BWPE control bit)

XASV instruction specification restrictions

When an XASV instruction is specified in the interleave mode, the instruction which updates the X address is restricted and cannot be specified in two or four successive pattern generation steps.

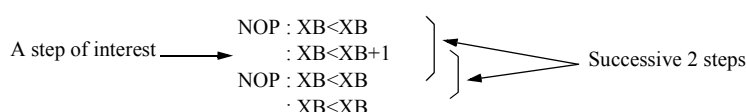
6. 4. 1 Concept of Successive Two or Four Steps

A condition "successive two or four steps" is often imposed as restrictions which are caused by a specification required for 2WAY or 4WAY mode. This occurs because the restrictions in the specified 2WAY or 4WAY mode are mainly generated by the combination of "successive two or four step" instructions.

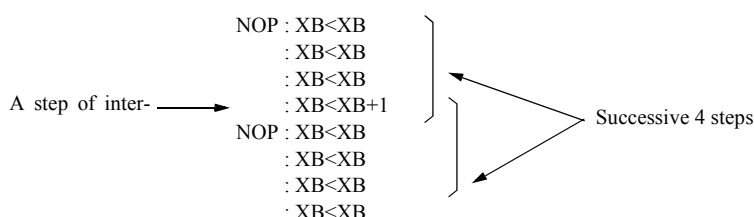
The phrase "successive two or four steps" indicates that two or four steps of instructions using fields B or C are in succession when the 2WAY or 4WAY mode is specified as shown below. That is, when taking notice of a specific step in the pattern program operating in the 2WAY mode, the phrase "successive two steps" means the specific step and the previous step, or the specific step and the following step.

In the 4WAY mode, the phrase "successive four steps" means the specific step and the previous three steps or the specific step and the following three steps.

◆ Successive Two Steps in the 2WAY Mode



◆ Successive Four Steps in the 4WAY Mode



The restriction on successive two or four steps requires that attention be paid to every pattern step in the pattern program as a step of interest. That is, an index loop, a jump, and subroutine call, (or return) are not exceptions.

In the following 2WAY mode example, it is necessary to confirm the restriction by changing the step of interest from Inst. (Instruction) A1, Inst. A2, Inst. B1, to Inst. C2. Also, the confirmation of the restriction on successive two steps is required for the sequence from Inst. A2 to Inst. A1 by the IDXII instruction, from Inst. B2 to Inst. A1 by the JNI2 instruction, and from Inst. C2 to Inst. A1 by the JNI3 instruction.

```

ST0:  IDXI1 #F : Inst. A1
      : Inst. A2
      JNI2 ST0 : Inst. B1
      : Inst. B2
      JNI3 ST0 : Inst. C1
      : Inst. C2

```

6. 4. 2 Restriction-related Instruction and Unrelated Instruction

Restriction-related instruction

Items written in the section of "Outline of Restrictions" are objects of restrictions. The following register-related instructions/operation instructions are objects.

- DSET instruction to the register for which a DSET instruction can be specified (example: XH<#FF)
 - Operation instruction on the XB register (example: XB<XB+1)
 - Operation instruction using the YB register (example: YB<YB+1)
 - Operation instruction using the Z register (example: Z<Z+1)
 - Operation instruction using the N register (example: N<N+1)
 - Operation instruction using the B register (example: B<B+1)
 - Operation instruction using the D1-6 registers (example: D3<D3+1, D4<D4*2)
 - Operation instruction using the XC, XS, and XK registers (example: XC<XC+1)
 - Operation instruction using the YC, YS, and YK registers (example: YC<YC+1)
 - Operation instruction using the TP and TP2 registers (example: TP<TP+1, TP2<TPH2)
 - Operation instruction using the DSD, RCD, and CCD registers (example: DSD<DSD+1)
 - Operation instruction using the RF register (example: RF<RF+1)
 - Operation instruction using the DSC register (example: DSC<DSC*2)
 - Operation instruction using the CBMA register (example: CBMA<CBMA+1)
 - SWAP instruction (e.g., LMAX<>HMAX)
 - RTN instruction
 - BWPE control bit (BDW, BCW, BMW)
 - XASV instruction (XASV)
- ➔ For more information, refer to [6. 5 "Restrictions on Each Instruction in Interleave Mode" on page 6-16.](#)

Restriction-unrelated instruction

These restrictions are not applied to each control bit not related to the register operation and instructions such as MUT signals. The following instructions are applied to the restrictions:

- Source field instructions (example: X<XB, Y<YB, D<TPXOR)
- Inversion field instructions (/X, /Y, /Z, /D, /D1, and /D2)
- FP function instructions (FP0, FP1, FP3, TP4, FP5, FP6, FP7, FP8, FP9, TP10, and FPX)
- TS (TS1-16)
- MUT control bits (R, W, M1, M2, and C0-23)
- ALPG control bits (XT, YT, SCROFF, PSCROFF, ARIOFF, and INHDBM)
- FM control bits (MMA, MMB, PM/2, and PM/4)
- Cycle Palette field (CYPA1-16 and CYPB1-16)
- XALD instruction (XALD)

6. 4. 3 Instructions whose Use is Prohibited in Interleave Mode

Instructions whose use is prohibited in interleave mode

- PD (Data Hold instruction)
- SET (SET instruction for sequence control instructions)
- FLGLI1 (Match Flag Sense instruction)

Instructions whose simultaneous use is prohibited in interleave mode

DSET (Direct data SET) instruction to D5 register and D5 & TP operating instructions, also DSET instruction to D6 register and D6 & TP2 operating instructions cannot be specified in the same step.

6. 5 Restrictions on Each Instruction in Interleave Mode

6. 5. 1 DSET (Direct data SET) Instruction (register name<m m:set value) Restriction

The following restrictions are applied in using a DSET instruction to assign a value directly to a register in the pattern program:

Restrictions on specifying registers

The DSET instruction can be specified for the following four register groups:

Only one register of each register groups can be specified in successive two or four steps.

When TP 36-bit mode (TPM 36) is specified, register groups C and D are treated as 1 register group.

Group A	Group B	Group C	Group D	Restriction on the number of specifications (one)
XH D1 D3B NH ND ZD	YH D2 D4B ZH BH BD	TPH1 D5	TPH2 D6	Specifying restriction
XOS XT	YOS YT Z	TP1 DCMR1 XMASK CBMA RCD DSD	TP2 DCMR2 YMASK DSC CCD	No specifying restriction

Restriction on the number of specifications (one)

There are two kinds of DSET instructions to the same register, one is the register which has the restriction to specify once in successive two or four steps, the other is the register which has no restriction.

Refer to the above table for the group of registers.

(The register, which is used with moving the setting value to other register, has the restriction on specifying.)

D5 and D6 register specifying restriction

The operating instructions of D5 & TP and D6 & TP2 registers (including the LOAD instruction) cannot be specified in the same step as a DSET instruction to D5 and D6 registers.

In successive two or four steps, DSET instruction to D5 and D6 registers cannot be specified after operating instructions of D5 & TP and D6 & TP2 (include LOAD instruction).

- Cannot be specified

- ◆ **Example 1:**

```
NOP:TP<D5 D5<#FF
:
```

- ◆ **Example 2:**

```
NOP:TP<TP+D5
:D5<#F
```

- Can be specified

- ◆ **Example 1:**

```
NOP:TP<TP D5<#FF
:TP<D5
```

- ◆ **Example 2:**

```
NOP:D5<#F
:TP<TP+D5
```

Notes

The CLEAR instruction to the register (example: F<0) is not the DSET instruction excluding DSC register (excluding TP<0, TP2<0 instruction).

The DSC<#F instruction to the DSC register is not the DSET instruction.

- Cannot be specified

- ◆ **Example**

```
NOP:TP2<#1 DSC<0
:
```

- Can be specified

- ◆ **Example**

```
NOP:TP2<#1 DSC<#F
:
```




Tip

If the interleave mode is not specified, ["Restrictions on specifying registers" on page 6-16](#) and ["Notes" on page 6-17](#) are applied.

6. 5. 2 XB and YB Register Operating Instruction Restriction

Restrictions when using XB and YB registers

Restrictions are imposed on the condition under which the XB and YB register values are changed in two or four successive steps and the XB and YB registers are used in the operation which uses the XC and YC registers.

➔ For more information, refer to [6. 5. 8 "XC and YC Register Operating Instruction Restriction" on page 6-29](#).

Restrictions when specifying operating instructions that include Carry (BX, CY)

For YB register operations, Restrictions are imposed when specifying operating instructions that include Carry (BX, CY).

➔ For more information, refer to [6. 5. 9 "Carry \(BX, CY\)-attached Operating Instruction Restriction" on page 6-35](#).

Restriction 1 when using D1 and D2 registers

Only one of the D1 and D2 registers can be specified for operations which uses the XB and YB registers in two and four successive steps.

- Cannot be specified

◆ Example

```
NOP : XB<XB+D1
      : XB<XB+D2
```

- Can be specified

◆ Example

```
NOP : XB<XB+D2
      : XB<XB+D2
```

Restriction 2 when using D1 and D2 registers

If D1 and D2 register values are changed and the old and new values of the D1 and D2 registers are used in the XB and YB registers' operation in two or four successive steps, the values of the D1 and D2 registers cannot be changed.

- Cannot be specified

◆ Example 1:

```

NOP : XB<XB+D1
      : XB<XB+D1 D1<#F
      : XB<XB+D1
      : XB<XB+D1

```

◆ Example 2:

```

NOP : XB<XB+D1
      :           D1<#F
      : XB<XB+D1
      : XB<XB+D1

```

- Can be specified

◆ Example 1:

```

NOP :           D1<#F
      : XB<XB+D1
      : XB<XB+D1
      : XB<XB+D1

```

◆ Example 2:

```

NOP : XB<XB+D1
      : XB<XB+D1
      : XB<XB+D1
      : XB<XB+D1 D1<#F

```

Restriction 3 when using D1 and D2 registers

Restrictions are imposed when specifying the D1A to D1H and D2A to D2D registers to be used for operations which use the XB and YB registers.

➔ For more information, refer to [6. 5. 6 "D1 to D4 Register Operating Instruction Restriction" on page 6-26](#).

6. 5. 3 Z Register Operating Instruction Restriction

Restriction on coexistence of different kinds of operation instructions

Mixture of an addition and subtraction operation instruction (example: $Z<Z\pm 1$), shift operation instruction (example: $Z<Z*2$), and inverse operation instruction (example: $Z*/Z$) cannot be specified in successive two steps or four steps.

- Cannot be specified

◆ Example

```
NOP: Z<Z+1
      : Z<Z*2
```

- Can be specified

◆ Example

```
NOP: Z<Z+1
      : Z<Z-1
```

Restriction on the number of specifications (one)

The inverse operating instruction ($Z</Z$) can be specified only once in successive two or four steps.

- Cannot be specified

◆ Example

```
NOP: Z</Z
      : Z</Z
```

- Can be specified

◆ Example

```
NOP: Z</Z
      : Z<ZH
NOP: Z</Z
      :
```

Restriction on use of the ZD register

If the ZD register value is changed and the old and new values of the ZD register are used in the Z register's operation in two or four successive steps, the value of the ZD register cannot be changed.

- Cannot be specified

◆ Example 1:

```
NOP : Z<Z+ZD
      : Z<Z+ZD ZD<#F
      : Z<Z+ZD
      : Z<Z+ZD
```

◆ **Example 2:**

```

NOP : Z<Z+ZD
      :          ZD<#F
      : Z<Z+ZD
      : Z<Z+ZD

```

- Can be specified

◆ **Example 1:**

```

NOP :          ZD<#F
      : Z<Z+ZD
      : Z<Z+ZD
      : Z<Z+ZD

```

◆ **Example 2:**

```

NOP : Z<Z+ZD
      : Z<Z+ZD
      : Z<Z+ZD
      : Z<Z+ZD ZD<#F

```

Restrictions on specifying operation instructions with Carry (BX, CY)

There is a restriction on specifying operation instructions with Carry (BX, CY).

- ➔ For more information, refer to [6. 5. 9 "Carry \(BX, CY\)-attached Operating Instruction Restriction" on page 6-35](#).

6. 5. 4 N Register Operating Instruction Restriction**Restriction on coexistence of different kinds of operation instructions**

An addition and subtraction instruction (example: $N < N \pm 1$) or $N < ND_n$ ($n=1$ to 7) cannot be used with an inverse operation instruction (example: $N < /N$) in two or four consecutive steps. However, this restriction does not apply if $N < ND_n$ is specified after the $N < /N$ instruction.

- Cannot be specified

◆ **Example 1:**

```

NOP : N<N+1
      : N</N

```

◆ **Example 2:**

```
NOP:N<ND1
      :N</N
```

- Can be specified

◆ **Example 1:**

```
NOP:N<N+1
      :N<N-1
```

◆ **Example 2:**

```
NOP:N</N
      :N<ND1
```

Restriction on the number of specifications (one)

The inverse operating instruction (example: N</N) can be specified only once in successive two or four steps.

- Cannot be specified

◆ **Example**

```
NOP:N</N
      :N</N
```

- Can be specified

◆ **Example**

```
NOP:N</N
      :N<NH
NOP:N</N
      :
```

Restriction 1 when using ND1 to ND7 registers

Only one of the ND1 to ND7 registers can be specified for operations which use the N register in two and four successive steps.

- Cannot be specified

◆ **Example**

```
NOP : N<N+ND1
      : N<N+ND3
```

- Can be specified

◆ **Example**

```
NOP : N<N+ND3
      : N<N+ND3
```

Restrictions when using the ND1 register

If the ND1 register value is changed and the old and new values of the ND1 register are used in the N register's operation in two or four successive steps, the value of the ND1 register cannot be changed.

- Cannot be specified

◆ **Example 1:**

```
NOP : N<N+ND1
      : N<N+ND1 ND1<#F
      : N<N+ND1
      : N<N+ND1
```

◆ **Example 2:**

```
NOP : N<N+ND1
      : ND1<#F
      : N<N+ND1
      : N<N+ND1
```

- Can be specified

◆ **Example 1:**

```
NOP : ND1<#F
      : N<N+ND1
      : N<N+ND1
      : N<N+ND1
```

◆ **Example 2:**

```

NOP : N<N+ND1
      : N<N+ND1
      : N<N+ND1
      : N<N+ND1 ND1<#F

```

6. 5. 5 B Register Operating Instruction Restriction

Restrictions on the coexistence of different types of operating instructions

An addition and subtraction instruction (example: $B<B\pm 1$) or $B<BDn$ ($n=1$ to 7) cannot be used with an inverse operation instruction (example: $B</B$) in two or four consecutive steps. However, this restriction does not apply if $B<BDn$ is specified after the $B</B$ instruction.

- Cannot be specified

◆ **Example 1:**

```

NOP: B<B+1
      : B</B

```

◆ **Example 2:**

```

NOP: B<BD1
      : B</B

```

- Can be specified

◆ **Example 1:**

```

NOP: B<B+1
      : B<B-1

```

◆ **Example 2:**

```

NOP: B</B
      : B<BD1

```

Restrictions on the number of specifications (one)

The inverse operation instruction (example: $B</B$) can be specified only once every two or four successive steps.

- Cannot be specified

◆ **Example**

```
NOP : B</B
      : B</B
```

- Can be specified

◆ **Example**

```
NOP : B</B
      : B<BH
NOP : B</B
      :
```

Restriction when using BD1 to BD7 registers

Only one of the BD1 to BD7 registers can be specified for operations which use the B register in two and four successive steps.

- Cannot be specified

◆ **Example**

```
NOP : B<B+BD1
      : B<B+BD3
```

- Can be specified

◆ **Example**

```
NOP : B<B+BD3
      : B<B+BD3
```

Restrictions when using the BD1 register

If the BD1 register value is changed and the old and new values of the BD1 register are used in the B register's operation in two or four successive steps, the value of the BD1 register cannot be changed.

- Cannot be specified

◆ **Example 1:**

```
NOP : B<B+BD1
      : B<B+BD1 BD1<#F
      : B<B+BD1
      : B<B+BD1
```


◆ Example 2:

```

NOP : B<B+BD1
      :          BD1<#F
      : B<B+BD1
      : B<B+BD1

```

- Can be specified

◆ Example 1:

```

NOP :          BD1<#F
      : B<B+BD1
      : B<B+BD1
      : B<B+BD1

```

◆ Example 2:

```

NOP : B<B+BD1
      : B<B+BD1
      : B<B+BD1
      : B<B+BD1 BD1<#F

```

6. 5. 6 D1 to D4 Register Operating Instruction Restriction

Restriction on pair specification

D3<D3*2L and D4<D4*2L must be coded in pairs.

In addition, D3<D3*2L and D4<D4*2L cannot be specified together with D3<D3*2 and D4<D4*2 in successive two or four steps.

- Cannot be specified

◆ Example

```

NOP:D3<D3*2L D4<D4*2L
      :D3<D3*2 D4<D4*2

```

- Can be specified

◆ Example

```

NOP:D3<D3*2L D4<D4*2L
      :D3<D3*2L D4<D4*2L

```

Restriction on coexistence of different kinds of operation instructions

Mixture of the addition and subtraction operation instruction ($D3 < D3 \pm 1$) and shift operation instruction ($D3 < D3 * 2$) cannot be specified in successive two steps or four steps.

- Cannot be specified

- ◆ **Example**

```
NOP: D3 < D3 + 1
      : D3 < D3 * 2
```

- Can be specified

- ◆ **Example**

```
NOP: D3 < D3 + 1
      : D3 < D3 B
NOP: D3 < D3 * 2
      :
```

D4 register operating Instruction

In the successive two or four steps, the $XC[S][K]$, or $YC[S][K]$ register's operation instructions using the D4 register cannot be specified following the $D3 < D3 * 2L$ $D4 < D4 * 2L$ operating instruction.

- Cannot be specified

- ◆ **Example**

```
NOP: D3 < D3 * 2L D4 < D4 * 2L
      : XC < XC + D4
```

- Can be specified

- ◆ **Example**

```
NOP: D3 < D3 * 2L D4 < D4 * 2L
      : XC < XC + D3
```

Restriction when using D1 and D2 registers

When operating on the XB, YB, XC, YC, XS, YS, XK, or YK register, any operating instruction that uses a different register among the D1A to D1H registers cannot be specified in two or four successive steps. The same applies for D2A to D2D registers.

- Cannot be specified

- ◆ **Example**

```
NOP : XB<XB+D1A
      : XB<XB+D1B
```

```
NOP : XB<XB+D1A
      : XC<XC+D1B
```

- Can be specified

- ◆ **Example**

```
NOP : XB<XB+D1A
      : XB<XB+D1A
```

```
NOP : XB<XB+D1A
      : XC<XC+D1A
```

6. 5. 7 D5 and D6 Register Specifying Restriction

The operating instructions of D5 & TP and D6 & TP2 registers (including the LOAD instruction) cannot be specified in the same step as a DSET instruction to D5 and D6 registers.

In successive two or four steps, DSET instruction to D5 and D6 registers cannot be specified after operating instructions of D5 & TP and D6 & TP2 (include LOAD instruction).

- Cannot be specified

- ◆ **Example 1:**

```
NOP:TP<D5 D5<#FF
      :
```

- ◆ **Example 2:**

```
NOP:TP<TP+D5
      :D5<#F
```

- Can be specified

- ◆ **Example 1:**

```
NOP:TP<TP D5<#FF
      :TP<D5
```

◆ **Example 2:**

```
NOP:D5<#F
      :TP<TP+D5
```

6. 5. 8 XC and YC Register Operating Instruction Restriction

XC and YC registers are different from other registers, that is, they have cases $F=A$ (example: $XC<XC+1$) and $F\neq A$ (example: $XC<XB+1$) in the general expression (example: $F<A+1$). The restriction is also different in these 2 cases.

Operating restriction among registers.

XC register has XS and XK, and YC register has YS and YK as saving registers respectively.

In successive two steps or four steps, the number of registers that data can be updated is only one from the three registers of XC, XS, and XK except the following data load instructions between two registers:

$XC<XC$, $XC<XS$, $XC<XK$

$XS<XS$, $XS<XC$

$XK<XK$, $XK<XC$

However, when the same operation results are stored by register simultaneous specifying format (XCS, XCK, XSK, XCSK, YCS, YCK, YSK and YCSK), they are counted as one.

The above description also applies to XC, YS, and YK registers.

In the following example of "cannot be specified," it is limited since two values of XC and XS registers are updated. In the following example of "can be specified," it is not limited since the XC is the data transfer of XK, and only one value of XS register is updated.

- Cannot be specified

◆ **Example**

```
NOP:XS<XB
      :XC<XC+1
```

- Can be specified

◆ **Example**

```
NOP:XS<XB    XC<XK
      :XS<XS+1
```

In the following example of "cannot be specified," it is limited since the register simultaneous specifying format of XCSK is specified, however, only the value of XC register is updated in the next step. In the following example of "can be specified," it is not limited since the same operation results are stored by the XCSK specification after the updating the value of XC register.

- Cannot be specified

- ◆ **Example**

```
NOP:XC<XC+1
      :XC<XC+1
```

- Can be specified

- ◆ **Example**

```
NOP:XC<XC+1
      :XC<XC+1
```

Restriction on coexistence of different kinds of operation instructions

F=A

Mixture of the addition and subtraction operation instruction (example: $F < A \pm 1$), the logical operation instruction (example: $F < A \& B$), the shift operation instruction ($F < A * 2$), and the inverse operation instruction ($F < /A$) cannot be specified in successive two or four steps.

- Cannot be specified

- ◆ **Example**

```
NOP:XC<XC+1
      :XC<XC*2
```

- Can be specified

- ◆ **Example**

```
NOP:XC<XC+1
      :XC<D1
NOP:XC<XC*2
      :
```

F≠A

If $F \neq A$, it is the LOAD instruction. Therefore, different kinds of operation instructions can be specified. However, if different registers are specified in the successive two or four steps, the restriction in ["Operating restriction among registers." on page 6-29](#) is applied.

- Cannot be specified

◆ Example

```
NOP:XC<XB+1
      :XS<XB*2
```

- Can be specified

◆ Example

```
NOP:XC<XB+1
      :XC<XB*2
```

Restriction on the number of specifications (one)

F=A

In the successive two or four steps, the logical operation instruction (example: F<A&B), and the inverse operation instruction (F</A) can be specified once.

- Cannot be specified

◆ Example

```
NOP:XC</XC
      :XC</XC
```

- Can be specified

◆ Example

```
NOP:XC</XC
      :XC<XB
NOP:XC</XC
      :
```

F≠A

The number of specifications is not limited on the inverse operation instruction. However, if different registers are specified in the successive two or four steps, the restriction on ["Operating restriction among registers." on page 6-29](#) is applied (The number of specifications is limited on the logical operation instruction).

- Cannot be specified

◆ Example

```
NOP:XC</XS
      :XS</XB
```

- Can be specified

◆ **Example**

```
NOP:XC</XS
      :XC</XB
```

Restriction on use of XB and YB register

Only addition and subtraction instruction can be used in the XC and YC register, if XB and YB registers are used to XC and YC registers' operation in the successive two or four steps and the addition and subtraction instruction (example: $F<A\pm 1$) is used in the XB and YB registers. The logical operation instruction (example: $F<A\&B$), the shift operation instruction ($F<A*2$), and the inverse operation instruction ($F</A$) cannot be used. (The LOAD instruction of XB and YB can be used.)

- Cannot be specified

◆ **Example**

```
NOP:XB<XB+1
      :XB<XB+1 XC<XB*2
```

- Can be specified

◆ **Example 1:**

```
NOP:XB<XB+1
      :XB<XB+1 XC<XB
```

◆ **Example 2:**

```
NOP:XB<XB+1
      :XB<XB+1 XC<XB+1
NOP:XB<XB+1 XC<XB+1
      :
```

Restriction on use of D1-4 registers

The following restrictions are imposed to the case where D1-4 registers are used to the XC and YC registers' operation in the successive two or four steps.

F=A (Restriction 1)

Only one register can be specified from registers D1-4.

- Cannot be specified

◆ **Example**

```
NOP:XC<XC+D1
      :XC<XC+D3
```

- Can be specified

◆ **Example 1:**

```
NOP:XC<XC+D3
      :XC<XC+D3
```

◆ **Example 2:**

```
NOP:XC<XB+D3
      :XC<XC+D3
```

F=A (Restriction 2)

If values of registers D1-4 are changed and the old values and the new values of D1-4 registers are used in the XC and YC registers' operation in the successive two or four steps, values of D1-4 registers cannot be changed.

- Cannot be specified

◆ **Example 1:**

```
NOP:XC<XC+D3
      :XC<XC+D3 D3<D3B
      :XC<XC+D3
      :XC<XC+D3
```

◆ **Example 2:**

```
NOP:XC<XC+D3
      :          D3<D3B
      :XC<XC+D3
      :XC<XC+D3
```

- Can be specified

◆ **Example 1:**

```

NOP:          D3<D3B
      :XC<XC+D3
      :XC<XC+D3
      :XC<XC+D3

```

◆ **Example 2:**

```

NOP:XC<XC+D3
      :XC<XC+D3
      :XC<XC+D3
      :XC<XC+D3 D3<D3B

```

Specification is possible only in the following case as an exception. (Subtraction is possible.)

◆ **Example 1:**

```

NOP:XC<XC+D3 D3<D3+1
      :XC<XC+D3

```

◆ **Example 2:**

```

NOP:XC<XC+D3
      :XC<XC+D3 D3<D3+1

```

◆ **Example 3:**

```

NOP:XC<XC+D3 D3<D3+1
      :XC<XC+D3 D3<D3+1

```

$F \neq A$

No restriction is imposed to the case of $F \neq A$.

Restriction on use of D4 register

In the successive two or four steps, the result of the right-shift operation instruction of the D4 register ($D4 \leftarrow D4/2$) cannot be used to the XC and YC registers' operation.

- Cannot be specified

◆ **Example**

```

NOP:          D4<D4/2
      :XC<XC+D4

```

- Can be specified

- ◆ **Example**

```
NOP:XC<XC+D4 D4<D4/2
      :          D4<D4/2
```

Restriction on specifying the D1 and D2 registers

Restrictions are imposed when specifying the D1A to D1H and D2A to D2D registers to be used for operations which use the XC and YC registers.

- For more information, refer to [6. 5. 6 "D1 to D4 Register Operating Instruction Restriction" on page 6-26](#).

Restriction on specifying Carry (BX.CY)-attached operating instructions

There is a restriction on specifying Carry (BX.CY)-attached operating instructions.

- For more information, refer to [6. 5. 9 "Carry \(BX, CY\)-attached Operating Instruction Restriction" on page 6-35](#).

6. 5. 9 Carry (BX, CY)-attached Operating Instruction Restriction

The following restrictions are imposed to the case where the Carry (BX, CY)-attached operation instruction is used in the YB, YC and Z registers.

Restriction on Carry generating condition

Carry (BX, CY) generation must be less than once in the successive two or four steps. Therefore, it is necessary to satisfy the following conditions.

- The settings for LMAX and HMAX must be #3 or greater in the 2WAY mode, or #7 or greater in the 4WAY mode.
- It is necessary to satisfy $D_n \leq LMAX/2$ in the 2WAY mode or $D_n \leq LMAX/4$ in the 4WAY mode, when Carry is generated in the operation using D_n ($n = 1$ to 4).

Restriction on pair specification

In using Carry-attached operating instructions, it must be specified in pairs with the operation instruction which generates the Carry.

However, if the specification is made in the order of the following example (Can be specified), this restriction is not applied. If so, the OS specially handles the $XC<XB\pm 1$ $YC<YB\pm 1\wedge BX$ instruction written in the step after the $XB<XB\pm 1$ $YB<YB\pm 1\wedge BX$ instruction. These two or four steps are not necessary to continue. If other Carry-attached operation instruction is written in the two steps, the OS does not perform the special handling.

- Cannot be specified

◆ **Example**

```
NOP:XC<XB+1 YC<YB+1^BX
:
```

- Can be specified

◆ **Example**

```
NOP:XB<XB+1 YB<YB+1^BX
:XC<XB+1 YC<YB+1^BX
```

The operation instruction that generates Carry is shown below. When the Carry-attached operation instruction is used in the Z register, the Carry-attached addition and subtraction instruction of YB registers or of YC registers should be written in the same step.

- Cannot be specified

◆ **Example**

```
NOP:XB<XB+1 YB<YB+1 Z<Z+1^BX
:
```

- Can be specified

◆ **Example**

```
NOP:XB<XB+1 YB<YB+1^BX Z<Z+1^BX
:
```

BX Carry

Addition and subtraction instruction of the XB register (Addition and subtraction instruction of the YB register)

CY Carry

Addition and subtraction instruction of the XC register (Addition and subtraction instruction of the YC register)

Restriction on pair specification

In using an operation instruction with Carry in the successive two or four steps, the operation instruction generating the Carry cannot be specified alone without specification of the operation instruction with Carry.

- Cannot be specified

◆ **Example 1:**

```
NOP:XB<XB+1
      :XB<XB+1 YB<YB+1^BX
```

◆ **Example 2:**

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XB+1
```

◆ **Example 3:**

```
NOP:XB<XB+1 YB<YH
      :XB<XB+1 YB<YB+1^BX
```

• Can be specified

◆ **Example 1:**

```
NOP:XB<0      YB<0
      :XB<XB+1 YB<YB+1^BX
```

◆ **Example 2:**

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XB+1 YB<YB+1^BX
```

◆ **Example 3:**

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XH   YB<YH
```

In other words, the operation instruction to generate Carry cannot be written one step before or after this step in the 2WAY mode (or three steps before or after this step in the 4WAY mode) as the example shown below.

```
NOP: (XB<XB)
      : XB<XB+1 YB<YB+1^BX
NOP: (XB<XB)
      :
```

When an operation instruction with Carry is specified in the first step of the successive two or four steps and then the register, which contains the result just obtained, is specified in the second step or second to fourth steps for the CLEAR and DSET instruction, or other instructions to load the register, an instruction to generate Carry can be specified in the second step or second to fourth steps.

- Cannot be specified

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XB+1 YB<YB
```

- Can be specified

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XB+1 YB<YH
```

Restriction on coexistence of operator codes

When an operation instruction with Carry is used in the successive two or four steps, the operators of the operation instruction with Carry and the instruction to generate Carry should be unified. However, this restriction applies only when the results of the Carry-attached instruction and the register to generate Carry are passed to the subsequent steps.

In the following example ("Cannot be specified"), both "+" and "-" operators are specified together. Therefore, this specification violates the restriction.

- Cannot be specified

◆ Example 1:

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XB+1 YB<YB-1^BX
```

◆ Example 2:

```
NOP:XC<XC+1 YC<YC+1+CY
      :XC<XC+1 YC<YC-1-CY
```

- Can be specified

◆ Example 1:

```
NOP:XB<XB+1 YB<YB+1^BX
      :XB<XB+1 YB<YB+1^BX
```

◆ **Example 2:**

```
NOP:XC<XC+1  YC<YC+1+CY
      :XC<XC+1  YC<YC+1+CY
```

◆ **Example 3:**

```
NOP:XB<XB+1  YB<YB+1^BX
      :XC<XC+1  XC<XC-1^CY
```

Restriction on coexistence of operator codes (Restriction on use of D1-4 registers)

The $F<A\pm 1^C$ instruction and the $F<A\pm B^C$ instruction cannot be specified together in the successive two or four steps.

The $F<A\pm 1\pm C$ instruction and the $F<A\pm B\pm C$ instruction can be used together in the successive 2 steps.

- Cannot be specified

◆ **Example**

```
NOP:XC<XC+1  YC<YC+1^CY
      :XC<XC+1  YC<YC+D3^CY
```

- Can be specified

◆ **Example**

```
NOP:XC<XC+1  YC<YC+1+CY
      :XC<XC+1  YC<YC+D3+CY
```

Restriction 1 on passing the results of the Carry-attached operation instruction

In the successive two or four steps, when the register F where the results of the Carry-attached operation instruction is stored is passed to the next and after operation instructions, the operator code should be the same between the Carry-attached operation instruction and the step where the register F is passed.

In the following example (that cannot be specified), the operator code of the Carry-attached operation instruction is "+" and the operator code of the step where the register F is passed is "-." Therefore, the specification is impossible.

- Cannot be specified

◆ Example

```
NOP:XB<XB+1 YB<YB+1^BX
      :          YC<YB-D3
```

- Can be specified

◆ Example

```
NOP:XB<XB+1 YB<YB+1^BX
      :          YC<YB+D3
```

To avoid this restriction, generate the disturbing cell address of the Butterfly Pattern and so on from the "YB+" side as in the above example (Can be specified).

Restriction 2 on passing the results of the Carry-attached operation instruction

In the successive two or four steps, when the results of the Carry-attached operation instruction are passed to the operation instruction with different Carry conditions in the subsequent steps, the register that generates Carry must be passed at the same time.

- Cannot be specified

◆ Example

```
NOP:XB<XB+1 YB<YB+1^BX
      :XC<XC+1 YC<YB+1^CY
```

- Can be specified

◆ Example

```
NOP:XB<XB+1 YB<YB+1^BX
      :XC<XB+1 YC<YB+1^CY
```

Restriction 1 on the YC[S][K] Carry-attached operation instruction

When the YB or D1-4 register is specified to the Carry-attached operation instruction of the YC[S][K] register and the YB or D1-4 register value is changed, the instruction to change the value of the YB or D1-4 register cannot be specified if the old and new values of the YB or D1-4 register are used in the successive two or four steps.

- Cannot be specified

◆ **Example 1:**

```

NOP:XC<XC+1 YC<YC+D3^CY D3<D3+1
      :XC<XC+1 YC<YC+D3^CY

```

◆ **Example 2:**

```

NOP:XB<XB+1 YB<YB+1 YC<YB+1^BX
      :XB<XB+1 YB<YB+1 YC<YB+1^BX

```

- Can be specified

◆ **Example 1:**

```

NOP:XC<XC+1 YC<YC+D3^CY
      :XC<XC+1 YC<YC+D3^CY D3<D3+1

```

◆ **Example 2:**

```

NOP :                               D3<D3+1
      :XC<XC+1 YC<YC+D3^CY

```

Restriction 2 on the YC[S][K] Carry-attached operation instruction

When the YC[S][K] Carry-attached operation instruction is successively written in the successive two or four steps, the YB register cannot be used for the YC[S][K] register operation.

- Cannot be specified

◆ **Example**

```

NOP:XC<XC+1 YC<YB+1^CY
      :XC<XC+1 YC<YB+1^CY

```

- Can be specified

◆ **Example**

```

NOP:XC<XC+1 YC<YC+1^CY
      :XC<XC+1 YC<YC+1^CY

```


Restriction on the YB Carry-attached operation instruction

When the D1 or D2 register is specified to the Carry-attached operation instruction of the YB register and the D1 or D2 register value is changed, the instruction to change the value of the D1 or D2 register cannot be specified if the old and new values of the D1 or D2 register are used in the successive two or four steps.

- Cannot be specified

- ◆ **Example**

```
NOP:XB<XB+1 YB<YB+D1^BX D1<#2
      :XB<XB+1 YB<YB+D1^BX
```

- Can be specified

- ◆ **Example 1:**

```
NOP:XB<XB+1 YB<YB+D1^BX
      :XB<XB+1 YB<YB+D1^BX D1<#2
```

- ◆ **Example 2:**

```
NOP : D1<#2
      :XB<XB+1 YB<YB+D1^BX
```

Restriction on the Z Carry-attached operation instruction

When the ZD register is specified to the Carry-attached operation instruction of the Z register and the ZD register value is changed, the instruction to change the value of the ZD register cannot be specified if the old and new values of the ZD register are used in the successive two or four steps.

- Cannot be specified

- ◆ **Example**

```
NOP:XB<XB+1 YB<YB+1^BX Z<Z+ZD^BX ZD<#2
      :XB<XB+1 YB<YB+1^BX Z<Z+ZD^BX
```

- Can be specified

- ◆ **Example 1:**

```
NOP:XB<XB+1 YB<YB+1^BX Z<Z+ZD^BX
      :XB<XB+1 YB<YB+1^BX Z<Z+ZD^BX ZD<#2
```

◆ **Example 2:**

```
NOP:                                ZD<#2
      :XB<XB+1 YB<YB+1^BX Z<Z+ZD^BX
```

6. 5.10 TP and TP2 Register Operating Instruction Restriction

Restrictions of TP and TP2 are the same.

(In the following restrictions, when TP is assigned to TP2, TPH is assigned to TPH2, TPS is assigned to TPS2, and D5 is assigned to D6, they are applied to the TP2 register.)

Restrictions on the coexistence of operator codes

Mixture of the addition and subtraction operation instruction (example: TP<TP+1), the shift operation instruction (example: TP<TP*2), the inverse operation instruction (example: TP</TP), and the logical operating instruction (example: TP<TP&D5) cannot be specified in successive two or four steps.

- Cannot be specified

◆ **Example**

```
NOP:TP<TP+1
      :TP<TP*2
```

- Can be specified

◆ **Example**

```
NOP:TP<TP+1
      :TP<TP
NOP:TP<TP*2
      :
```

Restriction on the number of specifications (one)

The inverse operation instruction (example: TP</TP) and the logical operation instruction (example: TP<TP&D5) can be specified only once in successive two or four steps.

- Cannot be specified

◆ **Example**

```
NOP:TP</TP
      :TP < / TP
```

- Can be specified

◆ **Example**

```
NOP:TP</TP
      :TP<TP
NOP:TP</TP
      :
```

Restriction on the number of specifications (one) of the TPS register

The operating instructions (TP<>TPS, TP<TPS, TPS<TP) which use the TPS register can be specified only once in successive two or four steps.

Restriction 1 when using the TPS register

When register operations are continuously specified in 2 lines of 2WAY interleaved mode or 4 lines of 4WAY interleaved mode, TP register statements except HOLD (TP<TP) cannot be specified before TPS<TP statement.

- Cannot be specified

◆ **Example**

```
NOP:TP<TP+1
      :TPS < TP
```

- Can be specified

◆ **Example**

```
NOP:TP<TP+1
      :TP < TPS
```

Restriction 2 when using the TPS register

When register operations are specified in continuous 4 lines of 4WAY interleaved mode, TP register statements except HOLD (TP<TP) can be specified once before or after TP<>TPS statement.

- Cannot be specified

◆ **Example**

```
NOP:TP<TP+1
      :TP <> TPS
      :TP<TP+1
      :
```

- Can be specified

◆ **Example**

```
NOP:TP<TP+1
      :TP <> TPS
      :
      :
```

D5 register specifying restriction 1

The operating instruction of D5 and TP registers cannot be specified in the same step as DSET instruction to D5 registers.

In successive two or four steps, DSET instruction to D5 registers cannot be specified after operating instructions of D5 and TP (include LOAD instruction).

- Cannot be specified

◆ **Example 1:**

```
NOP:TP<D5 D5<#FF
      :
```

◆ **Example 2:**

```
NOP:TP<TP+D5
      :D5<#F
```

- Can be specified

◆ **Example 1:**

```
NOP:TP<TP D5<#FF
      :TP<D5
```

◆ **Example 2:**

```
NOP:D5<#F
      :TP<TP+D5
```

D5 register specifying restriction 2

In successive two or four steps, addition and subtraction operating instructions and logical operating instruction of D5 and TP registers cannot be specified after the operating instructions of D5 and TP register (include LOAD instruction).

- Cannot be specified

◆ Example

```
NOP:TP<D5
      :TP<TP+D5
```

In successive two or four steps, a TP<TP*2 instruction cannot be specified after TP<D5 and TP</D5 instructions.

- Cannot be specified

◆ Example

```
NOP:TP<D5
      :TP<TP*2
```

- Can be specified

◆ Example

```
NOP:TP<TPH
      :TP<TP*2
```

6. 5.11 DSD, RCD, and CCD Register Operating Instruction Restriction

Specifying restriction on operating instruction and DSET instruction/CLEAR instruction

In successive two or four steps, DSD, RCD and CCD register operating instructions cannot be described after DSET instruction of DSD, RCD and CCD registers.

- Cannot be specified

◆ Example

```
NOP:CCD<#1
      :RCD<RCD+1
```

- Can be specified

◆ Example

```
NOP:
      :RCD<RCD+1
NOP:CCD<#1
      :
```

The DSD, RCD or CCD register operating instructions or CLEAR instruction cannot be set in address or data generation instructions (field B and field C) for the next sequence control instruction (field A) when the DSET instruction is set for the DSD, RCD or CCD register.

(When a program is branched with an instruction (such as JN1), these instructions cannot be set in a branch destination and the next line.)

- Cannot be specified

- ◆ **Normal Format**

```
NOP DSD<#F
NOP DSD<DSD+1
```

- ◆ **Separate Format**

```
NOP:DSD<#F
:
NOP:
:
```

- Can be specified

- ◆ **Normal Format**

```
NOP DSD<#F
NOP
NOP DSD<DSD+1
```

- ◆ **Separate Format**

```
NOP:
: DSD<#F
NOP:
:
NOP:
: DSD<DSD+1
```

Instruction restrictions when using the separate format

When using the separate format, set the DSD, RCD or CCD register operating instructions (DSD<DSD+1, RCD<RCD+1 or CCD<CCD+1) or the CLEAR instruction (DSD<0, RCD<0 or CCD<0) in the second step when in 2WAY interleaved mode, or in the fourth step when in 4WAY interleaved mode.

In the normal specification, DSD, RCD, and CCD register operating instructions or CLEAR instructions are processed as instructions specified in the second step of the separate specification when in 2WAY mode, or the fourth step of the separate specification when in 4WAY mode.

6. 5.12 RF Register Operating Instruction Restriction

Instruction restrictions when using the separate format

When editing the program by the separate instruction, the RF<RF+1 operation instruction and X<RF instruction must be set at the first of two steps.

- Cannot be specified

- ◆ **Separate Format**

```
NOP:RF<RF+1 X<RF
      :RF<RF+1 X<RF
```

- Can be specified

- ◆ **Separate Format**

```
NOP:RF<RF+1 X<RF
      :
```

- ◆ **Normal Format**

```
NOP RF<RF+1 X<RF
```

The separate instruction is equivalent to the normal specification.

Restriction on pair specification

In the interleave mode, the RF<RF+1 operation instructions must be written in pairs with the X<RF instruction.

(Specification of the X<RF instruction only is permitted.)

- Cannot be specified

- ◆ **Example**

```
NOP:RF<RF+1
      :
```

- Can be specified

- ◆ **Example**

```
NOP:X<RF
      :
```

Tip

When specifying $RF < RF+1$ $X < RF$, a dummy pattern is entered in the second step or the second to fourth steps due to the instruction restriction on the separate specification.

To remove the dummy pattern, there is a method of treating two or four patterns as one cycle by using the function that clock signals can be set up to the second or fourth pattern cycle RATE. The following describes the examples for 2WAY.

◆ Example of Specification in a Pattern Program

```
ISP01: IDX11 #3FD : RF<RF+1 X<RF C0 TS1
                : TS2 ] *1
RTN      : RF<RF+1 X<RF C0 TS1
                : TS2
```

◆ Example of Specification in a Test Plan Program

```
PD1-10 =IN1,XOR,ACLK1,BCLK1,CCLK1,PSM <X0-9,Y0-9>;A0-9
PD11   =IN1,/RZO,      BCLK2,CCLK2,      <C0>      ;/RAS
PD12   =IN1,/NRZA,ACLK2,      <C1>      ;/CAS
RATE   =65NS,65NS *2 RATE of the refresh cycle is 130 NS
ACLK1=0NS,OPEN *3*4
ACLK2=5NS,OPEN
BCLK1=10NS,OPEN
BCLK2=15NS,OPEN
CCLK1=50NS,OPEN
CCLK2=90NS,OPEN
```

*1 Specify a different timing set for the first and second steps of the pattern program.

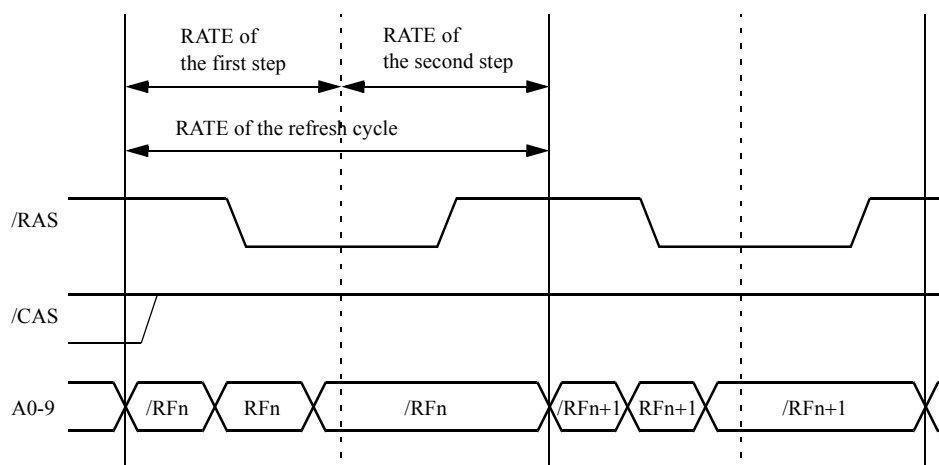
*2 Specify a RATE in such a way that the sum of the RATE of the first step and that of the second step is the RATE of the refresh cycle.

*3 Set ACLKn, BCLKn, and CCLKn for waveform shaping of the refresh cycle to the timing set to be specified for the first step.

The timing clocks must be less than $\{(First\ step\ test\ time) + (Second\ step\ test\ time) - 15.625\ ps\}$.

*4 Set OPEN to ACLKn, BCLKn, and CCLKn of the timing set to be specified for the second step.

◆ **Example of Operation**



Note that this method is used only when the RATE of the refresh cycle is 8 ns to 1 ms.

6. 5.13 DSC Register Operating Instruction Restriction

Operating instruction specifying restriction

DSC register operating instruction can be specified only once in successive two or four steps.

- Cannot be specified

◆ **Example 1:**

```
NOP:DSC<DSC*2
:DSC<DSC*2
```

◆ **Example 2:**

```
NOP:DSC<DSC*2
:DSC<DSC/2
```

- Can be specified

◆ **Example 1:**

```
NOP:DSC<#1
:DSC<#F
```

◆ **Example 2:**

```
NOP:DSC<#F
:DSC<#F
```

Specifying restriction on operating instruction and DSET instruction/CLEAR instruction

In successive two or four steps, DSC register operational instruction cannot be described after DSET instruction/CLEAR instruction of DSC register.

- Cannot be specified

◆ **Example 1:**

```
NOP:DSC<#1
:DSC<DSC*2
```

◆ **Example 2:**

```
NOP:DSC<#F
:DSC<DSC/2
```

- Can be specified

◆ **Example 1:**

```
NOP:DSC<DSC*2
:DSC<#1
```

◆ **Example 2:**

```
NOP:DSC<DSC/2
:DSC<#F
```

6. 5.14 CBM Register Operating Instruction Restriction

CBMA register operating instruction (CBMA<CBMA+1) cannot be specified after DSET instruction to CBMA register in successive two or four steps.

- Cannot be specified

◆ Example

```
NOP:CBMA<#10
      :CBMA<CBMA+1
```

- Can be specified

◆ Example

```
NOP:CBMA<CBMA+1
      :CBMA<#10
```

When MODE ECBM is specified, the specification of CBMA register operating instruction (CBMA<CBMA+1) is only once in successive two or four steps. In the step using CBM output, it is necessary to perform CBMA change once in two or four steps as shown below.

Also, the CBMA register operating instruction must be written in the second or fourth step of the separate specification.

- When MODE ECBM is specified

◆ Example

```
NOP :
      : CBMA<#10           ; CBMA=x
NOP :           D<CBM ; CBMA=#10,Data Adr.=20
      : CBMA<CBMA+1 D<CBM ; CBMA=#10,Data Adr.=21
NOP :           D<CBM ; CBMA=#11,Data Adr.=22
      : CBMA<CBMA+1 D<CBM ; CBMA=#11,Data Adr.=23
```

6. 5.15 SWAP Instruction (LMAX<>HMAX) Describing Restriction

Restriction on the number of specifications (one)

SWAP instruction can be specified only once in successive two or four steps.

- Cannot be specified

◆ **Example**

```
NOP: LMAX<>HMAX
      : LMAX<>HMAX
```

SWAP instruction specifying restriction 1

In successive two or four steps, a SWAP instruction cannot be specified after a D3<D3*2L D4<D4*2L instruction.

- Cannot be specified

◆ **Example**

```
NOP: D3<D3*2L D4<D4*2L
      : LMAX<>HMAX
```

- Can be specified

◆ **Example 1:**

```
NOP: LMAX<>HMAX
      : D3<D3*2L D4<D4*2L
```

◆ **Example 2:**

```
NOP: D3<D3*2L D4<D4*2L
      :
NOP: LMAX<>HMAX
      :
```

SWAP instruction specifying restriction 2

In successive two or four steps, SWAP instruction cannot be specified after a Carry (BX, CY)-attached operating instruction.

- Cannot be specified

◆ **Example**

```
NOP: XB<XB+1 YB<YB+1^BX
      : LMAX<>HMAX
```

- Can be specified

◆ **Example 1:**

```

NOP: LMAX<>HMAX
      :XB<XB+1 YB<YB+1^BX

```

◆ **Example 2:**

```

NOP:XB<XB+1 YB<YB+1^BX
      :
NOP: LMAX<>HMAX
      :

```

6. 5.16 Restriction on the RTN Instruction

In the first to second step or first to fourth step of the RTN instruction, only the RF<RF+1 register operation instruction can be specified.

(All sorts of control bits of event time 1 such as MUT, TS, etc. can be specified.)

- Cannot be specified

◆ **Example 1:**

```

RTN:XB<XB+1
      :

```

◆ **Example 2:**

```

RTN:
      :XB<XB+1

```

- Can be specified

◆ **Example**

```

RTN:RF<RF+1 X<RF
      :

```

If switch "-r" is specified for the compilation command, register operation instructions other than RF<RF+1 can be specified in the RTN instruction line.

trans -r source (Specify the source file name for source.)


Tip

If switch "-r" is specified for the compilation command, the JMP, JNCn and JET instructions cannot be specified in subroutines.

➔ For information about the syntax of the compilation command, refer to [XATL/U-51 Cross Compiler User's Manual\(No. : 8256237\)](#).

6. 5.17 Restriction on Refresh Test

Registers which are used for the refresh routine (from ISP to RTN) will not operate correctly during the refresh test (MODE REFRESHA/MODE REFRESHB) which is used with the timer when an operating instruction is executed by a routine other than the refresh routine.

The XC register does not operate with the pattern programmed below.

```

MODE REFRESHA
START #0
NOP T      :XC<0      XC<XC
            :XC<XC+1  XC<XC      [Operation instruction using the XC register executed in other than the refresh routine]
IDXI1 #FD  :X<XC
            :XC<XC+1  XC<XC

STPS

ISP:NOP     :XC<XC+1  XC<XC
            :XC<XC+1  XC<XC
RTN         :        X<XC
            :        X<XC      [Refresh routine]

```

6. 5.18 Restrictions on BWPE control bit

The following restrictions are applied when BDW, BCW, or BMW is specified for the 2WAY or 4WAY interleave mode:

Detailed description of restrictions

- For separate specification (a colon ":" is used as the field separator), restrictions (1) and (2) described below are applied.
- For normal specification (a colon is not used as the field separator), only restriction (2) described below is applied.
- Specify BDW, BCW, or BMW for every line with 2 continuous lines of 2WAY interleaved mode or 4 continuous lines of 4WAY interleaved mode.

2. The address values and data pattern of BDW, BCW, or BMW are saved in each REG-FILE.

BDW	Data	TP1
	Address	Selected address by BWPE ADDRESS statement. (Address applied to BD REG-FILE address bits 0 and 1)
BCW	Data	TP1
	Address	Selected address by BWPE ADDRESS statement. (Address applied to CD REG-FILE address bits 0 and 1)
BMW	Data	TP2
	Address	Selected address by BWPE ADDRESS statement. (Address applied to MD REG-FILE address bits 0 and 1)

These address data and data patterns cannot be changed in continuous two or four steps.
For normal specification, the address data and data patterns cannot be specified in a line where BDW, BCW or BMW are specified.

Restriction on compilation

BDW, BCW or BMW cannot be specified in separate specification.

When BDW, BCW or BMW is specified, pattern data generation and address generation statements cannot be specified in the same line.

- Cannot be specified

◆ Example 1:

```
NOP:BDW
:
```

◆ Example 2:

```
NOP BDW TP<#FFFFFFFF
```

- Can be specified

◆ Example

```
NOP BDW
```

The restrictions described in ["Detailed description of restrictions" on page 6-55](#) above apply when specifying BDW, BCW, or BMW. The restrictions described in ["Restriction on compilation" on page 6-56](#) are applied to check for programming errors when a compiler is used.

If your programming violates the restriction described in ["Restriction on compilation" on page 6-56](#) but not those in ["Detailed description of restrictions" on page 6-55](#), cancel the compilation error process for BDW, BCW, or BMW using the following compilation command before starting the compilation.

In this case, even if a pattern program can be compiled without compilation error, it will not be executed correctly if the pattern program violates any restriction described in ["Detailed description of restrictions" on page 6-55](#). Your thorough understanding of the restrictions is required before programming.

trans -t source (Specify the source file name for source.)

➔ For information about the compilation command, refer to [XATL/U-51 Cross Compiler User's Manual\(No. : 8256237\)](#).

6. 5.19 Restrictions when Specifying the XASV Instruction

When coding an XASV instruction, no instruction can be specified to update the X address in two or four successive steps. The following is a description of restrictions applied when specifying an instruction to update the X address.

Specification of the Source address field for the X side (X<*) cannot be changed in two or four successive steps.

- Cannot be specified

◆ Example

```
NOP : XASV  X<XC
      :      X<XS  ←Cannot be specified because the X address
                    is different between XC and XS
```

```
NOP : XASV  X<XC
      :      :      ←Cannot be specified because X<XB is
                    specified by default and the X address is
                    different between XC and XB
```

- Can be specified

◆ Example

```
NOP : XASV  X<XC
      :      X<XC  ←Can be specified because XC is not updated
```

```
NOP : XASV  X<XB
      :      :      ←Can be specified because X<XB is specified
                    by default.
```


The register used on the right side of the X-side Source address field (X<*) in the second step or second to fourth steps cannot be updated by an operation instruction in the first step or first to third step.

- Cannot be specified

◆ Example

```

NOP : XASV  X<XC  XC<XC+1
      :      X<XC  ←Cannot be specified because XC is
                   updated

NOP : XASV      XB<XB+1
      :          ←Cannot be specified because X<XB is
                   specified by default and XB is updated.

NOP : XASV  X<XB'Z  Z<Z+1
      :      X<XB'Z  ←Cannot be specified because Z is updated

```

- Can be specified

◆ Example

```

NOP : XASV  X<XC  XB<XB+1C
      :      X<XC  ←Can be specified because XC is not updated

NOP : XASV  X<XC
      :      X<XC  XC<XC+1  ←Can be specified because XC is
                           not updated

NOP : XASV
      :          XB<XB+1  ←Can be specified because XB is
                           not updated

```

No operation that uses the Z, N, or B register in the first step or first to third step can be specified and then let the B register interrupt the X address by an ADDRESS DEFINE statement.

- Cannot be specified

◆ Example

```

ADDRESS DEFINE
  X0-1=N0-1

NOP : XASV  N<N+1  ←Cannot be specified because N that
                   will interrupt X is updated
      :

```

- Can be specified

- ◆ Example

```
ADDRESS DEFINE
```

```
  X0-1=N0-1
```

```
  Y0-1=B0-1
```

```
NOP : XASV  B<B+1    ←Can be specified because B does
                        not interrupt X
```

The DSET instruction using the XOS and XT registers cannot be specified in the first step or first to third step.

- Cannot be specified

- ◆ Example

```
NOP : XASV    XOS<#1
      :                                     ←Cannot be specified because XOS
                                           is updated
```

```
NOP : XASV    XT<#1 XT
      :         XT<#2 XT    ←Cannot be specified because XT
                                           is updated
```

- Can be specified

- ◆ Example

```
NOP : XASV
      :         XOS<#1    ←Can be specified because XOS is
                           updated in the next step
```

```
NOP : XASV    XT
      :         XT<#1 XT    ←Can be specified because XT is
                           updated in the next step
```

Specification of /X, XT, SCROFF, or PSCROFF cannot be changed in successive two or four steps.

- Cannot be specified

◆ Example

```
NOP : XASV XT    ←Cannot be specified because XT is
                  output temporarily
```

```
NOP : XASV
      :        /X  ←Cannot be specified because X is
                  inverted temporarily
```

• Can be specified

◆ Example

```
NOP : XASV XT
      :        XT    ←Can be specified because XT is output in
                  both steps
```

```
NOP : XASV /X
      :        /X    ←Can be specified because X is
                  inverted temporarily in both steps
```

6. 6 Conversion from Normal Mode to Interleave Mode

The interleave mode can be specified by adding <IL-MODE> specification to the pattern program created in the normal mode. However, it is not enough to use the program as a pattern program for the interleave mode. The pattern program should be rewritten paying attention to the following points:

- Review the instruction with restriction on the number of specifications (one) in successive two or four steps, and other restrictions in each instruction.
 - Review the field A instruction (especially the index counter value) because the relationship between the field A instruction and field B and field C instructions changes from a 1 to 1 correspondence to a 1 to 2 or 1 to 4 correspondence.
- ➔ For the normal specification format, refer to [6. 3 "Programming Format for Interleave Mode" on page 6-8](#). For the restrictions on each instruction, refer to [6. 5 "Restrictions on Each Instruction in Interleave Mode" on page 6-16](#).

6. 7 Conversion from PCC 2 Mode to 2WAY Interleave Mode

The pattern program created in PCC 2 mode can be basically converted to 2WAY mode (interleave mode) program by adding <IL-MODE> specification and by changing the field separating character from a percent sign (%) to a colon (:). However, it is necessary to confirm the restrictions described in [6. 5 "Restrictions on Each Instruction in Interleave Mode" on page 6-16](#) because any of them might be applied.

The capacity of instruction memory is decreased to a half in PCC 2 mode, but the capacity in interleave mode is not changed. Therefore, 1024 words can be specified.

6. 8 Maximum Pattern Generating Frequency in Each Mode, Normal, PCC, or Interleave

The maximum pattern generating frequency in each mode, normal, PCC, or interleave, is as shown below. It is necessary to select a mode paying attention to the change of DUT speed because the compatibility of pattern programs does not exist among the normal, PCC, and interleave modes. (The capacity of instruction memory only in PCC mode is decreased to a half.)

Mode	Test system		
	T5593	T5592	T5591, T5581, and T5571P
Normal mode	266 MHz	133 MHz	125 MHz
PCC mode	Cannot be specified	133 MHz	125 MHz
2WAY interleave mode	533 MHz	266 MHz	250 MHz
4WAY interleave mode	Cannot be specified	533 MHz	500 MHz

6. 9 Coexistence Among Normal, PCC and Interleaved Modes

The PCC mode and interleave mode cannot coexist in a pattern program.

The following combination of modes is possible.

- Normal mode and PCC mode (The PCC mode cannot be specified on T5593.)
- Normal mode and interleave mode

Appendix 1. List of Pattern Program Instructions

This appendix describes how to write instructions used in pattern programs. It also describes the limitations of those instructions.

MODULE BEGIN			
ILMODE IL-mode			
MODE a1,a2,....	...<MAIN, SUB>	Operation mode specification	
MAIN MODE a1,a2,...	...<MAIN>		
SUB MODE a1,a2,...	...<SUB>		
RDX n		Specifying a cardinal number	
REGISTER	...<MAIN, SUB>	Initializing the register	
MAIN REGISTER	...<MAIN>		
SUB REGISTER	...<SUB>		
ARIRAM DEFINE	...<MAIN, SUB>	Specifying ARIRAM (area inversion)	
MAIN ARIRAM DEFINE	...<MAIN>		
SUB ARIRAM DEFINE	...<SUB>		
ADDRESS DEFINE	...<MAIN, SUB>	Specifying X, Y, PAGE address	
MAIN ADDRESS DEFINE	...<MAIN>		
SUB ADDRESS DEFINE	...<SUB>		
BURST BLOCK BEGIN (block-name)		Defining burst block	
BURST BLOCK END			
SDEF name=ALPG instruction		Definition statement	
SCRAM file-name	...<MAIN, SUB>	Address descramble file definition state-	
MAIN SCRAM file-name	...<MAIN>	ment	
SUB SCRAM file-name	...<SUB>		
PRESCRAM file-name	...<MAIN, SUB>	Prescramble file definition statement	
MAIN PRESCRAM file-name	...<MAIN>		Module
SUB PRESCRAM file-name	...<SUB>		block
PCC n	...<MAIN, SUB>	PCC mode setting statement	(MAX29)
MAIN PCC n	...<MAIN>		
SUB PCC n	...<SUB>		
DATA DEFINE		PDS data select definition statement	
MAIN DATA DEFINE	...<MAIN>		
SUB DATA DEFINE	...<SUB>		
BWPE ADDRESS a1,a2,...a6	...<MAIN, SUB>	Selection of RF ADDRESS SEL	
MAIN BWPE ADDRESS a1,a2,...a6	...<MAIN>		
SUB BWPE ADDRESS a1,a2,...a6	...<SUB>		
BANK SAVE ADDRESS	...<MAIN, SUB>	The specification of the row bank	
MAIN BANK SAVE ADDRESS	...<MAIN>	addresses which are used by the XASV	
SUB BANK SAVE ADDRESS	...<SUB>	instruction	
BANK LOAD ADDRESS	...<MAIN, SUB>	The specification of the column bank	
MAIN BANK LOAD ADDRESS	...<MAIN>	addresses which are used by the XALD	
SUB BANK LOAD ADDRESS	...<SUB>	instruction	
BYTEMASK DEFINE		Byte mask definition statement	
AFM PATTERN		Starting declaration statement of ROM	
		test pattern	
DBM PATTERN	...<MAIN, SUB>	Starting declaration statement of DBM	
MAIN DBM PATTERN	...<MAIN>	test pattern	
SUB DBM PATTERN	...<SUB>		
CBM PATTERN	...<MAIN, SUB>	Starting declaration statement of CBM	
MAIN CBM PATTERN	...<MAIN>	test pattern	
SUB CBM PATTERN	...<SUB>		
START 1			
Module section pattern program		ALPG pattern program	
MODULE END			
MODULE BEGIN			
MODULE END			
END			

Pattern program format

field A : PC Operation code, Operand, I, T, TM1, TM2, DBMINC Instruction

(Example: JMP n, NOP,)

field B : Main ALPG Instruction (Example: XB<XB+1, X<XB, /X)

field C : Sub ALPG Instruction (Example: XB<XB+1, X<XB, /X)

- In normal mode (1WAY)

```
PCC=0 or Default
  --- A ---    --- B --- ! --- C ---
```

```
PCC=1
  --- A --- % --- B --- ! --- C ---
```

```
PCC=2
  --- A --- % --- B --- ! --- C ---
                % --- B --- ! --- C ---
```

Tip

- When setting the PCC mode, separate field A from field B by a % mark.
- Field C (SUB PG instruction) cannot be specified for the T5592 and T5571P.
- Separate field B from field C by the symbol !.
- field B can be omitted. When omitted, the default value is inserted.
- Field C can be omitted. The defaults are as follows:
When there is no "!", in statements which begin with the START statement, field C is replaced by field B.
When "!" is written in statements which begin with the START statement, the default value is assigned to field C.
- When field C is omitted, "!" can also be omitted.

- In 2WAY interleave mode

```
--- A --- : --- B --- ! --- C ---
          : --- B --- ! --- C ---
```

Tip

- The field C (SUB PG instruction) can be described in the T5591 2WAY interleave mode or the T5593 2WAY interleave mode in which the MSM mode (MODE MSM) is specified.

- Separate field B from field C by the symbol !.
- When field B is omitted, the default value is entered. The default value is an instruction to put all the data on hold.

$$TS1 \ XB<XB \ YB<YB \ XC<XC \ XS<XS \ XK<XK \ YC<YC \ YS<YS \ YK<YK \ D3<D3 \ D4<D4 \ Z<Z$$

$$N<N \ RF<RF$$

$$X<XB \ Y<YB \ TP1<TP1 \ TP2<TP2 \ DSC<DSC \ CBMA<CBMA \ D<TP1 \ B<B$$
- Field C can be omitted. The defaults are as follows:
 When there is no "!", in statements which begin with the START statement, field C is replaced by field B.
 When "!" is written in statements which begin with the START statement, the default value is assigned to field C.
- When field C is omitted, "!" can also be omitted.
- In field B, if 2 steps are the same, the following can be specified, too.

--- A --- --- B --- ! --- C ---

- If a label is attached, write it in the field A line.

- In 4WAY interleave mode

--- A --- : --- B ---
 : --- B ---
 : --- B ---
 : --- B ---

Tip

- The field C (SUB PG instruction) cannot be specified in 4WAY interleave mode.
- When field B is omitted, the default value is entered. The default value is an instruction to put all the data on hold.

$$TS1 \ XB<XB \ YB<YB \ XC<XC \ XS<XS \ XK<XK \ YC<YC \ YS<YS \ YK<YK \ D3<D3 \ D4<D4 \ Z<Z$$

$$N<N \ RF<RF \ X<XB \ Y<YB \ TP1<TP1 \ TP2<TP2 \ DSC<DSC \ CBMA<CBMA \ D<TP1 \ B<B$$
- In field B, if 4 steps are the same, the following can be specified, too.

--- A --- --- B ---

- If a label is attached, write it in the field A line.

MODE (Setting ALPG operation mode)

For T5593

MODE	
MAIN MODE	MUX
	DATEN
	PAUSE
	REFRESHB
	REFRESHA
	PARITY
	TP2INV
	TPM
	ZSET1
	BWPEM
	DFCA
	DFCB
	ECBM
	DBMLP
	CYPC15
	DBFORM
	RDBM
	MSM
	<CR>

SUB MODE	 MUX			
	 DATEN			
	 PAUSE			
	 PARITY			
	 TP2INV			
	 TPM n		 n : 18, 38	
	 ZSET1			
	 BWPEM			
	 DFCA n		 n : 1 to 6	
	 DFCE n			
	 ECBM			
	 DBMFORM n		 n : 1 to 4	
	 <CR>			

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

MODE																																	
MAIN MODE	<table><tr><td>MUX</td><td></td></tr><tr><td>DATEN</td><td></td></tr><tr><td>PAUSE</td><td></td></tr><tr><td>REFRESHB</td><td>n</td></tr><tr><td>REFRESHA</td><td>n : 128,256,512,1024,2048,4096, 8192,16384,32768,65536</td></tr><tr><td>PARITY</td><td></td></tr><tr><td>TP2INV</td><td></td></tr><tr><td>TPM</td><td>n</td></tr><tr><td>ZSET1</td><td>n : 18,38</td></tr><tr><td>BWPEM</td><td></td></tr><tr><td>DFCA</td><td>n</td></tr><tr><td>DFCB</td><td>n</td></tr><tr><td>ECBM</td><td>n : 1 to 4</td></tr><tr><td>DBMLP</td><td></td></tr><tr><td>CYPC15</td><td></td></tr><tr><td><CR></td><td></td></tr></table>	MUX		DATEN		PAUSE		REFRESHB	n	REFRESHA	n : 128,256,512,1024,2048,4096, 8192,16384,32768,65536	PARITY		TP2INV		TPM	n	ZSET1	n : 18,38	BWPEM		DFCA	n	DFCB	n	ECBM	n : 1 to 4	DBMLP		CYPC15		<CR>	
MUX																																	
DATEN																																	
PAUSE																																	
REFRESHB	n																																
REFRESHA	n : 128,256,512,1024,2048,4096, 8192,16384,32768,65536																																
PARITY																																	
TP2INV																																	
TPM	n																																
ZSET1	n : 18,38																																
BWPEM																																	
DFCA	n																																
DFCB	n																																
ECBM	n : 1 to 4																																
DBMLP																																	
CYPC15																																	
<CR>																																	
SUB MODE	<table><tr><td>MUX</td><td></td></tr><tr><td>DATEN</td><td></td></tr><tr><td>PAUSE</td><td></td></tr><tr><td>PARITY</td><td></td></tr><tr><td>TP2INV</td><td></td></tr><tr><td>TPM</td><td>n</td></tr><tr><td>ZSET1</td><td>n : 18,38</td></tr><tr><td>BWPEM</td><td></td></tr><tr><td>DFCA</td><td>n</td></tr><tr><td>DFCB</td><td>n</td></tr><tr><td>ECBM</td><td>n : 1 to 4</td></tr><tr><td><CR></td><td></td></tr></table>	MUX		DATEN		PAUSE		PARITY		TP2INV		TPM	n	ZSET1	n : 18,38	BWPEM		DFCA	n	DFCB	n	ECBM	n : 1 to 4	<CR>									
MUX																																	
DATEN																																	
PAUSE																																	
PARITY																																	
TP2INV																																	
TPM	n																																
ZSET1	n : 18,38																																
BWPEM																																	
DFCA	n																																
DFCB	n																																
ECBM	n : 1 to 4																																
<CR>																																	

RDX n (Setting a cardinal number)

n=2, 8, 10, 16

Default = 16

REGISTER (Initializing the register)

REGISTER

MAIN REGISTER

SUB REGISTER

Table A 1-1 List of Registers (T5593)

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
TIMER	#FFFF	#100F	-----	-----	Time can be specified (100 ns to 409.5 s)
TREFRESH	#FFFF	#100F	-----	-----	Time can be specified (100 ns to 409.5 s)
TPAUSE	#FFFF	#100F	-----	-----	Time can be specified (100 ns to 409.5 s)
PC	#7FF	#0	-----	-----	-----
ISP	#7FF	#0	-----	-----	-----
IDXn	#FFFFFFF	#0	-----	-----	n=1 to 8
IDXm	#FFFFFFF	#0	-----	-----	m=9 to 16
CFLG	#FFFF	#0	-----	-----	-----
CPEn	Signal name ^{*1}	R	Signal name ^{*1}	R	n=1 to 4
DRE(DRE1)	Signal name ^{*1}	W	Signal name ^{*1}	W	Can be described as DRE1
DRE2	Signal name ^{*1}	W	Signal name ^{*1}	W	-----
RINV	Signal name ^{*1}	R	Signal name ^{*1}	R	-----
LMAX	#3FFFF	XMAX	#3FFFF	SUB XMAX	-----
HMAX	#3FFFF	YMAX	#3FFFF	SUB YMAX	-----
XMAX	#3FFFF	#FFF	#3FFFF	#FFF	-----
YMAX	#3FFFF	#FFF	#3FFFF	#FFF	-----
ZMAX	#3FFFF	#FF	#3FFFF	#FF	-----
XMASK	#3FFFF	#FFD	#3FFFF	#FFD	-----
YMASK	#3FFFF	#FFD	#3FFFF	#FFD	-----
XOS	#3FFFF	#0	#3FFFF	#0	-----

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
YOS	#3FFFF	#0	#3FFFF	#0	-----
XT	#3FFFF	#0	#3FFFF	#0	-----
YT	#3FFFF	#0	#3FFFF	#0	-----
XH	#3FFFF	#0	#3FFFF	#0	-----
YH	#3FFFF	#0	#3FFFF	#0	-----
ZH	#3FFFF	#0	#3FFFF	#0	-----
NH	#FF	#0	#FF	#0	-----
BH	#FF	#0	#FF	#0	
D1n	#3FFFF	#0	#3FFFF	#0	n=A, B, C, D, E, F, G, H (When n is omitted, D1A is assumed.)
D2n	#3FFFF	#0	#3FFFF	#0	n=A, B, C, D (When n is omitted, D2A is assumed.)
D3B	#3FFFF	#0	#3FFFF	#0	-----
D4B	#3FFFF	#0	#3FFFF	#0	-----
RCD	#3FFFF	#0	#3FFFF	#0	-----
CCD	#3FFFF	#0	#3FFFF	#0	-----
DSD	#3FFFF	#0	#3FFFF	#0	-----
TPH(TPH1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as TPH1
TPH2	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
DCMR (DCMR1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as DCMR1
DCMR2	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
D5	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
D6	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
DBMA	#3FFFFE	#0	-----	-----	The maximum value depends on the type of interleave mode, DBM board, and RDBMA mode.
DBMLPB	#3FFFFE	DBMA	-----	-----	The maximum value depends on the type of interleave mode, DBM board, and RDBMA mode.

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
DBMLPE	#3FFFFE	Maximum value	-----	-----	The maximum value depends on the type of interleave mode, DBM board, and RDBMA mode.
RLMAX	#3FFFF	#FF	#3FFFF	#FF	#3F, #7F, #FF, #1FF, #3FF, #7FF, #FFF, #1FFF, #3FFF, #7FFF, #FFFF, #1FFFF, and #3FFFF can be specified.
ZD	#3FFFF	#0	#3FFFF	#0	-----
NDn	#FF	#0	#FF	#0	n=1 to 7 (n defaults to 1.)
BDn	#FF	#0	#FF	#0	n=1 to 7 (n defaults to 1.)
RHZA	Signal name ^{*1}	FL	Signal name ^{*1}	FL	-----
RHZA	Signal name ^{*1}	FL	Signal name ^{*1}	FL	-----
ASCNT	Signal name ^{*1}	FL	-----	-----	-----
ASINT	Signal name ^{*1}	FL	-----	-----	-----
FPSEL	Address function name ^{*2}	DISABLE	Address function name ^{*2}	DISABLE	-----

"-----" indicates a register in which SUB PG cannot be specified independently.

^{*1} Specify one of the following signal names:

FIXL(FL), FIXH(FH), R(RD), W(WT), M1-2, C0-23

^{*2} Specify one of the following address function names or *DISABLE*:

FP0, FP1, FP3, FP4, FP5, FP6, FP7, FP8, FP9, FP10

Table A 1-2 List of Registers (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
TIMER	#FFFF	#100F	-----	-----	Time can be specified (100 ns to 409.5 s)
TREFRESH	#FFFF	#100F	-----	-----	Time can be specified (100 ns to 409.5 s)
TPAUSE	#FFFF	#100F	-----	-----	Time can be specified (100 ns to 409.5 s)
PC	#3FF	#0	-----	-----	Up to #1FF in PCC mode.
ISP	#3FF	#0	-----	-----	Up to #1FF in PCC mode.

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
BAR	#3FF	#0	----	----	Up to #1FF in PCC mode.
IDXn	#FFFFFF	#0	----	----	n=1 to 2
IDXm	#FFFFFF	#0	----	----	m=3 to 8
CFLG	#FF	#0	----	----	----
CPEn	Signal name ^{*1}	R	Signal name ^{*1}	R	n=1 to 4
DRE(DRE1)	Signal name ^{*1}	W	Signal name ^{*1}	W	Can be described as DRE1
DRE2	Signal name ^{*1}	W	Signal name ^{*1}	W	----
RINV	Signal name ^{*1}	R	Signal name ^{*1}	R	----
LMAX	#FFFF	XMAX	#FFFF	SUB XMAX	----
HMAX	#FFFF	YMAX	#FFFF	SUB YMAX	----
XMAX	#FFFF	#FFF	#FFFF	#FFF	----
YMAX	#FFFF	#FFF	#FFFF	#FFF	----
ZMAX	#FFFF	#FF	#FFFF	#FF	----
XMASK	#FFFF	#FFD	#FFFF	#FFD	----
YMASK	#FFFF	#FFD	#FFFF	#FFD	----
XOS	#FFFF	#0	#FFFF	#0	----
YOS	#FFFF	#0	#FFFF	#0	----
XT	#FFFF	#0	#FFFF	#0	----
YT	#FFFF	#0	#FFFF	#0	----
XH	#FFFF	#0	#FFFF	#0	----
YH	#FFFF	#0	#FFFF	#0	----
ZH	#FFFF	#0	#FFFF	#0	----
NH	#F	#0	#F	#0	----
D1	#FFFF	#0	#FFFF	#0	----
D2	#FFFF	#0	#FFFF	#0	----
D3B	#FFFF	#0	#FFFF	#0	----
D4B	#FFFF	#0	#FFFF	#0	----
RCD	#FFFF	#0	#FFFF	#0	----
CCD	#FFFF	#0	#FFFF	#0	----

Register	MAIN		SUB		Remarks
	Maximum value	Default value	Maximum value	Default value	
DSD	#FFFF	#0	#FFFF	#0	-----
TPH(TPH1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as TPH1
TPH2	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
DCMR (DCMR1)	#FFFFFFFF	#0	#FFFFFFFF	#0	Can be described as DCMR1
DCMR2	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
D5	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
D6	#FFFFFFFF	#0	#FFFFFFFF	#0	-----
DBMA	#3FFFC	#0	-----	-----	The maximum value depends on the type of interleave mode and DBM board.
DBMLPB	#3FFFC	DBMA	-----	-----	The maximum value depends on the type of interleave mode. Can be described in T5592.
DBMLPE	#3FFFC	Maximum value	-----	-----	The maximum value depends on the type of interleave mode. Can be described in T5592.
RLMAX	#FFFF	#FF	#FFFF	#FF	#3F, #7F, #FF, #1FF, #3FF, #7FF, #FFF, #1FFF, #3FFF, #7FFF, and #FFFF can be specified.

"-----" indicates a register for which SUB PG cannot be specified independently.

*1 Specify one of the following signal names:

FIXL (FL), FIXH (FH), R (RD), W (WT), M1-2, C0-15, PREF (For PREF, CPE1 to 4 only)

ARIRAM DEFINE (Setting area inversion memory)

ARIRAM DEFINE

MAIN ARIRAM DEFINE

SUB ARIRAM DEFINE

$X = X_{k_1} O_1 X_{k_2} \dots$

$Y = Y_{l_1} O_2 Y_{l_2} \dots$

$INV = X O_3 Y$

$0 \leq k_n \leq 15$ On: &=.AND.

$0 \leq l_n \leq 15$ "=.XOR.

'=.OR.

%=.NOT.

$D_n = X_{k_1} O_1 Y_{l_1} \dots$

T5593	For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P
$0 \leq n \leq 17$	$0 \leq n \leq 17$
$0 \leq k_n \leq 17$	$0 \leq k_n \leq 15$
$0 \leq l_n \leq 17$	$0 \leq l_n \leq 15$

On:&=.AND.

"= .XOR.

'= .OR.

%= .NOT.

$AR_{Ij} = X_{k_1} O_1 Y_{l_1} \dots$

$D_n = AR_{Ij}$

T5593	For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P
$0 \leq j \leq 7$	$0 \leq j \leq 7$
$0 \leq n \leq 17$	$0 \leq n \leq 17$
$0 \leq k_n \leq 17$	$0 \leq k_n \leq 15$
$0 \leq l_n \leq 17$	$0 \leq l_n \leq 15$

On:&=.AND.

"= .XOR.

'= .OR.

%= .NOT.

ADDRESS DEFINE

ADDRESS DEFINE

MAIN ADDRESS DEFINE

SUB ADDRESS DEFINE

$X_{l-m} = Z_{p-q}, N_{r-s}, B_{t-u}$

$Y_{l-m} = Z_{p-q}, N_{r-s}, B_{t-u}$

XY=ZN(ZNB) or XY .OR. ZN(ZNB)

T5593	For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P
$0 \leq l, m \leq 17$	$0 \leq l, m \leq 15$
$0 \leq p, q \leq 17$	$0 \leq p, q \leq 15$
$0 \leq r, s \leq 7$	$0 \leq r, s \leq 3$
$0 \leq t, u \leq 7$	Cannot be specified

BURST BLOCK

BURST BLOCK BEGIN (Block-name)

VS_n=vs, r (vr)

VS_n=vs

SEND EXT (address) data

WAIT TIME n n=100ns-409.5s

STISP m

STIn m n=1 to 8

BURST BLOCK END

SDEF

SDEF name=ALPG instruction

SCRAM (Specifying the scramble memory file)

PRESCRAM (Specifying the prescramble memory file)

SCRAM file-name

MAIN SCRAM file-name

SUB SCRAM file-name

PRESCRAM file-name

MAIN PRESCRAM file-name

SUB PRESCRAM file-name

PCC (Specifying the PC operation mode) (Only for T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P 1WAY)

PCC n (n=1-2)

DATA DEFINE

MAIN DATA DEFINE

SUB DATA DEFINE

PDS DATA = D/D

PDS DATA = CBM1/D

PDS DATA = FDA/D

PDS DATA = D/CBM2

PDS DATA = CBM1/CBM2

PDS DATA = FDA/CBM2

PDS DATA = D/FDB

PDS DATA = CBM1/FDB

PDS DATA = FDA/FDB

However, in the case of definition by SUB DATA DEFINE statement, FDA and FDB cannot be specified.

BWPE ADDRESS

BWPE ADDRESS a0, a1, a2, a3, a4, a5

MAIN BWPE ADDRESS a0, a1, a2, a3, a4, a5

SUB BWPE ADDRESS a0, a1, a2, a3, a4, a5

a0 to a5

For T5593

X0-17 or Y0-17 or 0 (Continuous specification is possible.)

For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P

X0-15 or Y0-15 or 0 (Continuous specification is possible.)

BANK SAVE ADDRESS

BANK SAVE ADDRESS a0, a1, a2, a3, a4, a5

a0 to a5 = X0 to 17 or Y0 to 17 or 0 (Continuous specification is possible.)

Can be executed on T5593.

BANK LOAD ADDRESS

BANK LOAD ADDRESS a0, a1, a2, a3, a4, a5

a0 to a5 = X0 to 17 or Y0 to 17 or 0 (Continuous specification is possible.)

Can be executed on T5593.

AFM PATTERN

AFM PATTERN

BITSIZE = m, n

m=8, 9 $1 \leq n \leq 4$

m=16, 18 $1 \leq n \leq 2$

m=32, 36 $1 \leq n \leq 1$

AMAX = x1, y1

$0 \leq x1, y1 \leq 65535$

AMIN = x2, y2

$0 \leq x2, y2 \leq 65535$ and $x2 \leq x1$ $y2 \leq y1$

FMA = x3, y3

$x2 \leq x3 \leq x1$ $y2 \leq y3 \leq y1$

ROM Test Pattern Data

DBM PATTERN

DBM PATTERN

MAIN DBM PATTERN

SUB DBM PATTERN

DBM CHANNEL i-j

If DBM is 80 bit width i, j = 0-79

If DBM is 96 bit width i, j = 0-95

Either continuous number specification or random number specification is available for i and j.

DBM PINSEL(CHTYPE_t, BITMODE_u) t=3, 4, 5, 9, 10, 12, 13 u=1 to 4

D_n=P_m

:

n=Bit defined by DBM CHANNEL statement.

m=1-512

DBMA n

For the 64 K DBM board

1WAY: #0 ≤ n ≤ #FFFF

2WAY: #0 ≤ n ≤ #1FFFE and is a multiple of 2.

4WAY: #0 ≤ n ≤ #3FFFC and is a multiple of 4.

For the 256 K DBM board

1WAY: #0 ≤ n ≤ #3FFFF

2WAY: #0 ≤ n ≤ #7FFFE and is a multiple of 2.

4WAY: #0 ≤ n ≤ #FFFFC and is a multiple of 4.

For the 1 M DBM board

1WAY: #0 ≤ n ≤ #FFFFF

2WAY: #0 ≤ n ≤ #1FFFFE and is a multiple of 2.

4WAY: #0 ≤ n ≤ #3FFFFC and is a multiple of 4.

For the 2 M DBM board

1WAY: #0 ≤ n ≤ #1FFFFF

2WAY: #0 ≤ n ≤ #3FFFFE and is a multiple of 2.

4WAY: #0 ≤ n ≤ #7FFFFC and is a multiple of 4.

(Falls within the 1 M board range when MODE RDBMA is specified.)

DBM STORAGE n

1WAY: #0 ≤ n ≤ #400000

2WAY: #0 ≤ n ≤ #800000 and is a multiple of 2.

4WAY: #0 ≤ n ≤ #1000000 and is a multiple of 4.

DBM pattern data

CBM PATTERN

CBM PATTERN

MAIN CBM PATTERN

SUB CBM PATTERN

CBM CHANNEL i-j

i, j=0-35 Continuous specification and random specification are available.

CBMA n

ECBM not specified

$\#0 \leq n \leq \#7FFF$

ECBM specified

2WAY: $\#0 \leq n \leq \#FFFE$ and is a multiple of 2.

4WAY: $\#0 \leq n \leq \#1FFFC$ and is a multiple of 4.

CBM pattern data

MODULE (Defining module)

MODULE BEGIN

(Setting the MODE in the module)

(pattern in the module)

MODULE END

field A

Table A 1-3 List of field A Instructions

field A	T5593	For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P
NOP	----	----
JMP l	l:#0 to #7FF	l:0 to #3FF
JSR l *1	l:#0 to #7FF	l:0 to #3FF
IDXIn m *1	n:1 to 8 m:#0 to #FFFFFF	n:1 to 8 m:#0 to #3FF
LDIn m	n:1 to 8 m:9 to 16 or m=n	Cannot be executed.
STIn m	n:1 to 8 m:#0 to #FFFFFF	Cannot be executed.
STISP m	m:#0 to #7FF	Cannot be executed.
JNIn l *1	n:1 to 8 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF
JNCn l *1	n:1 to 16 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF
FLGLI1 l *1	Cannot be executed.	l:#0 to #3FF Cannot be executed on T5592 and T5591.
RTN *1	----	----
JZD l	l:#0 to #7FF	l:#0 to #3FF
JET l	l:#0 to #7FF	l:#0 to #3FF
WAIT	----	----
OUT block-name	----	----
STPS	----	----

field A		T5593	For T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P
STFL		----	----
SET XH	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
YH	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
D1 (D1A)	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
D2 (D2A)	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
D3B	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
D4B	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
NH	dt2	dt2:#0 to #FF	dt2:#0 to #F
ZH	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
Z	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
XOS	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
YOS	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
XT	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
YT	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
XMASK	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
YMASK	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
RCD	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
CCD	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
DSD	dt1	dt1:#0 to #3FFFF	dt1:#0 to #FFFF
DSC	dt3	dt3:#0 to #F	dt3:#0 to #F
TP (TP1)	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
TP2	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
TPH (TPH1)	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
TPH2	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
D5	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
D6	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
DCMR (DCMR1)	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
DCMR2	dt4	dt4:#0 to #FFFFFFFF *2	dt4:#0 to #FFFFFFFF *2
CBMA	dt5	dt5:#0 to #1FFFC *3	dt5:#0 to #1FFFC *3
ND (ND1)	dt2	dt2:#0 to #FF	Cannot be executed.
BD (BD1)	dt2	dt2:#0 to #FF	Cannot be executed.
BH	dt2	dt2:#0 to #FF	Cannot be executed.
ZD	dt1	dt1:#0 to #3FFFF	Cannot be executed.
I		----	----
T		----	----
TM1		----	----
TM2		----	----
DBMINC		----	----
DBMSET (m)		m:#0 to #1FFFE *4	Cannot be executed.

*1 This instructions cannot be described simultaneously with TM2 or T (in PAUSE mode).

*2 When the TP18 bit mode (MODE TPM18) is specified, $0 \leq m \leq \#3FFFF$ can be specified for TP1, TP2, TPH1, TPH2, D5, D6, DCMR1, and DCMR2.

*3 The setting range for CBMA depends on the specification of the interleaved mode and MODE ECBM.

*4 The setting range for DBMSET depends on the type of the interleave mode and DBM board.

 Tip

- The field B or field C and I, T, TM1, TM2, DBMINC, or DBMSET cannot be described after any of the WAIT, STPS, and STFL instructions.
 - The FLGLH instruction cannot be executed on T5593, T5592, and T5591.
-

field B & field C (ALPG)

• XB & YB

XB<XB		A<D1A	*1
XB<XB+1		A<D1B	*1
XB<XB-1		A<D1C	*1
XB<XH		A<D1D	*1
XB<0		A<D1E	*1
XB<A	*1	A<D1F	*1
XB<XB+A	*1	A<D1G	*1
XB<XB-A	*1	A<D1H	*1
		A<D2A	*1
		A<D2B	*1
		A<D2C	*1
		A<D2D	*1
YB<YB		A<D1A	*1
YB<YB+1		A<D1B	*1
YB<YB-1		A<D1C	*1
YB<YH		A<D1D	*1
YB<YB+1^BX		A<D1E	*1
YB<YB-1^BX		A<D1F	*1
YB<YB+1+BX		A<D1G	*1
YB<YB-1-BX		A<D1H	*1
YB<0		A<D2A	*1
YB<A	*1	A<D2B	*1
YB<YB+A	*1	A<D2C	*1
YB<YB-A	*1	A<D2D	*1
YB<YB+A^BX	*1		
YB<YB-A^BX	*1		
YB<YB+A+BX	*1		
YB<YB-A-BX	*1		

*1 Can be executed on T5593.

- Z

Z<Z
 Z<Z+1
 Z<Z-1
 Z<ZH
 Z<Z*2
 Z</Z
 Z<0
 Z<Z+1^BX
 Z<Z-1^BX
 Z<Z+1^CY
 Z<Z-1^CY
 Z<Z+ZD *1
 Z<Z-ZD *1
 Z<Z+ZD^BX *1
 Z<Z-ZD^BX *1
 Z<Z+ZD^CY *1
 Z<Z-ZD^CY *1

*1 Can be executed on T5593.

- XC & XS & XK

XC<XC	XS<XS	XK<XK	F<A	A<XB	B<D1A	
XC<F	XS<XS	XK<XK	F<A+1	A<XC	B<D1B	*1
XC<XS	XS<XS	XK<XK	F<A-1	A<XS	B<D1C	*1
XC<XK	XS<XS	XK<XK	F<A*2	A<XK	B<D1D	*1
XC<XC	XS<F	XK<XK	F</A		B<D1E	*1
XC<XC	XS<XC	XK<XK	F<B		B<D1F	*1
XC<XC	XS<XS	XK<F	F<A+B		B<D1G	*1
XC<XC	XS<XS	XK<XC	F<A-B		B<D1H	*1
XCS<F		XK<XK	F<A&B		B<D2A	
XCK<F	XS<XS		F<A"B		B<D2B	*1
XCSK<F			F<A'B		B<D2C	*1
XC<XS<F		XK<XK	F<0		B<D2D	*1
XS<XC<F		XK<XK			B<D3	
XC<XK<F	XS<XS				B<D4	
XK<XC<F	XS<XS					
XC<>XS		XK<XK				
XC<>XK	XS<XS					

*1 Can be executed on T5593.

• YC & YS & YK

YC<YC	YS<YS	YK<YK	F<A	A<YB	B<D1A
YC<F	YS<YS	YK<YK	F<A+1	A<YC	B<D1B *1
YC<YS	YS<YS	YK<YK	F<A-1	A<YS	B<D1C *1
YC<YK	YS<YS	YK<YK	F<A*2	A<YK	B<D1D *1
YC<YC	YS<F	YK<YK	F</A		B<D1E *1
YC<YC	YS<YC	YK<YK	F<B		B<D1F *1
YC<YC	YS<YS	YK<F	F<A+B		B<D1G *1
YC<YC	YS<YS	YK<YC	F<A-B		B<D1H *1
YCS<F		YK<YK	F<A&B		B<D2A
YCK<F	YS<YS		F<A"B		B<D2B *1
YCSK<F			F<A'B		B<D2C *1
YC<YS<F		YK<YK	F<0		B<D2D *1
YS<YC<F		YK<YK	F<A+1+CY		B<D3
YC<YK<F	YS<YS		F<A-1-CY		B<D4
YK<YC<F	YS<YS		F<A+B+CY		
YC<Y>YS		YK<YK	F<A-B-CY		
YC<Y>YK	YS<YS		F<A+1+BX		
			F<A-1-BX		
			F<A+B+BX		
			F<A-B-BX		
			F<A+1^CY		
			F<A-1^CY		
			F<A+B^CY		
			F<A-B^CY		
			F<A+1^BX		
			F<A-1^BX		
			F<A+B^BX		
			F<A-B^BX		

*1 Can be executed on T5593.

• N

N<N	A<ND1 *1
N<N+1	A<ND2 *1
N<N-1	A<ND3 *1
N<NH	A<ND4 *1
N<0	A<ND5 *1
N</N	A<ND6 *1
N<A	A<ND7 *1
N<N+A *1	
N<N-A *1	

*1 Can be executed on T5593.

• B

B<B	A<BD1
B<B+1	A<BD2
B<B-1	A<BD3
B<BH	A<BD4
B<0	A<BD5
B</B	A<BD6
B<A	A<BD7
B<B+A	
B<B-A	

Can be executed on T5593.

- D3 & D4

D3<D3	D4<D4
D3<D3B	D4<D4B
D3<D3*2	D4<D4*2
D3<D3*2L	D4<D4*2L
D3<D3+1	
D3<D3-1	
D3<0	

- RF

RF<RF
RF<RF+1

- X & Y

X<XB	Y<YB
X<XC	Y<YC
X<XS	Y<YS
X<XK	Y<YK
X<YB	Y<XB
X<YC	Y<XC
X<YS	Y<XS
X<YK	Y<XK
X<XB'Z	Y<XB'Z
X<XC'Z	Y<XC'Z
X<XS'Z	Y<XS'Z
X<XK'Z	Y<XK'Z
X<Z	Y<Z
X<RF	

- /X & /Y & /Z

/X /Y /Z

- SWAP

LMAX<>HMAX

- TP

TP<TP
TP<TP+1
TP<TP-1
TP</TP
TP<0
TP<TP*2
TP<TPH
TP<D5
T</D5
TP<TP+D5
TP<TP-D5
TP<TP&D5
T<TP"D5
TP<TP'D5
TP<TPS
TP<>TPS
TPS<TP

- TP2

TP2<TP2
 TP2<TP2+1
 TP2<TP2-1
 TP2</TP2
 TP2<0
 TP2<TP2*2
 TP2<TPH2
 TP2<D6
 TP2</D6
 TP2<TP2+D6
 TP2<TP2-D6
 TP2<TP2&D6
 TP2<TP2"D6
 TP2<TP2'D6
 TP2<TPS2
 TP2<>TPS2
 TPS2<TP2

- Data select

D<TP
 D<TP2
 D<TPXOR
 D<CBM
 D<BR

- DSD & RCD & CCD

DSD<0
 RCD<0
 CCD<0
 DSD<DCD+1
 RCD<RCD+1
 CCD<CCD+1
 RCD<RCD+1 CCD<CCD+1

- /D & /D2

/D (/D1)
 /D2

- PD

PD

- XT & YT

XT
 YT

- XASV & XALD

XASV
 XALD

XASV and XALD can be executed on T5593.

- SCROFF & ARIOFF & PSCROFF

SCROFF
ARIOFF
PSCROFF

- PM/2 & PM/4

PM/2
PM/4

- DSET

XH<m
YH<m
D1<m
D2<m
D3B<m
D4B<m
NH<m
ZH<m
Z<m
XOS<m
YOS<m
XT<m
YT<m
TP1<m
TP2<m
TPH1<m
TPH2<m
D5<m
D6<m
DCMR1<m
DCMR2<m
XMASK<m
YMASK<m
CBMA<m
DSC<m
RCD<m
CCD<m
DSD<m
ND1<m
BD1<m
BH<m
ZD<m

➔ For the setting range for m, refer to the description of the Field A SET instruction.

- DSC

DSC<DSC
DSC<#F
DSC<DSC*2
DSC<DSC/2

- FP

FP0 (Fix0)
 FP1 (Checker board)
 FP3 (Diagonal)
 FP4 (Inversion diagonal)
 FP5 (Diagonal 2) *1
 FP6 (Inversion diagonal 2) *1
 FP7 (Mask parity)
 FP8 (Row bar)
 FP9 (Column bar)
 FP10 (Row/column bar)
 FPX (Universal FP) *1

**1 FP5, FP6, and FPX can be executed on T5593.*

- CBMA

CBMA<CBMA
 CBMA<CBMA+1

- Block Write Pseudo Emulator

BDW
 BMW
 BCW
 BDR
 BMR
 BCR

- Control bit

MMA
 MMB
 R W M1 M2 C0-C23

C16 to C23 can be executed on T5593.

- Timing set control

TSn [n=1 to 16]
 /Tn [n=1 to 16]

- Cycle palette

CYPAn [n=1 to 16]
 CYPBn [n=1 to 16]

CYPAn can be executed on T5593 and T5592.

CYPBn can be executed on T5593.

- INHDBM

INHDBM

INHDBM can be executed on T5593 and T5592.

List of Figures

Figure 1-1	ALPG Function Block Diagram (T5593)	1-3
Figure 1-2	ALPG Function Block Diagram (T5592, T5591, T5586, T5585, T5581T, T5581H, T5581P, and T5571P)	1-4
Figure 2-1	Specifiable Operation Modes (T5593)	2-9
Figure 2-2	Specifiable Operation Modes (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)	2-11
Figure 2-3	Block Diagram of Byte Mask Mode Outline	2-56
Figure 2-4	Format for COLM Packet and D Packet	2-62
Figure 2-5	Block Diagram of ALPG Outline	2-78
Figure 2-6	Function Block Diagram of Sequence Control Section	2-79
Figure 2-7	Address Generator Section Function Block Diagram (T5593)	2-96
Figure 2-8	Address Generator Section Function Block Diagram (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)	2-97
Figure 2-9	Data Pattern Generator Section Function Block Diagram (T5593)	2-146
Figure 2-10	Data Pattern Generator Section Function Block Diagram (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)	2-147
Figure 2-11	Outline of 8 Column × 8 I/O × 4 Byte Block Write Specification	2-167
Figure 2-12	Block Diagram of BWPE Outline	2-167
Figure 2-13	Read Operation of Multi-bank Operation	2-192
Figure 4-1	Example of VO Measurement Timing	4-10
Figure 4-2	Operation by Control Bit "I" in MODE REFRESHA n	4-21
Figure 4-3	Operation by Control Bit "I" in MODE REFRESHB n	4-21
Figure 4-4	Refresh Cycle	4-24
Figure 4-5	RAS Only Refresh Cycle	4-30
Figure 4-6	Refresh Cycle for Memory with a Refresh Control Terminal	4-37
Figure 4-7	VS Change during Bump Test	4-44
Figure 4-8	Burst Data Transfer and VS Change	4-46
Figure 4-9	Out Instruction and VS Change	4-49
Figure 5-1	Page Mode Time Chart	5-2
Figure 5-2	Timing Chart for 2 K × 8-bit Dual Port Static Memory	5-20
Figure 5-3	Timing Chart for Test Program (1)	5-23
Figure 5-4	Timing Chart for Test Program (2)	5-24

List of Tables

Table P-1	Models Adapted for Test System Descriptions	Preface-1
Table P-2	ALPG Configuration and Allocation of MAIN PG and SUB PG for Each Interleave Mode	Preface-1
Table P-3	Specification of the Interleave Mode and Corresponding Operation Frequency	Preface-3
Table 2-1	Relation between TP1 Data and FP Function and ARIRAM Data ..	2-19
Table 2-2	Relation between TP2 Data and FP Function and ARIRAM Data ..	2-20
Table 2-3	List of Registers (T5593)	2-22
Table 2-4	List of Registers (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)	2-25
Table 2-5	Specification Range for l, m, p, q, r, s, t, and u	2-36
Table 2-6	Number of Burst Data Words	2-40
Table 2-7	Allocation of Mask Bit (b00 - b37)	2-61
Table 2-8	Index Reference Instructions of the Sequence Control Section	2-82
Table 2-9	Subroutine Call Instructions	2-85
Table 2-10	Registers the SET Instruction can Set	2-86
Table 2-11	Condition Flag Reference Branch Instructions	2-89
Table 2-12	Data Flag Reference Branch Instructions	2-89
Table 2-13	Timer Flag Reference Branch Instructions	2-90
Table 2-14	Match Flag Sense Instruction	2-91
Table 2-15	Output Data to DA0 to 17 and DB0 to 17 when D is Specified for the Test Data	2-165
Table 2-16	Specification Ranges for n and m of CYPAn and CYPBm	2-188
Table 2-17	PIN Statement Specifications and Driver Enable Control	2-225
Table 2-18	PIN Statement Specifications and Comparator Enable Control	2-226
Table 4-1	Output Bits of the Refresh Counter (T5593)	4-18
Table 4-2	Output Bits of the Refresh Counter (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)	4-19
Table 4-3	Refresh Period (for REFRESH A)	4-25
Table 4-4	Refresh Period (for REFRESH B)	4-26
Table 4-5	Example of DRAM Data Topology	4-41
Table 4-6	Execution Time (t) of OUT Instruction	4-49
Table 4-7	OUT Instruction Transfer Time	4-50
Table 5-1	Match Flag Sense Instruction	5-18
Table A 1-1	List of Registers (T5593)	A1-9

Table A 1-2 List of Registers (T5592, T5591, T5586, T5585, T5581, T5581H, T5581P, and T5571P)	A1-11
Table A 1-3 List of field A Instructions	A1-18

Index

[Symbols]

%ENDC	2-71
%IFE	2-71
%IFN	2-71
%INSERT	2-75
%REMARK	2-71
/D	2-163
/D2	2-163

[A]

ADDRESS DEFINE	2-36
AFM PATTERN	2-49, 2-193
AMAX	2-195
AMIN	2-196
ARIOFF(control bit)	2-186
ARIRAM DEFINE	2-32
ASCNT	2-32
ASINI	2-32

[B]

BANK LOAD ADDRESS	2-54
BANK SAVE ADDRESS	2-52
BAR(REGISTER)	2-28
BCR	2-166
BCW	2-166
BD	2-31, 2-87
BDR	2-166
BDW	2-166
BH	2-87
BITSIZE	2-194
BMR	2-166
BMW	2-166
BURST BLOCK	2-39
BURST BLOCK BEGIN	2-40
BURST BLOCK END	2-40
BWPE	2-165
BWPE ADDRESS	2-50
BWPEM(MODE)	2-14
BYTEMASK DEFINE	2-56

[C]

CBM CHANNEL	2-232
CBM PATTERN	2-49
CBMA	2-169, 2-233
CBMA(SET)	2-87
CCD	2-162
CCD(REGISTER)	2-31
CCD(SET)	2-86
CFLG(REGISTER)	2-28
CPEn(REGISTER)	2-29
CYPAn	2-187
CYPBm	2-187
CYPC15(MODE)	2-18

[D]

D1(REGISTER)	2-29
D1(SET)	2-86
D2(REGISTER)	2-29
D2(SET)	2-86
D3B	2-98
D3B(REGISTER)	2-29
D3B(SET)	2-86
D4B	2-98
D4B(REGISTER)	2-29
D4B(SET)	2-86
D5	2-148
D5(REGISTER)	2-30
D5(SET)	2-87
D6	2-148
D6(REGISTER)	2-30
D6(SET)	2-87
DATA DEFINE	2-46
DATEN(MODE)	2-12
DBFORMn	2-18
DBM CHANNEL	2-203
DBM PATTERN	2-49
DBM PINSEL	2-204
DBM STORAGE	2-229
DBMA	2-218
DBMA(REGISTER)	2-31

DBMINC(control bit)	2-94	HMAX(REGISTER)	2-29
DBMLP(MODE)	2-17		
DBMLPB(REGISTER)	2-31	[I]	
DBMLPE(REGISTER)	2-31	I(control bit)	2-93
DBMSET(m)	2-94	IDXIn	2-83
DCMR	2-148	IDXm(REGISTER)	2-28
DCMR(REGISTER)	2-30	IDXn(REGISTER)	2-28
DCMR1(SET)	2-87	ILMODE	2-6
DCMR2	2-148	INHDBM(control bit)	2-189
DCMR2(SET)	2-87	INSERT	2-75
DFCA(MODE)	2-15	ISP(REGISTER)	2-28
DFCB(MODE)	2-15		
DFLG	2-164	[J]	
DPUTYPE	2-41	JET	2-90
DRE(REGISTER)	2-29	JNCn	2-88
DSC(SET)	2-87	JNIn	2-83
DSD	2-162	JSR	2-85
DSD(REGISTER)	2-31	JZD	2-89
DSD(SET)	2-86		
		[L]	
[E]		LDIn	2-83
ECBM(MODE)	2-17	LMAX	2-98
END	2-66	LMAX(REGISTER)	2-29
[F]		[M]	
FLGLI1	2-91	MAIN BWPE ADDRESS	2-50
FMA	2-196	MAIN MODE	2-7
FP	2-148	MAIN REGISTER	2-22
FP0	2-153	MEAS MPAT	3-6
FP1	2-154	MODE	2-7
FP10	2-160	MODULE BEGIN	2-48
FP3	2-154	MODULE END	2-48
FP4	2-155	MPAT	2-5
FP5	2-156	MSM	2-21
FP6	2-157	MUX(MODE)	2-12
FP7	2-159		
FP8	2-159	[N]	
FP9	2-160	N	2-98
FPSEL	2-32	ND	2-31, 2-87
FPX	2-161	NH	2-98
		NH(REGISTER)	2-29
[H]		NH(SET)	2-86
HMAX	2-98		

[O]		
OUT.....	2-92	SET 2-86
[P]		START 2-43
PARITY(MODE).....	2-13	STFL 2-93
PAUSE(MODE).....	2-12	STIn..... 2-83, 2-240
PC(REGISTER).....	2-28	STISP 2-239
PCC.....	2-44	STPS..... 2-93
PD.....	2-164	SUB BWPE ADDRESS 2-50
PDS DATA 2-46		SUB MODE 2-7
PKT 2-242		SUB REGISTER 2-22
PM(control bit).....	2-186	[T]
PRESCRAM.....	2-43	T(control bit)..... 2-93
PSCROFF(control bit).....	2-186	TIMER(REGISTER)..... 2-27
[R]		TM1(control bit)..... 2-93
RCD 2-162		TM2(control bit)..... 2-93
RCD(REGISTER).....	2-31	TP 2-147, 2-151
RCD(SET).....	2-86	TP1(SET)..... 2-87
RDBMA 2-21		TP2..... 2-147, 2-151
RDX.....	2-6	TP2(SET)..... 2-87
REFRESHA(MODE).....	2-13	TP2INV(MODE)..... 2-14
REFRESHB(MODE).....	2-13	TPAUSE(REGISTER)..... 2-27
REG MPAT 3-11		TPH 2-148
REGISTER.....	2-22	TPH(REGISTER)..... 2-29
REPEAT..... 2-222, 2-238		TPH1(SET)..... 2-87
RF.....	2-99	TPH2..... 2-148
RHZA.....	2-31	TPH2(SET)..... 2-87
RHZB.....	2-31	TPM(MODE)..... 2-14
RINV(REGISTER).....	2-29	TREFRESH(REGISTER)..... 2-27
RLMAX(REGISTER).....	2-31	TSn(control bit)..... 2-175
ROM.....	2-197	[W]
RTN.....	2-85	WAIT 2-93
[S]		WINDOW MODE 3-20
SCRAM 2-43		WINDOW XADRS..... 3-20
SCROFF(control bit).....	2-185	WINDOW YADRS..... 3-20
SDEF 2-42		[X]
SELECT MPAT 3-12		XALD..... 2-190
SELECT SCRDATA.....	3-16	XASV 2-190
SELECT SCRDATA DISABLE.....	3-16	XB..... 2-97
SELECT SCRDATA ENABLE.....	3-16	XC..... 2-98
SEND SCRDATA.....	3-16	XH..... 2-97
		XH(REGISTER)..... 2-29

XH(SET).....	2-86	ZMAX.....	2-99
XK.....	2-98	ZMAX(REGISTER).....	2-29
XMASK.....	2-148	ZSET1(MODE).....	2-14
XMASK(REGISTER)	2-30		
XMASK(SET).....	2-86		
XMAX	2-99		
XMAX(REGISTER)	2-29		
XOS	2-99		
XOS(REGISTER)	2-30		
XOS(SET).....	2-86		
XS.....	2-98		
XT	2-99		
XT(control bit)	2-185		
XT(REGISTER).....	2-30		
XT(SET)	2-86		

[Y]

YB.....	2-97
YC.....	2-98
YH.....	2-97
YH(REGISTER)	2-29
YH(SET).....	2-86
YK.....	2-98
YMASK.....	2-148
YMASK(REGISTER)	2-30
YMASK(SET).....	2-86
YMAX	2-99
YMAX(REGISTER)	2-29
YOS	2-99
YOS(REGISTER)	2-30
YOS(SET).....	2-86
YS.....	2-98
YT	2-99
YT(control bit)	2-185
YT(REGISTER).....	2-30
YT(SET)	2-86

[Z]

Z.....	2-97
Z(SET).....	2-86
ZD.....	2-31, 2-87
ZH.....	2-97
ZH(REGISTER).....	2-29
ZH(SET).....	2-86